PlussMic User Documentation

PlussMa Contributors

Jun 26, 2026

CONTENTS

1 PlussMic User Manual **3**
- 1.1 How to read this manual . . . 3
- 1.2 Example policy . . . 4
- 1.3 Verified examples . . . 4

2 Quickstart **103**
- 2.1 Include Paths . . . 103
- 2.2 Load A Table Fast . . . 103
- 2.3 Start With A Small In-Memory Frame . . . 103
- 2.4 Terminal vs Popup Preview . . . 103
- 2.5 Typical DataFrame Workflow . . . 104
- 2.6 Convert Between Models . . . 104
- 2.7 Save Results . . . 104

3 Demo Catalog **105**
- 3.1 Table Examples . . . 105
- 3.2 Kindle Examples . . . 105
- 3.3 Bundle Examples . . . 105

4 DataFrame Guide **107**
- 4.1 What DataFrame Is Good At . . . 107
- 4.2 Construction And Loading . . . 107
- 4.3 Inspect Rows, Features, And Cases . . . 107
- 4.4 Cast And Derive Features . . . 108
- 4.5 Filter, Query, And Sort . . . 109
- 4.6 Grouping, Pivoting, And Summaries . . . 109
- 4.7 Missing Data And Rolling Windows . . . 109
- 4.8 Join And Combine . . . 110
- 4.9 Time-Aware Table Verbs . . . 110
- 4.10 SQL And IO . . . 110

5 Time-Series Accessor Guide **111**
- 5.1 API Signature . . . 111
- 5.2 Notes . . . 111
- 5.3 DataFrame Surface . . . 111
- 5.4 Thread Surface . . . 113
- 5.5 Common Pitfall . . . 114

6 DataFrame Display And Popup Viewer **115**
- 6.1 Overview . . . 115
- 6.2 Which Methods Support `.show()` . . . 115

6.3 How The Display Policy Works . . . 116
6.4 What `.show()` Takes . . . 116
6.5 Useful `DisplayOptions` . . . 116
6.6 DataFrame Titles . . . 117
6.7 OS Support . . . 118
6.8 macOS Backend . . . 118
6.9 Linux Backend . . . 118
6.10 Windows Status . . . 119
6.11 Windows Build Setup . . . 119
6.12 Windows Runtime Setup . . . 119
6.13 Conda On Windows . . . 120
6.14 Are The Popup Windows Native . . . 120
6.15 Conda Notes . . . 120
6.16 Recommended User Guidance . . . 120
6.17 Demo . . . 121
6.18 External Setup References . . . 121

7 DataFrame API Reference 123

8 Practical DataFrame API Reference 125
8.1 API Reference Style . . . 125

9 ColumnRef API Reference 343

10 Practical ColumnRef API Reference 345
10.1 API Reference Style . . . 345

11 DataFramePlotAccessor API Reference 351

12 Practical DataFramePlotAccessor API Reference 353
12.1 API Reference Style . . . 353
12.2 `df.plot(...)` Defaults And Options . . . 353
12.3 Method-Level Options Reference . . . 354

13 DataFrameSqlAccessor API Reference 413

14 Practical DataFrameSqlAccessor API Reference 415
14.1 API Reference Style . . . 415

15 DataFrameSqlPrepared API Reference 417

16 Practical DataFrameSqlPrepared API Reference 419
16.1 API Reference Style . . . 419

17 Thread And Atlas Guide 423
17.1 Thread . . . 423
17.2 Row-Space Helpers . . . 423
17.3 Atlas . . . 423
17.4 Atlas To Kindle . . . 424

18 Thread API Reference 425

19 Practical Thread API Reference 427
19.1 API Reference Style . . . 427

20 ThreadValue API Reference 471

21 Practical ThreadValue API Reference **473**
21.1 API Reference Style . . . 473

22 Atlas API Reference **481**

23 Practical Atlas API Reference **483**
23.1 API Reference Style . . . 483

24 Kindle Model Guide **509**
24.1 When To Use Kindle Types . . . 509
24.2 Stream Basics . . . 509
24.3 Catalog Basics . . . 509
24.4 Text And Similarity Helpers . . . 510
24.5 Archive Basics . . . 510
24.6 Convert Back To Table Model . . . 510

25 Stream API Reference **511**

26 Practical Stream API Reference **513**
26.1 API Reference Style . . . 513

27 Catalog API Reference **549**

28 Practical Catalog API Reference **551**
28.1 API Reference Style . . . 551

29 Archive API Reference **643**

30 Practical Archive API Reference **645**
30.1 API Reference Style . . . 645

31 Bundle Model Guide **665**
31.1 When To Use Bundle Types . . . 665
31.2 Slot . . . 665
31.3 Registry . . . 665
31.4 Aisle . . . 666
31.5 Kindle Bridge . . . 666

32 Slot API Reference **667**

33 Practical Slot API Reference **669**
33.1 API Reference Style . . . 669

34 Registry API Reference **683**

35 Practical Registry API Reference **685**
35.1 API Reference Style . . . 685

36 Aisle API Reference **761**

37 Practical Aisle API Reference **763**
37.1 API Reference Style . . . 763

38 Build Outputs **779**
38.1 HTML . . . 779
38.2 PDF . . . 779
38.3 Notes . . . 779

PlussMic is the data-analysis layer in the Pluss stack. It gives you three related models:

- `table`: `DataFrame`, `Thread`, `Atlas`
- `kindle`: `Stream`, `Catalog`, `Archive`
- `bundle`: `Slot`, `Registry`, `Aisle`

This manual is organized the same way users usually approach the library:

- start with `DataFrame` for table-style analysis
- use `Thread` when you want typed column operations outside a frame
- use `Catalog` and `Archive` when streams/case-aligned data are a better fit
- use `Registry` and `Aisle` when you want heterogeneous record bundles

CHAPTER

ONE

PLUSSMIC USER MANUAL

This manual is the long-form guide for PlussMic, the data-analysis layer of the Pluss ecosystem. It is written for readers who want to use PlussMic as a daily tool, not only look up signatures.

PlussMic has three related models:

```
table  -> DataFrame, Thread, Atlas
kindle -> Stream, Catalog, Archive
bundle -> Slot, Registry, Aisle
```

Most users start with `DataFrame`, because it is the table abstraction used for CSV, Excel, JSON, Parquet, feature engineering, exploratory analysis, plotting, and export. `Thread` is the typed column model underneath `DataFrame`. `Atlas` stacks related frames. `Kindle` and `Bundle` are lower-level record/stream models used when the data is not naturally just a rectangular table.

1.1 How to read this manual

Read Part I and Part II in order. After that, jump to the chapters that match your task:

- Loading files: Part III.
- Loading and saving files: Part IV.
- Cleaning and transforming tables: Part V.
- Column-level operations: Part VI.
- Query expressions and SQL: Part VII.
- Time series: Part VIII.
- Plotting: Part IX.
- Multi-frame Atlas data: Part X.
- Kindle and Bundle models: Part XI and Part XII.
- Recipes: Part XIII.
- Production debugging: Part XIV.

The API reference pages remain the authority for exact overload lists. This manual explains how to choose and combine those APIs.

1.2 Example policy

This manual does not use one token example per feature. Important APIs are shown in several contexts: a minimal call, a realistic workflow, option and error cases, and combinations with adjacent APIs. A short example teaches the shape of a call; later examples teach how to use it in a program.

In-memory frames are created directly when the lesson is about table operations. I/O chapters use files such as `"sales.csv"` and `"transactions.parquet"`, assumed to be in the program's working directory. The manual shows the relevant input file contents before the code. Public examples never contain paths from the author's machine.

Pluss values are passed directly where `Thread` conversion is implicit:

```
DataFrame sales({
    {"city", pluss::make_note({"Accra", "Tema"})},
    {"units", pluss::make_series({10.0, 7.0})},
});
```

Explicit `Thread` construction appears only when the example needs Thread metadata, a validity mask, or a named standalone column.

1.3 Verified examples

Examples marked as verified are backed by this source file:

```
PlussMic/docs/sphinx/manual/examples/plussmic_manual_examples.cpp
```

It is compiled as an external SDK consumer with:

```
cmake -S PlussMic/docs/sphinx/manual/examples \
  -B /tmp/plussmic_manual_examples_build \
  -DCMAKE_PREFIX_PATH="/path/to/pluss-sdk;/path/to/cpp_env"

cmake --build /tmp/plussmic_manual_examples_build --target plussmic_manual_examples
/tmp/plussmic_manual_examples_build/plussmic_manual_examples
```

The printed outputs in this manual come from running the exact displayed operations against the current PlussMic build. A table shown after `df.to_string()` is its `to_string()` result; output from `head()`, `show()`, or another renderer is never substituted. When output reveals surprising behavior, the source and tests are checked first; source-level defects are fixed before the manual teaches the behavior as correct.

1.3.1 Part I: Orientation

PlussMic exists to make C++ data work practical. It gives you table loading, typed columns, missing-data handling, display, summarization, plotting, time-series operations, and conversions into more general record models.

What PlussMic is for

Use PlussMic when you want to:

- load CSV, TXT, Excel, JSON, or Parquet data into C++;
- inspect and summarize a dataset;
- select, rename, reorder, derive, and transform features;
- filter cases with masks or query expressions;
- handle missing numeric, text, and time values;

- group, aggregate, pivot, and correlate data;
- build plotting outputs from table data;
- represent a stack of related tables with `Atlas`;
- convert rectangular data into stream/catalog or registry/slot models;
- save cleaned data back to common file formats.

The three PlussMic layers

Table

The table layer is the normal user entry point:

```
#include <pluss/mic.h>

using pluss::table::DataFrame;

auto df = DataFrame::load_csv("sales.csv");
df.info();
df.describe();
```

The table layer has:

Type	Role
`Thread`	one typed column
`DataFrame`	a 2D table of named `Thread` features
`DataFrameView`	a display-oriented frame result returned by preview/summary operations
`Atlas`	a stack of related `DataFrame` objects

Kindle

Kindle is the stream/catalog model:

Type	Role
`Stream`	typed 1D data with optional cases and metadata
`Catalog`	named streams
`Archive`	stack of catalogs

Use Kindle when stream metadata and catalog-like organization matter more than DataFrame ergonomics.

Bundle

Bundle is the heterogeneous record model:

Type	Role
`Slot`	vector of heterogeneous values
`Registry`	named slots
`Aisle`	collection-oriented bundle layer

Use Bundle when a field can hold mixed values or when a dataset is closer to a record registry than a typed rectangular table.

The core table model

PlussMic uses this table hierarchy:

```
Atlas -> DataFrame -> Thread -> Series / Note / Timeline / TimeIndex
```

A `DataFrame` owns feature names and feature columns. Each feature is a `Thread`. A `Thread` wraps one core PlussQit payload kind:

- `Series` for numeric and boolean data;
- `Note` for text data;
- `Timeline` for datetime-like data;
- `TimeIndex` for time-index data.

This design is why a `DataFrame` can contain numeric, text, and time features without losing type-specific operations.

Cases and features

PlussMic documentation uses two axis names:

- A **case** is a row-like observation.
- A **feature** is a column-like variable.

Common shape calls:

```
std::int64_t rows = df.ncases();
std::int64_t cols = df.nfeatures();
const auto& names = df.feature_names();
```

Use this language when reading APIs:

- `features(...)` selects or returns columns.
- `cases(...)`, `cloc(...)`, `slice(...)`, and `take(...)` operate in row space.
- `set_cases(...)`, `reset_cases(...)`, and `align_cases(...)` manage the case axis.

Functional style

Most table operations return a new object:

```
auto cleaned = df.fillna(0.0);
auto sorted = cleaned.sort_values({"balance"});
```

Mutating operations use a trailing underscore:

```
df.fillna_(0.0);
df.rename_("old_name", "new_name");
```

Prefer the non-mutating form while learning. Use mutating methods only when you intentionally want to update an owned object.

What a beginner should learn first

1. Load a CSV with `DataFrame::load_csv`.
2. Inspect with `head`, `info`, and `describe`.
3. Select features with `features`.
4. Filter cases with `query` or `filter`.
5. Derive features with `assign` or `set_features`.
6. Summarize with `aggregate`, `group_by`, `mean`, `corr`, and `tally`.
7. Save with `save_csv`, `save_excel`, `save_json`, or `save_parquet`.

Everything else in PlussMic builds on those ideas.

1.3.2 Part II: Installation, Includes, and Build Setup

PlussMic is usually consumed through the Pluss SDK. In a downstream project, link `Pluss::SDK` and include either the Pluss umbrella header or the PlussMic module header.

Recommended include

For applications:

```
#include <pluss/pluss.h>
```

For code that only wants PlussMic:

```
#include <pluss/mic.h>
```

<pluss/mic.h> includes table, loader, Kindle, and Bundle surfaces:

```
#include "pluss/bundle/bundle.h"
#include "pluss/kindle/kindle.h"
#include "pluss/loader/loader.h"
#include "pluss/table/table.h"
```

It also includes PlussPl when available, which enables table plotting.

Minimal consumer CMake

```
cmake_minimum_required(VERSION 3.28)
project(plussmic_app LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

find_package(Pluss CONFIG REQUIRED)

add_executable(plussmic_app main.cpp)
target_link_libraries(plussmic_app PRIVATE Pluss::SDK)
```

Configure against a staged SDK:

```
cmake -S . -B build -DCMAKE_PREFIX_PATH=/path/to/pluss-sdk
cmake --build build
```

Minimal program

```cpp
#include <iostream>
#include <pluss/mic.h>

int main() {
  auto df = pluss::table::DataFrame({
      {"city", pluss::make_note({"Accra", "Tema", "Kumasi"})},
      {"revenue", pluss::make_series({120.0, 95.0, 140.0})},
  });

  df.head();
  df.describe();
  return 0;
}
```

Optional dependencies

PlussMic can expose functionality conditionally based on how the SDK was built.

Feature	Dependency
Excel loading/saving	Excel support in the PlussMic build, commonly via libzip-backed paths
Parquet loading/saving	Arrow/Parquet dependencies discovered during SDK build
Plot exports	PlussPl availability
Popup display	GTK/WebKit or supported platform display stack
URL loading	curl support in the SDK build

If an optional format is missing, rebuild the SDK in an environment where that dependency is available, then reconfigure your consumer project.

Development build inside this repository

From the repository root:

```
cmake --preset cpp_env
cmake --workflow --preset cpp_env_build
```

Useful PlussMic examples live under:

```
PlussMic/examples/table/
PlussMic/examples/kindle/
PlussMic/examples/bundle/
```

The examples are a good way to verify that a build has working table I/O, display, plotting, and model conversion.

Header choices

Use broad headers while learning:

```
#include <pluss/pluss.h>
```

Use module headers for production libraries with tighter dependencies:

```
#include <pluss/mic.h>
#include <pluss/table/table.h>
#include <pluss/loader/loader.h>
#include <pluss/kindle/kindle.h>
#include <pluss/bundle/bundle.h>
```

Use source-local headers only when working inside the Pluss repository.

1.3.3 Part III: The Table Model

This part explains how `Thread`, `DataFrame`, `DataFrameView`, and `Atlas` relate. Understanding these types makes the rest of PlussMic predictable.

Thread

A `Thread` is one typed column. It can hold one of four payload kinds:

```
pluss::table::Thread numeric(pluss::make_series({1.0, 2.0, 3.0}));
pluss::table::Thread text(pluss::make_note({"red", "green", "blue"}));
pluss::table::Thread dates(pluss::make_timeline({
    "2024-01-01T00:00:00Z",
    "2024-01-02T00:00:00Z",
    "2024-01-03T00:00:00Z",
}));
```

Inspect a thread:

```
thread.defined();
thread.size();
thread.kind();
thread.dtype_name();
thread.info();
```

Access the payload only after you know the kind:

```
if (thread.kind() == pluss::table::ThreadKind::Series) {
  auto values = thread.as_series();
}
```

Missingness and validity

`Thread` can carry a validity mask. Loader-created columns use this to represent missing values from input files.

```
auto missing = thread.isnull();
auto present = thread.notnull();
auto filled = thread.fillna(0.0);
auto complete = thread.dropna();
```

Numeric threads also support `isnan` and `isfinite`.

DataFrame

A `DataFrame` is a set of named `Thread` features with an optional case axis.

```
auto df = pluss::table::DataFrame({
    {"region", pluss::make_note({"north", "south"})},
    {"sales", pluss::make_series({10.0, 12.5})},
});
```

Shape and schema:

```
df.defined();
df.ncases();
df.nfeatures();
df.feature_names();
df.info();
```

Verified example: building a small frame

The following code creates two text features and two numeric features:

```
using pluss::table::DataFrame;

auto sales = DataFrame({
    {"city", pluss::make_note({"Accra", "Tema", "Kumasi", "Tamale"})},
    {"region", pluss::make_note({"south", "south", "middle", "north"})},
    {"units", pluss::make_series({10.0, 7.0, 12.0, 5.0})},
    {"price", pluss::make_series({3.0, 4.0, 2.5, 5.0})},
}).title("sales");

std::cout << sales.to_string() << '\n';
std::cout << "ncases=" << sales.ncases()
          << ", nfeatures=" << sales.nfeatures() << '\n';
std::cout << "features:";
for (const auto& name : sales.feature_names()) {
  std::cout << ' ' << name;
}
std::cout << '\n';
```

Output:

```
==================================
     | city    region  units  price
----------------------------------
0    | Accra   south   10     3
1    | Tema    south   7      4
2    | Kumasi  middle  12     2.5
3    | Tamale  north   5      5
----------------------------------
[4 cs x 4 feat]
ncases=4, nfeatures=4
features: city region units price
```

This table is the exact string returned by `sales.to_string()` in the current build. `head()` also renders a table because it returns a renderable `DataFrameView`, but its implementation is not used to produce this output. The display has four cases and four features. The default case axis is the integer axis shown at the left. `feature_names()` preserves insertion order: `city`, `region`, `units`, then `price`.

Feature selection:

```
auto sales = df.features("sales");                  // Thread
auto smaller = df.features({"region", "sales"});    // DataFrame
auto numeric = df.series_features();
auto text = df.note_features();
```

`features("name")` returns a `Thread` with display labels from the frame's case axis and a display name matching the feature. `features({"a", "b"})` returns a new `DataFrame` with the same case axis and the selected feature order.

Cases

The case axis is the row label/index axis. It may be default integer labels or loaded from a feature.

```
auto cases = df.cases();
auto first_case = df.cloc(0);
auto first_ten = df.cloc(0, 10);

auto indexed = df.set_cases("id", true);
auto reset = indexed.reset_cases("id");
```

`set_cases("id", true)` uses `id` as the case axis and drops it from features. Pass `false` to keep the feature as well.

DataFrameView

Some methods return `DataFrameView`:

```
df.head();
df.tail();
df.describe();
df.aggregate({{"mean_sales", "sales", DataFrame::SummaryOp::Mean}});
```

`DataFrameView` is still a `DataFrame`, but it also knows how to auto-render a preview in terminal or popup contexts. If you need a materialized `DataFrame`, pass `render=false` where the method supports it:

```
pluss::table::DataFrame first_ten = df.head(10, false);
```

Atlas

An `Atlas` stacks related frames:

```
auto january = df.query("month == 1");
auto february = df.query("month == 2");

pluss::table::Atlas atlas({january, february});
```

Use an `Atlas` for:

- repeated experiments;
- scenario outputs;

- time-sliced frames;
- multi-sheet workbook-like data;
- 3D plotting and depth aggregation.

Conversion model

PlussMic can convert between layers:

```
auto catalog = df.kindelize();
auto roundtrip_df = catalog.tabelize();

auto archive = atlas.kindelize();
auto roundtrip_atlas = pluss::table::Atlas::tabelize(archive);

auto registry = pluss::bundle::Registry::from_kindle(catalog);
auto back_catalog = registry.to_kindle();
```

Use these conversions when you need a different data model for a specific part of your workflow.

1.3.4 Part IV: Loading and Saving Data

PlussMic has two public I/O styles:

- `DataFrame::load_*` and `df.save_*` convenience methods.
- `pluss::loader::*` functions and option structs for lower-level control.

Most users should start with `DataFrame` convenience methods.

CSV

Minimal CSV load:

```
auto df = pluss::table::DataFrame::load_csv("data.csv");
```

This is the correct first call when the file is a conventional comma-separated file with a header row. PlussMic reads the header, infers feature types, creates default numeric case labels, and returns a `DataFrame`.

The loader has two public configuration styles:

- initializer-list config entries, which are compact and convenient;
- typed option structs, which are explicit and better for production code.

Limit cases and select features with config entries:

```
auto df = pluss::table::DataFrame::load_csv("bank-full.csv", {
    {"ncases", 1000},
    {"features", {"age", "job", "balance", "duration"}},
});
```

Use `ReadCsvOptions` for explicit control:

```
pluss::loader::ReadCsvOptions options;
options.sep = ';';
options.ncases = 1000;
options.usefeats = {"age", "job", "balance", "duration"};
```

(continues on next page)

(continued from previous page)

```
options.case_col = "job";
options.on_bad_lines = pluss::loader::BadLinePolicy::Warn;

auto df = pluss::table::DataFrame::load_csv("bank-full.csv", options);
```

Prefer `ReadCsvOptions` when the ingestion behavior matters. It is easier to read in code review, safer to extend, and maps directly to the source-level CSV loader API.

CSV mental model

CSV loading has five stages:

1. Read records from the source.
2. Decide the header and feature names.
3. Decide which input columns become features.
4. Decide whether one input column becomes the case axis.
5. Convert selected columns into typed `Thread` values.

Most surprising loading behavior comes from mixing up stages 3 and 4. If a column is selected as `case_col`, it becomes the DataFrame case axis and is not also emitted as a normal feature.

For example:

```
pluss::loader::ReadCsvOptions options;
options.case_col = "id";
options.usefeats = {"id", "city", "units"};
```

The `id` column is still used, but it is used as the case axis. The output features are `city` and `units`.

Header handling

For ordinary files:

```
options.header = true;
options.header_row = 0;
```

For files without a header:

```
options.header = false;
options.names = {"id", "city", "units"};
```

When `header` is false and you do not provide names, PlussMic generates feature names from column positions. For durable code, provide `names` explicitly.

Use `header_row` when the actual header is not the first physical row:

```
options.header = true;
options.header_row = 2;
```

Rows before the header row are skipped from data interpretation.

Feature projection

Use names when the file has stable headers:

```
options.usefeats = {"city", "units", "price"};
```

Use indices when working with headerless files or unstable external labels:

```
options.usefeats_idx = {0, 2, 4};
```

If both `usefeats` and `usefeats_idx` are empty, all non-case columns are loaded. If a requested name is missing, loading fails. If a requested index is out of range, loading fails.

Case columns

The case axis is PlussMic's row-label axis. It is similar to an index in other dataframe libraries, but the public method names use `cases`.

Use a named case column:

```
options.case_col = "id";
```

Use a positional case column:

```
options.case_col_idx = 0;
```

After loading, inspect the case axis:

```
std::cout << df.cases().dtype_name() << '\n';
```

Move a case axis back into the feature axis later with:

```
auto with_id_feature = df.reset_cases("id");
```

Verified example: semicolon CSV with case column

Assume the program and this `sales.csv` file are in the same directory:

```
id;city;region;units;price
A001;Accra;south;10;3.0
A002;Tema;south;7;4.0
A003;Kumasi;middle;12;2.5
A004;Tamale;north;5;5.0
```

The example loads only selected features, uses `id` as the case axis, and limits loading to three cases:

```
pluss::loader::ReadCsvOptions options;
options.sep = ';';
options.case_col = "id";
options.usefeats = {"city", "units", "price"};
options.ncases = 3;

auto df = DataFrame::load_csv("sales.csv", options);
std::cout << df.to_string() << '\n';
std::cout << "case dtype=" << df.cases().dtype_name() << '\n';
```

Output:

```
==========================
     | city    units  price
--------------------------
A001 | Accra   10     3
A002 | Tema    7      4
A003 | Kumasi  12     2.5
--------------------------
[3 cs x 3 feat]
case dtype=utf8
```

The `id` column does not appear as a feature because it became the case axis. The printed case labels are `A001`, `A002`, and `A003`, and the case dtype is `utf8`. The fourth input row is absent because `options.ncases = 3`.

Verified example: load records, infer dtypes, save CSV

This example uses `load_csv_records` instead of a file. It is useful for tests, examples, generated rows, or adapters that already have records in memory.

```cpp
const std::vector<std::vector<std::string>> records = {
    {"id", "city", "units", "active"},
    {"A001", "Accra", "10", "yes"},
    {"A002", "Tema", "", "no"},
    {"A003", "Kumasi", "12", "yes"},
};

pluss::loader::ReadCsvOptions read_options;
read_options.case_col = "id";
auto df = pluss::loader::load_csv_records(records, read_options);

std::cout << df.to_string() << '\n';
std::cout << "city dtype=" << df.features("city").dtype_name() << '\n';
std::cout << "units dtype=" << df.features("units").dtype_name() << '\n';
std::cout << "active dtype=" << df.features("active").dtype_name() << '\n';

pluss::loader::WriteCsvOptions write_options;
write_options.sep = ';';
write_options.add_cases = true;
write_options.case_label = "id";
write_options.na_rep = "";

df.save_csv("roundtrip.csv", write_options);

std::ifstream in("roundtrip.csv");
std::string line;
while (std::getline(in, line)) {
  std::cout << line << '\n';
}
```

Output:

```
============================
     | city    units  active
```

(continues on next page)

(continued from previous page)

```
---------------------------
A001 | Accra   10     true
A002 | Tema    NA     false
A003 | Kumasi  12     true
---------------------------
[3 cs x 3 feat]
city dtype=utf8
units dtype=int64
active dtype=bool
id;city;units;active
A001;Accra;10;true
A002;Tema;;false
A003;Kumasi;12;true
```

The loader inferred `city` as UTF-8 text, `units` as `int64`, and `active` as boolean because the default true/false sets include `yes` and `no`. The empty `units` cell in row `A002` becomes `NA` in the DataFrame. When saving, the example sets `na_rep = ""`, so that same missing value is written as an empty field between two semicolons.

`write_options.add_cases = true` writes the case axis as the first output column. `write_options.case_label = "id"` names that output column `id`.

Important CSV options:

Option	Purpose
`sep`	explicit separator
`auto_detect_sep`	infer separator when `sep` is not explicit
`engine`	`Auto`, `Parallel`, or `Serial`
`parallel_threads`	requested parallel parse threads
`profile`	emit parser timing diagnostics
`header`, `header_row`, `names`	header handling
`usefeats`, `usefeats_idx`	select input features
`case_col`, `case_col_idx`	choose case/index feature
`skipcases`, `skipcases_first_n`, `skipfooter`	skip rows
`ncases`	limit number of parsed cases
`parse_dates`, `parse_dates_idx`	datetime parsing
`na_values`, `keep_default_na`, `na_filter`	missing-value handling
`true_values`, `false_values`	boolean parsing
`thousands`, `decimal`	numeric locale hints
`on_bad_lines`	error, warn, or skip malformed rows

Type inference and explicit dtypes

CSV data starts as text. PlussMic converts each selected column into a typed Thread. Common inferred outputs are:

- integer Series for integral values;
- floating Series for decimal values;
- boolean Series for configured true/false tokens;
- Note for general text;
- Timeline or TimeIndex when date parsing is requested.

The default boolean token sets include values such as `true`, `false`, `1`, `0`, `yes`, and `no`.

Use `dtype` when a column must have a specific interpretation:

```
options.dtype["customer_id"] = "string";
options.dtype["units"] = "int64";
options.dtype["price"] = "float64";
```

Use `parse_dates` when text columns should become time payloads:

```
options.parse_dates = {"created_at", "updated_at"};
options.infer_datetime_format = true;
options.dayfirst = false;
```

Use explicit dtypes for identifiers that look numeric. Postal codes, account numbers, and product IDs should usually be loaded as text, not numbers.

Missing values

CSV missing-value handling is controlled by:

```
options.na_filter = true;
options.keep_default_na = true;
options.na_values.insert("missing");
options.na_values.insert("N/A");
```

`na_filter = false` disables missing-value detection. `keep_default_na = true` keeps the built-in missing tokens. `na_values` adds project-specific tokens.

Missingness becomes a Thread validity mask. That means a numeric feature can still have an integer or floating dtype even when some rows display as `NA`.

Bad lines

Malformed rows are rows whose field count does not match the expected layout. The policy is:

```
options.on_bad_lines = pluss::loader::BadLinePolicy::Error;
options.on_bad_lines = pluss::loader::BadLinePolicy::Warn;
options.on_bad_lines = pluss::loader::BadLinePolicy::Skip;
```

Use `Error` for production ingestion when malformed data should stop the job. Use `Warn` when you want diagnostics and can tolerate dropped rows. Use `Skip` only when silent row loss is acceptable in that context.

Engine selection

The CSV engine can be selected explicitly:

```
options.engine = pluss::loader::CsvEngine::Auto;
options.engine = pluss::loader::CsvEngine::Parallel;
options.engine = pluss::loader::CsvEngine::Serial;
```

`Auto` is the right default for most users. Use `Parallel` for large local files when throughput matters. Use `Serial` when debugging parser behavior or when deterministic step-by-step ingestion is more important than speed.

Related options:

```
options.parallel_threads = 8;
options.parallel_min_bytes = 4 * 1024 * 1024;
options.profile = true;
```

`profile = true` emits parser timing diagnostics through the diagnostics path.

TXT

TXT uses the same table loader surface as CSV:

```
auto df = pluss::table::DataFrame::load_txt("space_separated.txt", {
    {"sep", " "},
    {"ncases", 1000},
});
```

For strongly typed control, use CSV options with an explicit separator through the lower-level loader path.

Excel

Load the default sheet:

```
auto df = pluss::table::DataFrame::load_excel("report.xlsx");
```

Load by sheet name or index:

```
auto prices = pluss::table::DataFrame::load_excel("report.xlsx", "Prices");
auto first = pluss::table::DataFrame::load_excel("report.xlsx", 0);
```

Load a workbook:

```
auto loaded = pluss::table::DataFrame::load_excel("report.xlsx", nullptr, true);
for (const auto& name : loaded.sheet_names().values()) {
  loaded.sheets(name).head();
}
```

Use `ReadExcelOptions` for advanced reading:

```
pluss::loader::ReadExcelOptions options;
options.sheet_name = "Prices";
options.header = true;
options.start_row = 2;
options.end_row = 200;
options.start_col = 0;
options.end_col = 8;

auto prices = pluss::table::DataFrame::load_excel("report.xlsx", options);
```

Important Excel options:

Option	Purpose
`sheet_name`, `sheet_index`	single sheet selection
`sheet_names`, `sheet_indices`, `all_sheets`	workbook/multi-sheet selection
`header`, `header_row`, `header_rows`	header interpretation
`header_ffill`, `header_joiner`	multi-row header flattening
`start_row`, `end_row`, `start_col`, `end_col`	bounded region reads
`trim_outer_empty`	remove empty outer rows/columns
`auto_table_block`	infer a table block

JSON and Parquet

```
auto weather = pluss::table::DataFrame::load_json("weather.json");
auto flights = pluss::table::DataFrame::load_parquet("flights.parquet");
```

For dynamic format selection:

```
auto loaded = pluss::table::DataFrame::load("csv", "data.csv");
auto parquet = pluss::table::DataFrame::load("parquet", "flights.parquet");
```

Raw batches

Use raw or neutral batch APIs when you need to inspect parsed input before it becomes a `DataFrame`:

```
auto raw = pluss::loader::read_csv_raw_batch("data.csv", {{"ncases", 100}});
auto batch = pluss::loader::read_csv_batch("data.csv", {{"ncases", 100}});
```

Raw batches keep strings/notes close to file contents. Neutral batches carry `Thread` values but do not own a frame.

URL inputs

When the SDK has URL loader support, `load_csv`, `load_txt`, and `load_excel` can accept `http://` and `https://` sources. Treat remote reads as I/O operations that can fail, timeout, or produce changing data.

Save outputs

```
df.save_csv("clean.csv");
df.save_excel("clean.xlsx", "Clean");
df.save_json("clean.json");
df.save_parquet("clean.parquet");
df.save_txt("clean.txt");
```

Dynamic save:

```
df.save_as("csv", "clean.csv");
df.save_as("parquet", "clean.parquet");
```

CSV save options:

```
pluss::loader::WriteCsvOptions options;
options.add_cases = true;
options.case_label = "id";
options.na_rep = "";
options.float_format = "{:.6f}";

df.save_csv("clean.csv", options);
```

Important CSV write options:

Option	Purpose
`sep`	output delimiter
`header`	write feature names
`add_cases`	write the case axis as the first column
`case_label`	output name for the case column
`na_rep`	text written for missing values
`float_format`	optional floating-point format
`quotechar`	quote character
`doublequote`	escape quotes by doubling
`line_terminator`	row terminator
`decimal`	decimal character
`encoding`	output encoding label

When creating files for another system, set `sep`, `add_cases`, `case_label`, and `na_rep` deliberately. Defaults are convenient for exploration, but production exports should make these choices visible.

Excel save options:

```
pluss::loader::WriteExcelOptions options;
options.sheet_name = "Clean";
options.add_cases = true;
options.na_rep = "";

df.save_excel("clean.xlsx", options);
```

I/O checklist

- Use typed options for production ingestion.
- Set separators explicitly for non-comma files.
- Set `case_col` deliberately when a dataset has an ID column.
- Limit with `ncases` while developing.
- Use `on_bad_lines = Warn` or `Skip` only when dropping malformed rows is acceptable.
- Save cleaned outputs with explicit format-specific filenames.

1.3.5 Part V: DataFrame Workflows

This part covers the daily DataFrame operations: inspect, select, filter, sort, derive, transform, summarize, combine, and save.

Inspect a frame

```
df.head();
df.tail();
df.view(10, 20);
df.info();
df.describe();
```

Use shape and schema methods when writing logic:

```
if (!df.defined() || df.ncases() == 0) {
  throw std::runtime_error("empty input");
}

for (const auto& name : df.feature_names()) {
  std::cout << name << '\n';
}
```

Select features

```
auto one = df.features("balance");
auto some = df.features({"age", "job", "balance"});
auto numeric = df.series_features();
auto text = df.note_features();
auto temporal = df.timeline_features();
```

Verified example: numeric feature selection and reductions

```
auto sales = sales_frame();
auto numeric = sales.series_features();

std::cout << numeric.to_string() << '\n';
std::cout << "units mean=" << sales.features("units").mean().item<double>() << '\n';
std::cout << "price max=" << sales.features("price").max().item<double>() << '\n';
```

Output:

```
====================
     | units  price
--------------------
0    | 10     3
1    | 7      4
2    | 12     2.5
3    | 5      5
--------------------
[4 cs x 2 feat]
units mean=8.5
price max=5
```

`series_features()` removed the text features and kept only numeric `Series` features. The mean of `units` is `(10 + 7 + 12 + 5) / 4 = 8.5`; the maximum price is `5`.

Pattern-based selection:

```
auto costs = df.select_like("Cost", false);
auto monthly = df.select_regex(".*_month$");
```

Schema management:

```
auto renamed = df.rename("old", "new");
auto renamed_many = df.rename_features({{"age", "age_years"}, {"job", "role"}});
auto relabeled = df.relabel_features({"a", "b", "c"});
```

(continues on next page)

(continued from previous page)

```
auto prefixed = df.add_prefix("raw_");
auto suffixed = df.add_suffix("_clean");
auto reordered = df.reorder({"id", "date", "value"});
auto dropped = df.drop({"temporary_feature"});
```

`rename` changes one feature name and fails if the old name is missing or the new name already exists. `rename_features` applies several renames one after another. `relabel_features` replaces the entire feature axis, so the number of new names must match the current feature count. `reorder` also requires the full feature set; it is not a selector. Use `features({...})` or `drop({...})` when you want to keep only part of the frame.

Select cases

```
auto first = df.cloc(0);
auto first_ten = df.cloc(0, 10);
auto middle = df.slice(100, 200);
auto sampled = df.sample(100, 7);
auto taken = df.take({0, 2, 5, 9});
```

`head`, `tail`, and `view` are preview-oriented and return `DataFrameView`. `slice`, `take`, and `sample` return `DataFrame`.

`slice(start, stop)` uses half-open row positions: it keeps `start` and stops before `stop`. Negative slice bounds are normalized against the frame size by the implementation. `take(indices)` uses the exact row positions you provide and keeps them in that order. Both preserve the selected rows' original case labels.

Filter cases

Use query expressions for readable filters:

```
auto older = df.query("age >= 45");
auto known = df.query("job != 'unknown'");
auto band = df.query("age between 45 and 50");
auto selected = df.query("job in ('management', 'technician')");
auto combined = df.query("age >= 45 and balance > 0");
```

Use `filter` when you already have a boolean `Series` or boolean `Thread`:

```
auto mask = df.features("balance").gt(0.0).as_series();
auto positive = df.filter(mask);
```

Verified example: derive then query

```
auto enriched = sales.assign({{
    "revenue",
    [](const DataFrame& frame) {
      return frame.features("units") * frame.features("price");
    },
}});

auto high_value = enriched.query("revenue >= 30");
std::cout << high_value.to_string() << '\n';
```

Output:

```
=========================================
     | city    region  units  price  revenue
-----------------------------------------
0    | Accra   south   10     3      30
2    | Kumasi  middle  12     2.5    30
-----------------------------------------
[2 cs x 5 feat]
```

`assign` evaluates the lambda against the evolving output frame and adds the new `revenue` feature. The query keeps rows where `revenue >= 30`, so Accra and Kumasi remain. Tema has revenue `28`, and Tamale has revenue `25`, so both are filtered out.

Sort and rank

```
auto by_age = df.sort_values({"age"});
auto by_age_desc = df.sort_values({"age"}, false);

auto by_multiple = df.sort_values({
    {"age", true, true},
    {"duration", false, true},
});

auto top = df.top("balance", 10);
auto bottom = df.bottom("balance", 10);
```

`sort_values({"age"})` sorts ascending by default and puts null values last by default. Use `ArrangeSpec` when a sort needs per-feature direction or null placement. `top(feature, n)` is shorthand for descending sort plus the first `n` rows. `bottom(feature, n)` is shorthand for ascending sort plus the first `n` rows.

Verified example: shaping rows, schema, and case labels

```
auto sales = sales_frame();

auto sorted = sales.sort_values({DataFrame::ArrangeSpec{"price", false, true}})
                   .features({"city", "price"});
std::cout << sorted.to_string() << '\n';

std::cout << sales.slice(1, 3).to_string() << '\n';
std::cout << sales.take(std::vector<std::int64_t>{3, 0}).to_string() << '\n';

auto schema = sales.rename("units", "quantity")
                   .drop({"region"})
                   .reorder({"city", "price", "quantity"});
std::cout << schema.to_string() << '\n';

auto city_cases = sales.set_cases("city", true);
std::cout << city_cases.features({"region", "units"}).to_string() << '\n';

auto reset = city_cases.reset_cases("city_case").features({"city_case", "units"});
std::cout << reset.to_string() << '\n';
```

Output:

```
====================
     | city    price
--------------------
3    | Tamale  5
1    | Tema    4
0    | Accra   3
2    | Kumasi  2.5
--------------------
[4 cs x 2 feat]
==================================
     | city    region  units  price
----------------------------------
1    | Tema    south   7      4
2    | Kumasi  middle  12     2.5
----------------------------------
[2 cs x 4 feat]
==================================
     | city    region  units  price
----------------------------------
3    | Tamale  north   5      5
0    | Accra   south   10     3
----------------------------------
[2 cs x 4 feat]
============================
     | city    price  quantity
----------------------------
0    | Accra   3      10
1    | Tema    4      7
2    | Kumasi  2.5    12
3    | Tamale  5      5
----------------------------
[4 cs x 3 feat]
======================
       | region  units
----------------------
Accra  | south   10
Tema   | south   7
Kumasi | middle  12
Tamale | north   5
----------------------
[4 cs x 2 feat]
======================
     | city_case  units
----------------------
0    | Accra      10
1    | Tema       7
2    | Kumasi     12
3    | Tamale     5
----------------------
[4 cs x 2 feat]
```

The first table is sorted by `price` descending. The case labels still show the original row labels, so the first row displays case 3 because Tamale was the fourth row in the input.

The second table is `slice(1, 3)`. It keeps row positions 1 and 2 and stops before position 3. The third table is `take({3, 0})`; it keeps exactly Tamale then Accra because that is the order of the indices.

The fourth table demonstrates feature-axis work. `rename("units", "quantity")` changes the feature name, `drop({"region"})` removes the region feature, and `reorder({"city", "price", "quantity"})` arranges the full remaining feature set in the requested order.

The fifth table shows `set_cases("city", true)`. City values become case labels, and the original `city` feature is removed because `drop_feature` is `true`. The sixth table shows `reset_cases("city_case")`, which turns the case axis back into a normal feature named `city_case` and restores default numeric case labels.

Derive features

Use `assign` for one or more derived features:

```
auto enriched = df.assign({{
    "balance_per_second",
    [](const pluss::table::DataFrame& frame) {
      return frame.features("balance") / frame.features("duration");
    },
}});
```

Use `set_features` when the new feature is already computed:

```
auto doubled = df.set_features("balance_x2", df.features("balance") * 2.0);
```

Use mutable assignment only when mutation is intentional:

```
df.set_features_("balance_x2", df.features("balance") * 2.0);
```

Transform features

Transform all features:

```
auto transformed = df.transform([](const pluss::table::Thread& feature) {
  if (feature.kind() == pluss::table::ThreadKind::Series) {
    return feature + 1.0;
  }
  return feature;
});
```

Transform selected features:

```
auto adjusted = df.transform({
    {"age", [](const auto& feature) { return feature + 1.0; }},
    {"balance", [](const auto& feature) { return feature.clip(-1000.0, 1000.0); }},
});
```

Missing data

```
auto nulls = df.isna();
auto present = df.notna();
auto filled_numeric = df.fillna(0.0, {"balance", "duration"});
auto filled_text = df.fillna("unknown", {"job"});
auto forward = df.ffill();
```

(continues on next page)

(continued from previous page)

```
auto backward = df.bfill();
auto complete = df.dropna();
auto complete_selected = df.dropna_all({"balance", "duration"});
```

Count null rows:

```
std::int64_t null_rows = df.count_null_rows();
```

Verified example: missing data audit

```
const std::vector<std::vector<std::string>> records = {
    {"id", "city", "units", "price"},
    {"A001", "Accra", "10", "3.0"},
    {"A002", "Tema", "", "4.0"},
    {"A003", "", "12", ""},
};

pluss::loader::ReadCsvOptions options;
options.case_col = "id";
auto df = pluss::loader::load_csv_records(records, options);

std::cout << df.to_string() << '\n';
std::cout << "null rows=" << df.count_null_rows() << '\n';
std::cout << df.isna().to_string() << '\n';
std::cout << df.fillna(0.0, {"units", "price"})
               .fillna("unknown", {"city"})
               .to_string() << '\n';
```

Output:

```
==========================
     | city   units  price
--------------------------
A001 | Accra  10     3
A002 | Tema   NA     4
A003 | NA     12     NA
--------------------------
[3 cs x 3 feat]
null rows=2
==========================
     | city   units  price
--------------------------
A001 | false  false  false
A002 | false  true   false
A003 | true   false  true
--------------------------
[3 cs x 3 feat]
============================
     | city     units  price
----------------------------
A001 | Accra    10     3
A002 | Tema     0      4
```

(continues on next page)

(continued from previous page)

```
A003 | unknown  12     0
----------------------------
[3 cs x 3 feat]
```

The first table shows missing cells as `NA`. `count_null_rows()` reports `2` because rows `A002` and `A003` have at least one null value. The `isna()` table is a boolean mask with `true` exactly where the displayed input had `NA`.

The filled table keeps all existing non-null values. It replaces missing numeric cells in `units` and `price` with `0`, and replaces the missing `city` cell with `unknown`.

Arithmetic and comparisons

DataFrame arithmetic delegates to numeric features:

```
auto numeric = df.features({"age", "balance", "duration"});

numeric.add(1.0);
numeric.subtract(1.0);
numeric.multiply(2.0);
numeric.divide(2.0);
numeric.pow(2.0);
```

Reverse operations:

```
numeric.rsub(100.0);
numeric.rpow(2.0);
```

Comparisons:

```
auto ge_zero = numeric.ge(0.0);
auto same = numeric.eq(numeric);
auto cmp = numeric.compare(numeric.add(1.0));
```

Aggregation

Simple reductions:

```
df.series_features().mean(true);
df.series_features().std(true);
df.series_features().corr(true);
```

Named summaries:

```
df.aggregate({
    {"cases", "age", DataFrame::SummaryOp::Count},
    {"age_mean", "age", DataFrame::SummaryOp::Mean},
    {"job_cardinality", "job", DataFrame::SummaryOp::Nunique},
}, true);
```

Verified example: aggregate and group

```
auto sales = sales_frame().assign({{
    "revenue",
```

(continues on next page)

(continued from previous page)

```
    [](const DataFrame& frame) {
      return frame.features("units") * frame.features("price");
    },
}});

auto summary = sales.aggregate({
    {"total_units", "units", DataFrame::SummaryOp::Sum},
    {"mean_price", "price", DataFrame::SummaryOp::Mean},
    {"max_revenue", "revenue", DataFrame::SummaryOp::Max},
}, false);
std::cout << summary.to_string() << '\n';

auto grouped = sales.group_by(
    {"region"},
    {
        {"units_sum", "units", DataFrame::GroupOp::Sum},
        {"revenue_mean", "revenue", DataFrame::GroupOp::Mean},
    },
    true,
    false);
std::cout << grouped.to_string() << '\n';
```

Output:

```
=============================================
        | total_units  mean_price  max_revenue
---------------------------------------------
summary | 34           3.625       30
---------------------------------------------
[1 cs x 3 feat]
=====================================
     | region  units_sum  revenue_mean
-------------------------------------
0    | south   17         29
1    | middle  12         30
2    | north   5          25
-------------------------------------
[3 cs x 3 feat]
```

The aggregate result has one case named `summary`. The grouped result has one row per region. The south group combines Accra and Tema, so `units_sum` is `10 + 7 = 17`, and mean revenue is `(30 + 28) / 2 = 29`.

Grouped summaries:

```
df.group_by(
    {"job"},
    {
        {"count", "age", DataFrame::GroupOp::Count},
        {"mean_balance", "balance", DataFrame::GroupOp::Mean},
        {"max_duration", "duration", DataFrame::GroupOp::Max},
    },
    true,
    true);
```

Convenience group summaries:

```
df.count_by("job", 10, true, true);
df.mean_by("job", "balance", 10, 1, true, true);
```

Pivoting

```
df.pivot("date", "symbol", "close", true, true);
df.pivot_table("date", "symbol", "volume", DataFrame::GroupOp::Mean, true, true);
```

Use `pivot` when the case-feature and feature-feature pairs identify one value. Use `pivot_table` when duplicate pairs need aggregation.

Combining frames

```
auto joined = left.join(right, "_right");
auto updated = left.update(right, true);
auto combined = left.combine(right);

auto stitched = left.stitch_on("id", right, "id", "_right");
```

Use `join` when two frames already have compatible row counts and you want to append features. Use `stitch_on` when rows must be matched by key values. Use `update` when the right frame should patch values into the left frame. Use `combine` when you need the configurable primitive behind `join` and `update`.

The names are precise:

- `join` is feature-wise combination. It requires the same case count and appends right-hand features by row position. Duplicate feature names receive the collision suffix.
- `update` is schema-preserving patching. It only updates overlapping features and ignores right-hand features that do not exist on the left.
- `stitch_on` is key-based matching. It creates rows for matching key values and appends right-hand payload features.
- `combine` is the configurable primitive underneath `join` and `update`.

Verified example: join, update, and stitch_on

```
DataFrame left({
    {"sku", pluss::make_note({"A", "B", "C"})},
    {"qty", pluss::make_series({1.0, 2.0, 3.0})},
});
DataFrame right({
    {"sku", pluss::make_note({"B", "C", "D"})},
    {"qty", pluss::make_series({20.0, 30.0, 40.0})},
    {"cost", pluss::make_series({5.0, 8.0, 13.0})},
});
DataFrame patch({
    {"qty", pluss::make_series({10.0, 20.0, 30.0})},
    {"status", pluss::make_note({"new", "open", "closed"})},
});

std::cout << left.join(patch, "_patch").to_string() << '\n';
```

(continues on next page)

(continued from previous page)

```
std::cout << left.update(patch, true).to_string() << '\n';
std::cout << left.stitch_on("sku", right, "sku", "_right").to_string() << '\n';
```

Output:

```
===================================
     | sku   qty   qty_patch  status
-----------------------------------
0    | A     1     10         new
1    | B     2     20         open
2    | C     3     30         closed
-----------------------------------
[3 cs x 4 feat]
=================
     | sku   qty
-----------------
0    | A     10
1    | B     20
2    | C     30
-----------------
[3 cs x 2 feat]
=================================
     | sku   qty   qty_right  cost
---------------------------------
0    | B     2     20         5
1    | C     3     30         8
---------------------------------
[2 cs x 4 feat]
```

The first result is `join`. The left frame has `sku` and `qty`; the patch frame has `qty` and `status`. Because `qty` exists on both sides, PlussMic keeps the left `qty` and appends the right feature as `qty_patch`. `status` is appended without renaming.

The second result is `update`. Only overlapping features are considered, so `qty` is replaced by the patch values and `status` is ignored. This is useful when the right frame is a correction frame rather than a source of new columns.

The third result is `stitch_on("sku", right, "sku", "_right")`. This is the key-based operation. Only `B` and `C` appear because those keys exist in both frames. The right key feature is not duplicated because both key names are `sku`. The right `qty` collides with the left `qty`, so it becomes `qty_right`; `cost` is appended as a new payload feature.

Display and titles

```
auto named = df.title("Clean sales");
named.show();

pluss::table::display::DisplayOptions options;
named.show(options);
```

Preview methods return display-aware `DataFrameView` objects:

```
auto preview = df.head(20);
preview.show();
```

See *DataFrame Display* for terminal and popup behavior.

1.3.6 Part VI: Thread Workflows

`Thread` is PlussMic's column model. Every `DataFrame` feature is a named `Thread`, and many table operations delegate their real work to Thread methods. If `DataFrame` is the table abstraction, `Thread` is the typed one-dimensional array abstraction underneath it.

A Thread has four important pieces of state:

- a payload family: `Series`, `Note`, `Timeline`, or `TimeIndex`;
- an optional validity mask for missing values;
- optional display labels;
- an optional display name.

The payload family controls which operations are legal. Numeric arithmetic is for `Series` threads. Text cleaning is for `Note` threads. Calendar extraction is for `Timeline` and `TimeIndex` threads. Missingness is tracked separately through the validity mask, so a row can have a stored payload value and still display as `NA`.

Payload families

The public kind enum is:

```
enum class ThreadKind {
  Series,
  Note,
  Timeline,
  TimeIndex,
};
```

Use the family-specific accessors only after you know the kind:

```
if (thread.kind() == pluss::table::ThreadKind::Series) {
  const auto& values = thread.as_series();
}

if (thread.kind() == pluss::table::ThreadKind::Note) {
  const auto& text = thread.as_note();
}
```

Calling the wrong accessor is an error. For example, `as_series()` is only valid for a `Series` payload.

Construct threads

Numeric values normally enter a Thread through `pluss::make_series`:

```
pluss::table::Thread numeric = pluss::make_series({1.0, 2.0, 3.0});
```

Text values normally enter through `pluss::make_note`:

```
pluss::table::Thread text = pluss::make_note({"alpha", "beta", "gamma"});
```

You can attach a validity mask at construction time:

```
auto values = pluss::make_series({4.0, 1.0, 0.0, 3.0, 2.0});
auto valid = pluss::Series::zeros({5}, pluss::codem::ScalarType::Bool);
auto* valid_ptr = reinterpret_cast<bool*>(valid.raw_data());
```

(continues on next page)

(continued from previous page)

```
valid_ptr[0] = true;
valid_ptr[1] = true;
valid_ptr[2] = false;
valid_ptr[3] = true;
valid_ptr[4] = true;

pluss::table::Thread score(values, valid);
```

The third row stores `0.0` in the payload, but its mask value is false. Display and null-aware operations treat that row as missing.

Inspect a Thread

Use these methods when writing generic code:

```
thread.defined();
thread.unique();
thread.size();
thread.kind();
thread.dtype_name();
thread.has_validity_mask();
thread.is_valid(0);
```

`dtype_name()` reports the payload dtype, such as `float64` or `utf8`. It does not tell you whether the thread has missing values. Use `has_validity_mask()`, `isnull()`, `notnull()`, or `is_valid(row)` for that.

Display and debug methods:

```
thread.info();
thread.show();
std::cout << thread.to_string() << '\n';
```

`to_string()` is useful in tests, examples, and documentation because it uses the same display path a user sees.

Verified example: numeric and text operations

The following example is compiled and run by `PlussMic/docs/sphinx/manual/examples/plussmic_manual_examples.cpp`.

```
auto sales = sales_frame();
auto units = sales.features("units");
auto city = sales.features("city");

std::cout << "units dtype=" << units.dtype_name() << '\n';
std::cout << "units clipped:\n" << units.clip(6.0, 10.0).to_string() << '\n';
std::cout << "city lower:\n" << city.lower().to_string() << '\n';
std::cout << "city contains 'a':\n" << Thread(city.contains("a")).to_string() << '\n';
```

Output:

```
units dtype=float64
units clipped:
0  10
1  7
```

(continues on next page)

(continued from previous page)

```
2  10
3  6
dtype: Series
city lower:
0  accra
1  tema
2  kumasi
3  tamale
dtype: Note
city contains 'a':
0  true
1  true
2  true
3  true
dtype: Series
```

`features("units")` returns a numeric Series-backed Thread. `clip(6.0, 10.0)` clamps values below 6 upward and values above 10 downward. The original unit values are 10, 7, 12, and 5, so the result is 10, 7, 10, and 6.

`features("city")` returns a Note-backed Thread. `lower()` returns lower-case city names. `contains("a")` returns a boolean `Series`, which the example wraps as a `Thread` only so it prints in the same format.

Row-space operations

A Thread is one-dimensional, so row-space operations work directly on element positions:

```cpp
thread.head(5);
thread.tail(5);
thread.view(10, 20);
thread.slice(10, 20);
thread.take(std::vector<std::int64_t>{0, 3, 8});
thread.shift(1);
thread.sort_values(true, true);
thread.dropna();
thread.distinct(true);
```

`head`, `tail`, and `view` return `ThreadView`, which can auto-render. `slice`, `take`, `shift`, `sort_values`, `dropna`, and `distinct` return ordinary `Thread` values.

`slice(start, stop)` is half-open. `take(indices)` keeps exactly the requested positions in exactly that order. `shift(periods)` moves values by row offset and marks the newly exposed edge as missing. `sort_values(ascending, nulls_last)` sorts the valid values and places masked rows according to `nulls_last`.

Missingness

Missingness is mask-based. A Thread without a validity mask has no mask-level missing values. A Thread with a validity mask uses `true` for present and `false` for missing.

Common methods:

```cpp
auto missing = thread.isnull();
auto present = thread.notnull();
auto complete = thread.dropna();
auto forward = thread.ffill();
auto backward = thread.bfill();
```

Numeric checks are separate:

```
auto nan = thread.isnan();
auto finite = thread.isfinite();
```

`isnan()` checks floating-point NaN in a Series payload. `isnull()` checks the Thread validity mask. A value can be finite but masked out, and it will still display as `NA`.

Verified example: missingness and row operations

```
Thread score(
    pluss::make_series({4.0, 1.0, 0.0, 3.0, 2.0}),
    bool_series({true, true, false, true, true}));

std::cout << "score:\n" << score.to_string() << '\n';
std::cout << "isnull:\n" << score.isnull().to_string() << '\n';
std::cout << "fillna:\n" << score.fillna(99.0).to_string() << '\n';
std::cout << "dropna:\n" << score.dropna().to_string() << '\n';
std::cout << "shift(2):\n" << score.shift(2).to_string() << '\n';
std::cout << "sort asc nulls first:\n" << score.sort_values(true, false).to_string() <<
↪'\n';
std::cout << "take 4,1,0:\n" << score.take(std::vector<std::int64_t>{4, 1, 0}).to_
↪string() << '\n';
```

Output:

```
score:
0  4
1  1
2  NA
3  3
4  2
dtype: Series
isnull:
0  false
1  false
2  true
3  false
4  false
dtype: Series
fillna:
0  4
1  1
2  99
3  3
4  2
dtype: Series
dropna:
0  4
1  1
2  3
3  2
dtype: Series
shift(2):
```

(continues on next page)

(continued from previous page)

```
0  NA
1  NA
2  4
3  1
4  NA
dtype: Series
sort asc nulls first:
0  NA
1  1
2  2
3  3
4  4
dtype: Series
take 4,1,0:
0  2
1  1
2  4
dtype: Series
```

The `score` payload has five numeric values, but row 2 is masked as missing, so it displays as `NA`. `isnull()` is the logical inverse of the validity mask.

`fillna(99.0)` keeps the valid values `4`, `1`, `3`, and `2`, and replaces only the masked row with `99`. `dropna()` removes the masked row. `shift(2)` moves values down by two rows, marks rows 0 and 1 missing because there are no source values for them, and also carries the original missing row forward so row 4 is missing. `sort_values` with `nulls_last=false` places the missing value first, then sorts valid values ascending. `take({4, 1, 0})` keeps rows 4, 1, and 0, in that order.

Numeric operations

Series-backed Threads support arithmetic, comparisons, math transforms, rolling windows, and scalar reductions.

Arithmetic can be written through methods or operators:

```
auto plus_two = thread + 2.0;
auto doubled = thread * 2.0;
auto ratio = numerator / denominator;
auto remainder = thread % 2.0;
```

Comparison operators return boolean Series-backed Threads:

```
auto positive = thread > 0.0;
auto in_limit = thread <= 100.0;
auto same = left == right;
```

Named numeric transforms:

```
auto absolute = thread.abs();
auto clipped = thread.clip(0.0, 100.0);
auto logged = thread.log();
auto exponentiated = thread.exp();
```

Reductions return scalar `Series` values, not raw C++ scalars:

```
double mean = thread.mean().item<double>();
double total = thread.sum().item<double>();
double sd = thread.std(false).item<double>();
```

Available reductions include:

```
thread.sum();
thread.count();
thread.any();
thread.all();
thread.mean();
thread.median();
thread.min();
thread.max();
thread.var();
thread.std();
thread.corr(other);
```

`corr` accepts a method:

```
double pearson = x.corr(y);
double spearman = x.corr(y, pluss::table::ThreadCorrMethod::Spearman);
double kendall = x.corr(y, pluss::table::ThreadCorrMethod::KendallTau);
```

Comparisons and bands

Use `compare` when the comparison operator is data-driven:

```
auto gt_zero = thread.compare(0.0, pluss::table::ThreadBinaryCompareOp::Gt);
auto le_other = thread.compare(other, pluss::table::ThreadBinaryCompareOp::Le);
```

Use `btw` for interval checks:

```
auto inclusive = thread.btw(10.0, 20.0, true);
auto half_open = thread.btw(10.0, 20.0, true, false);
auto text_band = text.btw("a", "m", true);
```

For DataFrame workflows, `df.query("age between 10 and 20")` is often more readable. Direct Thread comparisons are useful when you are constructing masks in C++.

Rolling windows

Use `window` for simple rolling aggregations over numeric Series payloads:

```
auto rolling_sum = thread.window(3, pluss::table::ThreadWindowOp::RollingSum, 1);
auto rolling_mean = thread.window(3, pluss::table::ThreadWindowOp::RollingMean, 2);
auto rolling_min = thread.window(5, pluss::table::ThreadWindowOp::RollingMin, 3);
auto rolling_max = thread.window(5, pluss::table::ThreadWindowOp::RollingMax, 3);
```

The arguments are:

- `window`: number of rows in the trailing window;
- `op`: rolling operation;
- `min_periods`: minimum number of valid observations required for an output value.

If a row does not have enough valid observations, the output row is masked and prints as `NA`.

Thread time-series accessor

`thread.ts()` is for one numeric value stream. It does not need a separate date feature for operations such as returns, lag, lead, rolling mean, or expanding statistics.

```
auto returns = price.ts().returns(1);
auto log_returns = price.ts().log_returns(1);
auto diff = price.ts().diff(1);
auto lagged = price.ts().lag(1);
auto leading = price.ts().lead(1);
auto rolling = price.ts().rolling_mean(20, 10);
auto z = price.ts().zscore_rolling(20, 10);
```

Use DataFrame time-series APIs when you need to resample or align one or more value features against an explicit time feature.

Casting

Use `cast` to change payload family:

```
auto as_note = thread.cast(pluss::table::ThreadCastTarget::Note);
auto as_series = text.cast(pluss::table::ThreadCastTarget::Series,
                           pluss::codem::ScalarType::Float64);
auto as_time = date_text.cast(pluss::table::ThreadCastTarget::Timeline);
auto as_index = date_text.cast(pluss::table::ThreadCastTarget::TimeIndex);
```

Use `astype` as a shorthand for Series dtype conversion:

```
auto as_i64 = numeric.astype(pluss::codem::ScalarType::Int64);
auto as_f64 = numeric.astype(pluss::codem::ScalarType::Float64);
```

Casting rules are family-specific:

- Series to Series changes numeric dtype.
- Series to Note formats each scalar as text.
- Series to Timeline treats numeric values as either epoch nanoseconds or `YYYYMMDD` values when all valid rows look like dates.
- Note to Series parses bools or numbers depending on the requested dtype.
- Note to Timeline parses date/time text.
- Timeline and TimeIndex can be converted to Note or numeric epoch-nanosecond Series.

Parsing failures become invalid rows where supported by the implementation.

Text operations

Note-backed Threads support text cleaning:

```
auto lowered = text.lower();
auto replaced = text.replace("old", "new");
auto mask = text.contains("needle");
```

`contains` returns a `Series`, not a `Thread`. Wrap it in `Thread(...)` when you want Thread-style display.

Verified example: windows, casts, text, and returns

```
Thread values = pluss::make_series({1.0, 2.0, 4.0, 8.0, 16.0});
Thread labels = pluss::make_note({"10", "20", "bad", "40"});
Thread words = pluss::make_note({"North-East", "South", "north-west", "Central"});

std::cout << "values + 3:\n" << (values + 3.0).to_string() << '\n';
std::cout << "between 2 and 8 inclusive:\n" << values.btw(2.0, 8.0, true).to_string() <<
↪'\n';
std::cout << "rolling mean window 3 min 2:\n"
          << values.window(3, pluss::table::ThreadWindowOp::RollingMean, 2).to_string() <
↪< '\n';
std::cout << "returns:\n" << values.ts().returns(1).to_string() << '\n';
std::cout << "label cast to float64:\n"
          << labels.cast(pluss::table::ThreadCastTarget::Series,
                         pluss::codem::ScalarType::Float64).to_string()
          << '\n';
std::cout << "words replace/lower:\n" << words.replace("-", " ").lower().to_string() <<
↪'\n';
std::cout << "words contains north:\n" << Thread(words.lower().contains("north")).to_
↪string() << '\n';
```

Output:

```
values + 3:
0  4
1  5
2  7
3  11
4  19
dtype: Series
between 2 and 8 inclusive:
0  false
1  true
2  true
3  true
4  false
dtype: Series
rolling mean window 3 min 2:
0  NA
1  1.5
2  2.333333333
3  4.666666667
4  9.333333333
dtype: Series
returns:
0  nan
1  1
2  1
3  1
4  1
dtype: Series
label cast to float64:
0  10
```

(continues on next page)

(continued from previous page)

```
1  20
2  NA
3  40
dtype: Series
words replace/lower:
0  north east
1  south
2  north west
3  central
dtype: Note
words contains north:
0  true
1  false
2  true
3  false
dtype: Series
```

`values + 3` performs elementwise numeric addition. `btw(2.0, 8.0, true)` marks the values `2`, `4`, and `8` as true because the interval is inclusive.

The rolling mean uses a trailing window of three rows and requires at least two observations. Row 0 has only one observation, so it prints `NA`. Row 1 averages `1` and `2`, giving `1.5`. Row 4 averages `4`, `8`, and `16`, giving `9.333333333`.

`returns(1)` computes one-period fractional returns. The first row has no previous value and prints `nan`; every later value doubles from the previous row, so each return is `1`.

Casting `labels` to a Float64 Series parses `"10"`, `"20"`, and `"40"`. The string `"bad"` cannot be parsed as a number, so that row is invalid and prints `NA`.

The text example first replaces hyphens with spaces and then lowercases. The contains check is performed on the lowercased values, so it matches both `North-East` and `north-west`.

Time operations

Timeline and TimeIndex payloads support calendar extraction:

```
auto years = dates.year();
auto months = dates.month();
auto days = dates.day();
auto hours = dates.hour();
auto weekdays = dates.to_note_weekday();
```

These return Threads so they can be inserted back into a DataFrame:

```
auto enriched = df.set_features("year", df.features("timestamp").year());
```

If you need to parse string dates, cast a Note-backed Thread to Timeline first:

```
auto timeline = text_dates.cast(pluss::table::ThreadCastTarget::Timeline);
auto months = timeline.month();
```

Display labels and names

Display metadata is useful for presentation, but it should not be treated as schema truth.

```
auto named = thread.with_display_name("balance");
auto labeled = named.with_display_labels(labels);

named.show();
std::cout << named.to_string() << '\n';
```

For DataFrames, feature names are the schema. Thread display names are just presentation metadata.

Functional and mutating methods

Thread transforms are functional by default:

```
auto filled = thread.fillna(0.0);
auto clipped = thread.clip(0.0, 1.0);
auto shifted = thread.shift(1);
```

The original Thread is unchanged unless you use a mutating method with a trailing underscore:

```
thread.fillna_(0.0);
thread.with_validity_mask_(mask);
thread.clear_validity_mask_();
```

Use mutating methods when mutation is intentional and local. Use functional methods when building pipelines or returning values from helper functions.

Choosing Thread or DataFrame APIs

Use Thread directly when:

- you are transforming one feature;
- you need a reusable mask;
- you are writing a DataFrame `assign` lambda;
- you need payload-specific methods such as `lower`, `contains`, `year`, or `window`;
- you want scalar reductions such as `mean().item<double>()`.

Use DataFrame methods when:

- the operation uses multiple features;
- you need to preserve feature names;
- you need table-shaped output;
- you are loading, saving, joining, grouping, or pivoting.

The common pattern is to use DataFrame for structure and Thread for feature logic:

```
auto enriched = df.assign({{
    "clean_city",
    [](const pluss::table::DataFrame& frame) {
      return frame.features("city").replace("-", " ").lower();
    },
}});
```

1.3.7 Part VII: Query Expressions and SQL

PlussMic gives you three practical ways to choose rows:

- `DataFrame::query(...)` evaluates a text predicate against the current frame.
- `DataFrame::filter(...)` applies an explicit boolean mask that you build in C++.
- `DataFrame::sql(...).execute(...)` parses a small SQL `SELECT` language for projection, joins, parameters, ordering, and limits.

The important design point is that these are not three unrelated systems. The SQL executor parses the SQL statement, builds an intermediate `DataFrame`, and sends the SQL `WHERE` clause to `DataFrame::query`. That means SQL `WHERE` supports the same predicate language as `query`, while SQL itself is responsible for `SELECT`, `FROM`, `JOIN`, `ORDER BY`, `LIMIT`, `OFFSET`, and parameter replacement.

Use `query` when you only need row filtering. Use SQL when you also want column projection, source binding, a join, a limit, or late-bound parameters. Use an explicit mask when your predicate is easier or safer to express as ordinary C++ code.

Query expressions

`DataFrame::query` takes a predicate string and returns a new frame containing only the cases for which that predicate is true. The result keeps the same feature set as the input frame unless you later select or drop features.

```
auto older = df.query("age >= 45");
auto positive = df.query("balance > 0");
auto known = df.query("job != 'unknown'");
auto band = df.query("age between 45 and 50");
auto selected = df.query("job in ('management', 'technician')");
auto combined = df.query("age >= 45 and balance > 0");
```

A query string names features directly. Numeric comparisons use numeric features. String comparisons use quoted string literals. Boolean combinations use words such as `and` and `or`.

Use query expressions for:

- filtering a frame without changing its columns;
- readable conditions in analysis code;
- direct comparisons against literals;
- membership checks;
- range checks;
- null filtering through helper predicates.

Examples:

```
auto clean_balance = df.query("isnotnull(balance)");
auto clean_job = df.query("isnotnull(job) and job != ''");
auto adults_with_income = df.query("age >= 18 and income > 0");
```

Because `query` works from feature names, treat those names as part of your data contract. If a CSV column is renamed from `balance` to `account_balance`, every query using `balance` must change.

Query result shape

A query filters cases, not features. If the input frame has 20 features and the predicate matches 5 cases, the output has 5 cases and the same 20 features.

This is why a common workflow is:

```
auto filtered = df.query("age >= 45 and balance > 0");
auto compact = filtered.features({"age", "job", "balance"});
```

The first line chooses rows. The second line chooses columns.

Explicit masks

Use a boolean mask when the condition is easier to express in C++ than in the text predicate language. A mask must have one boolean value for each case in the frame. `true` keeps the row. `false` removes it.

```
auto values = df.features("age").as_series().to_vec<std::int64_t>();
auto mask = pluss::Series::zeros(
    {static_cast<std::int64_t>(values.size())},
    pluss::codem::ScalarType::Bool);

auto* keep = reinterpret_cast<bool*>(mask.raw_data());
for (std::size_t i = 0; i < values.size(); ++i) {
  keep[i] = values[i] >= 45 && values[i] <= 50;
}

auto filtered = df.filter(mask);
```

This style is more verbose than `df.query("age between 45 and 50")`, but it gives you full control. It is useful when the predicate calls your own C++ functions, depends on external state, or must be debugged step by step.

Use masks when:

- the predicate is a C++ function;
- the predicate depends on a lookup table or an object that is not a feature;
- you already computed a boolean `Series`;
- you want to inspect the mask before filtering;
- you need precise control over exceptional cases.

SQL is a PlussMic SELECT language

`DataFrame::sql()` returns a `DataFrameSqlAccessor`. The accessor executes `SELECT` statements over the calling frame and optional bound sources.

```
auto result = df.sql().execute(
    "SELECT age, job, balance FROM df WHERE age >= 45");
```

The current SQL implementation supports `SELECT` statements only. Non-`SELECT` statements such as `DELETE FROM df` are rejected. This SQL layer is for shaping in-memory frames, not mutating databases.

The calling frame is named `df` by default:

```
SELECT age, job FROM df
```

You may alias the calling frame:

```
SELECT customers.age FROM df AS customers
```

The parser supports:

- `SELECT *`
- `SELECT feature`
- `SELECT feature AS alias`
- qualified names such as `df.age`
- `FROM df`
- `FROM df AS alias`
- `JOIN name ON left_feature = right_feature`
- `WHERE ...`
- `ORDER BY feature ASC`
- `ORDER BY feature DESC`
- `ORDER BY feature ASC NULLS FIRST`
- `ORDER BY feature ASC NULLS LAST`
- `LIMIT n`
- `OFFSET n`
- scalar parameters such as `{min_age}`
- identifier parameters such as `{@feature_name}`

There are deliberate limits. `ORDER BY` accepts one feature key. Projection items are feature names or aliases, not arbitrary expressions. `WHERE` expressions are handled by `DataFrame::query`, so use the query predicate language there rather than expecting a full external SQL dialect.

Verified SQL example

The following example is compiled and run by `PlussMic/docs/sphinx/manual/examples/plussmic_manual_examples.cpp`.

```cpp
void example_09_sql() {
  section("09 sql");
  auto sales = sales_frame().assign({{
      "revenue",
      [](const DataFrame& frame) {
        return frame.features("units") * frame.features("price");
      },
  }});

  auto top = sales.sql().execute(
      "SELECT city, revenue FROM df WHERE revenue >= 28 ORDER BY revenue DESC LIMIT 2");
  std::cout << top.to_string() << '\n';

  DataFrame::SqlOptions options;
  options.params["min_revenue"] = std::int64_t{28};
  options.identifier_params["value_col"] = "revenue";
  auto parameterized = sales.sql(std::move(options)).execute(
```

(continues on next page)

(continued from previous page)

```
      "SELECT city, {@value_col} FROM df WHERE revenue >= {min_revenue} ORDER BY revenue␣
↪DESC LIMIT 3");
  std::cout << parameterized.to_string() << '\n';

  auto prepared = sales.prepare_sql(
      "SELECT city FROM df WHERE revenue >= {min_revenue} ORDER BY revenue DESC LIMIT
↪{limit_n}");
  auto prepared_result = prepared.execute({
      {"min_revenue", std::int64_t{28}},
      {"limit_n", std::int64_t{2}},
  });
  std::cout << prepared_result.to_string() << '\n';
}
```

The `sales_frame()` helper builds this input:

```
city    region  units  price
Accra   south   10     3
Tema    south   7      4
Kumasi  middle  12     2.5
Tamale  north   5      5
```

The example first derives `revenue = units * price`, then runs three SQL variants over the enriched frame.

Verified output:

```
=== 09 sql ===
======================
     | city    revenue
----------------------
0    | Accra   30
2    | Kumasi  30
----------------------
[2 cs x 2 feat]
======================
     | city    revenue
----------------------
0    | Accra   30
2    | Kumasi  30
1    | Tema    28
----------------------
[3 cs x 2 feat]
=============
     | city
-------------
0    | Accra
2    | Kumasi
-------------
[2 cs x 1 feat]
```

The first result has two features, `city` and `revenue`, because the SQL projection is `SELECT city, revenue`. The `WHERE revenue >= 28` predicate keeps Accra, Kumasi, and Tema, then `ORDER BY revenue DESC LIMIT 2` returns only the first two after sorting. Accra and Kumasi both have revenue `30`, so both appear before Tema.

The second result uses two different parameter systems. `{min_revenue}` is a scalar parameter and becomes the numeric value `28`. `{@value_col}` is an identifier parameter and becomes the feature name `revenue`. Identifier parameters are for column names, not values. They are rendered as escaped SQL identifiers by PlussMic before parsing.

The third result uses a prepared SQL statement. The statement template is created once with `prepare_sql`, and the values are supplied when `execute` is called. The projection only asks for `city`, so the output has one feature.

SQL options

`DataFrame::SqlOptions` configures how SQL statements are rendered and executed.

```
pluss::table::DataFrame::SqlOptions options;
options.params["min_age"] = std::int64_t{45};
options.identifier_params["value_col"] = "balance";
options.max_result_rows = 1000;
options.timeout_ms = 5000;
options.diagnostics_explain = true;

auto result = df.sql(options).execute(
    "SELECT age, job, {@value_col} FROM df WHERE age >= {min_age}");
```

Important options:

Option	Purpose
`sources`	Additional named `DataFrame` sources available to joins.
`params`	Scalar placeholders like `{min_age}`. Values may be null, bool, `std::int64_t`, double, or string.
`identifier_params`	Identifier placeholders like `{@feature}`. Use these for feature or source names.
`missing_param_policy`	Choose whether missing parameters are errors or render as SQL null.
`reject_unused_params`	Fail if a supplied scalar or identifier parameter is never used.
`collision_policy`	Control duplicate projection names: error, suffix, or overwrite.
`max_intermediate_rows`	Guard intermediate frames after joins, filtering, ordering, or projection.
`max_result_rows`	Cap the returned result.
`memory_budget_mb`	Estimate and guard result memory.
`timeout_ms`	Check execution time at SQL execution stages.
`diagnostics_explain`	Print a statement plan through Pluss diagnostics.
`diagnostics_profile`	Print stage timing.
`diagnostics_trace_tok`	Print tokenization output.
`diagnostics_trace_pla`	Print the parsed plan.
`enable_plan_cache`	Enable or disable parsed statement caching.
`plan_cache_capacity`	Set the statement plan cache capacity.
`result_mode`	Request materialized or chunked behavior. Current `execute()` still returns a materialized `DataFrame`.
`chunk_rows`	Chunk size used with chunked result mode diagnostics.
`case_axis_aliases`	Names that expose the case axis as a projected output feature.

Scalar parameters

Scalar parameters are placeholders wrapped in braces:

```
DataFrame::SqlOptions options;
options.params["min_age"] = std::int64_t{45};
options.params["blocked_job"] = std::string{"unknown"};
```

(continues on next page)

(continued from previous page)

```
options.params["limit_n"] = std::int64_t{4};

auto out = df.sql(std::move(options)).execute(
    "SELECT age, job FROM df "
    "WHERE age >= {min_age} and job != {blocked_job} "
    "ORDER BY age DESC LIMIT {limit_n}");
```

The SQL renderer turns numbers into SQL numeric tokens, strings into quoted SQL string literals, bools into `true` or `false`, and null values into `null`. This happens before parsing, so the parser receives a normal SQL-like statement after parameter substitution.

Use scalar parameters for values only. Do not use them for column names. For column names, use identifier parameters.

Identifier parameters

Identifier parameters are placeholders with `@` after the opening brace:

```
DataFrame::SqlOptions options;
options.identifier_params["col"] = "age";
options.params["limit_n"] = std::int64_t{2};

auto out = df.sql(std::move(options)).execute(
    "SELECT {@col} FROM df LIMIT {limit_n}");
```

This is safer and clearer than concatenating a feature name into the SQL string yourself. PlussMic validates identifier parameter values before rendering them.

Use identifier parameters for:

- selected feature names;
- order-by feature names;
- source names when they are part of a supported identifier position;
- aliases where supported by the parser.

Prepared SQL

Use prepared SQL when the statement shape stays the same and only values change.

```
auto prepared = df.prepare_sql(
    "SELECT age FROM df WHERE age >= {min_age} ORDER BY age DESC LIMIT {limit_n}");

auto first = prepared.execute({
    {"min_age", std::int64_t{40}},
    {"limit_n", std::int64_t{3}},
});

auto second = prepared.execute({
    {"min_age", std::int64_t{55}},
    {"limit_n", std::int64_t{10}},
});
```

Prepared execution keeps the statement template separate from runtime parameters. It also lets you bind identifier parameters at execution time:

```
auto prepared = df.prepare_sql(
    "SELECT {@col} FROM df WHERE age >= {min_age} LIMIT {limit_n}");

auto out = prepared.execute(
    {
        {"min_age", std::int64_t{45}},
        {"limit_n", std::int64_t{5}},
    },
    {
        {"col", "balance"},
    });
```

Case-axis projection

The case axis is not a normal feature unless you ask SQL to expose it. Configure `case_axis_aliases` when you want case labels in the result:

```
auto with_case = df.sql(DataFrame::SqlOptions{
    .case_axis_aliases = {"case", "idx"},
}).execute("SELECT idx, age FROM df ORDER BY age DESC LIMIT 3");
```

In this example, `idx` becomes a projected output feature representing the original case labels. Without `case_axis_aliases`, `SELECT idx` would be treated like a normal feature lookup and would fail unless the frame actually had a feature named `idx`.

Joins

Additional frames are supplied through `SqlOptions::sources`. The calling frame must still participate in the `FROM` or `JOIN` sources.

```
DataFrame metrics({
    {"age", df.features("age")},
    {"score", df.features("balance")},
});

auto joined = df.sql(DataFrame::SqlOptions{
    .sources = {{"m", &metrics}},
}).execute("SELECT df.age, m.score FROM df JOIN m ON df.age = m.age LIMIT 6");
```

The join source is named `m`, so the SQL statement can refer to `m.age` and `m.score`. Joined right-hand columns are prefixed internally with the joined source alias, which is why qualified projections such as `m.score` are useful.

Keep joins narrow. Set `max_intermediate_rows` when joining data that could expand unexpectedly.

Collision policy

SQL projection names must be unique by default:

```
SELECT age as v, balance as v FROM df LIMIT 3
```

This projection asks for two output features with the same name, `v`, so the default collision policy rejects it.

Use suffixing if you want PlussMic to keep both:

```
auto out = df.sql(DataFrame::SqlOptions{
    .collision_policy = DataFrame::SqlOptions::CollisionPolicy::Suffix,
}).execute("SELECT age as v, balance as v FROM df LIMIT 3");
```

The result has `v` and `v_2`.

Use overwrite only when replacing an earlier duplicate is intentional:

```
auto out = df.sql(DataFrame::SqlOptions{
    .collision_policy = DataFrame::SqlOptions::CollisionPolicy::Overwrite,
}).execute("SELECT age as v, balance as v FROM df LIMIT 3");
```

The result keeps one `v`.

Choosing the right tool

Use `query` when:

- you only need to filter rows;
- you want the output to keep all features;
- the predicate is short and readable;
- you do not need joins or SQL parameters.

Use SQL when:

- you want projection and filtering in one expression;
- you need `ORDER BY`, `LIMIT`, or `OFFSET`;
- you need to bind scalar or identifier parameters;
- you need to join against additional named frames;
- you want execution guards such as `max_result_rows`, `max_intermediate_rows`, `memory_budget_mb`, or `timeout_ms`.

Use explicit masks when:

- the predicate is ordinary C++ logic;
- you need custom functions;
- you need external state;
- you already have a boolean `Series`;
- you want to debug every intermediate value.

Common mistakes

Do not use SQL for mutation:

```
df.sql().execute("DELETE FROM df"); // rejected
```

Do not use scalar parameters for feature names:

```
// Wrong: renders the string as a SQL string literal, not an identifier.
options.params["col"] = std::string{"age"};
df.sql(options).execute("SELECT {col} FROM df");
```

Use an identifier parameter instead:

```
options.identifier_params["col"] = "age";
df.sql(options).execute("SELECT {@col} FROM df");
```

Do not expect projection expressions:

```
SELECT units * price AS revenue FROM df
```

Derive the feature first, then select it:

```
auto enriched = df.assign({{
    "revenue",
    [](const DataFrame& frame) {
      return frame.features("units") * frame.features("price");
    },
}});

auto result = enriched.sql().execute("SELECT revenue FROM df");
```

Do not assume SQL `WHERE` is an external database dialect. It is PlussMic's DataFrame query predicate language inside a SQL statement.

1.3.8 Part VIII: Time-Series Workflows

Time-series work in PlussMic is built from two ideas:

1. A time axis is data, usually a `Timeline` or `TimeIndex` stored in a `Thread`.
2. Time-series calculations are ordinary table transformations that return either a `Thread` or a `DataFrame`.

Use the Thread accessor when you are working on one numeric value stream:

```
auto returns = df.features("close").ts().returns(1);
```

Use the DataFrame accessor when you want to transform one or more named features and keep a table-shaped result:

```
auto returns = df.ts({"close", "volume"}).returns(1);
```

Use DataFrame resampling and reindexing when the operation needs a time axis:

```
auto daily = df.ts({"close", "volume"}).resample(
    "date", "D", pluss::subsp::stats::ts::ResampleOp::Mean);
```

The accessors do not guess that a feature named `"date"` is time-like. The feature must actually be a `Timeline` or `TimeIndex` for time-axis methods.

The time data model

PlussMic represents time in three common ways:

Representation	Meaning
`Timeline`	A sequence of timestamps. Use it for date/time columns.
`TimeIndex`	A timeline with index semantics. Use it when alignment and as-of operations matter.
Numeric `Series`	A numeric value stream measured over time, such as close price, volume, demand, or latency.

Time-series value transforms require numeric `Series` threads. Time-axis methods require a `Timeline` or `TimeIndex` thread.

This split is deliberate. A price column and a timestamp column have different semantics even though both can be represented internally as arrays.

Creating time features

When you construct a frame in C++, use `pluss::make_timeline`:

```
DataFrame df({
    {"date", pluss::make_timeline({
        "2024-01-01T00:00:00Z",
        "2024-01-01T12:00:00Z",
        "2024-01-02T00:00:00Z"})},
    {"close", pluss::make_series({100.0, 105.0, 103.0})},
});
```

When you load from CSV, parse or cast time columns early:

```
pluss::loader::ReadCsvOptions options;
options.parse_dates = {"date"};

auto df = DataFrame::load_csv("prices.csv", options);
```

If a column loaded as text or as a numeric `YYYYMMDD` series, cast it:

```
auto with_time = df.cast_timeline({"date"});
```

The current source supports conversion of integer `YYYYMMDD` values through `cast_timeline`, which is useful for market-data files that store dates as `19900102` instead of ISO strings.

Sorting is your responsibility

Time-series calculations use row order unless the method explicitly resamples or aligns by a time axis. PlussMic will not sort a frame for you before computing a return, lag, rolling mean, or line plot.

For ordered calculations, sort first:

```
auto sorted = df.sort_values({DataFrame::ArrangeSpec{"date", true, true}});
auto returns = sorted.ts("close").returns(1);
```

This matters in production. A return computed on unsorted prices is not a slightly messy return; it is a different calculation.

Thread time-series accessor

`Thread::ts()` creates a `TimeSeriesAccessor` for one numeric `Series` payload:

```
auto close = df.features("close");
auto r = close.ts().returns(1);
```

The source thread must be defined, must be `ThreadKind::Series`, and must be numeric. If you call `ts()` on a `Note`, `Timeline`, or non-numeric payload, the operation throws when you call the accessor method.

Thread accessor methods

Method	Return	Meaning
`returns(periods)`	`Thread`	Fractional return over `periods`.
`log_returns(periods)`	`Thread`	Log return over `periods`.
`diff(periods)`	`Thread`	Difference from the value `periods` rows earlier.
`lag(periods)`	`Thread`	Shift values backward by `periods`; early rows become undefined numeric positions.
`lead(periods)`	`Thread`	Shift values forward by `periods`; trailing rows become undefined numeric positions.
`rolling_mean(window, min_periods)`	`Thread`	Rolling mean.
`rolling_std(window, min_periods)`	`Thread`	Rolling standard deviation.
`rolling_sum(window, min_periods)`	`Thread`	Rolling sum.
`rolling_var(window, min_periods)`	`Thread`	Rolling variance.
`rolling_min(window, min_periods)`	`Thread`	Rolling minimum.
`rolling_max(window, min_periods)`	`Thread`	Rolling maximum.
`ema(alpha)`	`Thread`	Exponential moving average.
`ewm_mean(alpha)`	`Thread`	Exponentially weighted mean.
`ewm_std(alpha, epsilon)`	`Thread`	Exponentially weighted standard deviation.
`expanding_mean(min_periods)`	`Thread`	Expanding mean.
`expanding_std(min_periods, epsilon)`	`Thread`	Expanding standard deviation.
`zscore_rolling(window, min_periods, epsilon)`	`Thread`	Rolling z-score.
`fill_gaps(mode, constant)`	`Thread`	Fill numeric gaps according to a gap-fill mode.
`resample(axis, freq, op, step)`	`TimeSeriesRes`	Resample one value stream against a time axis.

The return type is important. `Thread::ts()` methods return `Thread`, except for `resample`, which returns:

```
struct TimeSeriesResample {
  Thread labels;
  Thread values;
};
```

Use this form when you do not need a full DataFrame:

```
auto result = df.features("close").ts().resample(
    df.features("date"),
    "D",
    pluss::subsp::stats::ts::ResampleOp::Last);

auto labels = result.labels;
auto values = result.values;
```

Returns, differences, lag, and lead

The simplest time-series methods operate on row order:

```
auto close = df.features("close");

auto simple_returns = close.ts().returns(1);
auto log_returns = close.ts().log_returns(1);
auto one_row_diff = close.ts().diff(1);
auto lagged = close.ts().lag(1);
auto led = close.ts().lead(1);
```

The `periods` argument is a row count. It is not a calendar duration. For daily data, `periods = 1` usually means one trading day or one calendar day depending on your rows. For intraday data, `periods = 1` means one observation.

Current numeric time-series transforms use floating `nan` for positions that cannot be computed, such as the first one-period return or the first one-period lag. Do not document or test those positions as table `NA` unless you have explicitly converted them to a validity-mask representation.

Rolling windows

Rolling methods use observation windows:

```
auto rolling_mean = close.ts().rolling_mean(20, 10);
auto rolling_std = close.ts().rolling_std(20, 10);
auto rolling_sum = close.ts().rolling_sum(20, 10);
auto rolling_var = close.ts().rolling_var(20, 10);
auto rolling_min = close.ts().rolling_min(20, 10);
auto rolling_max = close.ts().rolling_max(20, 10);
```

`window` is the number of rows in the rolling window. `min_periods` is the minimum number of observations required before a value is emitted.

Use `min_periods = window` when you want strict full-window results:

```
auto strict = close.ts().rolling_mean(20, 20);
```

Use a smaller `min_periods` when early estimates are acceptable:

```
auto warmup = close.ts().rolling_mean(20, 5);
```

Rolling windows are observation-based, not calendar-based. If you need calendar windows, resample first or construct the exact frame you want before rolling.

Expanding and exponentially weighted metrics

Expanding metrics grow from the start of the series:

```
auto expanding_mean = close.ts().expanding_mean(5);
auto expanding_std = close.ts().expanding_std(5);
```

Exponentially weighted metrics give more weight to recent values:

```
auto ema = close.ts().ema(0.1);
auto ewm_mean = close.ts().ewm_mean(0.1);
auto ewm_std = close.ts().ewm_std(0.1);
```

`alpha` controls responsiveness. Values closer to `1.0` react faster. Values closer to `0.0` are smoother.

Rolling z-scores

Rolling z-scores are useful for anomaly detection and normalized signals:

```
auto z = close.ts().zscore_rolling(20, 10, 1e-12);
```

The `epsilon` argument protects divisions by very small standard deviations. Use a domain-appropriate value if your data scale is unusual.

Gap filling

The Thread and DataFrame accessors expose `fill_gaps`:

```
auto forward = close.ts().fill_gaps(
    pluss::subsp::stats::ts::GapFillMode::Forward);

auto zeros = close.ts().fill_gaps(
    pluss::subsp::stats::ts::GapFillMode::Zero);

auto constant = close.ts().fill_gaps(
    pluss::subsp::stats::ts::GapFillMode::Constant, 100.0);
```

Choose the fill mode based on the domain:

- Forward fill can be appropriate for last-observation-carried-forward prices or states.
- Backward fill can be useful for planned schedules where the next known value should apply.
- Zero fill is appropriate only when missing means zero.
- Constant fill should be a deliberate modeling choice.

DataFrame time-series accessor

`DataFrame::ts(...)` creates a `DataFrameTsAccessor` for one or more selected numeric features:

```
auto one = df.ts("close");
auto many = df.ts({"close", "volume"});
```

The selected feature names must exist and must be unique. Most accessor methods return a DataFrame whose cases come from the source frame. Resampling and reindexing methods return a DataFrame whose cases come from the time labels or target axis.

DataFrame accessor methods

Method	Return	Meaning
`returns(periods)`	`DataFra`	Returns for selected features.
`log_returns(periods)`	`DataFra`	Log returns for selected features.
`diff(periods)`	`DataFra`	Differences for selected features.
`lag(periods)`	`DataFra`	Lag selected features.
`lead(periods)`	`DataFra`	Lead selected features.
`rolling_mean(window, min_periods)`	`DataFra`	Rolling means.
`rolling_std(window, min_periods)`	`DataFra`	Rolling standard deviations.
`rolling_sum(window, min_periods)`	`DataFra`	Rolling sums.
`rolling_var(window, min_periods)`	`DataFra`	Rolling variances.
`rolling_min(window, min_periods)`	`DataFra`	Rolling minimums.
`rolling_max(window, min_periods)`	`DataFra`	Rolling maximums.
`ema(alpha)`	`DataFra`	Exponential moving averages.
`ewm_mean(alpha)`	`DataFra`	Exponentially weighted means.
`ewm_std(alpha, epsilon)`	`DataFra`	Exponentially weighted standard deviations.
`expanding_mean(min_periods)`	`DataFra`	Expanding means.
`expanding_std(min_periods, epsilon)`	`DataFra`	Expanding standard deviations.
`zscore_rolling(window, min_periods, epsilon)`	`DataFra`	Rolling z-scores.
`fill_gaps(mode, constant)`	`DataFra`	Fill numeric gaps in selected features.
`resample(time_feature, freq, op, step)`	`DataFra`	Aggregate selected features into time bins.
`asfreq(time_feature, freq, step, fill_mode, constant, op)`	`DataFra`	Resample, then fill gaps.
`insert_gaps(time_feature, freq, step)`	`DataFra`	Align to regular bins and insert gaps where exact observations are missing.
`align_to(time_feature, target_axis, mode)`	`DataFra`	Align selected features to a target time axis.
`reindex_time(time_feature, target_axis, mode, fill, constant)`	`DataFra`	Align to a target axis and optionally fill gaps.
`calendar(time_feature)`	`DataFra`	Extract calendar features from a time axis.
`lag_block(lags)`	`DataFra`	Create multiple lagged features.
`rolling_block(windows, options)`	`DataFra`	Create multiple rolling features.

Resampling

Resampling groups observations into time bins and aggregates values:

```
auto daily = df.ts({"close", "volume"}).resample(
    "date",
    "D",
    pluss::subsp::stats::ts::ResampleOp::Mean,
    1);
```

Arguments:

Argument	Meaning
`time_feature`	Name of a `Timeline` or `TimeIndex` feature.
`freq`	Frequency key, such as `"D"` for daily bins.
`op`	Aggregation operation, such as mean or last.
`step`	Positive frequency step.

The resulting DataFrame uses the resample labels as cases. In the verified example later in this chapter, four half-day observations become two daily rows.

The older `DataFrame::resample(time_feature, value_features, freq, op, step, render)` API still exists and returns a `DataFrameView`. Prefer `df.ts(features).resample(...)` when you are writing pipeline code and want a plain `DataFrame`.

As-frequency conversion

`asfreq` resamples and then fills gaps:

```
auto hourly = df.ts("close").asfreq(
    "date",
    "H",
    1,
    pluss::subsp::stats::ts::GapFillMode::Forward,
    std::nullopt,
    pluss::subsp::stats::ts::ResampleOp::Last);
```

Use it when your downstream code expects a regular time grid.

Inserting gaps

`insert_gaps` builds regular bins and aligns exact observations onto them:

```
auto with_gaps = df.ts("close").insert_gaps("date", "H", 1);
```

Rows with no exact source observation are emitted as undefined numeric positions. This is useful before plotting because it makes data outages visible instead of connecting across missing intervals.

Aligning and reindexing to a target axis

Use `align_to` when you already have the target timeline:

```
auto aligned = df.ts({"close", "volume"}).align_to(
    "date",
    target_axis,
    pluss::table::TsAlignMode::Exact);
```

`TsAlignMode` controls how target timestamps map to source timestamps:

Mode	Meaning
`Exact`	Only exact timestamp matches are valid.
`Backward`	Use the nearest source timestamp at or before the target.
`Forward`	Use the nearest source timestamp at or after the target.
`Nearest`	Use the nearest source timestamp in either direction.

Use `reindex_time` when you also want to fill missing aligned values:

```
auto reindexed = df.ts("close").reindex_time(
    "date",
    target_axis,
    pluss::table::TsAlignMode::Exact,
    pluss::table::TsReindexFill::Forward);
```

`TsReindexFill` controls post-alignment filling:

Fill	Meaning
`None`	Keep missing aligned values.
`Forward`	Forward-fill aligned gaps.
`Backward`	Backward-fill aligned gaps.
`Zero`	Fill aligned gaps with zero.
`Constant`	Fill aligned gaps with the supplied constant.

If you use `Constant`, pass the constant value:

```
auto filled = df.ts("close").reindex_time(
    "date",
    target_axis,
    pluss::table::TsAlignMode::Exact,
    pluss::table::TsReindexFill::Constant,
    100.0);
```

Calendar features

`calendar(time_feature)` extracts useful calendar columns from a time axis:

```
auto cal = df.ts("close").calendar("date");
```

It returns:

- `date_year`
- `date_month`
- `date_day`
- `date_hour`
- `date_weekday`

The prefix comes from the time feature name. For a feature named `"timestamp"`, the output columns are `timestamp_year`, `timestamp_month`, and so on.

Calendar features are often used for grouping, joins, or model features:

```
auto enriched = df.combine(cal, {DataFrame::CombineOp::Columns, "_cal", true});
```

Lag blocks

`lag_block` creates several lagged features at once:

```
auto lags = df.ts({"close", "volume"}).lag_block({1, 2, 5});
```

For features `close` and `volume`, that emits:

- `close_lag1`
- `close_lag2`
- `close_lag5`
- `volume_lag1`

- volume_lag2
- volume_lag5

Use lag blocks for supervised learning datasets, event studies, or feature engineering.

Rolling blocks

rolling_block creates many rolling features in one pass:

```
pluss::table::TsRollingBlockOptions options;
options.include_mean = true;
options.include_std = true;
options.include_min = true;
options.include_max = true;
options.min_periods = 5;

auto block = df.ts({"close", "volume"}).rolling_block({5, 20}, options);
```

TsRollingBlockOptions contains:

Option	Default	Meaning
include_mean	true	Emit rolling mean features.
include_std	true	Emit rolling standard deviation features.
include_min	false	Emit rolling minimum features.
include_max	false	Emit rolling maximum features.
include_sum	false	Emit rolling sum features.
include_var	false	Emit rolling variance features.
min_periods	1	Minimum observations required for each rolling value.

The output naming convention is:

```
<feature>_roll_mean_w<window>
<feature>_roll_std_w<window>
<feature>_roll_min_w<window>
<feature>_roll_max_w<window>
<feature>_roll_sum_w<window>
<feature>_roll_var_w<window>
```

For example, close with window 20 can emit close_roll_mean_w20 and close_roll_std_w20.

Verified example: returns, resampling, calendar, and lag blocks

This example is compiled and run by the manual example project:

```
DataFrame df({
    {"date", pluss::make_timeline({
        "2024-01-01T00:00:00Z",
        "2024-01-01T12:00:00Z",
        "2024-01-02T00:00:00Z",
        "2024-01-02T12:00:00Z"})},
    {"close", pluss::make_series({100.0, 105.0, 103.0, 109.0})},
    {"volume", pluss::make_series({10.0, 20.0, 30.0, 40.0})},
});
```

(continues on next page)

(continued from previous page)

```
std::cout << "returns:\n"
          << df.features("close").ts().returns(1).to_string() << '\n';
std::cout << "rolling mean window 2:\n"
          << df.features("close").ts().rolling_mean(2, 2).to_string()
          << '\n';

auto daily = df.ts({"close", "volume"}).resample(
    "date", "D", pluss::subsp::stats::ts::ResampleOp::Mean);
std::cout << "daily mean:\n" << daily.to_string() << '\n';

auto calendar = df.ts("close").calendar("date");
std::cout << "calendar:\n"
          << calendar.features({"date_year", "date_month",
                                "date_weekday"}).to_string() << '\n';

auto lags = df.ts({"close", "volume"}).lag_block({1, 2});
std::cout << "lag block:\n" << lags.to_string() << '\n';
```

The verified output is:

```
=== 16 time series ===
returns:
0  nan
1  0.05
2  -0.01904761905
3  0.05825242718
dtype: Series
rolling mean window 2:
0  nan
1  102.5
2  104
3  106
dtype: Series
daily mean:
==========================
           | close  volume
--------------------------
2024-01-01 | 102.5  15
2024-01-02 | 106    35
--------------------------
[2 cs x 2 feat]
calendar:
=============================================
     | date_year  date_month  date_weekday
---------------------------------------------
0    | 2024       1           2024-01-01 (Mon)
1    | 2024       1           2024-01-01 (Mon)
2    | 2024       1           2024-01-02 (Tue)
3    | 2024       1           2024-01-02 (Tue)
---------------------------------------------
[4 cs x 3 feat]
```

(continues on next page)

(continued from previous page)

```
lag block:
=======================================================
     | close_lag1  close_lag2  volume_lag1  volume_lag2
-------------------------------------------------------
0    | nan         nan         nan          nan
1    | 100         nan         10           nan
2    | 105         100         20           10
3    | 103         105         30           20
-------------------------------------------------------
[4 cs x 4 feat]
```

The output demonstrates several important details:

- The first one-period return is `nan` because no previous value exists.
- The strict two-row rolling mean is `nan` for the first row because `min_periods = 2`.
- Daily resampling uses the resample labels as DataFrame cases.
- Calendar extraction keeps the original case count.
- Lag blocks create one output feature for each selected feature and lag.

Choosing the right API

Need	Use
One transformed value stream	`df.features("x").ts()`
Several transformed value streams	`df.ts({"x", "y"})`
Resample one value stream and handle labels manually	`Thread::ts().resample(axis, ...)`
Resample several value streams into a new frame	`DataFrame::ts(features).resample(...)`
Add calendar model features	`DataFrame::ts(feature).calendar(time_feature)`
Build lagged model features	`DataFrame::ts(features).lag_block(...)`
Build rolling model features	`DataFrame::ts(features).rolling_block(...)`
Align data to an existing schedule	`align_to` or `reindex_time`
Make missing intervals visible	`insert_gaps`

Production checklist

Before using a time-series result in a report, model, or downstream pipeline:

- Confirm the time feature is `Timeline` or `TimeIndex`.
- Sort by the time feature before order-dependent calculations.
- Know whether `periods` means rows, not calendar units.
- Choose `min_periods` deliberately.
- Decide whether early undefined numeric positions should remain `nan` or be converted to table missingness.
- Use resampling when observations need calendar bins.
- Use reindexing when outputs must match a target schedule.
- Keep the original time feature available unless the resampled labels have replaced it as cases.
- Validate output row counts after resampling or reindexing.

Common mistakes

Calling the wrong accessor

This returns a `Thread`:

```
auto out = df.features("close").ts().returns(1);
```

This returns a `DataFrame`:

```
auto out = df.ts("close").returns(1);
```

Choose based on the return shape you need.

Treating row periods as calendar periods

This is a one-row return:

```
auto out = df.ts("close").returns(1);
```

It is not necessarily a one-day return. If the frame has hourly rows, it is a one-hour return. If the frame has irregular rows, it is a one-observation return. Resample first when the calendar interval matters.

Resampling a non-time feature

This fails when `"date"` is not a `Timeline` or `TimeIndex`:

```
auto daily = df.ts("close").resample("date", "D");
```

Fix the schema first:

```
auto fixed = df.cast_timeline({"date"});
auto daily = fixed.ts("close").resample("date", "D");
```

Rolling unsorted data

This compiles but can be wrong:

```
auto rolling = df.ts("close").rolling_mean(20, 20);
```

If `df` is not sorted by time, the rolling window follows row order. Sort first:

```
auto sorted = df.sort_values({DataFrame::ArrangeSpec{"date", true, true}});
auto rolling = sorted.ts("close").rolling_mean(20, 20);
```

1.3.9 Part IX: Plotting

PlussMic plotting turns a table workflow into a report workflow. You clean a `DataFrame`, choose the features that should become axes, groups, or values, and receive a PlussPl `Studio` object that can be inspected, saved, or further customized. This chapter teaches the table-level plotting API. It is not a replacement for the lower-level PlussPl manual; it is the layer you use when the data already lives in PlussMic.

The most important idea is that PlussMic does not plot directly from a CSV file, a SQL query, or an aggregation object. It plots from a `DataFrame` or `Atlas`. That means all the table skills from the earlier chapters still apply:

1. Load the data into a `DataFrame`.

2. Convert columns to the correct `Thread` kind.
3. Clean missing values deliberately.
4. Filter, aggregate, pivot, or derive features.
5. Call `plot()`, `plot3d()`, or `Atlas::plot()`.
6. Save the returned `pluss::pl::Studio`.

The plotting API is intentionally high-level. You ask for a line plot, histogram, heatmap, scatter matrix, or surface plot, and PlussMic builds the appropriate PlussPl trace objects.

Plotting namespaces and includes

Most user programs include the umbrella header:

```
#include <pluss/pluss.h>
```

The plotting classes used most often are:

```
pluss::table::DataFrame;
pluss::table::Thread;
pluss::table::DataFramePlotOptions;
pluss::table::LinePlotOptions;
pluss::table::ScatterPlotOptions;
pluss::table::BarPlotOptions;
pluss::table::HistPlotOptions;
pluss::table::HeatmapPlotOptions;
pluss::table::CorrHeatmapPlotOptions;
pluss::table::ScatterMatrixPlotOptions;
pluss::table::ParallelPlotOptions;
pluss::table::AndrewsCurvesPlotOptions;
pluss::table::RadvizPlotOptions;
pluss::table::LagPlotOptions;
pluss::table::ActPlotOptions;
pluss::table::BootstrapPlotOptions;
pluss::table::BriefPlotOptions;
pluss::table::Scatter3DPlotOptions;
pluss::table::Line3DPlotOptions;
pluss::table::Hist3DPlotOptions;
pluss::table::Surface3DPlotOptions;
pluss::table::Wireframe3DPlotOptions;
pluss::pl::Studio;
```

All `DataFrame::plot()` and `DataFrame::plot3d()` methods return `pluss::pl::Studio`. A `Studio` is a figure container. It can have one or more panes, and each pane can contain one or more traces.

The plotting accessors

There are three user-facing entry points:

```
auto two_d = df.plot();
auto two_d_with_options = df.plot(options);
auto three_d = df.plot3d();
auto three_d_with_options = df.plot3d(options);
auto atlas_plots = atlas.plot();
```

Use `df.plot()` for ordinary 2D charts: line, scatter, bar, histogram, density, heatmap, correlation, scatter matrix, parallel coordinates, Andrews curves, radviz, lag plots, autocorrelation, bootstrap plots, pie plots, and brief table summaries.

Use `df.plot3d()` for 3D traces: 3D scatter, 3D line, 3D histogram, surface, and wireframe.

Use `atlas.plot()` when the depth axis matters. An `Atlas` contains multiple frames with compatible shape. That is useful for comparing scenarios, dates, simulation paths, or model versions.

What PlussMic expects from the data

Plotting methods use `Thread` kinds to decide how a feature can participate in a plot.

Thread kind	Plotting role
`Series`	Numeric y-values, x-values, histogram input, heatmap values, correlation input, 3D coordinates.
`Note`	Labels, categories, hue groups, case labels, and tick labels.
`Timeline`	Labeled temporal axes; numeric coordinates are epoch nanoseconds internally.
`TimeIndex`	Labeled temporal axes through the contained timeline.

Most numeric plot families require `Series` features. If you pass a `Note` feature to a method that requires a numeric feature, PlussMic throws an error instead of silently producing a misleading chart. A few axis paths can cast to labels because labeled axes are expected to be textual.

The distinction matters. In a CSV file, `2026-01-01` might be inferred as text, a timeline, or a case label depending on the loader options and schema. A line plot will behave differently in each case:

```
auto by_date = df.plot().line("close", "date");
```

If `date` is a `Timeline`, the x-axis uses positional coordinates with rendered date or datetime tick labels. If `date` is a numeric `Series`, the x-axis uses the numeric values directly. If `date` is a `Note`, the x-axis uses positional coordinates with note tick labels.

Missing values in plots

PlussMic keeps table-level missingness through validity masks. For plotting, numeric missing values are converted to floating-point `NaN` before PlussPl receives the values. That makes the missing point explicit to the plotting engine while preserving the original table data.

This behavior is useful, but it is not a substitute for cleaning. Before plotting, decide whether missing values should be dropped, filled, or kept as gaps:

```
auto report = raw
    .dropna({"timestamp", "close"})
    .fillna(0.0, {"volume"});

auto fig = report.plot().line({"close", "volume"}, "timestamp");
```

Do not let plotting be the first time you think about null semantics. For a production report, missingness is part of the analysis.

Figure and axis defaults

`DataFramePlotOptions` configures the accessor. It does not mutate the frame. You pass it once to `plot(options)` or `plot3d(options)`, and every plot created by that accessor inherits those defaults.

```
pluss::table::DataFramePlotOptions options;
options.default_x_feature = "date";
options.default_hue_feature = "symbol";
options.default_bins = 30;
options.width = 900.0;
options.height = 520.0;
options.use_tight_layout = true;
options.tight_layout_padding = 8.0;

auto plots = df.plot(options);
auto close = plots.line("close");
auto volume = plots.hist("volume");
auto spread = plots.scatter("bid", "ask");
```

In that example, `line("close")` uses `"date"` as the x feature because `default_x_feature` is set. `scatter("bid", "ask")` uses `"symbol"` as the hue feature because `default_hue_feature` is set and no hue feature was passed to `scatter`.

DataFramePlotOptions reference

Option	Default	Meaning
`default_x_featur`	unset	Feature used as x-axis when a method accepts an optional x feature and none is passed.
`default_hue_feat`	unset	Feature used for scatter grouping when no explicit hue feature is passed.
`default_bins`	`20`	Histogram bin fallback. Used when a histogram call passes `20` and this option has a different value, or when bins are non-positive.
`datetime_label_v`	`Full`	Rendering preference for timeline labels: full datetime, date only, or time only.
`max_labeled_tick`	`14`	Maximum labeled ticks sampled from label-heavy axes.
`x_tick_rotation_`	unset	Explicit x tick label rotation in degrees.
`y_tick_rotation_`	unset	Explicit y tick label rotation in degrees.
`x_label_rotation`	unset	Explicit x-axis label rotation in degrees.
`y_label_rotation`	unset	Explicit y-axis label rotation in degrees.
`x_tick_format`	unset	PlussPl tick format for x-axis numeric rendering.
`y_tick_format`	unset	PlussPl tick format for y-axis numeric rendering.
`x_tick_label_gap`	unset	Gap between x ticks and x tick labels.
`y_tick_label_gap`	unset	Gap between y ticks and y tick labels.
`x_label_gap_poin`	unset	Gap between x-axis and x-axis label.
`y_label_gap_poin`	unset	Gap between y-axis and y-axis label.
`labeled_tick_rot`	unset	Legacy x-axis tick rotation option. Prefer `x_tick_rotation_deg`.
`width`	current figure width	Output figure width.
`height`	current figure height	Output figure height.
`use_tight_layout`	`false`	Enables PlussPl tight layout for the returned `Studio`.
`tight_layout_pad`	`6.0`	Padding passed to tight layout when enabled.

The accessor applies `width`, `height`, and tight layout to the `Studio` before returning it. Axis options are applied to panes as the plot family creates them.

Inspecting returned figures

Because every plotting method returns a `Studio`, you can inspect the figure without opening a GUI:

```
auto fig = df.plot().line({"close", "open"}, "date");

std::cout << "panes=" << fig.pane_count() << '\n';
if (!fig.panes().empty()) {
  const auto& pane = fig.panes().front();
  std::cout << "first pane traces=" << pane.trace_count() << '\n';
}
```

This is a useful testing technique. A documentation example, unit test, or CI job can verify that a line plot produced two trend traces without having to compare rendered pixels.

A verified plotting example

The following program builds a small frame, creates several 2D and 3D plots, prints figure summaries, and saves one SVG. This is the plotting section from the manual example program in `manual/examples/plussmic_manual_examples.cpp`.

```
std::string trace_kind_name(pluss::pl::TraceKind kind) {
  switch (kind) {
    case pluss::pl::TraceKind::Trend: return "Trend";
    case pluss::pl::TraceKind::Cloud: return "Cloud";
    case pluss::pl::TraceKind::Cloud3: return "Cloud3";
    case pluss::pl::TraceKind::Fieldmap: return "Fieldmap";
    case pluss::pl::TraceKind::Bucket: return "Bucket";
    case pluss::pl::TraceKind::Sheet: return "Sheet";
    default: return "Other";
  }
}

void print_figure_summary(const std::string& label,
                          const pluss::pl::Studio& figure) {
  std::cout << label << ": panes=" << figure.pane_count();
  if (!figure.panes().empty()) {
    const auto& pane = figure.panes().front();
    std::cout << ", first pane traces=" << pane.trace_count();
    if (!pane.traces().empty()) {
      std::cout << ", first trace="
                << trace_kind_name(pane.traces().front().kind());
    }
  }
  std::cout << '\n';
}

void plotting_example() {
  auto df = DataFrame({
      {"x", pluss::make_series({0.0, 1.0, 2.0, 3.0})},
      {"y", pluss::make_series({1.0, 1.4, 2.2, 3.0})},
      {"z", pluss::make_series({0.8, 1.0, 1.6, 2.4})},
      {"grp", pluss::make_note({"alpha", "alpha", "beta", "beta"})},
  });
```

(continues on next page)

(continued from previous page)

```
  pluss::table::DataFramePlotOptions plot_options;
  plot_options.default_hue_feature = "grp";
  plot_options.width = 720.0;
  plot_options.height = 420.0;
  plot_options.use_tight_layout = true;

  auto line = df.plot(plot_options).line({"y", "z"}, "x");
  auto scatter = df.plot(plot_options).scatter("x", "y");
  auto hist = df.plot().hist({"x", "y"}, 6);
  auto corr = df.plot().corr_heatmap({"x", "y", "z"});
  auto brief = df.plot().brief({"x", "y", "z"});
  auto scatter3d = df.plot3d().scatter("x", "y", "z");

  auto grid = DataFrame({
      {"gx", pluss::make_series({0.0, 1.0, 0.0, 1.0})},
      {"gy", pluss::make_series({0.0, 0.0, 1.0, 1.0})},
      {"gz", pluss::make_series({1.0, 2.0, 3.0, 4.0})},
  });
  auto surface = grid.plot3d().surface("gx", "gy", "gz");

  print_figure_summary("line", line);
  print_figure_summary("scatter default hue", scatter);
  print_figure_summary("hist multi", hist);
  print_figure_summary("corr heatmap", corr);
  std::cout << "brief: panes=" << brief.pane_count()
            << ", caption characters="
            << brief.panes().front().caption_text().size() << '\n';
  print_figure_summary("scatter3d", scatter3d);
  print_figure_summary("surface3d", surface);

  line.save_svg("line.svg");
  std::cout << "saved svg bytes=" << std::filesystem::file_size("line.svg")
            << '\n';
}
```

The verified output from this build is:

```
=== 15 plotting ===
line: panes=1, first pane traces=2, first trace=Trend
scatter default hue: panes=1, first pane traces=2, first trace=Cloud
hist multi: panes=2, first pane traces=1, first trace=Bucket
corr heatmap: panes=1, first pane traces=1, first trace=Fieldmap
brief: panes=1, caption characters=265
scatter3d: panes=1, first pane traces=1, first trace=Cloud3
surface3d: panes=1, first pane traces=1, first trace=Sheet
saved svg bytes=9101
saved gallery files=14
```

The `brief` plot reports zero traces because it stores its table summary as caption text on the pane rather than as a geometric trace. That is expected.

The SVG byte count is included to prove the export occurred in this build. It can change after renderer or theme changes,

so use it as an example output, not as a stable cross-version test assertion.

Verified plot gallery

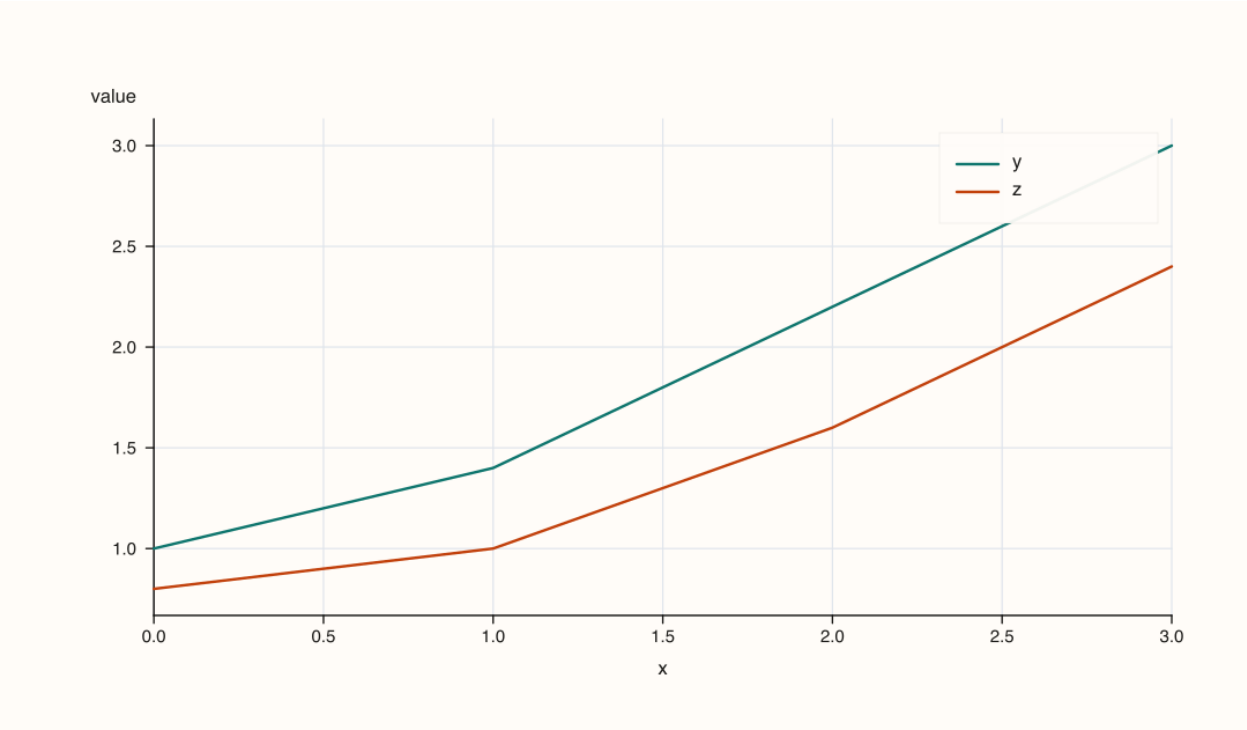
The same executable saves each figure in SVG and PNG form:

```
const std::vector<std::pair<std::string, const pluss::pl::Studio*>> gallery = {
    {"line", &line},
    {"scatter", &scatter},
    {"hist", &hist},
    {"corr_heatmap", &corr},
    {"brief", &brief},
    {"scatter3d", &scatter3d},
    {"surface3d", &surface},
};

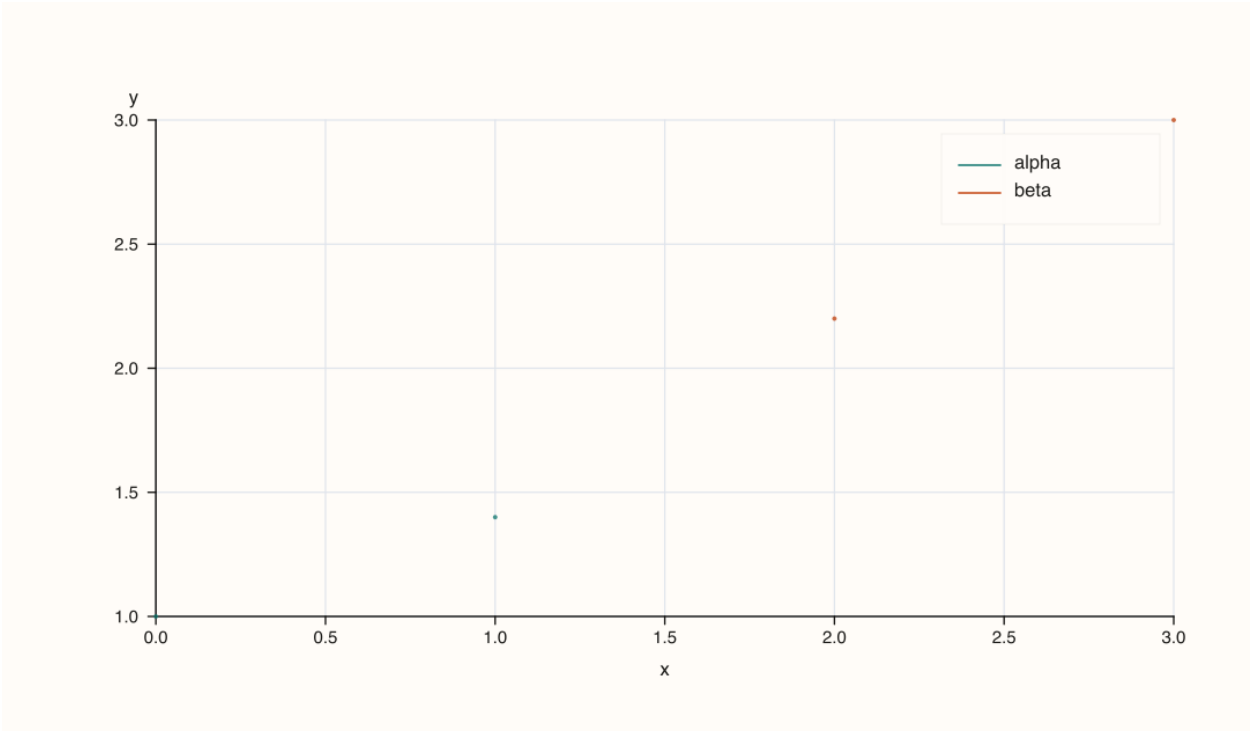
for (const auto& [name, figure] : gallery) {
  figure->save_svg(name + ".svg");
  figure->save_png(name + ".png", 144);
}
```

These are rendered artifacts from that run, not illustrative substitutes.

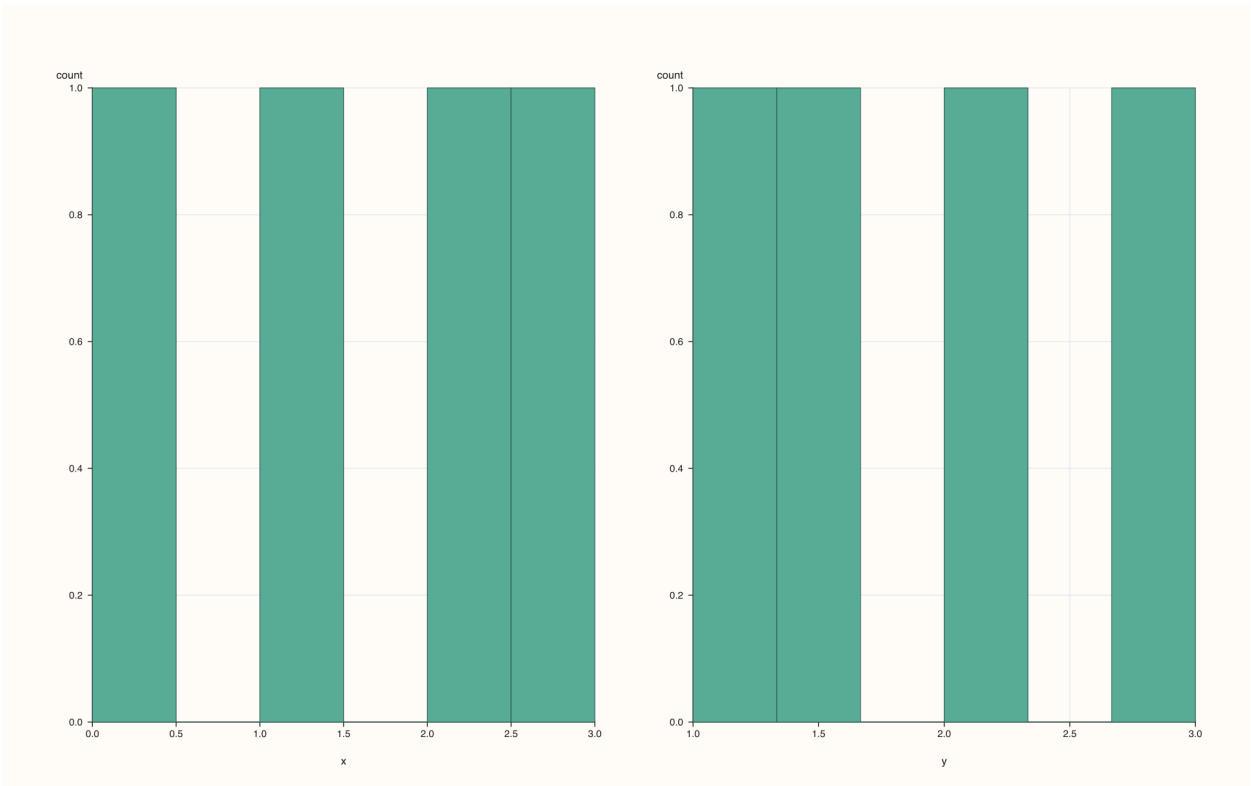
Two-series line plot



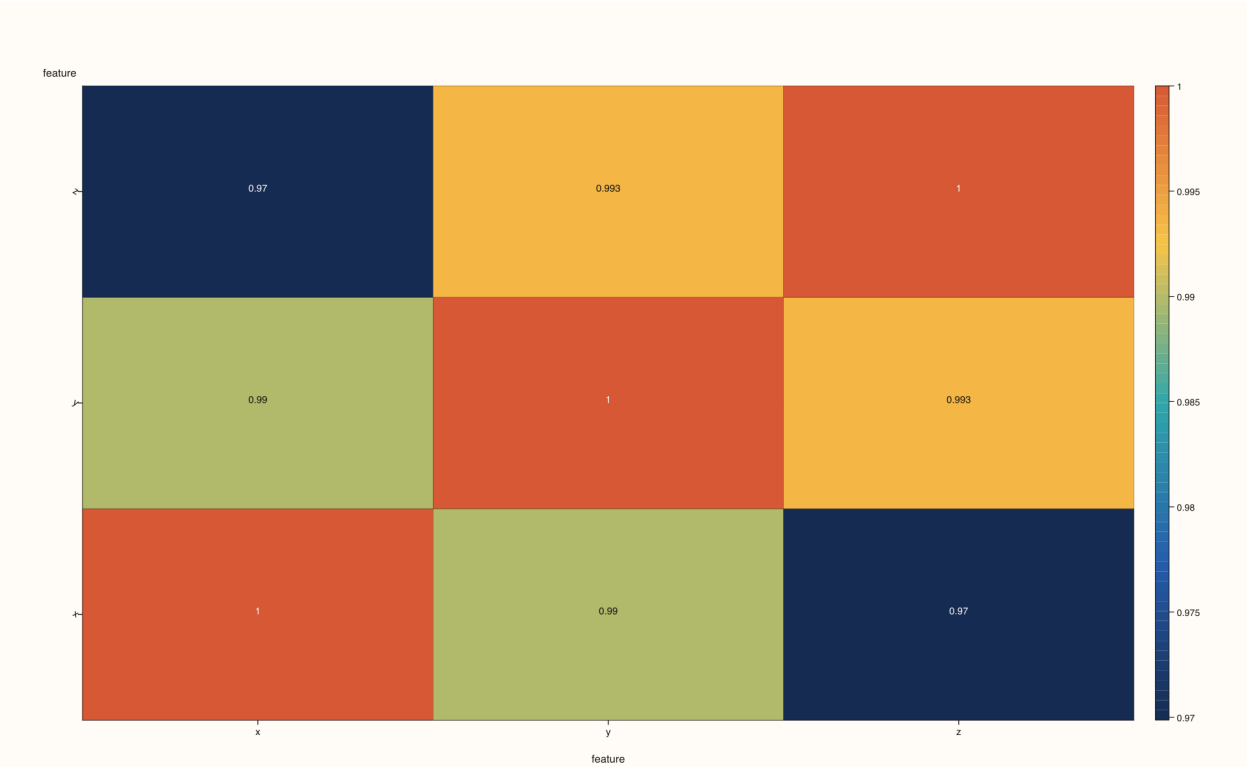
Scatter plot with hue groups



Multiple histograms



Correlation heatmap

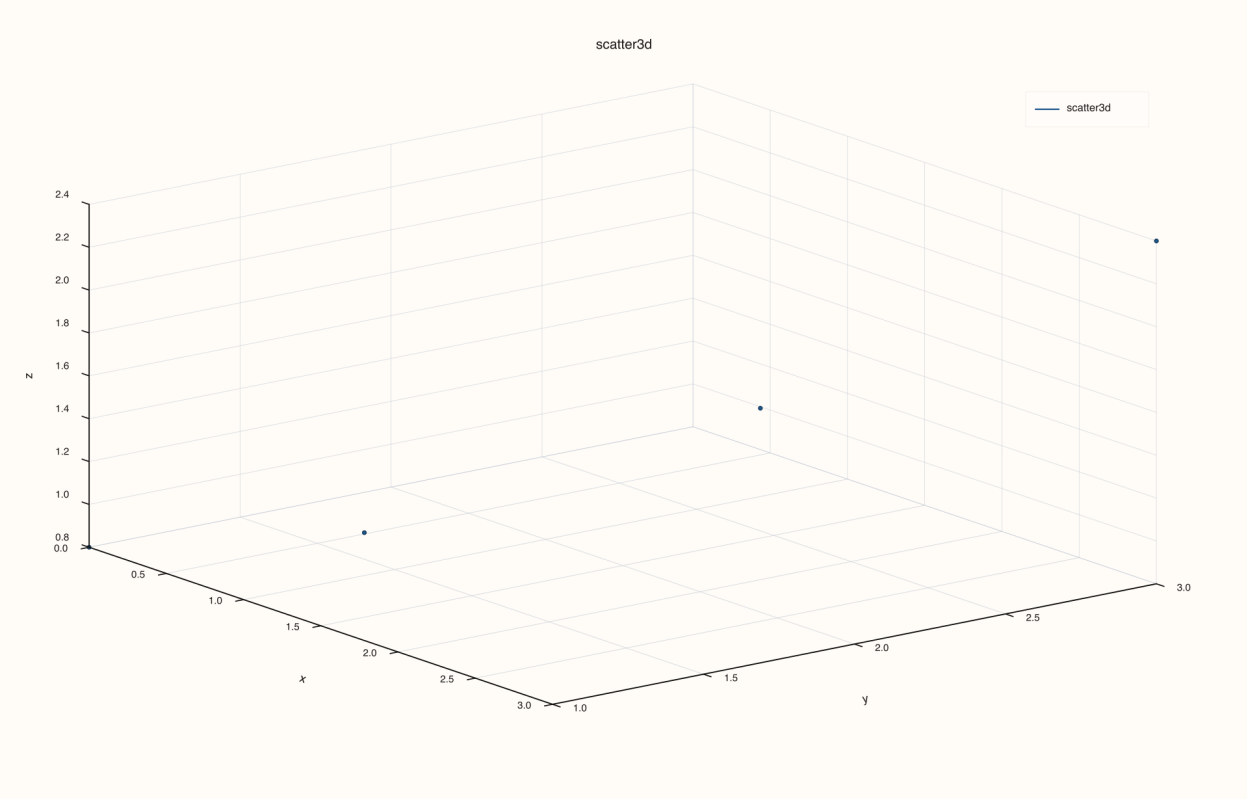


Statistical brief

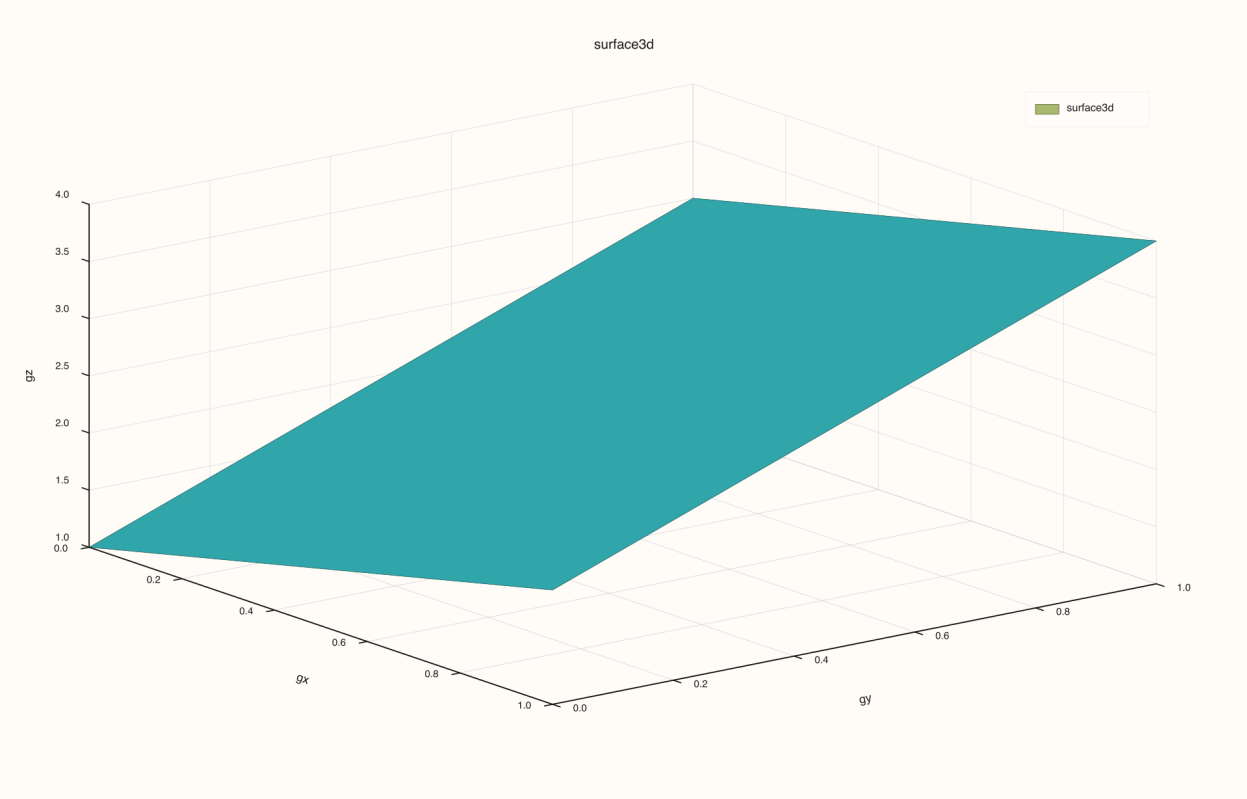
table

case	x	y	z
0	0.0000	1.0000	0.8000
1	1.0000	1.4000	1.0000
2	2.0000	2.2000	1.6000
3	3.0000	3.0000	2.4000

3D scatter plot



3D surface



Line plots

Use a line plot when the x-axis has an ordering and the y feature changes over that order. Common examples include prices over dates, cumulative quantities over event number, or metrics over training step.

```
auto fig = df.plot().line("close", "date");
```

For multiple y features, pass an initializer list:

```
auto fig = df.plot().line({"open", "high", "low", "close"}, "date");
```

Each y feature becomes one `Trend` trace. If there is more than one y feature, the pane shows a legend. The y-axis label is the feature name for a single y feature and `"value"` for multiple y features.

`LinePlotOptions` controls line style:

```
pluss::table::LinePlotOptions line_options;
line_options.alpha = 0.85;
line_options.line_width = 2.0;
line_options.y_limit_method = pluss::pl::LimitMethod::Padded;

auto fig = df.plot().line({"actual", "forecast"}, "date", line_options);
```

Option	Default	Meaning
alpha	1.0	Trace opacity, clamped by styling logic.
line_width	1.6	Stroke width.
y_limit_method	Padded	Optional PlussPl y-axis limit behavior.

Line plots should usually be sorted by the x feature before plotting. PlussMic does not reorder rows inside `line()`.

Step plots

Use a step plot when values remain constant until the next observation: account balances, inventory levels, regime labels encoded numerically, or state-machine outputs.

```
auto fig = df.plot().step("inventory", "date");
```

Multiple y features are supported:

```
auto fig = df.plot().step({"warehouse_a", "warehouse_b"}, "date");
```

`StepPlotOptions` contains:

Option	Default	Meaning
alpha	1.0	Trace opacity.
line_width	1.5	Stroke width.

As with line plots, PlussMic uses the row order you give it. Sort the frame if the x-axis is time-like.

Area plots

Use an area plot when the magnitude under the curve matters. Examples include volume, exposure, cumulative usage, or demand.

```
auto fig = df.plot().area("volume", "date");
```

For several independent filled curves:

```
auto fig = df.plot().area({"organic", "paid"}, "date");
```

`AreaPlotOptions` contains:

Option	Default	Meaning
alpha	0.88	Fill opacity.
line_width	1.3	Boundary stroke width.

Stacked area plots

Use a stacked area plot when the y features are components of a total:

```
auto fig = df.plot().area_stacked({"desktop", "mobile", "tablet"}, "date");
```

All y features should be non-negative unless you have a very specific reason to stack signed values. The implementation builds cumulative heights from the features in the order you pass them, so the first feature is the base layer and later features stack on top.

`AreaStackedPlotOptions` contains:

Option	Default	Meaning
`alpha`	`0.7`	Fill opacity.
`line_width`	`1.2`	Boundary stroke width.

Scatter plots

Use a scatter plot to inspect relationships between two numeric features.

```
auto fig = df.plot().scatter("temperature", "sales");
```

Add a hue feature to split the points into groups:

```
auto fig = df.plot().scatter("temperature", "sales", "region");
```

Or configure a default hue feature once:

```
pluss::table::DataFramePlotOptions options;
options.default_hue_feature = "region";

auto fig = df.plot(options).scatter("temperature", "sales");
```

When hue is used, PlussMic creates one `Cloud` trace per hue label. Hue features should be `Note`, `Timeline`, or `TimeIndex` threads. The typical hue feature is a `Note` category such as region, class, symbol, status, or cohort.

`ScatterPlotOptions` contains:

Option	Default	Meaning
`alpha`	`0.92`	Marker opacity.
`line_width`	`1.1`	Marker stroke width.

Scatter plots do not aggregate. If the data has millions of points, sample or aggregate first.

Bar plots

Use a bar plot when one axis is categorical or discrete and the other is a numeric value.

```
auto fig = df.plot().bar("revenue", "region");
```

For grouped bars, pass several y features:

```
auto fig = df.plot().bar({"revenue", "cost"}, "region");
```

`BarPlotOptions` contains:

Option	Default	Meaning
`ci_enabled`	`false`	Adds confidence-interval whiskers for repeated x groups.
`ci_level`	`0.95`	Confidence level used when CI is enabled.
`horizontal`	`false`	Draws horizontal bars when true.

A confidence-interval bar plot groups repeated x positions, computes a mean, and adds a whisker trace. This is useful when each category has multiple observations:

```
pluss::table::BarPlotOptions options;
options.ci_enabled = true;
options.ci_level = 0.95;

auto fig = df.plot().bar("latency_ms", "endpoint", options);
```

For horizontal bars:

```
pluss::table::BarPlotOptions options;
options.horizontal = true;

auto fig = df.plot().bar("revenue", "region", options);
```

Horizontal bars are often easier to read when labels are long.

Stacked bar plots

Use a stacked bar plot when each y feature is part of the same total:

```
auto fig = df.plot().bar_stacked({"new", "returning", "reactivated"}, "month");
```

`BarStackedPlotOptions` contains:

Option	Default	Meaning
`alpha`	`0.65`	Bar opacity.
`line_width`	`1.0`	Boundary stroke width.

As with stacked area plots, feature order matters. Put the most foundational or largest component first if you want the chart to read naturally.

Box plots

Use a box plot to compare distributions across numeric features.

```
auto fig = df.plot().box({"control", "treatment"});
```

If you pass no features, PlussMic uses all numeric `Series` features:

```
auto fig = df.plot().box();
```

`BoxPlotOptions` contains:

Option	Default	Meaning
alpha	0.95	Trace opacity.
line_width	1.1	Stroke width.

Box plots ignore non-finite numeric values when building the distribution.

Violin plots

Use a violin plot to compare distributions while showing density shape.

```
auto fig = df.plot().violin({"control", "treatment"});
```

You can also plot one value feature grouped by a category:

```
auto fig = df.plot().violin("latency_ms", "endpoint");
```

The grouped form is usually what you want for long-form data. The multi-feature form is useful for wide data where each feature already represents one group.

`ViolinPlotOptions` contains:

Option	Default	Meaning
alpha	0.92	Trace opacity.
line_width	1.1	Stroke width.
width	0.8	Violin width.
bw_adjust	1.0	Kernel bandwidth adjustment.
cut	2.0	Density extension beyond observed data.
scale	Area	Violin scaling mode.
inner	Box	Inner summary drawing.
side	Full	Full or sided violin rendering.

Use box plots for a compact statistical summary. Use violin plots when the shape of the distribution matters.

Histograms

Use a histogram to inspect a numeric feature's distribution.

```
auto fig = df.plot().hist("latency_ms", 30);
```

For several features, pass a feature list:

```
auto fig = df.plot().hist({"latency_ms", "queue_ms"}, 30);
```

The multi-feature histogram creates one pane per feature, which keeps each distribution readable. If you call `hist(30)` with only a bin count, PlussMic uses all numeric `Series` features:

```
auto fig = df.plot().hist(30);
```

`HistPlotOptions` contains:

Option	Default	Meaning
`bins`	`20`	Number of histogram bins.
`alpha`	`0.95`	Bucket opacity.
`line_width`	`1.0`	Bucket boundary width.

You can pass the option object instead of a raw bin count:

```
pluss::table::HistPlotOptions options;
options.bins = 40;
options.alpha = 0.8;

auto fig = df.plot().hist("latency_ms", options);
```

Density plots

Use a density plot when you want a smooth distribution estimate.

```
auto fig = df.plot().density("latency_ms", 256);
```

For several features:

```
auto fig = df.plot().density({"latency_ms", "queue_ms"}, 256);
```

`DensityPlotOptions` contains:

Option	Default	Meaning
`points`	`128`	Number of evaluation points in the density curve.
`alpha`	`1.0`	Trace opacity.
`line_width`	`1.5`	Curve stroke width.

Histograms show counts in bins. Density plots show a smoothed estimate. Use both when you are validating a statistical report.

Pie plots

Use a pie plot when values are parts of a whole and the number of categories is small:

```
auto fig = df.plot().pie("revenue", "region");
```

`value_feature` must be numeric. `label_feature` is optional. If the label feature is omitted, PlussMic uses generated labels.

`PiePlotOptions` controls label text:

Option	Default	Meaning
`show_label`	`true`	Include the category label.
`show_percent`	`true`	Include percent of total.
`show_value`	`false`	Include raw numeric value.
`precision`	`1`	Decimal precision for percentages.
`separator`	`` `” ``	“`

Pie charts are easy to overuse. Prefer a bar chart when there are many categories or when users need to compare values precisely.

Heatmaps

PlussMic has two heatmap styles:

1. Feature matrix heatmaps.
2. Pivot heatmaps.

A feature matrix heatmap uses several numeric features as columns and cases as rows:

```
auto fig = df.plot().heatmap({"bid", "ask", "mid"});
```

A pivot heatmap uses three columns: row category, column category, and numeric value:

```
auto fig = df.plot().heatmap(
    "maturity",
    "delta_bucket",
    "implied_vol",
    DataFrame::GroupOp::Mean);
```

The pivot form groups repeated `(case_feature, feature_feature)` pairs using the requested `DataFrame::GroupOp`. The default operation is mean.

`HeatmapPlotOptions` contains:

Option	Default	Meaning
`annotate_levels`	`false`	Print cell values on the heatmap.
`scale_guide`	`true`	Show the color scale guide.
`borderless`	`false`	Use borderless cells.

Use annotation only when the heatmap is small enough for text to remain legible.

Correlation heatmaps

Use `corr_heatmap()` to visualize pairwise correlations among numeric features:

```
auto fig = df.plot().corr_heatmap({"open", "high", "low", "close"});
```

If you pass no features, PlussMic uses all numeric `Series` features:

```
auto fig = df.plot().corr_heatmap();
```

`CorrHeatmapPlotOptions` contains:

Option	Default	Meaning
`annotate_levels`	`true`	Print correlation values in cells.
`scale_guide`	`true`	Show the color scale guide.
`borderless`	`false`	Use borderless cells.

The plotting method uses the DataFrame correlation implementation and then renders the selected features as a square matrix.

Scatter matrices

Use a scatter matrix to inspect pairwise relationships among several numeric features:

```
auto fig = df.plot().scatter_matrix({"sepal_length", "sepal_width",
                                     "petal_length", "petal_width"});
```

The default full matrix can become large. For compact reports, render only the lower triangle:

```
pluss::table::ScatterMatrixPlotOptions options;
options.corner = true;

auto fig = df.plot().scatter_matrix({"x", "y", "z"}, options);
```

You can also configure the underlying PlussPl pair plot options:

```
pluss::table::ScatterMatrixPlotOptions options;
options.pair.triangle = pluss::table::ScatterMatrixTriangle::Lower;
options.pair.auto_tick_decimals = 1;

auto fig = df.plot().scatter_matrix({"x", "y", "z"}, options);
```

`ScatterMatrixPlotOptions` contains:

Option	Default	Meaning
`corner`	`false`	Convenience flag that switches a full matrix to the lower triangle.
`pair`	default pair options	Underlying PlussPl pair plot options.

Scatter matrices require at least two numeric features.

Parallel coordinates

Use parallel coordinates when you want to compare rows across many numeric features:

```
auto fig = df.plot().parallel({"a", "b", "c", "d"}, "class");
```

Each sampled case becomes a polyline across feature axes. The optional class feature groups or colors the lines.

`ParallelPlotOptions` contains:

Option	Default	Meaning
`labels`	empty	Axis labels. Empty means use feature names.
`normalize`	`true`	Normalize feature scales before plotting.
`max_points`	`1200`	Maximum rows drawn.
`alpha`	`0.24`	Line opacity.
`trace_width`	`0.9`	Line width.
`show_grid`	`true`	Show axis/grid guides.
`show_legend`	`true`	Show class legend.

Keep `normalize` enabled unless all features already share the same units.

Andrews curves

Andrews curves turn each row into a curve derived from the numeric feature values. They are useful for spotting clusters and outliers:

```
auto fig = df.plot().andrews_curves({"a", "b", "c", "d"}, "class");
```

`AndrewsCurvesPlotOptions` contains:

Option	Default	Meaning
`labels`	empty	Feature labels. Empty means use feature names.
`normalize`	`true`	Normalize feature scales before plotting.
`points`	`220`	Evaluation points per curve.
`max_curves`	`600`	Maximum rows drawn.
`alpha`	`0.28`	Curve opacity.
`trace_width`	`0.9`	Curve width.
`show_grid`	`true`	Show grid.
`show_legend`	`true`	Show class legend.

Andrews curves become unreadable quickly. Use sampling and class grouping.

Radviz

Radviz places features around a circle and positions rows by their normalized feature values:

```
auto fig = df.plot().radviz({"a", "b", "c", "d"}, "class");
```

`RadvizPlotOptions` contains:

Option	Default	Meaning
`labels`	empty	Anchor labels. Empty means use feature names.
`normalize`	`true`	Normalize feature values before layout.
`max_points`	`3000`	Maximum rows drawn.
`marker_area`	`18.0`	Marker area.
`trace_width`	`0.55`	Marker stroke width.
`show_grid`	`true`	Show circular guide/grid.
`show_anchors`	`true`	Show feature anchors.
`show_legend`	`true`	Show class legend.

Radviz is best for exploratory data analysis, not precise numeric reading.

Lag plots

Use a lag plot to compare a feature against a shifted version of itself:

```
auto fig = df.plot().lag("close");
```

The default lag is one row. Increase it when the domain has a meaningful period:

```
pluss::table::LagPlotOptions options;
options.lag = 7;
options.max_points = 2500;
```

(continues on next page)

(continued from previous page)

```
auto fig = df.plot().lag("close", options);
```

`LagPlotOptions` contains:

Option	Default	Meaning
`lag`	`1`	Row lag used for the shifted comparison.
`max_points`	`2500`	Maximum points drawn.
`marker_area`	`16.0`	Marker area.
`trace_width`	`0.55`	Marker stroke width.

Sort the frame before lag plotting. A lag plot on unsorted time series data is usually meaningless.

Autocorrelation plots

Use `act()` to plot autocorrelation-like behavior across lags:

```
auto fig = df.plot().act("close");
```

`ActPlotOptions` contains:

Option	Default	Meaning
`max_lag`	`40`	Maximum lag drawn.
`show_confidence_band`	`true`	Show confidence guide.
`confidence_level`	`0.95`	Confidence level for the guide.
`trace_width`	`1.2`	Spike trace width.

This plot family emits a `Spike` trace first.

Bootstrap plots

Use `bootstrap()` to visualize the sampling distribution of a statistic:

```
auto fig = df.plot().bootstrap("latency_ms");
```

Configure the statistic and simulation parameters:

```
pluss::table::BootstrapPlotOptions options;
options.samples = 1000;
options.sample_size = 0;
options.seed = 42;
options.bins = 30;
options.statistic = pluss::table::BootstrapStatistic::Median;

auto fig = df.plot().bootstrap("latency_ms", options);
```

`sample_size = 0` means use the available sample size. `seed` makes the bootstrap reproducible.

`BootstrapPlotOptions` contains:

Option	Default	Meaning
`samples`	`300`	Number of bootstrap samples.
`sample_size`	`0`	Sample size per bootstrap draw; zero means default to the data size.
`seed`	`7`	Random seed.
`bins`	`24`	Histogram bins.
`statistic`	`Mean`	`Mean`, `Median`, or `Std`.
`trace_width`	`0.85`	Bucket stroke width.

Brief table plots

`brief()` creates a figure containing a compact table summary as pane caption text:

```
auto fig = df.plot().brief({"city", "revenue", "margin"});
```

Because it is caption text rather than a geometric chart, the pane can have zero traces. That is normal.

`BriefPlotOptions` contains:

Option	Default	Meaning
`title`	`"table"`	Summary title.
`precision`	`4`	Numeric precision.
`include_header`	`true`	Include feature header.
`max_cases`	`10`	Maximum rows displayed.
`transpose`	`false`	Transpose the table summary.
`include_case_index`	`true`	Include case labels.
`features_per_table`	`0`	Split features into multiple tables; zero means no forced split.
`tables_per_row`	`1`	Tables per row when split.
`table_gap_spaces`	`4`	Space between split tables.

You can also add a brief summary to an existing figure:

```
auto fig = df.plot().line("revenue", "date");
df.plot().brief(fig, {"revenue"});
```

Use this for report figures where the chart and a small tabular summary belong together.

Auto plots

`auto_plot()` chooses a simple plot based on the selected numeric features:

```
auto fig = df.plot().auto_plot({"x", "y", "z"});
```

Auto plotting is useful for exploration and examples. For production reports, prefer an explicit plot family so a schema change does not silently change the visual design.

3D plotting

`plot3d()` returns `DataFramePlot3DAccessor`. It shares the top-level `DataFramePlotOptions` for figure size and layout, then each 3D method accepts family-specific options.

3D scatter

Use 3D scatter for point clouds:

```
auto fig = df.plot3d().scatter("x", "y", "z");
```

Optional size and color features are supported:

```
auto fig = df.plot3d().scatter("x", "y", "z", "market_cap", "sector_code");
```

`Scatter3DPlotOptions` contains:

Option	Default	Meaning
`scene`	default scene options	Camera, lighting, theme, grid, and scene presets.
`trace`	default trace options	Trace preset, shade, material, line width, transparency.
`default_marker_size`	`12.0`	Marker size when no size feature is passed.

3D line

Use 3D line plots for paths:

```
auto fig = df.plot3d().line("x", "y", "z");
```

`Line3DPlotOptions` contains `scene` and `trace` option groups.

3D histogram

Use a 3D histogram for bivariate density or counts:

```
pluss::table::Hist3DPlotOptions options;
options.bins_x = 30;
options.bins_y = 24;
options.normalize = true;

auto fig = df.plot3d().hist("x", "y", options);
```

`Hist3DPlotOptions` contains:

Option	Default	Meaning
`bins_x`	`26`	Number of x bins.
`bins_y`	`20`	Number of y bins.
`normalize`	`false`	Normalize counts.
`smooth`	`false`	Smooth the histogram surface.
`smooth_passes`	`1`	Number of smoothing passes.
`as_surface`	`false`	Render as a surface.
`as_wireframe`	`false`	Render as a wireframe.
`scene`	default scene options	Camera, lighting, theme, grid, and scene presets.
`trace`	default trace options	Trace preset, shade, material, line width, transparency.

Surfaces and wireframes

Use a surface when x and y coordinates define a grid or a pivotable set of points:

```
auto fig = df.plot3d().surface("strike", "maturity", "implied_vol");
```

Use a wireframe for the same data when the grid structure matters more than the filled surface:

```
auto fig = df.plot3d().wireframe("strike", "maturity", "implied_vol");
```

Both methods accept a `DataFrame::GroupOp` for repeated coordinate pairs:

```
auto fig = df.plot3d().surface(
    "strike",
    "maturity",
    "implied_vol",
    DataFrame::GroupOp::Mean);
```

`Surface3DPlotOptions` and `Wireframe3DPlotOptions` contain `scene` and `trace` option groups.

3D scene and trace options

The 3D option structs share these nested groups:

```
pluss::table::Plot3DSceneOptions scene;
pluss::table::Plot3DTraceOptions trace;
```

`Plot3DSceneOptions` can hold:

Option	Meaning
`preset`	PlussPl scene preset.
`theme`	3D theme preset.
`camera_preset`	Camera preset.
`camera`	Full camera options.
`lighting`	Lighting options.
`grid_style`	Grid style override.

`Plot3DTraceOptions` can hold:

Option	Meaning
`preset`	Trace preset.
`shade`	Shading mode.
`lighting`	Lighting mode.
`material`	Material profile.
`line_width`	Line width override.
`transparent`	Transparency override.

Use the presets for most work. Drop to full PlussPl configuration only when a publication or product UI needs exact visual control.

Atlas plotting

An `Atlas` is a stack of frames. Plotting an atlas is useful when the depth dimension is meaningful:

```
Thread scenarios = pluss::make_note({"base", "stress", "upside"});
pluss::table::Atlas atlas({base, stressed, optimistic}, scenarios);

auto fig = atlas.plot().line("revenue", "date");
```

Good atlas plotting cases include:

- scenarios in risk analysis;
- repeated simulations;
- one frame per model version;
- one frame per date or reporting period;
- one frame per cohort.

The same data hygiene rules apply. The frames should have compatible features, case counts, and feature meanings. If the depth labels are meaningful, keep them as `Note`, `Timeline`, or `TimeIndex` data so plots and legends can explain the depth axis.

Exporting plots

The returned `Studio` can be saved directly:

```
auto fig = df.plot().line("close", "date");

fig.save_svg("close.svg");
fig.save_pdf("close.pdf");
fig.save_png("close.png", 180);
```

Use SVG for documentation, web pages, and source-controlled report assets. Use PDF for print workflows. Use PNG when the target system cannot render vector graphics or when you need a fixed raster result.

You can also get serialized output in memory:

```
auto svg = fig.to_svg();
auto pdf = fig.to_pdf();
```

For reliable report builds, write plots into a generated output directory:

```
const std::filesystem::path out_dir = "build/report-plots";
std::filesystem::create_directories(out_dir);

df.plot().line("close", "date").save_svg((out_dir / "close.svg").string());
df.plot().corr_heatmap({"open", "high", "low", "close"})
    .save_svg((out_dir / "ohlc-corr.svg").string());
```

Treat exported plots as build artifacts unless your project intentionally checks them in.

Choosing the right plot

Task	Good first plot
One value over ordered x	`line()`
Value changes that hold until next event	`step()`
Magnitude over ordered x	`area()`
Components of a total over ordered x	`area_stacked()`
Relationship between two numeric features	`scatter()`
Numeric value per category	`bar()`
Components per category	`bar_stacked()`
Compare distributions across features	`box()` or `violin()`
Inspect one distribution	`hist()` or `density()`
Values over two categorical axes	pivot `heatmap()`
Pairwise correlations	`corr_heatmap()`
Pairwise relationships among many features	`scatter_matrix()`
Rows across many numeric dimensions	`parallel()`
Cluster/outlier exploration	`andrews_curves()` or `radviz()`
Time-series lag behavior	`lag()` or `act()`
Sampling distribution	`bootstrap()`
Compact table-in-figure summary	`brief()`
3D point cloud	`plot3d().scatter()`
3D path	`plot3d().line()`
3D bivariate counts	`plot3d().hist()`
3D grid surface	`plot3d().surface()`
3D grid skeleton	`plot3d().wireframe()`

Production plotting checklist

Before sending a plot to users, check:

- The frame is sorted when the plot depends on order.
- Missing values are intentionally dropped, filled, or left as gaps.
- Numeric features are really `Series` threads.
- Categorical labels are `Note` threads or are safely convertible to labels.
- Large datasets are sampled or aggregated before high-density plots.
- Long category labels use rotated ticks or horizontal bars.
- `width`, `height`, and tight layout are set for the target document.
- Export format matches the destination: SVG/PDF for vector, PNG for raster.
- The plot can be regenerated from source data and code.

Common plotting failures

Missing feature names

This fails:

```
auto fig = df.plot().scatter("x", "missing_feature");
```

The accessor asks the `DataFrame` for `"missing_feature"` and the frame throws. Check `df.feature_names()` before building dynamic plotting code.

Non-numeric features in numeric methods

This fails when `status` is a `Note`:

```
auto fig = df.plot().hist("status");
```

A histogram needs numeric values. Use `bar()` on an aggregated count table, or encode the category intentionally before plotting numeric distributions.

Unsorted time series

This compiles but may lie:

```
auto fig = df.plot().line("close", "date");
```

If `df` is not sorted by `date`, the line connects points in row order. Sort first:

```
auto sorted = df.sort_values({DataFrame::ArrangeSpec{"date", true, true}});
auto fig = sorted.plot().line("close", "date");
```

Too many labels

When an axis has labels, PlussMic samples labeled ticks using `max_labeled_ticks`. If you still have crowding, rotate ticks or aggregate:

```
pluss::table::DataFramePlotOptions options;
options.max_labeled_ticks = 8;
options.x_tick_rotation_deg = 45.0;

auto fig = df.plot(options).bar("revenue", "customer");
```

Too many points

Scatter plots and high-dimensional plots can become unreadable and slow. Use domain-aware filtering:

```
auto sample = df.query("event_rank < 5000");
auto fig = sample.plot().scatter("x", "y", "class");
```

Or aggregate:

```
auto grouped = df.group_by(
    {"region"},
    {{"revenue_sum", "revenue", DataFrame::GroupOp::Sum}},
    true,
    false);

auto fig = grouped.plot().bar("revenue_sum", "region");
```

When to use PlussPl directly

Use the DataFrame plotting API when the chart maps directly from table features. Use PlussPl directly when you need:

- custom multi-pane layouts not represented by the DataFrame API;
- manual trace construction;

- custom annotations beyond `brief()`;
- precise theme and mark control;
- composition of non-table graphics with table plots.

A good workflow is to use PlussMic for data preparation, then pass the resulting vectors, labels, or summaries into PlussPl for final composition.

1.3.10 Part X: Atlas

`Atlas` represents a stack of related `DataFrame` objects.

Construct an Atlas

```
auto frame0 = df.slice(0, 100);
auto frame1 = df.slice(100, 200);

pluss::table::Atlas atlas({frame0, frame1});
```

With depth labels:

```
pluss::table::Thread labels = pluss::make_note({"train", "valid"});
pluss::table::Atlas atlas({train, valid}, labels);
```

Verified example: split a frame into an Atlas

```
auto sales = sales_frame();
auto south = sales.query("region == 'south'");
auto other = sales.query("region != 'south'");
Thread labels = pluss::make_note({"south", "other"});

pluss::table::Atlas atlas({south, other}, labels);
std::cout << "depth=" << atlas.depth()
          << ", ncases=" << atlas.ncases()
          << ", nfeatures=" << atlas.nfeats() << '\n';
std::cout << atlas.loc("south").to_string() << '\n';

auto catalog = sales.kindelize();
auto roundtrip = catalog.tabelize();
std::cout << "roundtrip equals original="
          << (roundtrip.equals(sales) ? "true" : "false") << '\n';
```

Output:

```
depth=2, ncases=2, nfeatures=4
==================================
     | city   region  units  price
----------------------------------
0    | Accra  south   10     3
1    | Tema   south   7      4
----------------------------------
[2 cs x 4 feat]
roundtrip equals original=false
```

The Atlas has two frames: one for the south region and one for all other regions. `atlas.loc("south")` uses the depth labels and returns the first frame. `ncases()` is 2 because both frames in this example have two rows.

The final line demonstrates an important practical detail: converting through `Catalog` and back can produce a table that displays similarly but is not strictly `equals(...)` to the original frame in this build. Equality checks are stricter than user-facing display equivalence, so use them when exact internal round-trip identity matters.

Inspect

```
atlas.defined();
atlas.empty();
atlas.depth();
atlas.ncases();
atlas.nfeats();
atlas.column_names();
atlas.axis0_labels();
atlas.show();
```

All frames in an Atlas should have compatible schemas. Keep feature names and case semantics aligned unless you are intentionally modeling heterogeneous frames.

Access frames

```
auto first = atlas.frame(0);
auto also_first = atlas.dloc(0);
auto by_label = atlas.loc("train");
```

`loc` supports string, integer, and timeline labels when the depth labels support those lookup modes.

Slice and sort depth

```
auto first_five = atlas.head(5);
auto last_five = atlas.tail(5);
auto subset = atlas.slice_frames(10, 20);
auto sorted = atlas.sort_depth(true, true);
```

Relabel or align depth:

```
auto relabeled = atlas.relabel_depth(new_labels);
auto aligned = atlas.align_depth(target_labels);
```

Map frames

Apply a DataFrame transform to every frame:

```
auto cleaned = atlas.map_frames([](const pluss::table::DataFrame& frame) {
  return frame.fillna(0.0);
});
```

Aggregate across frames:

```
auto summary = atlas.aggregate_over_frames(
    [](const std::vector<pluss::table::DataFrame>& frames) {
      return frames.front().copy();
    });
```

Use `aggregate_over_frames` when the aggregation requires seeing all frames at once.

Squeeze and unsqueeze

Create an Atlas from a DataFrame:

```
auto atlas = df.unsqueeze();
```

Return to a DataFrame when depth is reducible:

```
auto frame = atlas.squeeze();
```

Convert to Archive

```
auto archive = atlas.kindelize();
auto roundtrip = pluss::table::Atlas::tabelize(archive);
```

This is the Atlas equivalent of `DataFrame <-> Catalog` conversion.

Atlas plotting

```
atlas.plot();
```

Atlas plots are useful for:

- depth overlays;
- scenario comparisons;
- frame facets;
- depth surfaces;
- rolling depth analysis.

See *Atlas API Reference* and the Atlas plot examples under `PlussMic/examples/table/atlas_plot_demo.cpp`.

1.3.11 Part XI: Kindle: Stream, Catalog, and Archive

Kindle is the stream/catalog model in PlussMic. It is useful when your data is still typed and column-like, but you want metadata and catalog operations instead of strict DataFrame ergonomics.

Stream

A `Stream` is similar to a `Thread`, with optional cases and metadata.

```
auto stream = pluss::kindle::Stream::from_thread(df.features("balance"));
```

Common stream operations:

```
stream.size();
stream.kind();
stream.dtype_name();

stream.isnull();
stream.notnull();
stream.fillna(0.0);
```

(continues on next page)

(continued from previous page)

```
stream.dropna();
stream.ffill();
stream.bfill();

stream.head(10);
stream.tail(10);
stream.slice(10, 20);
stream.take({0, 2, 5});
stream.filter(mask);
```

Numeric stream reductions:

```
stream.sum();
stream.mean();
stream.min();
stream.max();
stream.var();
stream.std();
```

Metadata:

```
auto annotated = stream.with_meta("source", "raw_import");
auto source = annotated.meta("source");
auto stripped = annotated.drop_meta("source");
```

Cases:

```
auto with_cases = stream.with_cases(df.cases());
auto no_cases = with_cases.clear_cases();
```

Catalog

A `Catalog` is a named set of streams. Convert from DataFrame:

```
auto catalog = df.kindelize();
```

Convert back:

```
auto frame = catalog.tabelize();
```

Catalog operations include stream selection, mutation, merge, transform, text preprocessing, and export.

```
auto cleaned = catalog.apply("balance", [](const pluss::kindle::Stream& s) {
  return s.fillna(0.0);
});

auto table = cleaned.to_table();
```

Use Catalog when:

- stream metadata matters;
- you want named-stream transformations outside a DataFrame;
- your workflow moves between typed streams and table display;

- text-oriented catalog operations are useful.

Archive

An `Archive` is a stack of catalogs. It mirrors `Atlas` in the Kindle model.

```
auto archive = atlas.kindelize();
auto atlas2 = archive.tabelize();
```

Common archive operations:

```
archive.depth();
archive.empty();
archive.catalog(0);
archive.dloc(0);
archive.loc("scenario_a");
archive.head(5);
archive.tail(5);
archive.slice_depth(0, 3);
archive.sort_depth();
```

Map catalogs:

```
auto cleaned = archive.map_catalogs([](const pluss::kindle::Catalog& catalog) {
  return catalog.apply_all([](const pluss::kindle::Stream& stream) {
    return stream.dropna();
  });
});
```

Kindle and DataFrame policy

Conversion functions accept policy objects:

```
auto catalog = df.kindelize(policy);
auto frame = catalog.tabelize(policy);
```

Use policies when you need to control how cases, metadata, and stream types map between the table and Kindle models.

See *Kindle*, *Stream API Reference*, *Catalog API Reference*, and *Archive API Reference*.

1.3.12 Part XII: Bundle: Slot, Registry, and Aisle

Bundle is PlussMic's heterogeneous record model. It is useful when values are not naturally represented as one typed `Thread`.

Slot

A `Slot` is a vector of values. Values can represent multiple scalar/value families depending on the Bundle API surface.

Common Slot operations:

```
slot.defined();
slot.size();
slot.empty();
slot.values();
slot.at(0);
```

(continues on next page)

(continued from previous page)

```
slot.head(10);
slot.tail(10);
slot.slice(10, 20);
slot.take(indices);
slot.filter(predicate);
slot.transform(fn);
```

Slot mutation style follows the Pluss convention:

```
auto next = slot.push(value);
slot.push_(value);
```

Use non-mutating operations in pipelines and mutating operations when updating owned state.

Registry

A `Registry` is a named collection of slots.

```
auto registry = pluss::bundle::Registry::from_kindle(catalog);
auto catalog2 = registry.to_kindle();
```

Load and save registry data:

```
auto reg = pluss::bundle::Registry::load_csv("data.csv");
reg.save_json("registry.json");
```

Inspect:

```
registry.nslots();
registry.nfields();
registry.keys();
registry.has("field");
registry.slot("field");
registry.get("field");
registry.slot_sizes();
registry.type_profiles();
```

Select and reshape:

```
auto first = registry.head(10);
auto rows = registry.slice(10, 20);
auto one_row = registry.cloc(0);
auto selected = registry.select({"name", "score"});
auto dropped = registry.drop("temporary");
auto renamed = registry.rename("old", "new");
```

Transform:

```
auto changed = registry.apply("score", [](const pluss::bundle::Slot& slot) {
  return slot;
});
```

(continues on next page)

(continued from previous page)

```
auto all = registry.apply_all([](const pluss::bundle::Slot& slot) {
  return slot;
});
```

Update fields:

```
auto with_slot = registry.set_slot("new_field", slot);
auto pushed = registry.push("events", value);
```

Combine:

```
auto merged = registry.combine(other, "_other");
```

Stream compatibility

Some Registry operations require slots that can map cleanly to streams.

```
if (registry.is_stream_compatible()) {
  auto catalog = registry.to_kindle();
}
```

Use `ensure_stream_compatible` when failure should be explicit:

```
registry.ensure_stream_compatible("export");
```

Aisle

`Aisle` is the higher collection-oriented bundle type. Use it when you need to organize registries/slots into a broader bundle workflow. See the *Aisle API Reference* for exact constructors and operations.

Bundle conversion strategy

DataFrame path:

```
auto catalog = df.kindelize();
auto registry = pluss::bundle::Registry::from_kindle(catalog);
```

Back to table:

```
auto catalog = registry.to_kindle();
auto df = catalog.tabelize();
```

Use Bundle for heterogeneous data, but return to DataFrame when you need rectangular analysis, group-by, plotting, or file export.

1.3.13 Part XIII: Recipes

This part gives task-oriented recipes. Copy the pattern, then adapt feature names and options.

Load, clean, summarize, save

```
using pluss::table::DataFrame;

auto raw = DataFrame::load_csv("input.csv", {
    {"ncases", 10000},
});

auto clean = raw
    .fillna(0.0)
    .dropna_all({"id"})
    .sort_values({"id"});

clean.describe();
clean.save_csv("outputs/clean.csv");
```

Load selected features

```
pluss::loader::ReadCsvOptions options;
options.usefeats = {"id", "date", "price", "volume"};
options.parse_dates = {"date"};
options.case_col = "id";

auto df = DataFrame::load_csv("trades.csv", options);
```

Feature engineering pipeline

```
auto features = df
    .fillna(0.0, {"price", "volume"})
    .assign({{
        "notional",
        [](const DataFrame& frame) {
          return frame.features("price") * frame.features("volume");
        },
    }})
    .assign({{
        "log_volume",
        [](const DataFrame& frame) {
          return frame.features("volume").log();
        },
    }});
```

Grouped report

```
df.group_by(
    {"sector"},
    {
        {"n", "price", DataFrame::GroupOp::Count},
        {"avg_price", "price", DataFrame::GroupOp::Mean},
        {"max_volume", "volume", DataFrame::GroupOp::Max},
    },
    true,
    true);
```

Top-N report

```
auto top = df
    .sort_values({"revenue"}, false)
    .head(25, false);

top.save_csv("outputs/top_revenue.csv");
```

Missing-value audit

```
std::cout << "null rows: " << df.count_null_rows() << '\n';
df.isna().head(20);
df.tally({"category"}, 20, false, true);
```

Text cleanup

```
auto cleaned = df.transform({
    {"name", [](const auto& t) { return t.lower().replace(" inc.", ""); }},
    {"city", [](const auto& t) { return t.lower(); }},
});
```

Time-series features

```
auto sorted = df.sort_values({"date"});
auto engineered = sorted.assign({
    {"return_1", [](const DataFrame& frame) {
      return frame.features("close").ts().returns(1);
    }},
    {"close_20_mean", [](const DataFrame& frame) {
      return frame.features("close").ts().rolling_mean(20, 10);
    }},
});
```

Daily resample

```
df.resample("timestamp", {"price", "volume"}, "D", "mean", 1, true);
```

Excel workbook ingestion

```
auto workbook = DataFrame::load_excel("report.xlsx", nullptr, true);

for (const auto& name : workbook.sheet_names().values()) {
  auto sheet = workbook.sheets(name);
  sheet.info();
}
```

Multi-scenario Atlas

```
auto base = DataFrame::load_csv("base.csv");
auto stress = DataFrame::load_csv("stress.csv");
```

(continues on next page)

(continued from previous page)

```
auto optimistic = DataFrame::load_csv("optimistic.csv");

pluss::table::Thread labels =
    pluss::make_note({"base", "stress", "optimistic"});
pluss::table::Atlas scenarios({base, stress, optimistic}, labels);

auto cleaned = scenarios.map_frames([](const DataFrame& frame) {
  return frame.fillna(0.0);
});
```

Round-trip through Kindle and Bundle

```
auto catalog = df.kindelize();
auto registry = pluss::bundle::Registry::from_kindle(catalog);

registry.type_profiles();

auto back = registry.to_kindle().tabelize();
```

Save a report bundle

```
auto clean = df.fillna(0.0);
auto summary = clean.describe();
auto grouped = clean.group_by(
    {"group"},
    {{"mean_value", "value", DataFrame::GroupOp::Mean}},
    true,
    false);

clean.save_csv("outputs/clean.csv");
summary.save_csv("outputs/summary.csv");
grouped.save_csv("outputs/grouped.csv");
```

1.3.14 Part XIV: Troubleshooting and Production Notes

This part covers common failure modes and how to diagnose them.

Loader cannot open a file

Check:

- the path is absolute or relative to the process working directory;
- the file exists;
- the process has read permissions;
- URL support is enabled if loading `http://` or `https://`;
- optional dependencies are present for Excel or Parquet.

Prefer logging the resolved path before loading:

```
std::filesystem::path path = "data/input.csv";
std::cout << "loading " << std::filesystem::absolute(path) << '\n';
auto df = pluss::table::DataFrame::load_csv(path.string());
```

Separator inferred incorrectly

Set the separator explicitly:

```
pluss::loader::ReadCsvOptions options;
options.sep = ';';
auto df = pluss::table::DataFrame::load_csv("bank-full.csv", options);
```

Header parsed incorrectly

Use header options:

```
pluss::loader::ReadCsvOptions options;
options.header = true;
options.header_row = 1;
options.names = {"a", "b", "c"};
```

For Excel, use:

```
pluss::loader::ReadExcelOptions options;
options.header_row = 2;
options.header_rows = 2;
options.header_joiner = "_";
```

Wrong column types

First inspect:

```
df.info();
df.describe();
```

Then cast:

```
auto typed = df.astype({{"amount", pluss::codem::ScalarType::Float64}});
auto text = typed.cast_note({"category"});
auto dates = typed.cast_timeline({"date"});
```

Verified output does not match what you expected

The manual examples are compiled and run against the current SDK. If an output looks surprising, do not assume the manual should normalize it. Check the source path, add a focused regression test, and fix source-level defects before teaching the behavior as correct.

For missing values, the expected `fillna` contract is:

- existing non-null values remain unchanged;
- null values in selected features are replaced;
- the validity mask is cleared when all null values in the filled thread are resolved.

When writing production code around missing values, keep a small regression test for the exact operation and version you depend on.

Missing values break math

Audit, then fill or drop:

```
df.isna().head(20);
auto clean = df.fillna(0.0, {"amount"});
auto complete = clean.dropna();
```

Query returns no rows

Debug in smaller pieces:

```
df.query("age >= 45").head();
df.query("job != 'unknown'").head();
df.query("age >= 45 and job != 'unknown'").head();
```

Check feature names exactly:

```
for (const auto& name : df.feature_names()) {
  std::cout << "[" << name << "]\n";
}
```

Group-by output is unexpected

Check:

- grouping features have the expected type;
- null group keys are handled by `dropna_keys`;
- the selected value feature is numeric for numeric operations;
- `GroupOp::Count` vs `GroupOp::Nunique` semantics match your intent.

```
df.tally("group", 20, false, true);
df.group_by({"group"}, {{"n", "value", DataFrame::GroupOp::Count}}, false, true);
```

Plot is empty or too dense

Check:

- selected features contain finite values;
- x/y features have the same length;
- missing values are handled;
- high-cardinality groups are aggregated first;
- max point options are set for dense plot families.

```
auto plot_df = df
    .features({"date", "close"})
    .dropna();

plot_df.info();
plot_df.plot();
```

Excel or Parquet API fails at runtime

These formats depend on optional build dependencies. Rebuild the SDK with the needed dependency available, then rebuild the consumer project.

Popup display does not open

PlussMic falls back to terminal display when popup display support is not available. Use terminal output as the reliable path in headless environments:

```
df.show(std::cout);
```

Large CSV is slow

Try:

```
pluss::loader::ReadCsvOptions options;
options.engine = pluss::loader::CsvEngine::Parallel;
options.parallel_threads = 8;
options.usefeats = {"only", "needed", "columns"};
options.ncases = 100000; // while developing
options.profile = true;
```

Also avoid loading columns you will drop immediately.

Production checklist

- Use typed option structs for non-trivial I/O.
- Validate schema with `info()` after loading.
- Keep feature names stable and documented.
- Decide case-axis semantics early.
- Prefer non-mutating transforms in reusable code.
- Save cleaned and intermediate outputs in deterministic locations.
- Keep plotting outputs separate from data outputs.
- Use API reference pages for exact overloads.

1.3.15 Part XV: API Map

This part maps tasks to API areas. It is not a replacement for the API reference; it tells you where to look.

DataFrame API areas

Task	Methods
Load	load, load_csv, load_txt, load_excel, load_json, load_parquet
Save	save_csv, save_txt, save_excel, save_json, save_parquet, save_as
Inspect	defined, ncases, nfeatures, feature_names, info, describe, to_string
Select features	features, select, select_like, select_regex, series_features, note_features, timeline_features
Manage features	insert, drop, reorder, rename, rename_features, set_axis, add_prefix, add_suffix
Manage cases	cases, cases_series, set_cases, reset_cases, align_cases, reindex_cases
Row selection	head, tail, view, cloc, slice, take, sample
Filter	where, filter, query
Derive	assign, set_features, set
Transform	transform, cast, astype, cast_series, cast_note, cast_timeline, cast_timeindex
Sort/rank	sort_values, sort_cases, top, bottom
Missing data	isna, isnull, notna, notnull, fillna, dropna, dropna_all, ffill, bfill
Arithmetic	add, sub, mul, div, truediv, floordiv, mod, pow, reverse variants
Comparison	eq, ne, lt, le, gt, ge, compare
Reductions	sum, mean, median, min, max, std, var, count, corr, count_distinct
Grouping	aggregate, group_by, count_by, mean_by, pivot, pivot_table
Time	ts, resample, rolling, window, tz_convert, tz_localize, to_utc, floor_time, ceil_time, round_time, shift_time
Combine	join, update, combine, stitch_on, conform_to, reindex_like
Plot	plot, plot3d
SQL	sql, prepare_sql
Convert	kindelize, tabelize, unsqueeze

See *DataFrame API Reference*.

Thread API areas

Task	Methods
Inspect	defined, size, kind, dtype_name, info, to_string
Missing	isnull, notnull, isnan, isfinite, fillna, dropna, ffill, bfill
Row operations	head, tail, view, shift, slice, take, filter, sort_values, distinct
Numeric	abs, clip, log, exp, reductions
Text	lower, replace, contains
Time	year, month, day, hour, to_note_weekday, ts
Compare	compare, btw, compare_rows
Cast	cast, astype
Payload	as_series, as_note, as_timeline, as_timeindex
Display	show, with_display_labels, with_display_name

See *Thread API Reference*.

Loader API areas

Task	APIs
CSV options	`ReadCsvOptions`, `WriteCsvOptions`
Excel options	`ReadExcelOptions`, `WriteExcelOptions`
Raw ingest	`read_csv_raw_batch`, `read_excel_raw_batch`, `read_json_raw_text`
Neutral ingest	`read_csv_batch`, `read_excel_batch`
DataFrame load	`load_csv`, `load_txt`, `load_excel`, `load_json`, `load_parquet`, `load`
Save	`save_csv`, `save_txt`, `save_excel`, `save_json`, `save_parquet`, `save_as`
Workbook	`LoadResult`, `Workbook`, `SheetNames`

Atlas API areas

Task	Methods
Inspect	`defined`, `empty`, `depth`, `ncases`, `nfeats`, `column_names`
Access	`frame`, `dloc`, `loc`, `frames`, `axis0_labels`
Slice depth	`head`, `tail`, `slice_frames`, `sort_depth`
Align depth	`relabel_depth`, `align_depth`
Transform	`map_frames`, `aggregate_over_frames`
Convert	`kindelize`, `tabelize`, `squeeze`
Plot	`plot`

See *Atlas API Reference*.

Kindle API areas

Type	Reference
`Stream`	*Stream API Reference*
`Catalog`	*Catalog API Reference*
`Archive`	*Archive API Reference*

Bundle API areas

Type	Reference
`Slot`	*Slot API Reference*
`Registry`	*Registry API Reference*
`Aisle`	*Aisle API Reference*

Example source map

Example	What it teaches
`dataframe_initial_demo.cpp`	loading, display, `head`, `tail`, `info`, `describe`, Atlas basics
`dataframe_versatile_methods_demo.cpp`	query, transform, aggregate, arithmetic, missing data, casts, rolling
`dataframe_plot_demo.cpp`	DataFrame plot families
`time_series_accessor_demo.cpp`	time-series accessors
`atlas_plot_demo.cpp`	Atlas plotting
`loader_saver_parity_demo.cpp`	load/save round trips
`kindle_model_demo.cpp`	Kindle model
`bundle_model_demo.cpp`	Bundle model

CHAPTER

TWO

QUICKSTART

2.1 Include Paths

```
#include <pluss/pluss.h>
// or, when you only want PlussMic:
#include <pluss/mic.h>
```

2.2 Load A Table Fast

```
auto df = pluss::table::DataFrame::load_csv(
    "path/to/data.csv",
    {{"nrows", 16}});

std::cout << df << "\n";
df.info();
df.describe();
```

2.3 Start With A Small In-Memory Frame

```
auto sales = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"revenue", pluss::table::Thread(pluss::make_series({120.0, 95.0, 140.0}))},
    {"opened_at", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:00:00Z",
        "2024-01-02T09:00:00Z",
        "2024-01-03T09:00:00Z",
    }))},
});

sales.head();
```

2.4 Terminal vs Popup Preview

Table preview methods such as `head`, `tail`, `view`, `describe`, `aggregate`, and `group_by` return a lightweight `DataFrameView`.

```
df.head();          // terminal preview
df.head().show();   // popup if GTK/WebKit is available, else terminal preview
df.describe().show();

auto preview = df.head(10);  // preview is a DataFrameView
preview.show();              // explicit popup/terminal fallback
```

If you need a plain `DataFrame`, disable the deferred preview:

```
pluss::table::DataFrame materialized = df.head(10, false);
```

For the full popup viewer guide, OS support matrix, dependency setup, and `title(...)` naming rules, see `dataframe_display.md`.

2.5 Typical DataFrame Workflow

```
auto accidents = pluss::table::DataFrame::load_csv(
    "path/to/data.csv",
    {{"nrows", 64}})
    .fillna(0.0, {"Lifecycle Cost", "Cost Variance (%)"})
    .query("isnotnull(Agency Name)")
    .sort_values({"Lifecycle Cost"}, false)
    .head(10, false);

accidents.show();
```

2.6 Convert Between Models

```
auto df = pluss::table::DataFrame::load_csv("path/to/data.csv", {{"nrows", 8}});
auto catalog = df.kindelize();
auto roundtrip_df = catalog.tabelize();

auto registry = pluss::bundle::Registry::from_kindle(catalog);
auto back_catalog = registry.to_kindle();
```

2.7 Save Results

```
auto out = pluss::table::DataFrame::load_json("path/to/data.json", {{"nrows", 12}});

out.save_csv("path/to/output.csv");
out.save_excel("path/to/output.xlsx");
out.save_json("path/to/output.json");
```

CHAPTER

THREE

DEMO CATALOG

3.1 Table Examples

- `PlussMic/examples/table/dataframe_initial_demo.cpp`: loading CSV files, viewing rows, info, describe, and a simple `Atlas`
- `PlussMic/examples/table/dataframe_describe_demo.cpp`: summary statistics plus CSV loader benchmarking
- `PlussMic/examples/table/dataframe_view_demo.cpp`: row windows and quick inspection patterns
- `PlussMic/examples/table/dataframe_thread_values_demo.cpp`: working with typed `Thread` columns
- `PlussMic/examples/table/dataframe_versatile_methods_demo.cpp`: selection, transforms, grouping, joins, and reshaping verbs
- `PlussMic/examples/table/loader_saver_parity_demo.cpp`: CSV, TXT, JSON, Excel, and Parquet loader/saver parity
- `PlussMic/examples/table/dataframe_plot_demo.cpp`: plotting accessors and integration with PlussPl
- `PlussMic/examples/table/display_policy_demo.cpp`: terminal display policy and compact rendering
- `PlussMic/examples/table/eda_us_accidents.cpp`: end-to-end exploratory analysis workflow on the accidents dataset
- `PlussMic/examples/table/us_accidents_eda_verbs_demo.cpp`: richer EDA verbs and derived-feature workflow

3.2 Kindle Examples

- `PlussMic/examples/kindle/kindle_model_demo.cpp`: `Stream`, `Catalog`, and `Archive` workflows

3.3 Bundle Examples

- `PlussMic/examples/bundle/bundle_model_demo.cpp`: `Slot`, `Registry`, and `Aisle` workflows

CHAPTER

FOUR

DATAFRAME GUIDE

4.1 What DataFrame Is Good At

`pluss::table::DataFrame` is the main table-analysis surface in PlussMic. Use it when you want:

- named features with mixed typed columns
- a case axis that can be numeric, note-based, timeline-based, or time-index based
- table verbs such as `query`, `assign`, `group_by`, `pivot_table`, `join`, `fillna`, and `describe`
- import/export across CSV, TXT, JSON, Excel, and Parquet

4.2 Construction And Loading

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"opened", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T08:00:00Z",
        "2024-01-02T08:00:00Z",
        "2024-01-03T08:00:00Z",
    }))},
});

auto csv_df = pluss::table::DataFrame::load_csv(
    "path/to/data.csv",
    {{"nrows", 32}});

auto excel_df = pluss::table::DataFrame::load_excel(
    "path/to/data.xlsx",
    {{"sheetindex", 0}, {"header", true}});

auto json_df = pluss::table::DataFrame::load_json(
    "path/to/data.json",
    {{"nrows", 24}});
```

4.3 Inspect Rows, Features, And Cases

```
df.head();
df.tail();
```

(continues on next page)

(continued from previous page)

```
df.info();
df.describe();

auto city = df.features("city");
auto sales = df.features("sales");
auto case_axis = df.cases();

auto first_row = df.iloc(0);
auto first_value = df.at(0, "city");
```

Notes:

- `features("name")` returns a typed `Thread`
- `cases()` is the row axis, not just a display index
- `describe()` is the fastest way to check whether numeric casts and missing values look right

4.3.1 Display Views

The table-style preview verbs return a `DataFrameView`, not a plain `DataFrame`. That is what makes this pattern work:

```
df.head();          // terminal preview
df.head().show();   // popup if GTK/WebKit exists, else terminal preview
df.view(10, 20).show();
df.describe().show();
```

If you keep the result, the stored type is still the preview wrapper:

```
auto preview = df.head(5);
preview.show();
std::cout << preview << "\n";
```

If you explicitly want a materialized `DataFrame`, use `render=false`:

```
pluss::table::DataFrame head_df = df.head(5, false);
```

For OS support, popup backends, `DisplayOptions`, and `title(...)` / `title_(...)`, see `dataframe_display.md`.

4.4 Cast And Derive Features

```
auto work = csv_df
    .cast_timeline({"Business Case ID"})
    .astype({{"Lifecycle Cost", pluss::codem::ScalarType::Float64}})
    .assign({
        pluss::table::DataFrame::DeriveSpec{
            "Cost_plus_buffer",
            [](const pluss::table::DataFrame& f) {
              return pluss::table::Thread(f.features("Lifecycle Cost").as_series().
↪add(500.0));
            },
        },
    });
```

4.5 Filter, Query, And Sort

```
auto positive = df.filter(df.features("sales").as_series().gt(95.0));

auto queried = df.query("sales > 95");

auto sorted = df.sort_values({"sales"}, false);

auto top3 = df.top("sales", 3);
```

Notes:

- use `filter(bool_mask)` when you already have a boolean `Series`
- use `query("...")` when the filter is easier to read as a table expression
- `top` and `bottom` are good for ranking tasks without writing extra sort/slice steps

4.6 Grouping, Pivoting, And Summaries

```
auto grouped = df.group_by(
    {"city"},
    {{"avg_sales", "sales", pluss::table::DataFrame::GroupOp::Mean}},
    true,
    false);

auto pivoted = df.pivot_table(
    "city",
    "city",
    "sales",
    pluss::table::DataFrame::GroupOp::Mean,
    true,
    false);

auto stats = df.aggregate({
    {"sales_mean", "sales", pluss::table::DataFrame::SummaryOp::Mean},
    {"sales_max", "sales", pluss::table::DataFrame::SummaryOp::Max},
}, false);
```

4.7 Missing Data And Rolling Windows

```
auto cleaned = df.fillna(0.0, {"sales"});
auto dropped = df.dropna();
auto carried = df.ffill();

auto rolling = df.window({
    {"sales_roll3", "sales", pluss::table::ThreadWindowOp::RollingMean, 3, 1},
});
```

4.8 Join And Combine

```
auto left = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0}))},
});

auto right = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema"}))},
    {"manager", pluss::table::Thread(pluss::make_note({"Ama", "Kojo"}))},
});

auto joined = left.stitch_on({"city"}, right, {"city"}, "_r");
auto updated = left.update(right, true);
```

4.9 Time-Aware Table Verbs

```
auto schedule = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"value", pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}))},
});

auto hourly = schedule.resample("ts", {"value"}, "1H", "mean", 1, false);
auto floored = schedule.floor_time({"ts"}, "hour");
auto utc = schedule.to_utc({"ts"});
```

4.10 SQL And IO

```
auto result = df.sql().query("SELECT city, sales FROM self ORDER BY sales DESC");

result.save_csv("path/to/output.csv");
result.save_excel("path/to/output.xlsx");
result.save_json("path/to/output.json");
```

CHAPTER

FIVE

TIME-SERIES ACCESSOR GUIDE

This page documents the `.ts()` surface for `DataFrame` and `Thread`.

5.1 API Signature

- `DataFrame::ts(feature: string_view) -> DataFrameTsAccessor`
- `DataFrame::ts(features: {string_view...}) -> DataFrameTsAccessor`
- `Thread::ts() -> TimeSeriesAccessor`

5.2 Notes

Use `DataFrame::ts(...)` when you want transforms returned as a `DataFrame` (single or multi-feature).
Use `Thread::ts()` when you want transforms returned as a `Thread` for one column.

Inputs:

- Numeric `Series`-kind features.
- For time-axis methods (`resample`, `asfreq`, `align_to`, `reindex_time`), a timeline/timeindex column is required.

Output:

- `DataFrameTsAccessor` methods return `DataFrame`.
- `TimeSeriesAccessor` methods return `Thread` (or `{labels, values}` for `resample`).

5.3 DataFrame Surface

5.3.1 API Signature

- `returns(periods: int64 = 1) -> DataFrame`
- `log_returns(periods: int64 = 1) -> DataFrame`
- `diff(periods: int64 = 1) -> DataFrame`
- `lag(periods: int64 = 1) -> DataFrame`
- `lead(periods: int64 = 1) -> DataFrame`
- `rolling_mean(window: int64, min_periods: int64 = 1) -> DataFrame`
- `rolling_std(window: int64, min_periods: int64 = 1) -> DataFrame`
- `rolling_sum(window: int64, min_periods: int64 = 1) -> DataFrame`

- rolling_var(window: int64, min_periods: int64 = 1) -> DataFrame
- rolling_min(window: int64, min_periods: int64 = 1) -> DataFrame
- rolling_max(window: int64, min_periods: int64 = 1) -> DataFrame
- ema(alpha: double) -> DataFrame
- ewm_mean(alpha: double) -> DataFrame
- ewm_std(alpha: double, epsilon: double = 1e-12) -> DataFrame
- expanding_mean(min_periods: int64 = 1) -> DataFrame
- expanding_std(min_periods: int64 = 1, epsilon: double = 1e-12) -> DataFrame
- zscore_rolling(window: int64, min_periods: int64 = 1, epsilon: double = 1e-12) -> DataFrame
- fill_gaps(mode: GapFillMode = Forward, constant: optional<double> = nullopt) -> DataFrame
- resample(time_feature: string_view, freq: string_view, op: ResampleOp = Mean, step: int64 = 1) -> DataFrame
- asfreq(time_feature: string_view, freq: string_view, step: int64 = 1, fill_mode: GapFillMode = Forward, constant: optional<double> = nullopt, op: ResampleOp = Last) -> DataFrame
- insert_gaps(time_feature: string_view, freq: string_view, step: int64 = 1) -> DataFrame
- align_to(time_feature: string_view, target_axis: Thread, mode: TsAlignMode = Exact) -> DataFrame
- reindex_time(time_feature: string_view, target_axis: Thread, mode: TsAlignMode = Exact, fill: TsReindexFill = None, constant: optional<double> = nullopt) -> DataFrame
- calendar(time_feature: string_view) -> DataFrame
- lag_block(lags: {int64...}) -> DataFrame
- rolling_block(windows: {int64...}, options: TsRollingBlockOptions = {}) -> DataFrame

5.3.2 Example

```
#include <pluss/mic.h>

using pluss::table::DataFrame;

DataFrame df = DataFrame::load_csv("prices.csv")
                 .cast_timeline({"date"});

auto lr = df.ts("Adj Close").log_returns();
auto lag = df.ts({"Adj Close", "Volume"}).lag_block({1, 2, 5});
auto roll = df.ts({"Adj Close"}).rolling_mean(20, 5);
auto daily = df.ts({"Adj Close", "Volume"}).resample(
    "date", "D", pluss::subsp::stats::ts::ResampleOp::Mean, 1);
```

5.3.3 Expected output

- `lr`, `lag`, `roll`, and `daily` are all `DataFrame` objects.
- Selected features are transformed and returned with aligned cases/time axis.

5.4 Thread Surface

5.4.1 API Signature

- `returns(periods: int64 = 1) -> Thread`
- `log_returns(periods: int64 = 1) -> Thread`
- `diff(periods: int64 = 1) -> Thread`
- `lag(periods: int64 = 1) -> Thread`
- `lead(periods: int64 = 1) -> Thread`
- `rolling_mean(window: int64, min_periods: int64 = 1) -> Thread`
- `rolling_std(window: int64, min_periods: int64 = 1) -> Thread`
- `rolling_sum(window: int64, min_periods: int64 = 1) -> Thread`
- `rolling_var(window: int64, min_periods: int64 = 1) -> Thread`
- `rolling_min(window: int64, min_periods: int64 = 1) -> Thread`
- `rolling_max(window: int64, min_periods: int64 = 1) -> Thread`
- `ema(alpha: double) -> Thread`
- `ewm_mean(alpha: double) -> Thread`
- `ewm_std(alpha: double, epsilon: double = 1e-12) -> Thread`
- `expanding_mean(min_periods: int64 = 1) -> Thread`
- `expanding_std(min_periods: int64 = 1, epsilon: double = 1e-12) -> Thread`
- `zscore_rolling(window: int64, min_periods: int64 = 1, epsilon: double = 1e-12) -> Thread`
- `fill_gaps(mode: GapFillMode = Forward, constant: optional<double> = nullopt) -> Thread`
- `resample(axis: Thread, freq: string_view, op: ResampleOp = Mean, step: int64 = 1) -> TimeSeriesResample`

5.4.2 Example

```
#include <pluss/mic.h>

using pluss::table::DataFrame;

DataFrame df = DataFrame::load_csv("prices.csv").cast_timeline({"date"});

auto s = df.features("Adj Close");
auto lr = s.ts().log_returns();
auto z = s.ts().zscore_rolling(30, 10, 1e-12);
```

(continues on next page)

(continued from previous page)

```
auto rs = s.ts().resample(df.features("date"), "W",␣
↪pluss::subsp::stats::ts::ResampleOp::Last, 1);
```

5.4.3 Expected output

- `lr` and `z` are `Thread`.
- `rs.labels` is timeline/timeindex-like labels and `rs.values` is resampled values.

5.5 Common Pitfall

If you want a transformed **DataFrame**, use `df.ts(...)...`.
If you call `df.features(\"col\").ts()...`, you are on the **Thread** surface and the return type is `Thread`.

CHAPTER

SIX

DATAFRAME DISPLAY AND POPUP VIEWER

6.1 Overview

PlussMic supports two display paths for table-style inspection:

- terminal rendering
- popup rendering through `.show()`

The popup viewer is opt-in. Nothing opens automatically unless the user explicitly calls `.show()`.

6.2 Which Methods Support `.show()`

`.show()` is available on methods that return `pluss::table::DataFrameView`. Those are the table-rendering methods.

Current examples include:

- `head()`
- `tail()`
- `view()`
- `describe()`
- `transpose()`
- `aggregate()`
- `group_by()`
- `pivot()`
- `pivot_table()`
- `count_by()`
- `mean_by()`
- `tally()`
- `resample()`
- `sum()`
- `mean()`
- `min()`
- `max()`

- std()
- var()
- count()

Examples:

```
df.head();                // terminal preview
df.head().show();         // popup if available, else terminal fallback

df.describe();            // terminal preview
df.describe().show();     // popup if available, else terminal fallback

df.group_by({"city"}, {{"avg_sales", "sales", pluss::table::DataFrame::GroupOp::Mean}})
  .show();
```

If a method returns a plain DataFrame instead of DataFrameView, then it does not participate in this deferred popup-preview flow.

6.3 How The Display Policy Works

The behavior is:

```
df.head();          // prints in the terminal
df.head().show();   // suppresses terminal auto-preview and requests popup instead
```

This works because head(), describe(), and the other preview verbs return a lightweight DataFrameView. That wrapper delays the terminal render until the expression ends. If .show() is called first, the pending terminal render is consumed and the popup path is used instead.

6.4 What .show() Takes

The primary overload is:

```
view.show();
```

There is also an overload that accepts DisplayOptions:

```
pluss::table::display::DisplayOptions opts;
view.show(opts);
```

There is also a stream-oriented overload:

```
view.show(std::cout, opts);
```

In practice, the appropriate argument for .show() is DisplayOptions.

6.5 Useful DisplayOptions

The most relevant display options for popup/table preview are:

- backend
- viewer_width

- `viewer_height`
- `window_title`
- `html_sortable`
- `html_search`
- `html_copy_button`

Example:

```
pluss::table::display::DisplayOptions opts;
opts.viewer_width = 1440;
opts.viewer_height = 900;
opts.html_search = true;
opts.html_sortable = true;

df.describe().show(opts);
```

Notes:

- `backend = DisplayBackend::Terminal` forces terminal rendering
- `backend = DisplayBackend::NativeWebView` requests the GUI path when a native backend exists
- `window_title` is available, but for most users the better pattern is to name the frame with `title(...)` or `title_(...)`

6.6 DataFrame Titles

Frames can carry display metadata:

```
auto prices = df.title("prices");
prices.head().show();
prices.describe().show();
```

Or mutate in place:

```
df.title_("prices");
df.head().show();
```

Behavior:

- untitled frame: `DataFrame - head`
- untitled frame: `DataFrame - describe`
- titled frame: `prices - head`
- titled frame: `prices - describe`

The title is replaceable metadata.

```
df.title_("some title");
df.title_("some other title");
```

Immutable behavior:

```
auto prices = df.title("prices");   // df unchanged
auto other = prices.title("derived"); // prices unchanged
```

6.7 OS Support

Current popup support is:

- macOS: supported
- Linux: supported
- Windows: supported when WebView2 support is compiled in and the WebView2 Runtime is installed

If the popup backend is unavailable, `.show()` falls back to the normal terminal rendering path.

6.8 macOS Backend

On macOS, the popup viewer uses native Apple frameworks:

- `Cocoa`
- `WebKit`
- `QuartzCore`
- `UniformTypeIdentifiers`

That means the current macOS popup path does not depend on Conda GUI packages. It uses OS-provided frameworks.

Practical consequence:

- a Conda environment is fine for the compiler/toolchain
- the popup viewer itself is not coming from Conda on macOS

6.9 Linux Backend

On Linux, the popup viewer uses:

- `gtk+-3.0`
- `webkit2gtk-4.1`
- fallback to `webkit2gtk-4.0`

Those are discovered through `pkg-config`.

Typical system packages:

Ubuntu:

```
sudo apt install pkg-config libgtk-3-dev libwebkit2gtk-4.1-dev
```

Fedora:

```
sudo dnf install pkgconf-pkg-config gtk3-devel webkit2gtk4.1-devel
```

Linux Conda environments can often provide the GTK/WebKit pieces, but the most reliable path is still the system packages plus `pkg-config`.

6.10 Windows Status

On Windows, the popup viewer uses:

- native Win32 windowing
- Microsoft Edge WebView2 for the embedded HTML table

That means the host window is native Windows UI, and the table itself is still rendered from the same HTML/CSS/JavaScript payload used on the other platforms.

There are two separate Windows requirements:

- build-time requirement: the WebView2 SDK headers and loader library must be available when compiling PlussMic from source
- runtime requirement: the Microsoft Edge WebView2 Runtime must be installed on the target machine

If either one is missing, `.show()` falls back to terminal rendering.

6.11 Windows Build Setup

The Windows viewer backend is enabled when CMake can find the WebView2 SDK.

PlussMic looks for:

- cache variable: `PLUSSMIC_WEBVIEW2_SDK_DIR`
- environment variable: `WEBVIEW2_SDK_DIR`
- common NuGet cache location under the user profile

Typical configure command:

```
cmake -S . -B build ^
  -DPLUSSMIC_WEBVIEW2_SDK_DIR="C:/path/to/Microsoft.Web.WebView2.<version>"
```

If you want to disable Windows popup support explicitly:

```
cmake -S . -B build -DPLUSSMIC_ENABLE_WINDOWS_WEBVIEW2=OFF
```

What the SDK directory should contain:

- `build/native/include/WebView2.h`
- `build/native/x64/WebView2LoaderStatic.lib` or the equivalent `x86` / `arm64` loader library for the target architecture

6.12 Windows Runtime Setup

Even if PlussMic builds with WebView2 support, the target machine still needs the WebView2 Runtime.

That runtime is often already present on modern Windows systems, but it should still be treated as a requirement for popup viewing.

If the runtime is missing:

- `.show()` does not crash
- it simply falls back to terminal rendering

6.13 Conda On Windows

Conda is not the source of the Windows popup backend.

On Windows:

- your Conda environment can still provide the compiler toolchain and normal build environment
- the popup viewer itself comes from Windows + WebView2
- users should not expect a Conda package to supply the full native popup stack

So the practical rule is:

- Conda can help with building
- WebView2 SDK and Runtime are the Windows viewer requirements

6.14 Are The Popup Windows Native

Partly.

The host window is native to the platform backend:

- macOS: native Cocoa window
- Linux: GTK window
- Windows: native Win32 window with embedded WebView2

The table content itself is rendered as HTML/CSS/JavaScript inside the embedded web view.

So the actual behavior is:

- window chrome feels native to the OS/toolkit
- table content stays visually consistent across platforms because it is HTML-driven

6.15 Conda Notes

Conda support differs by platform.

- macOS: no GTK/WebKitGTK package is needed for the current implementation
- Linux: Conda may provide the needed GTK/WebKit packages, but system packages are usually simpler and more reliable
- Windows: Conda does not provide the native viewer stack; use WebView2 SDK + Runtime

6.16 Recommended User Guidance

The simplest explanation for users is:

```
df.head();          // terminal
df.head().show();   // popup if supported, else terminal
```

If they want better popup titles:

```
auto prices = df.title("prices");
prices.head().show();      // prices - head
prices.describe().show();  // prices - describe
```

6.17 Demo

See:

- `PlussMic/examples/table/display_policy_demo.cpp`

That demo shows:

- terminal preview
- popup preview with `.show()`
- explicit frame titles with `title("prices")`

6.18 External Setup References

Official references that are useful when setting up popup support:

- WebView2 overview and installation: https://learn.microsoft.com/en-us/microsoft-edge/webview2/
- WebView2 Runtime distribution: https://developer.microsoft.com/en-us/microsoft-edge/webview2/
- WebView2 SDK package: https://www.nuget.org/packages/Microsoft.Web.WebView2
- Conda GTK3 package: https://anaconda.org/conda-forge/gtk3
- Conda WebKitGTK package: https://anaconda.org/conda-forge/webkit2gtk4.1
- Ubuntu `libwebkit2gtk-4.1-dev`: https://packages.ubuntu.com/noble/libwebkit2gtk-4.1-dev
- Ubuntu `libgtk-3-dev`: https://packages.ubuntu.com/noble/libgtk-3-dev
- Fedora `webkit2gtk4.1-devel`: https://packages.fedoraproject.org/pkgs/webkitgtk/webkit2gtk4.1-devel
- Fedora `gtk3-devel`: https://packages.fedoraproject.org/pkgs/gtk3/gtk3-devel

CHAPTER

SEVEN

DATAFRAME API REFERENCE

CHAPTER

EIGHT

PRACTICAL DATAFRAME API REFERENCE

8.1 API Reference Style

This page documents the full public `DataFrame` surface with usage-first examples generated from the current table API.

8.1.1 add

Signature

```
DataFrame add(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame add(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.add(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.2 add

Signature

```
DataFrame add(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame add(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.add(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.3 add_prefix

Signature

```
DataFrame add_prefix(self, std::string_view prefix) const
```

Parameters

`prefix` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Prepends the supplied text to every feature name. Row values, row order, and case labels stay unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.add_prefix("feat_");
```

Output

```
df.head():
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi

out.head():
feat_id  feat_sales  feat_city
1        100.0       Accra
2        120.0       Tema
3         90.0       Kumasi
```

8.1.4 add_suffix

Signature

```
DataFrame add_suffix(self, std::string_view suffix) const
```

Parameters

`suffix` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Appends the supplied text to every feature name. Row values, row order, and case labels stay unchanged.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.add_suffix("_2024");
```

Output

```
df.head():
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi

out.head():
id_2024  sales_2024  city_2024
1        100.0       Accra
2        120.0       Tema
3         90.0       Kumasi
```

8.1.5 aggregate

Signature

```
DataFrameView aggregate(self, std::initializer_list<SummarySpec> specs, bool render =␣
↪true) const
```

Parameters

`specs` : `std::initializer_list<SummarySpec>` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Builds a compact summary frame from one or more aggregation specs, so you can inspect computed metrics instead of the full row set.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.aggregate({{"sales_mean", "sales",␣
↪pluss::table::DataFrame::SummaryOp::Mean}}, false);
```

Output

```
metric       value
sales_mean    115.0
```

8.1.6 align_cases

Signature

```
DataFrame align_cases(self, const Thread& new_cases) const
```

Parameters

`new_cases` : `const Thread&` Replacement case labels in the exact order you want the rows to follow.

Returns

`DataFrame` Value produced by this call.

Notes

Reorders rows so their case labels match the supplied `new_cases` order. Every requested case must already exist, and the output case axis is rewritten to that order.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df = df.with_cases(pluss::table::Thread(pluss::make_note({"cs0", "cs1", "cs2"})));
auto new_cases = pluss::table::Thread(pluss::make_note({"cs1", "cs0", "cs2"}));
auto out = df.align_cases(new_cases);
```

Output

```
df.cases():
["cs0", "cs1", "cs2"]
```

(continues on next page)

(continued from previous page)

```
out.cases():
["cs1", "cs0", "cs2"]

out.head():
id  sales  city
2   120.0  Tema
1   100.0  Accra
3    90.0  Kumasi
```

8.1.7 assign

Signature

```
DataFrame assign(self, std::initializer_list<DeriveSpec> specs) const
```

Parameters

specs : std::initializer_list<DeriveSpec> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Adds or replaces derived features computed from the current frame. Use this when the new column depends on one or more existing columns.

See also

- DataFrame assign(self, std::initializer_list<std::pair<std::string, Thread>> assignments) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.assign({{.feature = "profit", .expr = [](const pluss::table::DataFrame&␣
↪frame) { return pluss::table::Thread(frame.features("sales").as_series().sub(60)); }}}
↪);
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

8.1.8 assign

Signature

```
DataFrame assign(self, std::initializer_list<std::pair<std::string, Thread>>␣
↪assignments) const
```

Parameters

`assignments` : `std::initializer_list<std::pair<std::string, Thread>>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Adds or replaces derived features computed from the current frame. Use this when the new column depends on one or more existing columns.

See also

- `DataFrame assign(self, std::initializer_list<DeriveSpec> specs) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.assign({{"profit", pluss::table::Thread(pluss::make_series({40.0, 35.0, 30.
↪0}))}});
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

8.1.9 astype

Signature

```
DataFrame astype(self,  std::initializer_list<std::pair<std::string_view,␣
↪pluss::codem::ScalarType>> feature_dtypes) const
```

Parameters

`feature_dtypes` : `std::initializer_list<std::pair<std::string_view, pluss::codem::ScalarType>>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

See also

- `DataFrame astype(self, std::initializer_list<std::string_view> features, pluss::codem::ScalarType dtype) const`

Examples

```
auto df = pluss::table::DataFrame({{"sales", pluss::table::Thread(pluss::make_note({"100.
↪0", "120.0", "90.0"}))}});

auto out = df.astype({{"sales", pluss::codem::ScalarType::Float64}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.10 astype

Signature

```
DataFrame astype(self, std::initializer_list<std::string_view> features,␣
↪pluss::codem::ScalarType dtype) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`dtype` : `pluss::codem::ScalarType` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

See also

- DataFrame astype(self, std::initializer_list<std::pair<std::string_view, pluss::codem::ScalarType>> feature_dtypes) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.astype();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.11 at

Signature

```
std::string at(self, std::int64_t row, std::string_view feature_name) const
```

Parameters

row : std::int64_t See the signature for the supported argument form.

feature_name : std::string_view Name of the target feature to read, rewrite, sort by, or promote to cases.

Returns

std::string Rendered string result.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.at(0, "city");
```

Output

```
1
```

8.1.12 bfill

Signature

```
DataFrame bfill(self) const
```

Parameters

None

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.bfill();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.13 bottom

Signature

```
DataFrame bottom(self, std::string_view feature, std::int64_t n) const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

Returns

`DataFrame` Value produced by this call.

Notes

Returns the rows with the smallest values in the selected feature.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.bottom("sales", 2);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.14 cases

Signature

```
const Thread& cases(self) const
```

Parameters

None

Returns

`const Thread&` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- `DataFrame cases(self, std::int64_t row) const`
- `DataFrame cases(self, std::pair<std::int64_t, std::int64_t> row_span) const`
- `std::string cases(self, std::int64_t row, std::size_t feature_idx) const`
- `std::string cases(self, std::int64_t row, std::string_view feature_name) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cases(0);
```

8.1.15 cases

Signature

```
DataFrame cases(self, std::int64_t row) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- `const Thread& cases(self) const`
- `DataFrame cases(self, std::pair<std::int64_t, std::int64_t> row_span) const`
- `std::string cases(self, std::int64_t row, std::size_t feature_idx) const`
- `std::string cases(self, std::int64_t row, std::string_view feature_name) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cases(0);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.16 cases

Signature

```
DataFrame cases(self, std::pair<std::int64_t, std::int64_t> row_span) const
```

Parameters

row_span : std::pair<std::int64_t, std::int64_t> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- const Thread& cases(self) const
- DataFrame cases(self, std::int64_t row) const
- std::string cases(self, std::int64_t row, std::size_t feature_idx) const
- std::string cases(self, std::int64_t row, std::string_view feature_name) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cases({0, 2});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.17 cases

Signature

```
std::string cases(self, std::int64_t row, std::size_t feature_idx) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

`feature_idx` : `std::size_t` See the signature for the supported argument form.

Returns

`std::string` Rendered string result.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- `const Thread& cases(self) const`
- `DataFrame cases(self, std::int64_t row) const`
- `DataFrame cases(self, std::pair<std::int64_t, std::int64_t> row_span) const`
- `std::string cases(self, std::int64_t row, std::string_view feature_name) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cases(0, 0);
```

Output

```
1
```

8.1.18 cases

Signature

```
std::string cases(self, std::int64_t row, std::string_view feature_name) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

`feature_name` : `std::string_view` Name of the target feature to read, rewrite, sort by, or promote to cases.

Returns

`std::string` Rendered string result.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- `const Thread& cases(self) const`
- `DataFrame cases(self, std::int64_t row) const`
- `DataFrame cases(self, std::pair<std::int64_t, std::int64_t> row_span) const`
- `std::string cases(self, std::int64_t row, std::size_t feature_idx) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cases(0, "city");
```

Output

```
1
```

8.1.19 cases_series

Signature

```
pluss::Series cases_series(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

Returns the current case axis as a `Series`, which is useful when you need the row labels as data.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cases_series();
```

Output

```
[0, 1, 2]
```

8.1.20 cast

Signature

```
DataFrame cast(self,  std::initializer_list<std::pair<std::string_view,␣
↪pluss::codem::ScalarType>> feature_dtypes) const
```

Parameters

`feature_dtypes` : `std::initializer_list<std::pair<std::string_view, pluss::codem::ScalarType>>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

See also

- `DataFrame cast(self, std::initializer_list<CastSpec> specs) const`

Examples

```
auto df = pluss::table::DataFrame({{"sales", pluss::table::Thread(pluss::make_note({"100.
↪0", "120.0", "90.0"}))}});

auto out = df.astype({{"sales", pluss::codem::ScalarType::Float64}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.21 cast

Signature

```
DataFrame cast(self, std::initializer_list<CastSpec> specs) const
```

Parameters

`specs` : `std::initializer_list<CastSpec>` See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

See also

- DataFrame cast(self, std::initializer_list<std::pair<std::string_view, pluss::codem::ScalarType>> feature_dtypes) const

Examples

```
auto df = pluss::table::DataFrame({{"sales", pluss::table::Thread(pluss::make_note({"100.
↪0", "120.0", "90.0"}))}});

auto out = df.cast({{.feature = "sales", .target =␣
↪pluss::table::ThreadCastTarget::Series, .series_dtype =␣
↪pluss::codem::ScalarType::Float64}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.22 cast_note

Signature

```
DataFrame cast_note(self, std::initializer_list<std::string_view> features) const
```

Parameters

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto out = df.cast_note({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.23 cast_series

Signature

```
DataFrame cast_series(self, std::initializer_list<std::string_view> features,␣
↪std::optional<pluss::codem::ScalarType> dtype = std::nullopt) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`dtype` : `std::optional<pluss::codem::ScalarType>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto df = pluss::table::DataFrame({{"sales", pluss::table::Thread(pluss::make_note({"100.
↪0", "120.0", "90.0"}))}});

auto out = df.cast_series({"sales"}, pluss::codem::ScalarType::Float64);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.24 cast_timeindex

Signature

```
DataFrame cast_timeindex(self, std::initializer_list<std::string_view> features) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.cast_timeindex({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.25 `cast_timeline`

Signature

```
DataFrame cast_timeline(self, std::initializer_list<std::string_view> features) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto df = pluss::table::DataFrame({{"ts", pluss::table::Thread(pluss::make_note({"2024-
↪01-01T09:10:00Z", "2024-01-01T09:40:00Z"}))}});

auto out = df.cast_timeline({"ts"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.26 ceil_time

Signature

```
DataFrame ceil_time(self, std::initializer_list<std::string_view> features, std::string_
↪view unit) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`unit` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.ceil_time({"ts"}, "hour");
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

8.1.27 combine

Signature

```
DataFrame combine(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Combines this frame with another frame according to the supplied combine policy, such as feature-wise update or join-like alignment.

See also

- `DataFrame combine(self, const DataFrame& other, const CombineSpec& spec) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.combine(right);
```

Output

```
id  sales  cost
1   100.0  60.0
2   120.0  80.0
3    90.0  55.0
```

8.1.28 `combine`

Signature

```
DataFrame combine(self, const DataFrame& other, const CombineSpec& spec) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

`spec` : `const CombineSpec&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Combines this frame with another frame according to the supplied combine policy, such as feature-wise update or join-like alignment.

See also

- `DataFrame combine(self, const DataFrame& other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

pluss::table::DataFrame::CombineSpec spec;
spec.op = pluss::table::DataFrame::CombineOp::Update;
auto out = left.combine(right, spec);
```

Output

```
id  sales  cost
1   100.0  60.0
2   120.0  80.0
3    90.0  55.0
```

8.1.29 compare

Signature

```
DataFrame compare(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame compare(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.compare(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.30 compare

Signature

```
DataFrame compare(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame compare(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.compare(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.31 conform_to

Signature

```
DataFrame conform_to(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Reorders and aligns this frame so it is compatible with another frame for downstream joins or arithmetic.

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.conform_to(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.32 copy

Signature

```
DataFrame copy(self) const
```

Parameters

None

Returns

`DataFrame` Value produced by this call.

Notes

Returns a detached copy of the frame so later edits to the result do not affect the source object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.copy();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.33 `corr`

Signature

```
DataFrameView corr(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Applies `corr` to the current `DataFrame` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `DataFrameView corr(self, CorrMethod method, bool render = true) const`
- `DataFrameView corr(self, std::initializer_list<std::string_view> features, CorrMethod method, bool render = true) const`

- `DataFrameView corr(self, std::initializer_list<std::string_view> features, bool render = true) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.corr();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.34 corr

Signature

```
DataFrameView corr(self, CorrMethod method, bool render = true) const
```

Parameters

`method` : `CorrMethod` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Applies `corr` to the current `DataFrame` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `DataFrameView corr(self, bool render = true) const`
- `DataFrameView corr(self, std::initializer_list<std::string_view> features, CorrMethod method, bool render = true) const`
- `DataFrameView corr(self, std::initializer_list<std::string_view> features, bool render = true) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.corr();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.35 corr

Signature

```
DataFrameView corr(self, std::initializer_list<std::string_view> features, bool render =␣
↪true) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Applies `corr` to the current `DataFrame` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `DataFrameView corr(self, CorrMethod method, bool render = true) const`
- `DataFrameView corr(self, bool render = true) const`
- `DataFrameView corr(self, std::initializer_list<std::string_view> features, CorrMethod method, bool render = true) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto out = df.corr();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.36 corr

Signature

```
DataFrameView corr(self, std::initializer_list<std::string_view> features, CorrMethod␣
↪method, bool render = true) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`method` : `CorrMethod` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Applies `corr` to the current `DataFrame` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `DataFrameView corr(self, CorrMethod method, bool render = true) const`
- `DataFrameView corr(self, bool render = true) const`
- `DataFrameView corr(self, std::initializer_list<std::string_view> features, bool render = true) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.corr();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.37 count

Signature

```
DataFrameView count(self, bool render = true) const
```

Parameters

render : bool See the signature for the supported argument form.

Returns

DataFrameView Value produced by this call.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.count(false);
```

Output

```
feature  value
sales    310.0
```

8.1.38 count_by

Signature

```
DataFrameView count_by(self, std::string_view feature, std::int64_t top_n = 0, bool␣
↪dropna = true, bool render = true) const
```

Parameters

feature : std::string_view See the signature for the supported argument form.

top_n : std::int64_t See the signature for the supported argument form.

dropna : bool See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Groups by one feature and returns per-group row counts.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.count_by("city", 3, true, false);
```

Output

```
city    count
Accra   2
Tema    2
```

8.1.39 count_null_rows

Signature

```
std::int64_t count_null_rows(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Counts how many rows contain at least one null value.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.count_null_rows();
```

Output

```
3
```

8.1.40 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.defined();
```

Output

```
true
```

8.1.41 describe

Signature

```
DataFrameView describe(self, bool render = true) const
```

Parameters

render : bool See the signature for the supported argument form.

Returns

DataFrameView Value produced by this call.

Notes

Returns descriptive summary statistics for the selected numeric features.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.describe(false);
```

Output

```
feature  count  mean   min   max
sales    3      103.3  90.0  120.0
```

8.1.42 div

Signature

```
DataFrame div(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame div(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.div(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.43 div

Signature

```
DataFrame div(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame div(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.div(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.44 divide

Signature

```
DataFrame divide(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame divide(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.divide(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.45 divide

Signature

```
DataFrame divide(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame divide(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.divide(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.46 drop

Signature

```
DataFrame drop(self, std::initializer_list<std::string_view> names) const
```

Parameters

`names` : `std::initializer_list<std::string_view>` Collection of feature, slot, or stream names supplied in positional order.

Returns

`DataFrame` Value produced by this call.

Notes

Removes one or more features and returns the remaining frame.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.drop({"city"});
```

Output

```
id  sales
1   100.0
2   120.0
3    90.0
```

8.1.47 dropna

Signature

```
DataFrame dropna(self) const
```

Parameters

None

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.dropna();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.48 dropna_all

Signature

```
DataFrame dropna_all(self, std::initializer_list<std::string_view> features =
```

Parameters

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.dropna_all({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.49 eq

Signature

```
DataFrame eq(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame eq(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.eq(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.50 eq

Signature

```
DataFrame eq(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame eq(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.eq(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.51 equals

Signature

```
bool equals(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`bool` Boolean status or membership result.

Notes

Checks whether two frames have the same structure, labels, and values.

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.equals(right);
```

Output

```
true
```

8.1.52 feature_names

Signature

```
const std::vector<std::string>& feature_names(self) const
```

Parameters

None

Returns

`const std::vector<std::string>&` Borrowed list of names.

Notes

Returns the current labels used to address fields, slots, rows, or depth items.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.feature_names();
```

Output

```
["id", "sales", "city"]
```

8.1.53 feature_ref

Signature

```
ColumnRef feature_ref(self, std::string_view name) const
```

Parameters

name : std::string_view See the signature for the supported argument form.

Returns

ColumnRef Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.feature_ref("sales");
```

8.1.54 features

Signature

```
DataFrame features(self, std::initializer_list<std::string_view> names)
```

Parameters

names : std::initializer_list<std::string_view> Collection of feature, slot, or stream names supplied in positional order.

Returns

DataFrame Value produced by this call.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

See also

- std::vector<Thread> features(self) const
- Thread features(self, std::string_view name)
- Thread features(self, std::string_view name) const
- DataFrame features(self, std::initializer_list<std::string_view> names) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.features({"id", "sales"});
```

Output

```
Thread([100.0, 120.0, 90.0])
```

8.1.55 features

Signature

```
DataFrame features(self, std::initializer_list<std::string_view> names) const
```

Parameters

names : std::initializer_list<std::string_view> Collection of feature, slot, or stream names supplied in positional order.

Returns

DataFrame Value produced by this call.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

See also

- std::vector<Thread> features(self) const
- Thread features(self, std::string_view name)

- Thread features(self, std::string_view name) const
- DataFrame features(self, std::initializer_list<std::string_view> names)

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.features({"id", "sales"});
```

Output

```
Thread([100.0, 120.0, 90.0])
```

8.1.56 features

Signature

```
std::vector<Thread> features(self) const
```

Parameters

None

Returns

std::vector<Thread> Materialized collection of thread objects.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

See also

- Thread features(self, std::string_view name)
- Thread features(self, std::string_view name) const
- DataFrame features(self, std::initializer_list<std::string_view> names)
- DataFrame features(self, std::initializer_list<std::string_view> names) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.features();
```

Output

```
[Thread(id), Thread(sales), Thread(city)]
```

8.1.57 features

Signature

```
Thread features(self, std::string_view name)
```

Parameters

name : std::string_view See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

See also

- std::vector<Thread> features(self) const
- Thread features(self, std::string_view name) const
- DataFrame features(self, std::initializer_list<std::string_view> names)
- DataFrame features(self, std::initializer_list<std::string_view> names) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.features("sales");
```

Output

```
Thread([100.0, 120.0, 90.0])
```

8.1.58 features

Signature

```
Thread features(self, std::string_view name) const
```

Parameters

name : std::string_view See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

See also

- std::vector<Thread> features(self) const
- Thread features(self, std::string_view name)
- DataFrame features(self, std::initializer_list<std::string_view> names)
- DataFrame features(self, std::initializer_list<std::string_view> names) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.features("sales");
```

Output

```
Thread([100.0, 120.0, 90.0])
```

8.1.59 ffill

Signature

```
DataFrame ffill(self) const
```

Parameters

None

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.ffill();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.60 fillna

Signature

```
DataFrame fillna(self, const char* value) const
```

Parameters

`value` : `const char*` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame fillna(self, scalar value) const`
- `DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, std::string_view value) const`
- `DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::Timeline& value) const`
- `DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const`
- `DataFrame fillna(self, const pluss::TimeIndex& value) const`

- DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.61 fillna

Signature

```
DataFrame fillna(self, const char* value, std::initializer_list<std::string_view>␣
↪features) const
```

Parameters

value : const char* See the signature for the supported argument form.

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame fillna(self, scalar value) const
- DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, std::string_view value) const
- DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const char* value) const
- DataFrame fillna(self, const pluss::Timeline& value) const
- DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::TimeIndex& value) const

- DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.62 fillna

Signature

```
DataFrame fillna(self, const pluss::TimeIndex& value) const
```

Parameters

value : const pluss::TimeIndex& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame fillna(self, scalar value) const
- DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, std::string_view value) const
- DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const char* value) const
- DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::Timeline& value) const
- DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const

- DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const

Examples

```
auto df = pluss::table::DataFrame({{"idx", pluss::table::Thread(pluss::make_timeindex({
↪"2024-01-01T09:10:00Z", "2024-01-01T09:40:00Z"}))}});

auto out = df.fillna(pluss::make_timeindex("2024-01-01T00:00:00Z"));
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.63 fillna

Signature

```
DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_
↪view> features) const
```

Parameters

value : const pluss::TimeIndex& See the signature for the supported argument form.

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame fillna(self, scalar value) const
- DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, std::string_view value) const
- DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const char* value) const
- DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::Timeline& value) const

- DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::TimeIndex& value) const

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.64 fillna

Signature

```
DataFrame fillna(self, const pluss::Timeline& value) const
```

Parameters

value : const pluss::Timeline& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame fillna(self, scalar value) const
- DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, std::string_view value) const
- DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const char* value) const
- DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::TimeIndex& value) const

- DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const

Examples

```
auto df = pluss::table::DataFrame({{"ts", pluss::table::Thread(pluss::make_timeline({
↪"2024-01-01T09:10:00Z", "2024-01-01T09:40:00Z"}))}});

auto out = df.fillna(pluss::make_timeline("2024-01-01T00:00:00Z"));
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.65 fillna

Signature

```
DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_
↪view> features) const
```

Parameters

value : const pluss::Timeline& See the signature for the supported argument form.

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame fillna(self, scalar value) const
- DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, std::string_view value) const
- DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const char* value) const
- DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::Timeline& value) const
- DataFrame fillna(self, const pluss::TimeIndex& value) const

- DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.66 fillna

Signature

```
DataFrame fillna(self, scalar value) const
```

Parameters

value : scalar See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, std::string_view value) const
- DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const char* value) const
- DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::Timeline& value) const
- DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::TimeIndex& value) const

- DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.fillna(0.0, {"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.67 fillna

Signature

```
DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features)␣
↪const
```

Parameters

value : scalar See the signature for the supported argument form.

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame fillna(self, scalar value) const
- DataFrame fillna(self, std::string_view value) const
- DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const char* value) const
- DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::Timeline& value) const

- DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::TimeIndex& value) const
- DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.68 fillna

Signature

```
DataFrame fillna(self, std::string_view value) const
```

Parameters

value : std::string_view See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame fillna(self, scalar value) const
- DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const char* value) const
- DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::Timeline& value) const

- DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::TimeIndex& value) const
- DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.69 fillna

Signature

```
DataFrame fillna(self, std::string_view value, std::initializer_list<std::string_view>␣
↪features) const
```

Parameters

`value` : `std::string_view` See the signature for the supported argument form.

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame fillna(self, scalar value) const
- DataFrame fillna(self, scalar value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, std::string_view value) const
- DataFrame fillna(self, const char* value) const
- DataFrame fillna(self, const char* value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::Timeline& value) const

- DataFrame fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> features) const
- DataFrame fillna(self, const pluss::TimeIndex& value) const
- DataFrame fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> features) const

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

auto out = df.fillna("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.70 fillna_

Signature

```
DataFrame& fillna_(self, const char* value)
```

Parameters

value : const char* See the signature for the supported argument form.

Returns

DataFrame& Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame& fillna_(self, scalar value)
- DataFrame& fillna_(self, std::string_view value)
- DataFrame& fillna_(self, const pluss::Timeline& value)
- DataFrame& fillna_(self, const pluss::TimeIndex& value)

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

df.fillna_("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.71 fillna_

Signature

```
DataFrame& fillna_(self, const pluss::TimeIndex& value)
```

Parameters

`value` : `const pluss::TimeIndex&` See the signature for the supported argument form.

Returns

`DataFrame&` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame& fillna_(self, scalar value)`
- `DataFrame& fillna_(self, std::string_view value)`
- `DataFrame& fillna_(self, const char* value)`
- `DataFrame& fillna_(self, const pluss::Timeline& value)`

Examples

```
auto df = pluss::table::DataFrame({{"idx", pluss::table::Thread(pluss::make_timeindex({
↪"2024-01-01T09:10:00Z", "2024-01-01T09:40:00Z"}))}});

df.fillna_(pluss::make_timeindex("2024-01-01T00:00:00Z"));
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.72 fillna_

Signature

```
DataFrame& fillna_(self, const pluss::Timeline& value)
```

Parameters

`value` : `const pluss::Timeline&` See the signature for the supported argument form.

Returns

`DataFrame&` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `DataFrame& fillna_(self, scalar value)`
- `DataFrame& fillna_(self, std::string_view value)`
- `DataFrame& fillna_(self, const char* value)`
- `DataFrame& fillna_(self, const pluss::TimeIndex& value)`

Examples

```
auto df = pluss::table::DataFrame({{"ts", pluss::table::Thread(pluss::make_timeline({
↪"2024-01-01T09:10:00Z", "2024-01-01T09:40:00Z"}))}});

df.fillna_(pluss::make_timeline("2024-01-01T00:00:00Z"));
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.73 `fillna_`

Signature

```
DataFrame& fillna_(self, scalar value)
```

Parameters

`value` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame&` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame& fillna_(self, std::string_view value)
- DataFrame& fillna_(self, const char* value)
- DataFrame& fillna_(self, const pluss::Timeline& value)
- DataFrame& fillna_(self, const pluss::TimeIndex& value)

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.fillna_(0.0);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.74 fillna_

Signature

```
DataFrame& fillna_(self, std::string_view value)
```

Parameters

value : std::string_view See the signature for the supported argument form.

Returns

DataFrame& Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- DataFrame& fillna_(self, scalar value)
- DataFrame& fillna_(self, const char* value)
- DataFrame& fillna_(self, const pluss::Timeline& value)
- DataFrame& fillna_(self, const pluss::TimeIndex& value)

Examples

```
auto df = pluss::table::DataFrame({{"city", pluss::table::Thread(pluss::make_note({"Accra
↪", "Tema", "Kumasi"}))}});

df.fillna_("Unknown");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.75 filter

Signature

```
DataFrame filter(self, const pluss::Series& bool_mask) const
```

Parameters

bool_mask : const pluss::Series& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Keeps only the rows where the supplied boolean mask is true.

See also

- DataFrame filter(self, const Thread& bool_mask) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = df.filter(mask);
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

8.1.76 filter

Signature

```
DataFrame filter(self, const Thread& bool_mask) const
```

Parameters

bool_mask : const Thread& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Keeps only the rows where the supplied boolean mask is true.

See also

- DataFrame filter(self, const pluss::Series& bool_mask) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = df.filter(mask);
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

8.1.77 floor_time

Signature

```
DataFrame floor_time(self, std::initializer_list<std::string_view> features, std::string_
↪view unit) const
```

Parameters

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

unit : std::string_view See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.floor_time({"ts"}, "hour");
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

8.1.78 floordiv

Signature

```
DataFrame floordiv(self, const DataFrame& other) const
```

Parameters

other : const DataFrame& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- DataFrame floordiv(self, scalar other) const

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
```

(continues on next page)

(continued from previous page)

```
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.floordiv(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.79 `floordiv`

Signature

```
DataFrame floordiv(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame floordiv(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.floordiv(10);
```

Output

```
id  sales  city
11  110.0  Accra
```

(continues on next page)

(continued from previous page)

```
12  130.0  Tema
13  100.0  Kumasi
```

8.1.80 ge

Signature

```
DataFrame ge(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame ge(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.ge(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.81 ge

Signature

```
DataFrame ge(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame ge(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.ge(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.82 `get`

Signature

```
std::optional<Thread> get(self, std::string_view feature_name) const
```

Parameters

`feature_name` : `std::string_view` Name of the target feature to read, rewrite, sort by, or promote to cases.

Returns

`std::optional<Thread>` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.get("sales");
```

Output

```
Thread([100.0, 120.0, 90.0])
```

8.1.83 group_by

Signature

```
DataFrameView group_by(self, std::initializer_list<std::string_view> keys,␣
↪std::initializer_list<GroupSpec> specs, bool dropna_keys = true, bool render = true)␣
↪const
```

Parameters

`keys` : `std::initializer_list<std::string_view>` Collection of slot or stream names to keep, drop, or rewrite together.

`specs` : `std::initializer_list<GroupSpec>` See the signature for the supported argument form.

`dropna_keys` : `bool` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Groups rows by one or more key features and computes the requested aggregations for each group.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.group_by({"city"}, {{"sales_sum", "sales",␣
↪pluss::table::DataFrame::GroupOp::Sum}}, true, false);
```

Output

```
city    sales_sum
Accra   220.0
Tema    240.0
```

8.1.84 gt

Signature

```
DataFrame gt(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame gt(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.gt(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.85 gt

Signature

```
DataFrame gt(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame gt(self, const DataFrame& other) const`

Examples

```cpp
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.gt(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.86 has_feature

Signature

```cpp
bool has_feature(self, std::string_view name) const
```

Parameters

`name` : `std::string_view` See the signature for the supported argument form.

Returns

`bool` Boolean status or membership result.

Notes

Checks whether a feature name exists in the frame.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.has_feature("sales");
```

Output

```
true
```

8.1.87 head

Signature

```
DataFrameView head(self, std::int64_t n = 5, bool render = true) const
```

Parameters

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Returns the first `n` rows of the frame.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.head(2, false);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.88 iat

Signature

```
std::string iat(self, std::int64_t row, std::size_t feature_idx) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

`feature_idx` : `std::size_t` See the signature for the supported argument form.

Returns

`std::string` Rendered string result.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.iat(0, 0);
```

Output

```
1
```

8.1.89 iloc

Signature

```
DataFrame iloc(self, std::int64_t row) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- DataFrame iloc(self, std::int64_t start, std::int64_t stop) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.iloc(1);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.90 iloc

Signature

```
DataFrame iloc(self, std::int64_t start, std::int64_t stop) const
```

Parameters

start : std::int64_t Inclusive starting position of the slice.

stop : std::int64_t Exclusive ending position of the slice.

Returns

DataFrame Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- DataFrame iloc(self, std::int64_t row) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.iloc(0, 2);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.91 info

Signature

```
void info(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- `void info(self, std::ostream& os, const display::DisplayOptions& options =`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.info();
```

Output

```
rows: 3
features: 3
numeric: id, sales
note: city
```

8.1.92 info

Signature

```
void info(self, std::ostream& os, const display::DisplayOptions& options =
```

Parameters

`os` : `std::ostream&` See the signature for the supported argument form.

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- `void info(self, const display::DisplayOptions& options =`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.info();
```

Output

```
rows: 3
features: 3
numeric: id, sales
note: city
```

8.1.93 info_string

Signature

```
std::string info_string(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.info_string();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.94 insert

Signature

```
DataFrame insert(self, std::int64_t feature_index, std::string name, const Thread&␣
↪feature) const
```

Parameters

`feature_index` : `std::int64_t` See the signature for the supported argument form.

`name` : `std::string` See the signature for the supported argument form.

`feature` : `const Thread&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Inserts a new feature at the requested position and shifts the later features to the right.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.insert(1, "profit", pluss::table::Thread(pluss::make_series({40.0, 35.0,␣
↪30.0})));
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

8.1.95 isna

Signature

```
DataFrame isna(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.isna({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.96 isnull

Signature

```
DataFrame isnull(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.isnull({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.97 join

Signature

```
DataFrame join(self, const DataFrame& other, std::string collision_suffix = "_right")␣
↪const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

`collision_suffix` : `std::string` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Adds compatible features from another frame, aligning rows by the shared case axis.

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
```

(continues on next page)

(continued from previous page)

```
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.join(right);
```

Output

```
id  sales  cost
1   100.0  60.0
2   120.0  80.0
3    90.0  55.0
```

8.1.98 kindelize

Signature

```
pluss::kindle::Catalog kindelize(self) const
```

Parameters

None

Returns

`pluss::kindle::Catalog` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `pluss::kindle::Catalog kindelize(self, const pluss::kindle::KindelizePolicy& policy) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.kindelize();
```

8.1.99 kindelize

Signature

```
pluss::kindle::Catalog kindelize(self, const pluss::kindle::KindelizePolicy& policy)␣
↪const
```

Parameters

`policy` : `const pluss::kindle::KindelizePolicy&` See the signature for the supported argument form.

Returns

`pluss::kindle::Catalog` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `pluss::kindle::Catalog kindelize(self) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.kindelize();
```

8.1.100 `le`

Signature

```
DataFrame le(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame le(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.le(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.101 le

Signature

```
DataFrame le(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame le(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.le(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.102 length

Signature

```
std::int64_t height(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

Applies height to the current DataFrame object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.height();
```

8.1.103 load

Signature

```
static DataFrame load(std::string_view source_type, std::string_view filepath)
```

Parameters

source_type : std::string_view Storage format selector such as "csv", "xlsx", "json", or "parquet".

filepath : std::string_view File location to read from or write to. Use a neutral project path such as path/to/data.csv.

Returns

DataFrame Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)

- static DataFrame load(std::string_view source_type, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries)
- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::ReadCsvOptions& options)
- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =
- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)

Examples

```
auto out = pluss::table::DataFrame::load("csv", "path/to/data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.104 load

Signature

```
static DataFrame load(std::string_view source_type, std::string_view filepath, char sep,␣
↪std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =
```

Parameters

source_type : std::string_view Storage format selector such as "csv", "xlsx", "json", or "parquet".

filepath : std::string_view File location to read from or write to. Use a neutral project path such as path/to/data.csv.

sep : char Column separator used for delimited text files such as CSV.

ncases : std::optional<std::size_t> See the signature for the supported argument form.

usefeats : std::vector<std::string> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load(std::string_view source_type, std::string_view filepath)
- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load(std::string_view source_type, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries)
- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::ReadCsvOptions& options)
- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)

Examples

```
auto out = pluss::table::DataFrame::load("csv", "path/to/data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.105 load

Signature

```
static DataFrame load(std::string_view source_type, std::string_view filepath, char sep,␣
↪std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats,␣
↪std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_
↪bad_lines = pluss::loader::BadLinePolicy::Error)
```

Parameters

source_type : std::string_view Storage format selector such as "csv", "xlsx", "json", or "parquet".

filepath : std::string_view File location to read from or write to. Use a neutral project path such as path/to/data.csv.

sep : char Column separator used for delimited text files such as CSV.

ncases : std::optional<std::size_t> See the signature for the supported argument form.

usefeats : std::initializer_list<std::string_view> See the signature for the supported argument form.

case_col : std::optional<std::string> See the signature for the supported argument form.

on_bad_lines : pluss::loader::BadLinePolicy See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load(std::string_view source_type, std::string_view filepath)
- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load(std::string_view source_type, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries)
- static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::ReadCsvOptions& options)
- static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =

Examples

```
auto out = pluss::table::DataFrame::load("csv", "path/to/data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.106 load

Signature

```
static DataFrame load(std::string_view source_type, std::string_view filepath, const␣
↪pluss::loader::Config& config)
```

Parameters

source_type : std::string_view Storage format selector such as "csv", "xlsx", "json", or "parquet".

filepath : std::string_view File location to read from or write to. Use a neutral project path such as path/to/data.csv.

config : const pluss::loader::Config& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load(std::string_view source_type, std::string_view filepath)`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries)`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.107 load

Signature

```
static DataFrame load(std::string_view source_type, std::string_view filepath, const␣
↪pluss::loader::ReadCsvOptions& options)
```

Parameters

`source_type` : `std::string_view` Storage format selector such as `"csv"`, `"xlsx"`, `"json"`, or `"parquet"`.

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`options` : `const pluss::loader::ReadCsvOptions&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load(std::string_view source_type, std::string_view filepath)`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries)`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
pluss::loader::ReadCsvOptions options;
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", options);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.108 load

Signature

```
static DataFrame load(std::string_view source_type, std::string_view filepath,␣
↪std::initializer_list<pluss::loader::ConfigEntry> config_entries)
```

Parameters

`source_type` : `std::string_view` Storage format selector such as `"csv"`, `"xlsx"`, `"json"`, or `"parquet"`.

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load(std::string_view source_type, std::string_view filepath)`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =`
- `static DataFrame load(std::string_view source_type, std::string_view filepath, char sep, std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.109 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.110 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t>
↪ ncases = std::nullopt, std::vector<std::string> usefeats =
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`sep` : `char` Column separator used for delimited text files such as CSV.

`ncases` : `std::optional<std::size_t>` See the signature for the supported argument form.

`usefeats` : `std::vector<std::string>` See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load_csv(std::string_view filepath)
- static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)
- static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt
config_entries)
- static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)
- static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)
- static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.111 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t>
↪ ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string>␣
↪case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines =␣
↪pluss::loader::BadLinePolicy::Error)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`sep` : `char` Column separator used for delimited text files such as CSV.

`ncases` : `std::optional<std::size_t>` See the signature for the supported argument form.

`usefeats` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`case_col` : `std::optional<std::string>` See the signature for the supported argument form.

`on_bad_lines` : `pluss::loader::BadLinePolicy` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_csv(std::string_view filepath)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =`

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.112 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_csv(std::string_view filepath)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.113 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions&
↪ options)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`options` : `const pluss::loader::ReadCsvOptions&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_csv(std::string_view filepath)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
pluss::loader::ReadCsvOptions options;
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", options);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.114 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_csv(std::string_view filepath)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.115 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_
↪t ncases)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`header` : `std::string_view` Controls whether feature names are written to, or expected from, the file header row.

`ncases` : `std::size_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_csv(std::string_view filepath)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)`
- `static DataFrame load_csv(std::string_view filepath, std::string_view header, std::string_view case_feature = "none", std::string_view engine = "auto", std::optional<std::size_t> ncases = std::nullopt)`
- `static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =`

- static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.116 load_csv

Signature

```
static DataFrame load_csv(std::string_view filepath, std::string_view header,␣
↪std::string_view case_feature = "none", std::string_view engine = "auto", std::optional
↪<std::size_t> ncases = std::nullopt)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`header` : `std::string_view` Controls whether feature names are written to, or expected from, the file header row.

`case_feature` : `std::string_view` See the signature for the supported argument form.

`engine` : `std::string_view` See the signature for the supported argument form.

`ncases` : `std::optional<std::size_t>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load_csv(std::string_view filepath)
- static DataFrame load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)
- static DataFrame load_csv(std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load_csv(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt
config_entries)

- static DataFrame load_csv(std::string_view filepath, std::string_view header, std::size_t ncases)
- static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases = std::nullopt, std::vector<std::string> usefeats =
- static DataFrame load_csv(std::string_view filepath, char sep, std::optional<std::size_t> ncases, std::initializer_list<std::string_view> usefeats, std::optional<std::string> case_col = std::nullopt, pluss::loader::BadLinePolicy on_bad_lines = pluss::loader::BadLinePolicy::Error)

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.117 load_excel

Signature

```
static DataFrame load_excel(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)
- static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)
- static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)

- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("path/to/data.xlsx");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.118 load_excel

Signature

```
static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config&
↪config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`

- static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)
- static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)
- static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.119 load_excel

Signature

```
static DataFrame load_excel(std::string_view filepath, const␣
↪pluss::loader::ReadExcelOptions& options)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`options` : `const pluss::loader::ReadExcelOptions&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
pluss::loader::ReadExcelOptions options;
auto out = pluss::table::DataFrame::load_excel("path/to/data.xlsx", options);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.120 load_excel

Signature

```
static DataFrame load_excel(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.121 load_excel

Signature

```
static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool␣
↪header = true)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`sheet_index` : `std::int64_t` Zero-based worksheet position inside an Excel workbook.

`header` : `bool` Controls whether feature names are written to, or expected from, the file header row.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("path/to/data.xlsx", 0, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.122 load_excel

Signature

```
static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool␣
↪header = true)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`sheet_name` : `std::string_view` Worksheet name to read from or write to inside an Excel workbook.

`header` : `bool` Controls whether feature names are written to, or expected from, the file header row.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`

- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("path/to/data.xlsx", "Sheet1", true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.123 load_excel

Signature

```
static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector
↪<std::int64_t>& sheet_indices, bool header = true)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`sheet_indices` : `const std::vector<std::int64_t>&` See the signature for the supported argument form.

`header` : `bool` Controls whether feature names are written to, or expected from, the file header row.

Returns

`pluss::loader::LoadResult` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`

- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("path/to/data.xlsx");
```

8.1.124 load_excel

Signature

```
static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector
↪<std::string>& sheet_names, bool header = true)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`sheet_names` : `const std::vector<std::string>&` See the signature for the supported argument form.

`header` : `bool` Controls whether feature names are written to, or expected from, the file header row.

Returns

`pluss::loader::LoadResult` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`

- static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)

Examples

```
auto out = pluss::table::DataFrame::load_excel("path/to/data.xlsx", "Sheet1", true);
```

8.1.125 load_excel

Signature

```
static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_
↪list<std::int64_t> sheet_indices, bool header = true)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`sheet_indices` : `std::initializer_list<std::int64_t>` See the signature for the supported argument form.

`header` : `bool` Controls whether feature names are written to, or expected from, the file header row.

Returns

`pluss::loader::LoadResult` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load_excel(std::string_view filepath)
- static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)
- static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)
- static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)
- static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)

- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("path/to/data.xlsx");
```

8.1.126 load_excel

Signature

```
static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_
↪list<std::string_view> sheet_names, bool header = true)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`sheet_names` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`header` : `bool` Controls whether feature names are written to, or expected from, the file header row.

Returns

`pluss::loader::LoadResult` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`

- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)`
- `static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)`

Examples

```
auto out = pluss::table::DataFrame::load_excel("path/to/data.xlsx", "Sheet1", true);
```

8.1.127 load_excel

Signature

```
static pluss::loader::LoadResult load_excel(std::string_view filepath, std::nullptr_t,␣
↪bool header = true)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`arg2` : `std::nullptr_t` See the signature for the supported argument form.

`header` : `bool` Controls whether feature names are written to, or expected from, the file header row.

Returns

`pluss::loader::LoadResult` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_excel(std::string_view filepath)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static DataFrame load_excel(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_excel(std::string_view filepath, std::initializer_list<pluss::loader::ConfigE config_entries)`
- `static DataFrame load_excel(std::string_view filepath, std::string_view sheet_name, bool header = true)`
- `static DataFrame load_excel(std::string_view filepath, std::int64_t sheet_index, bool header = true)`

- static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::string>& sheet_names, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::string_view> sheet_names, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, const std::vector<std::int64_t>& sheet_indices, bool header = true)
- static pluss::loader::LoadResult load_excel(std::string_view filepath, std::initializer_list<std::int64_t> sheet_indices, bool header = true)

Examples

```
auto out = pluss::table::DataFrame::load_excel("path/to/data.xlsx", nullptr, true);
```

8.1.128 load_json

Signature

```
static DataFrame load_json(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load_json(std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load_json(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEr config_entries)

Examples

```
auto out = pluss::table::DataFrame::load_json("path/to/data.json");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.129 load_json

Signature

```
static DataFrame load_json(std::string_view filepath, const pluss::loader::Config&␣
↪config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_json(std::string_view filepath)`
- `static DataFrame load_json(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEn config_entries)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.130 load_json

Signature

```
static DataFrame load_json(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load_json(std::string_view filepath)
- static DataFrame load_json(std::string_view filepath, const pluss::loader::Config& config)

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.131 load_parquet

Signature

```
static DataFrame load_parquet(std::string_view filepath)
```

Parameters

filepath : std::string_view File location to read from or write to. Use a neutral project path such as path/to/data.csv.

Returns

DataFrame Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- static DataFrame load_parquet(std::string_view filepath, const pluss::loader::Config& config)
- static DataFrame load_parquet(std::string_view filepath, std::initializer_list<pluss::loader::Confi config_entries)

Examples

```
auto out = pluss::table::DataFrame::load_parquet("path/to/data.parquet");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.132 load_parquet

Signature

```
static DataFrame load_parquet(std::string_view filepath, const pluss::loader::Config&␣
↪config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_parquet(std::string_view filepath)`
- `static DataFrame load_parquet(std::string_view filepath, std::initializer_list<pluss::loader::Confi config_entries)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.133 load_parquet

Signature

```
static DataFrame load_parquet(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_parquet(std::string_view filepath)`
- `static DataFrame load_parquet(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.134 load_txt

Signature

```
static DataFrame load_txt(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_txt(std::string_view filepath, const pluss::loader::Config& config)`
- `static DataFrame load_txt(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)`

Examples

```
auto out = pluss::table::DataFrame::load_txt("path/to/data.txt");
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.135 `load_txt`

Signature

```
static DataFrame load_txt(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_txt(std::string_view filepath)`
- `static DataFrame load_txt(std::string_view filepath, std::initializer_list<pluss::loader::ConfigEnt config_entries)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.136 load_txt

Signature

```
static DataFrame load_txt(std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static DataFrame load_txt(std::string_view filepath)`
- `static DataFrame load_txt(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::table::DataFrame::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.137 `lt`

Signature

```
DataFrame lt(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame lt(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.lt(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.138 `lt`

Signature

```
DataFrame lt(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame lt(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.lt(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.139 `matrix`

Signature

```
MatrixValues matrix(self) const
```

Parameters

None

Returns

`MatrixValues` Value produced by this call.

Notes

Applies `matrix` to the current `DataFrame` object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
```

(continues on next page)

(continued from previous page)

```
});

auto out = df.matrix();
```

8.1.140 max

Signature

```
DataFrameView max(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Computes per-feature maxima for the selected numeric features.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.max(false);
```

Output

```
feature  value
sales    310.0
```

8.1.141 mean

Signature

```
DataFrameView mean(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Computes per-feature means for the selected numeric features.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.mean(false);
```

Output

```
feature  value
sales    310.0
```

8.1.142 mean_by

Signature

```
DataFrameView mean_by(self, std::string_view group_feature, std::string_view value_
↪feature, std::int64_t top_n = 0, std::int64_t min_count = 1, bool dropna_group = true,␣
↪bool render = true) const
```

Parameters

`group_feature` : `std::string_view` See the signature for the supported argument form.

`value_feature` : `std::string_view` See the signature for the supported argument form.

`top_n` : `std::int64_t` See the signature for the supported argument form.

`min_count` : `std::int64_t` See the signature for the supported argument form.

`dropna_group` : `bool` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Groups by one feature and returns the mean of another feature within each group.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});
```

(continues on next page)

(continued from previous page)

```
auto out = df.mean_by("city", "sales", 3, 1, true, false);
```

Output

```
city    sales_mean
Accra   110.0
Tema    120.0
```

8.1.143 min

Signature

```
DataFrameView min(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Computes per-feature minima for the selected numeric features.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.min(false);
```

Output

```
feature  value
sales    310.0
```

8.1.144 mod

Signature

```
DataFrame mod(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame mod(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.mod(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.145 mod

Signature

```
DataFrame mod(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame mod(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.mod(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.146 `mul`

Signature

```
DataFrame mul(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame mul(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.mul(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.147 mul

Signature

```
DataFrame mul(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame mul(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.mul(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.148 multiply

Signature

```
DataFrame multiply(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame multiply(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.multiply(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.149 multiply

Signature

```
DataFrame multiply(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- DataFrame multiply(self, const DataFrame& other) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.multiply(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.150 ncases

Signature

```
std::int64_t ncases(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

Applies ncases to the current DataFrame object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
```

(continues on next page)

(continued from previous page)

```
});

auto out = df.ncases();
```

8.1.151 ne

Signature

```
DataFrame ne(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame ne(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.ne(right);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.152 ne

Signature

```
DataFrame ne(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `DataFrame ne(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.ne(10);
```

Output

```
id  sales
1   true
2   true
3   false
```

8.1.153 `nfeats`

Signature

```
std::int64_t nfeats(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Applies `nfeats` to the current `DataFrame` object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.nfeats();
```

8.1.154 nfeatures

Signature

```
std::int64_t nfeatures(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.nfeatures();
```

Output

```
3
```

8.1.155 note_features

Signature

```
DataFrame note_features(self) const
```

Parameters

None

Returns

DataFrame Value produced by this call.

Notes

Returns only the note-backed features.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.note_features();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.156 notna

Signature

```
DataFrame notna(self, std::initializer_list<std::string_view> features =
```

Parameters

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.notna({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.157 notnull

Signature

```
DataFrame notnull(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.notnull({"sales"});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.158 pivot

Signature

```
DataFrameView pivot(self, std::string_view case_feature, std::string_view feature_
↪feature, std::string_view value_feature, bool dropna_keys = true, bool render = true)␣
↪const
```

Parameters

`case_feature` : `std::string_view` See the signature for the supported argument form.

`feature_feature` : `std::string_view` See the signature for the supported argument form.

`value_feature` : `std::string_view` See the signature for the supported argument form.

`dropna_keys` : `bool` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Reshapes long-form rows into a wider table indexed by one feature and expanded by another.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.pivot("city", "product", "sales", true, false);
```

Output

```
city   A      B
Accra  100.0  120.0
Tema    90.0  150.0
```

8.1.159 pivot_table

Signature

```
DataFrameView pivot_table(self, std::string_view case_feature, std::string_view feature_
↪feature, std::string_view value_feature, GroupOp op = GroupOp::Mean, bool dropna_keys␣
↪= true, bool render = true) const
```

Parameters

`case_feature` : `std::string_view` See the signature for the supported argument form.

`feature_feature` : `std::string_view` See the signature for the supported argument form.

`value_feature` : `std::string_view` See the signature for the supported argument form.

`op` : `GroupOp` See the signature for the supported argument form.

`dropna_keys` : `bool` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Builds a pivoted summary table using an aggregation function when multiple rows map to the same cell.

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.pivot_table("city", "product", "sales",␣
↪pluss::table::DataFrame::GroupOp::Mean, true, false);
```

Output

```
city   A      B
Accra  100.0  120.0
Tema    90.0  150.0
```

8.1.160 plot

Signature

```
DataFramePlotAccessor plot(self) const
```

Parameters

None

Returns

`DataFramePlotAccessor` Value produced by this call.

Notes

This opens the plotting accessor for the current data object.

See also

- `DataFramePlotAccessor plot(self, const DataFramePlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto fig = df.plot().line("sales");
```

Output

```
Figure(line/scatter/bar/hist)
```

8.1.161 plot

Signature

```
DataFramePlotAccessor plot(self, const DataFramePlotOptions& options) const
```

Parameters

`options` : `const DataFramePlotOptions&` See the signature for the supported argument form.

Returns

`DataFramePlotAccessor` Value produced by this call.

Notes

This opens the plotting accessor for the current data object.

See also

- `DataFramePlotAccessor plot(self) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto fig = df.plot().line("sales");
```

Output

```
Figure(line/scatter/bar/hist)
```

8.1.162 plot3d

Signature

```
DataFramePlot3DAccessor plot3d(self) const
```

Parameters

None

Returns

DataFramePlot3DAccessor Value produced by this call.

Notes

Applies plot3d to the current DataFrame object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- DataFramePlot3DAccessor plot3d(self, const DataFramePlotOptions& options) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.plot3d();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.163 plot3d

Signature

```
DataFramePlot3DAccessor plot3d(self, const DataFramePlotOptions& options) const
```

Parameters

options : const DataFramePlotOptions& See the signature for the supported argument form.

Returns

DataFramePlot3DAccessor Value produced by this call.

Notes

Applies plot3d to the current DataFrame object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- DataFramePlot3DAccessor plot3d(self) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.plot3d();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.164 pow

Signature

```
DataFrame pow(self, const DataFrame& other) const
```

Parameters

other : const DataFrame& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- DataFrame pow(self, scalar other) const

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});
```

(continues on next page)

(continued from previous page)

```
auto out = left.pow(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.165 pow

Signature

```
DataFrame pow(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame pow(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.pow(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.166 prepare_sql

Signature

```
DataFrameSqlPrepared prepare_sql(self, std::string_view statement) const
```

Parameters

`statement` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrameSqlPrepared` Value produced by this call.

Notes

Use this when the transformation is easier to express as SQL than as a chained sequence of table verbs.

See also

- `DataFrameSqlPrepared prepare_sql(self, std::string_view statement, SqlOptions options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto stmt = df.prepare_sql("SELECT id, sales FROM self WHERE sales > {min_sales}");
```

Output

```
SQL accessor / prepared statement created
```

8.1.167 prepare_sql

Signature

```
DataFrameSqlPrepared prepare_sql(self, std::string_view statement, SqlOptions options)␣
↪const
```

Parameters

`statement` : `std::string_view` See the signature for the supported argument form.

`options` : `SqlOptions` See the signature for the supported argument form.

Returns

`DataFrameSqlPrepared` Value produced by this call.

Notes

Use this when the transformation is easier to express as SQL than as a chained sequence of table verbs.

See also

- `DataFrameSqlPrepared prepare_sql(self, std::string_view statement) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto stmt = df.prepare_sql("SELECT id, sales FROM self WHERE sales > {min_sales}");
```

Output

```
SQL accessor / prepared statement created
```

8.1.168 query

Signature

```
DataFrame query(self, std::string_view expression) const
```

Parameters

`expression` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Keeps only the rows that satisfy the supplied filter expression or mask-producing query.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.query("sales > 95");
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

8.1.169 radd

Signature

```
DataFrame radd(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.radd(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.170 rdiv

Signature

```
DataFrame rdiv(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rdiv(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.171 `reindex_cases`

Signature

```
DataFrame reindex_cases(self, const Thread& new_cases) const
```

Parameters

`new_cases` : `const Thread&` Replacement case labels in the exact order you want the rows to follow.

Returns

`DataFrame` Value produced by this call.

Notes

Builds a new case axis from the supplied labels and aligns rows onto that case order.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto cases = pluss::table::Thread(pluss::make_note({"cs1", "cs0", "cs2"}));
auto out = df.reindex_cases(cases);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.172 reindex_like

Signature

```
DataFrame reindex_like(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Realigns this frame so its cases match another frame.

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.reindex_like(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.173 relabel_features

Signature

```
DataFrame relabel_features(self, std::initializer_list<std::string_view> feature_names)␣
↪const
```

Parameters

`feature_names` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Replaces the full feature label set with the supplied names in positional order.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.relabel_features({"revenue", "location", "id_new"});
```

Output

```
id  revenue  city
1   100.0    Accra
2   120.0    Tema
3    90.0    Kumasi
```

8.1.174 `rename`

Signature

```
DataFrame rename(self, std::string_view old_name, std::string new_name) const
```

Parameters

`old_name` : `std::string_view` See the signature for the supported argument form.

`new_name` : `std::string` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Renames one feature without changing its values.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rename("sales", "revenue");
```

Output

```
id  revenue  city
1   100.0    Accra
2   120.0    Tema
3    90.0    Kumasi
```

8.1.175 rename_

Signature

```
DataFrame& rename_(self, std::string_view old_name, std::string new_name)
```

Parameters

`old_name` : `std::string_view` See the signature for the supported argument form.

`new_name` : `std::string` See the signature for the supported argument form.

Returns

`DataFrame&` Value produced by this call.

Notes

Renames one feature in place without changing its values.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.rename_("sales", "revenue");
```

Output

```
id  revenue  city
1   100.0    Accra
2   120.0    Tema
3    90.0    Kumasi
```

8.1.176 rename_features

Signature

```
DataFrame rename_features(self,  std::initializer_list<std::pair<std::string_view,␣
↪std::string_view>> renames) const
```

Parameters

renames : std::initializer_list<std::pair<std::string_view, std::string_view>> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Renames several features in one call.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rename_features({{"sales", "revenue"}, {"city", "location"}});
```

Output

```
id  revenue  city
1   100.0    Accra
2   120.0    Tema
3    90.0    Kumasi
```

8.1.177 reorder

Signature

```
DataFrame reorder(self, std::initializer_list<std::string_view> names) const
```

Parameters

names : std::initializer_list<std::string_view> Collection of feature, slot, or stream names supplied in positional order.

Returns

DataFrame Value produced by this call.

Notes

Reorders the feature columns without changing any row values.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.reorder({"sales", "id", "city"});
```

Output

```
sales  id  city
100.0  1  Accra
120.0  2  Tema
90.0   3  Kumasi
```

8.1.178 repeated_cases

Signature

```
pluss::Series repeated_cases(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::Series` Value produced by this call.

Notes

Returns the rows whose current case labels are duplicates of an earlier row.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.repeated_cases();
```

8.1.179 resample

Signature

```
DataFrameView resample(self, std::string_view time_feature, std::initializer_list
↪<std::string_view> value_features, std::string_view freq, std::string_view op = "mean",
↪ std::int64_t step = 1, bool render = true) const
```

Parameters

`time_feature` : `std::string_view` See the signature for the supported argument form.

`value_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`freq` : `std::string_view` See the signature for the supported argument form.

`op` : `std::string_view` See the signature for the supported argument form.

`step` : `std::int64_t` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.resample("ts", {"load"}, "1h", "mean", 1, false);
```

Output

```
ts                   load
2024-01-01T09:00:00Z  15.0
2024-01-01T10:00:00Z  15.0
```

8.1.180 `reset_cases`

Signature

```
DataFrame reset_cases(self, std::string case_feature = "case") const
```

Parameters

`case_feature` : `std::string` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Materializes the current case axis as a normal feature, then restores the default positional cases `0..n-1`.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto with_cases = df.set_cases("city", true);
auto out = with_cases.reset_cases("case");
```

Output

```
with_cases.cases():
["Accra", "Tema", "Kumasi"]

out.cases():
[0, 1, 2]

out.head():
id  sales  case
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.181 `rfloordiv`

Signature

```
DataFrame rfloordiv(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rfloordiv(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.182 `rmod`

Signature

```
DataFrame rmod(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rmod(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.183 rmul

Signature

```
DataFrame rmul(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rmul(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.184 rolling

Signature

```
DataFrame rolling(self, std::initializer_list<WindowSpec> specs, bool keep_input = true)␣
↪const
```

Parameters

`specs` : `std::initializer_list<WindowSpec>` See the signature for the supported argument form.

`keep_input` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Computes rolling-window features and returns them as a derived frame.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rolling({{.output = "sales_roll", .feature = "sales", .op =␣
↪pluss::table::ThreadWindowOp::RollingMean, .window = 2, .min_periods = 1}}, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.185 round_time

Signature

```
DataFrame round_time(self, std::initializer_list<std::string_view> features, std::string_
↪view unit) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`unit` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
```

(continues on next page)

(continued from previous page)

```
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.round_time({"ts"}, "hour");
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

8.1.186 rpow

Signature

```
DataFrame rpow(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rpow(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.187 rsub

Signature

```
DataFrame rsub(self, scalar other) const
```

Parameters

other : scalar See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rsub(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.188 rtruediv

Signature

```
DataFrame rtruediv(self, scalar other) const
```

Parameters

other : scalar See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.rtruediv(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.189 sample

Signature

```
DataFrame sample(self, std::int64_t n, std::optional<std::uint64_t> random_state =␣
↪std::nullopt) const
```

Parameters

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

`random_state` : `std::optional<std::uint64_t>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Returns a random sample of rows.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.sample(2, 7);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.190 save_as

Signature

```
void save_as(self, std::string_view target_type, std::string_view filepath, const␣
↪pluss::loader::Config& config =
```

Parameters

`target_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame using the explicitly supplied output format selector rather than a dedicated saver overload.

See also

- `void save_as(self, std::string_view target_type, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_as("json", "path/to/output.json");
```

Output

```
saved: out.*
```

8.1.191 save_as

Signature

```
void save_as(self, std::string_view target_type, std::string_view filepath,␣
↪std::initializer_list<pluss::loader::ConfigEntry> config_entries) const
```

Parameters

`target_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame using the explicitly supplied output format selector rather than a dedicated saver overload.

See also

- `void save_as(self, std::string_view target_type, std::string_view filepath, const pluss::loader::Config& config =`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_as("json", "path/to/output.json");
```

Output

```
saved: out.*
```

8.1.192 save_csv

Signature

```
void save_csv(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to CSV. Set `add_cases=true` when you want the current case axis exported as a regular column so row labels survive the round trip.

See also

- void save_csv(self, std::string_view filepath, bool add_cases, bool header = true, char sep = ',', std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt, std::optional<std::string> float_format = std::nullopt, char quotechar = '"', bool scalarquote = true, std::string line_terminator = "\n", char decimal = '.', std::string encoding = "utf-8") const
- void save_csv(self, std::string_view filepath, const pluss::loader::WriteCsvOptions& options) const
- void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const
- void save_csv(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

8.1.193 save_csv

Signature

```
void save_csv(self, std::string_view filepath, bool add_cases, bool header = true, char␣
↪sep = ',', std::string na_rep = "NA", std::optional<std::string> case_label =␣
↪std::nullopt, std::optional<std::string> float_format = std::nullopt, char quotechar =
↪'"', bool scalarquote = true, std::string line_terminator = "\n", char decimal = '.',␣
↪std::string encoding = "utf-8") const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`add_cases` : `bool` When `true`, the current case axis is exported as its own column so row labels can be reconstructed when the file is loaded again.

`header` : `bool` Controls whether feature names are written to, or expected from, the file header row.

`sep` : `char` Column separator used for delimited text files such as CSV.

`arg5` : `'` See the signature for the supported argument form.

`na_rep` : `std::string` String written anywhere a null or missing value must be serialized.

`case_label` : `std::optional<std::string>` Optional column label to use for the exported case axis when `add_cases=true`.

`float_format` : `std::optional<std::string>` Optional numeric formatting string used when floating-point values are written to text output.

`quotechar` : `char` Quote character used when writing delimited text output.

`doublequote` : `bool` When `true`, embedded quote characters are escaped by doubling them in CSV-style output.

`line_terminator` : `std::string` Line ending written between serialized rows in text-based export.

`decimal` : `char` Decimal marker used when formatting floating-point output.

`encoding` : `std::string` Text encoding used when reading or writing text-based files.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to CSV. Set `add_cases=true` when you want the current case axis exported as a regular column so row labels survive the round trip.

See also

- `void save_csv(self, std::string_view filepath) const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::WriteCsvOptions& options) const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_csv(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

8.1.194 save_csv

Signature

```
void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to CSV. Set `add_cases=true` when you want the current case axis exported as a regular column so row labels survive the round trip.

See also

- `void save_csv(self, std::string_view filepath) const`
- `void save_csv(self, std::string_view filepath, bool add_cases, bool header = true, char sep = ',', std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt, std::optional<std::string> float_format = std::nullopt, char quotechar = '"', bool scalarquote = true, std::string line_terminator = "\n", char decimal = '.', std::string encoding = "utf-8") const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::WriteCsvOptions& options) const`
- `void save_csv(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

8.1.195 save_csv

Signature

```
void save_csv(self, std::string_view filepath, const pluss::loader::WriteCsvOptions&␣
↪options) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`options` : `const pluss::loader::WriteCsvOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to CSV. Set `add_cases=true` when you want the current case axis exported as a regular column so row labels survive the round trip.

See also

- `void save_csv(self, std::string_view filepath) const`
- `void save_csv(self, std::string_view filepath, bool add_cases, bool header = true, char sep = ',', std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt, std::optional<std::string> float_format = std::nullopt, char quotechar = '"', bool scalarquote = true, std::string line_terminator = "\n", char decimal = '.', std::string encoding = "utf-8") const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_csv(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

8.1.196 save_csv

Signature

```
void save_csv(self, std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to CSV. Set `add_cases=true` when you want the current case axis exported as a regular column so row labels survive the round trip.

See also

- `void save_csv(self, std::string_view filepath) const`
- `void save_csv(self, std::string_view filepath, bool add_cases, bool header = true, char sep = ',', std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt, std::optional<std::string> float_format = std::nullopt, char quotechar = '"', bool scalarquote = true, std::string line_terminator = "\n", char decimal = '.', std::string encoding = "utf-8") const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::WriteCsvOptions& options) const`
- `void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

8.1.197 save_excel

Signature

```
void save_excel(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to an Excel sheet. Set `add_cases=true` when you want the current case axis exported as a regular column.

See also

- `void save_excel(self, std::string_view filepath, std::string sheet_name, bool add_cases = false, bool header = true, std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::WriteExcelOptions& options) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_excel(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

8.1.198 save_excel

Signature

```
void save_excel(self, std::string_view filepath, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to an Excel sheet. Set `add_cases=true` when you want the current case axis exported as a regular column.

See also

- `void save_excel(self, std::string_view filepath) const`
- `void save_excel(self, std::string_view filepath, std::string sheet_name, bool add_cases = false, bool header = true, std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::WriteExcelOptions& options) const`
- `void save_excel(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

8.1.199 save_excel

Signature

```
void save_excel(self, std::string_view filepath, const pluss::loader::WriteExcelOptions&␣
↪options) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`options` : `const pluss::loader::WriteExcelOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to an Excel sheet. Set `add_cases=true` when you want the current case axis exported as a regular column.

See also

- `void save_excel(self, std::string_view filepath) const`
- `void save_excel(self, std::string_view filepath, std::string sheet_name, bool add_cases = false, bool header = true, std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_excel(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

8.1.200 save_excel

Signature

```
void save_excel(self, std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to an Excel sheet. Set `add_cases=true` when you want the current case axis exported as a regular column.

See also

- `void save_excel(self, std::string_view filepath) const`
- `void save_excel(self, std::string_view filepath, std::string sheet_name, bool add_cases = false, bool header = true, std::string na_rep = "NA", std::optional<std::string> case_label = std::nullopt) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::WriteExcelOptions& options) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

8.1.201 save_excel

Signature

```
void save_excel(self, std::string_view filepath, std::string sheet_name, bool add_cases␣
↪= false, bool header = true, std::string na_rep = "NA", std::optional<std::string>␣
↪case_label = std::nullopt) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`sheet_name` : `std::string` Worksheet name to read from or write to inside an Excel workbook.

`add_cases` : `bool` When `true`, the current case axis is exported as its own column so row labels can be reconstructed when the file is loaded again.

`header` : `bool` Controls whether feature names are written to, or expected from, the file header row.

`na_rep` : `std::string` String written anywhere a null or missing value must be serialized.

`case_label` : `std::optional<std::string>` Optional column label to use for the exported case axis when `add_cases=true`.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to an Excel sheet. Set `add_cases=true` when you want the current case axis exported as a regular column.

See also

- `void save_excel(self, std::string_view filepath) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::WriteExcelOptions& options) const`
- `void save_excel(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_excel(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

8.1.202 save_json

Signature

```
void save_json(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to JSON using the table export route for later reload or interchange.

See also

- void save_json(self, std::string_view filepath, const pluss::loader::Config& config) const
- void save_json(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_json("path/to/output.json");
```

Output

```
saved: out.*
```

8.1.203 save_json

Signature

```
void save_json(self, std::string_view filepath, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to JSON using the table export route for later reload or interchange.

See also

- void save_json(self, std::string_view filepath) const
- void save_json(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_json("path/to/output.json");
```

Output

```
saved: out.*
```

8.1.204 save_json

Signature

```
void save_json(self, std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to JSON using the table export route for later reload or interchange.

See also

- `void save_json(self, std::string_view filepath) const`
- `void save_json(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_json("path/to/output.json");
```

Output

```
saved: out.*
```

8.1.205 save_parquet

Signature

```
void save_parquet(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to Parquet for compact typed storage.

See also

- `void save_parquet(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_parquet(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_parquet("path/to/output.parquet");
```

Output

```
saved: out.*
```

8.1.206 save_parquet

Signature

```
void save_parquet(self, std::string_view filepath, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to Parquet for compact typed storage.

See also

- `void save_parquet(self, std::string_view filepath) const`
- `void save_parquet(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_parquet("path/to/output.parquet");
```

Output

```
saved: out.*
```

8.1.207 save_parquet

Signature

```
void save_parquet(self, std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to Parquet for compact typed storage.

See also

- `void save_parquet(self, std::string_view filepath) const`
- `void save_parquet(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_parquet("path/to/output.parquet");
```

Output

```
saved: out.*
```

8.1.208 `save_txt`

Signature

```
void save_txt(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to a plain-text table preview suitable for inspection or lightweight logging.

See also

- `void save_txt(self, std::string_view filepath, const pluss::loader::Config& config) const`
- `void save_txt(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_txt("path/to/output.txt");
```

Output

```
saved: out.*
```

8.1.209 save_txt

Signature

```
void save_txt(self, std::string_view filepath, const pluss::loader::Config& config) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to a plain-text table preview suitable for inspection or lightweight logging.

See also

- `void save_txt(self, std::string_view filepath) const`
- `void save_txt(self, std::string_view filepath, std::initializer_list<pluss::loader::ConfigEntry> config_entries) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_txt("path/to/output.txt");
```

Output

```
saved: out.*
```

8.1.210 save_txt

Signature

```
void save_txt(self, std::string_view filepath, std::initializer_list
↪<pluss::loader::ConfigEntry> config_entries) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config_entries` : `std::initializer_list<pluss::loader::ConfigEntry>` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the frame to a plain-text table preview suitable for inspection or lightweight logging.

See also

- `void save_txt(self, std::string_view filepath) const`
- `void save_txt(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.save_txt("path/to/output.txt");
```

Output

```
saved: out.*
```

8.1.211 series_features

Signature

```
DataFrame series_features(self) const
```

Parameters

None

Returns

DataFrame Value produced by this call.

Notes

Returns only the numeric series-backed features.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.series_features();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.212 set

Signature

```
LetSlot set(self, std::string_view name)
```

Parameters

name : std::string_view See the signature for the supported argument form.

Returns

LetSlot Value produced by this call.

Notes

Opens an assignment helper so you can write df.set(...) = ... and replace or create features declaratively.

See also

- LetSlots set(self, std::initializer_list<std::string_view> names)

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.set("profit") = pluss::table::Thread(pluss::make_series({40.0, 35.0, 30.0}));
```

8.1.213 set

Signature

```
LetSlots set(self, std::initializer_list<std::string_view> names)
```

Parameters

names : std::initializer_list<std::string_view> Collection of feature, slot, or stream names supplied in positional order.

Returns

LetSlots Value produced by this call.

Notes

Opens an assignment helper so you can write df.set(...) = ... and replace or create features declaratively.

See also

- LetSlot set(self, std::string_view name)

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.set({"profit", "cost"}) = {pluss::table::Thread(pluss::make_series({40.0, 35.0, 30.0}
↪)), pluss::table::Thread(pluss::make_series({60.0, 85.0, 60.0}))};
```

8.1.214 set_axis

Signature

```
DataFrame set_axis(self, std::initializer_list<std::string_view> feature_names) const
```

Parameters

`feature_names` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Replaces the full feature label set with the supplied names in positional order.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.set_axis({"revenue", "location", "id_new"});
```

Output

```
id  revenue  city
1   100.0    Accra
2   120.0    Tema
3    90.0    Kumasi
```

8.1.215 set_cases

Signature

```
DataFrame set_cases(self, std::string_view feature_name, bool drop_feature = true) const
```

Parameters

`feature_name` : `std::string_view` Name of the target feature to read, rewrite, sort by, or promote to cases.

`drop_feature` : `bool` When `true`, the feature promoted to cases is removed from the ordinary feature columns.

Returns

`DataFrame` Value produced by this call.

Notes

Promotes one feature to become the case axis. With the default `drop_feature=true`, that feature stops being a normal column and becomes the row labels instead.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.set_cases("city");
```

Output

```
df.head():
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi

out.cases():
["Accra", "Tema", "Kumasi"]

out.head():
id  sales
1   100.0
2   120.0
3    90.0
```

8.1.216 set_feature_axis

Signature

```
DataFrame set_feature_axis(self, std::initializer_list<std::string_view> feature_names)␣
↪const
```

Parameters

`feature_names` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Replaces the full feature label set with the supplied names in positional order.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
```

(continues on next page)

(continued from previous page)

```
});

auto out = df.set_feature_axis({"revenue", "location", "id_new"});
```

Output

```
id  revenue  city
1   100.0    Accra
2   120.0    Tema
3    90.0    Kumasi
```

8.1.217 set_features

Signature

```
DataFrame set_features(self, std::initializer_list<std::string_view> names,␣
↪std::initializer_list<Thread> values) const
```

Parameters

names : std::initializer_list<std::string_view> Collection of feature, slot, or stream names supplied in positional order.

values : std::initializer_list<Thread> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Returns a new frame with one added or replaced feature.

See also

- DataFrame set_features(self, std::string_view name, const Thread& value) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.set_features("profit", pluss::table::Thread(pluss::make_series({40.0, 35.0,
↪ 30.0})));
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

8.1.218 set_features

Signature

```
DataFrame set_features(self, std::string_view name, const Thread& value) const
```

Parameters

`name` : `std::string_view` See the signature for the supported argument form.

`value` : `const Thread&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Returns a new frame with one added or replaced feature.

See also

- `DataFrame set_features(self, std::initializer_list<std::string_view> names, std::initializer_list<Thread> values) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.set_features("profit", pluss::table::Thread(pluss::make_series({40.0, 35.0,
↪ 30.0})));
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

8.1.219 set_features_

Signature

```
DataFrame& set_features_(self, std::initializer_list<std::string_view> names,␣
↪std::initializer_list<Thread> values)
```

Parameters

`names` : `std::initializer_list<std::string_view>` Collection of feature, slot, or stream names supplied in positional order.

`values` : `std::initializer_list<Thread>` See the signature for the supported argument form.

Returns

`DataFrame&` Value produced by this call.

Notes

Adds or replaces one feature in place.

See also

- `DataFrame& set_features_(self, std::string_view name, const Thread& value)`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.set_features_("profit", pluss::table::Thread(pluss::make_series({40.0, 35.0, 30.0})));
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

8.1.220 set_features_

Signature

```
DataFrame& set_features_(self, std::string_view name, const Thread& value)
```

Parameters

`name` : `std::string_view` See the signature for the supported argument form.

`value` : `const Thread&` See the signature for the supported argument form.

Returns

DataFrame& Value produced by this call.

Notes

Adds or replaces one feature in place.

See also

- DataFrame& set_features_(self, std::initializer_list<std::string_view> names, std::initializer_list<Thread> values)

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

df.set_features_("profit", pluss::table::Thread(pluss::make_series({40.0, 35.0, 30.0})));
```

Output

```
id  sales  city    profit
1   100.0  Accra   40.0
2   120.0  Tema    35.0
3    90.0  Kumasi  30.0
```

8.1.221 shift

Signature

```
DataFrame shift(self, std::initializer_list<std::string_view> features, std::int64_t␣
↪periods = 1) const
```

Parameters

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

periods : std::int64_t See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Applies shift to the current DataFrame object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- DataFrame shift(self, std::int64_t periods = 1) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.shift();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.222 shift

Signature

```
DataFrame shift(self, std::int64_t periods = 1) const
```

Parameters

periods : std::int64_t See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Applies shift to the current DataFrame object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- DataFrame shift(self, std::initializer_list<std::string_view> features, std::int64_t periods = 1) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.shift();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.223 shift_time

Signature

```
DataFrame shift_time(self, std::initializer_list<std::string_view> features, int periods,
↪ std::string_view freq) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`periods` : `int` See the signature for the supported argument form.

`freq` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.shift_time({"ts"}, 1, "hour");
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

8.1.224 show

Signature

```
void show(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- `void show(self, std::ostream& os, const display::DisplayOptions& options =`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.show();
```

8.1.225 show

Signature

```
void show(self, std::ostream& os, const display::DisplayOptions& options =
```

Parameters

`os` : `std::ostream&` See the signature for the supported argument form.

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- `void show(self, const display::DisplayOptions& options =`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.show();
```

8.1.226 slice

Signature

```
DataFrame slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` Inclusive starting position of the slice.

`stop` : `std::int64_t` Exclusive ending position of the slice.

Returns

`DataFrame` Value produced by this call.

Notes

Returns a contiguous row slice `[start, stop)` from the frame.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.slice(0, 2);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.227 sort_cases

Signature

```
DataFrame sort_cases(self, bool ascending = true, bool nulls_last = true) const
```

Parameters

`ascending` : `bool` See the signature for the supported argument form.

`nulls_last` : `bool` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Sorts rows by the current case axis rather than by a data feature.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto with_cases = df.with_cases(pluss::table::Thread(pluss::make_note({"cs2", "cs0", "cs1
↪"})));
auto out = with_cases.sort_cases(true, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.228 sort_values

Signature

```
DataFrame sort_values(self, std::initializer_list<ArrangeSpec> specs) const
```

Parameters

`specs` : `std::initializer_list<ArrangeSpec>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Reorders rows using one or more feature values as sort keys.

See also

- DataFrame sort_values(self, std::initializer_list<std::string_view> features, bool ascending = true, bool nulls_last = true) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.sort_values({{.feature = "sales", .ascending = false}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.229 sort_values

Signature

```
DataFrame sort_values(self, std::initializer_list<std::string_view> features, bool␣
↪ascending = true, bool nulls_last = true) const
```

Parameters

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

ascending : bool See the signature for the supported argument form.

nulls_last : bool See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Reorders rows using one or more feature values as sort keys.

See also

- DataFrame sort_values(self, std::initializer_list<ArrangeSpec> specs) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
```

(continues on next page)

(continued from previous page)

```
});

auto out = df.sort_values({"sales"}, false, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.230 sql

Signature

```
DataFrameSqlAccessor sql(self) const
```

Parameters

None

Returns

`DataFrameSqlAccessor` Value produced by this call.

Notes

Use this when the transformation is easier to express as SQL than as a chained sequence of table verbs.

See also

- `DataFrameSqlAccessor sql(self, SqlOptions options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.sql().execute("SELECT id, sales FROM self WHERE sales > 95");
```

Output

```
SQL accessor / prepared statement created
```

8.1.231 sql

Signature

```
DataFrameSqlAccessor sql(self, SqlOptions options) const
```

Parameters

`options` : `SqlOptions` See the signature for the supported argument form.

Returns

`DataFrameSqlAccessor` Value produced by this call.

Notes

Use this when the transformation is easier to express as SQL than as a chained sequence of table verbs.

See also

- `DataFrameSqlAccessor sql(self) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.sql().execute("SELECT id, sales FROM self WHERE sales > 95");
```

Output

```
SQL accessor / prepared statement created
```

8.1.232 `std`

Signature

```
DataFrameView std(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Computes per-feature standard deviations for the selected numeric features.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.std(false);
```

Output

```
feature  value
sales    310.0
```

8.1.233 stitch_on

Signature

```
DataFrame stitch_on(self, std::initializer_list<std::string_view> left_features, const␣
↪DataFrame& right, std::initializer_list<std::string_view> right_features =
```

Parameters

`left_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`right` : `const DataFrame&` See the signature for the supported argument form.

`right_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Matches rows from two frames using key features and appends the matched features from the right-hand frame.

See also

- `DataFrame stitch_on(self, std::string_view left_feature, const DataFrame& right, std::string_view right_feature = "", std::string right_suffix = "_right") const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
```

(continues on next page)

(continued from previous page)

```
});

auto out = left.stitch_on({"id"}, right, {"id"}, "_right");
```

Output

```
id  sales  cost
1   100.0  60.0
2   120.0  80.0
3    90.0  55.0
```

8.1.234 stitch_on

Signature

```
DataFrame stitch_on(self, std::string_view left_feature, const DataFrame& right,␣
↪std::string_view right_feature = "", std::string right_suffix = "_right") const
```

Parameters

`left_feature` : `std::string_view` See the signature for the supported argument form.

`right` : `const DataFrame&` See the signature for the supported argument form.

`right_feature` : `std::string_view` See the signature for the supported argument form.

`right_suffix` : `std::string` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Matches rows from two frames using key features and appends the matched features from the right-hand frame.

See also

- `DataFrame stitch_on(self, std::initializer_list<std::string_view> left_features, const DataFrame& right, std::initializer_list<std::string_view> right_features =`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.stitch_on("id", right, "id", "_right");
```

Output

```
id  sales  cost
1   100.0  60.0
2   120.0  80.0
3    90.0  55.0
```

8.1.235 sub

Signature

```
DataFrame sub(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame sub(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.sub(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.236 sub

Signature

```
DataFrame sub(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame sub(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.sub(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.237 `subtract`

Signature

```
DataFrame subtract(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame subtract(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.subtract(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.238 subtract

Signature

```
DataFrame subtract(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame subtract(self, const DataFrame& other) const`

Examples

```cpp
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.subtract(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.239 sum

Signature

```cpp
DataFrameView sum(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Computes per-feature sums for the selected numeric features.

Examples

```cpp
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.sum(false);
```

Output

```
feature  value
sales    310.0
```

8.1.240 tabelize

Signature

```
static DataFrame tabelize(const pluss::kindle::Catalog& catalog)
```

Parameters

`catalog` : `const pluss::kindle::Catalog&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `static DataFrame tabelize(const pluss::kindle::Catalog& catalog, const pluss::kindle::TabelizePolicy& policy)`

Examples

```
using namespace pluss::kindle::literals;
auto catalog = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_
↪series({10.0, 20.0, 30.0}))});
auto out = pluss::table::DataFrame::tabelize(catalog);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.241 tabelize

Signature

```
static DataFrame tabelize(const pluss::kindle::Catalog& catalog, const␣
↪pluss::kindle::TabelizePolicy& policy)
```

Parameters

`catalog` : `const pluss::kindle::Catalog&` See the signature for the supported argument form.

`policy` : `const pluss::kindle::TabelizePolicy&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `static DataFrame tabelize(const pluss::kindle::Catalog& catalog)`

Examples

```
using namespace pluss::kindle::literals;
auto catalog = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_
↪series({10.0, 20.0, 30.0}))});
auto out = pluss::table::DataFrame::tabelize(catalog);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.242 `tail`

Signature

```
DataFrameView tail(self, std::int64_t n = 5, bool render = true) const
```

Parameters

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Returns the last `n` rows of the frame.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.tail(2, false);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.243 take

Signature

```
DataFrame take(self, std::span<const std::int64_t> indices) const
```

Parameters

`indices` : `std::span<const std::int64_t>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Returns the rows at the requested integer positions.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

std::vector<std::int64_t> picks{0, 2};
auto out = df.take(picks);
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

8.1.244 tally

Signature

```
DataFrameView tally(self, std::initializer_list<std::string_view> features, std::int64_t␣
↪top_n = 0, bool dropna = true, bool render = true) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`top_n` : `std::int64_t` See the signature for the supported argument form.

`dropna` : `bool` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Counts how many rows fall into each distinct value of the grouping feature.

See also

- `DataFrameView tally(self, std::int64_t top_n = 0, bool dropna = true, bool render = true) const`
- `DataFrameView tally(self, std::string_view feature, std::int64_t top_n = 0, bool dropna = true, bool render = true) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.tally("city", 3, true, false);
```

Output

```
city    count
Accra   2
Tema    2
```

8.1.245 `tally`

Signature

```
DataFrameView tally(self, std::int64_t top_n = 0, bool dropna = true, bool render =␣
↪true) const
```

Parameters

`top_n` : `std::int64_t` See the signature for the supported argument form.

`dropna` : `bool` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Counts how many rows fall into each distinct value of the grouping feature.

See also

- `DataFrameView tally(self, std::initializer_list<std::string_view> features, std::int64_t top_n = 0, bool dropna = true, bool render = true) const`
- `DataFrameView tally(self, std::string_view feature, std::int64_t top_n = 0, bool dropna = true, bool render = true) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.tally("city", 3, true, false);
```

Output

```
city    count
Accra   2
Tema    2
```

8.1.246 `tally`

Signature

```
DataFrameView tally(self, std::string_view feature, std::int64_t top_n = 0, bool dropna
↪= true, bool render = true) const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`top_n` : `std::int64_t` See the signature for the supported argument form.

`dropna` : `bool` See the signature for the supported argument form.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Counts how many rows fall into each distinct value of the grouping feature.

See also

- `DataFrameView tally(self, std::int64_t top_n = 0, bool dropna = true, bool render = true) const`
- `DataFrameView tally(self, std::initializer_list<std::string_view> features, std::int64_t top_n = 0, bool dropna = true, bool render = true) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Accra", "Tema", "Tema"}))},
    {"product", pluss::table::Thread(pluss::make_note({"A", "B", "A", "B"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0, 150.0}))},
});

auto out = df.tally("city", 3, true, false);
```

Output

```
city    count
Accra   2
Tema    2
```

8.1.247 timeline_features

Signature

```
DataFrame timeline_features(self) const
```

Parameters

None

Returns

`DataFrame` Value produced by this call.

Notes

Returns only the timeline-backed features.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.timeline_features();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.248 `title`

Signature

```
DataFrame title(self, std::string title) const
```

Parameters

`title` : `std::string` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Applies `title` to the current `DataFrame` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `std::string title(self) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.title();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.249 `title`

Signature

```
std::string title(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

Applies `title` to the current `DataFrame` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `DataFrame title(self, std::string title) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.title();
```

8.1.250 `title_`

Signature

```
DataFrame& title_(self, std::string title)
```

Parameters

`title` : `std::string` See the signature for the supported argument form.

Returns

`DataFrame&` Value produced by this call.

Notes

Applies `title_` to the current `DataFrame` object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.title_();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.251 to_string

Signature

```
std::string to_string(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.to_string();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.252 to_utc

Signature

```
DataFrame to_utc(self, std::initializer_list<std::string_view> features) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.to_utc({"ts"});
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

8.1.253 top

Signature

```
DataFrame top(self, std::string_view feature, std::int64_t n) const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

Returns

`DataFrame` Value produced by this call.

Notes

Returns the rows with the largest values in the selected feature.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto out = df.top("sales", 2);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.254 `transform`

Signature

```
DataFrame transform(self, const std::function<Thread(const Thread&)>& fn) const
```

Parameters

`fn` : `const std::function<Thread(const Thread&)>&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Rewrites one feature, several features, or all features by applying a user function to each targeted thread.

See also

- `DataFrame transform(self, std::string_view feature, const std::function<Thread(const Thread&)>& fn) const`
- `DataFrame transform(self, std::initializer_list<TransformSpec> specs) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.transform([](const pluss::table::Thread& thread) { return thread.kind() ==␣
↪pluss::table::ThreadKind::Series ? thread.clip(0.0, 110.0) : thread; });
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.255 transform

Signature

```
DataFrame transform(self, std::initializer_list<TransformSpec> specs) const
```

Parameters

specs : std::initializer_list<TransformSpec> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Rewrites one feature, several features, or all features by applying a user function to each targeted thread.

See also

- DataFrame transform(self, const std::function<Thread(const Thread&)>& fn) const
- DataFrame transform(self, std::string_view feature, const std::function<Thread(const Thread&)>& fn) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.transform({{.feature = "sales", .expr = [](const pluss::table::Thread&␣
↪thread) { return thread.clip(0.0, 110.0); }}});
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.256 transform

Signature

```
DataFrame transform(self, std::string_view feature, const std::function<Thread(const␣
↪Thread&)>& fn) const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`fn` : `const std::function<Thread(const Thread&)>&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Rewrites one feature, several features, or all features by applying a user function to each targeted thread.

See also

- `DataFrame transform(self, const std::function<Thread(const Thread&)>& fn) const`
- `DataFrame transform(self, std::initializer_list<TransformSpec> specs) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.transform("sales", [](const pluss::table::Thread& thread) { return thread.
↪clip(0.0, 110.0); });
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.257 `transpose`

Signature

```
DataFrameView transpose(self, bool render = false) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Swaps rows and features so former feature names become row labels.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.transpose(false);
```

Output

```
feature  row0   row1   row2
id       1      2      3
sales    100.0  120.0  90.0
```

8.1.258 truediv

Signature

```
DataFrame truediv(self, const DataFrame& other) const
```

Parameters

`other` : `const DataFrame&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame truediv(self, scalar other) const`

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.truediv(right);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.259 truediv

Signature

```
DataFrame truediv(self, scalar other) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `DataFrame truediv(self, const DataFrame& other) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.truediv(10);
```

Output

```
id  sales  city
11  110.0  Accra
12  130.0  Tema
13  100.0  Kumasi
```

8.1.260 tz_convert

Signature

```
DataFrame tz_convert(self, std::initializer_list<std::string_view> features, std::string_
↪view tz_name) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`tz_name` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.tz_convert({"ts"}, "America/Edmonton");
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

8.1.261 `tz_localize`

Signature

```
DataFrame tz_localize(self, std::initializer_list<std::string_view> features,␣
↪std::string_view tz_name, int fold = 0, std::string_view nonexistent = "raise") const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`tz_name` : `std::string_view` See the signature for the supported argument form.

`fold` : `int` See the signature for the supported argument form.

`nonexistent` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto df = pluss::table::DataFrame({
    {"ts", pluss::table::Thread(pluss::make_timeline({
        "2024-01-01T09:10:00Z",
        "2024-01-01T09:40:00Z",
        "2024-01-01T10:05:00Z",
    }))},
    {"load", pluss::table::Thread(pluss::make_series({10.0, 20.0, 15.0}))},
});

auto out = df.tz_localize({"ts"}, "America/Edmonton", 0, "shift_forward");
```

Output

```
ts
2024-01-01T09:00:00Z
2024-01-01T10:00:00Z
```

8.1.262 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.unique();
```

Output

```
true
```

8.1.263 unique_cases

Signature

```
DataFrame unique_cases(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Keeps only the first occurrence of each case label and drops later duplicates.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.unique_cases();
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.264 unsqueeze

Signature

```
Atlas unsqueeze(self) const
```

Parameters

None

Returns

`Atlas` Value produced by this call.

Notes

Promotes the frame into a depth-1 `Atlas` so it can participate in multi-frame workflows.

See also

- `Atlas unsqueeze(self, const Thread& depth_labels) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.unsqueeze();
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

8.1.265 unsqueeze

Signature

```
Atlas unsqueeze(self, const Thread& depth_labels) const
```

Parameters

`depth_labels` : `const Thread&` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

Promotes the frame into a depth-1 `Atlas` so it can participate in multi-frame workflows.

See also

- Atlas unsqueeze(self) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto labels = pluss::table::Thread(pluss::make_note({"train"}));
auto out = df.unsqueeze(labels);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

8.1.266 update

Signature

```
DataFrame update(self, const DataFrame& other, bool overwrite = true) const
```

Parameters

other : const DataFrame& See the signature for the supported argument form.

overwrite : bool See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Overlays matching features from another frame onto this frame using aligned cases.

Examples

```
auto left = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
});
auto right = pluss::table::DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"cost", pluss::table::Thread(pluss::make_series({60.0, 80.0, 55.0}))},
});

auto out = left.update(right);
```

Output

```
id  sales  cost
1   100.0  60.0
2   120.0  80.0
3    90.0  55.0
```

8.1.267 values

Signature

```
std::vector<Thread> values(self) const
```

Parameters

None

Returns

std::vector<Thread> Materialized collection of thread objects.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.values();
```

Output

```
[Thread(id), Thread(sales), Thread(city)]
```

8.1.268 var

Signature

```
DataFrameView var(self, bool render = true) const
```

Parameters

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Computes per-feature variances for the selected numeric features.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.var(false);
```

Output

```
feature  value
sales    310.0
```

8.1.269 `view`

Signature

```
DataFrameView view(self, std::int64_t start, std::int64_t stop, bool render = true) const
```

Parameters

`start` : `std::int64_t` Inclusive starting position of the slice.

`stop` : `std::int64_t` Exclusive ending position of the slice.

`render` : `bool` See the signature for the supported argument form.

Returns

`DataFrameView` Value produced by this call.

Notes

Returns a lightweight row view of the requested slice.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
```

(continues on next page)

(continued from previous page)

```
});

auto out = df.view(0, 2, false);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.270 where

Signature

```
DataFrame where(self, const pluss::Series& bool_mask) const
```

Parameters

bool_mask : const pluss::Series& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Keeps only the rows where the supplied boolean mask is true.

See also

- DataFrame where(self, const Thread& bool_mask) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = df.where(mask);
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

8.1.271 where

Signature

```
DataFrame where(self, const Thread& bool_mask) const
```

Parameters

bool_mask : const Thread& See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Keeps only the rows where the supplied boolean mask is true.

See also

- DataFrame where(self, const pluss::Series& bool_mask) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = df.where(mask);
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

8.1.272 window

Signature

```
DataFrame window(self, std::initializer_list<WindowSpec> specs, bool keep_input = true)␣
↪const
```

Parameters

specs : std::initializer_list<WindowSpec> See the signature for the supported argument form.

keep_input : bool See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Computes one or more window-based features such as rolling means over selected columns.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = df.window({{.output = "sales_roll", .feature = "sales", .op =␣
↪pluss::table::ThreadWindowOp::RollingMean, .window = 2, .min_periods = 1}}, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.273 with_cases

Signature

```
DataFrame with_cases(self, const Thread& cases) const
```

Parameters

cases : const Thread& Case-label thread to attach to the current object.

Returns

DataFrame Value produced by this call.

Notes

Replaces the current case axis with the supplied thread. The row values stay in the same order, but later case-aware operations now use the new labels.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto cases = pluss::table::Thread(pluss::make_note({"cs0", "cs1", "cs2"}));
auto out = df.with_cases(cases);
```

Output

```
df.cases():
[0, 1, 2]

out.cases():
["cs0", "cs1", "cs2"]

out.head():
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

8.1.274 with_cases_

Signature

```
DataFrame& with_cases_(self, const Thread& cases)
```

Parameters

`cases` : `const Thread&` Case-label thread to attach to the current object.

Returns

`DataFrame&` Value produced by this call.

Notes

Replaces the current case axis in place. The row values stay in the same order, but later case-aware operations now use the new labels.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto cases = pluss::table::Thread(pluss::make_note({"cs0", "cs1", "cs2"}));
df.with_cases_(cases);
```

Output

```
df.cases():
["cs0", "cs1", "cs2"]
```

(continues on next page)

(continued from previous page)

```
df.head():
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

CHAPTER

NINE

COLUMNREF API REFERENCE

CHAPTER

TEN

PRACTICAL COLUMNREF API REFERENCE

10.1 API Reference Style

This page documents the public `ColumnRef` helper returned by DataFrame column-reference calls.

10.1.1 head

Signature

```
Thread head(self, std::int64_t n) const
```

Parameters

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

Returns

`Thread` Value produced by this call.

Notes

Returns the first `n` values from the referenced column as a thread.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = df.feature_ref("sales");

auto out = ref.head(2);
```

Output

```
[100.0, 120.0]
```

10.1.2 mean

Signature

```
pluss::Series mean(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

Computes the mean of the referenced numeric column.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = df.feature_ref("sales");

auto out = ref.mean();
```

Output

```
103.3
```

10.1.3 name

Signature

```
const std::string& name(self) const
```

Parameters

None

Returns

`const std::string&` Value produced by this call.

Notes

Returns the feature name associated with this column reference.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = df.feature_ref("sales");

auto out = ref.name();
```

Output

```
sales
```

10.1.4 thread

Signature

```
const Thread& thread(self) const
```

Parameters

None

Returns

`const Thread&` Value produced by this call.

Notes

Returns the referenced column as a `Thread` object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = df.feature_ref("sales");

auto out = ref.thread();
```

Output

```
[10.0, 20.0, 30.0]
```

10.1.5 value

Signature

```
ThreadValue value(self) const
```

Parameters

None

Returns

`ThreadValue` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = df.feature_ref("sales");

auto out = ref.value();
```

Output

```
100.0
```

10.1.6 values

Signature

```
Thread values(self) const
```

Parameters

None

Returns

`Thread` Value produced by this call.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto ref = df.feature_ref("sales");

auto out = ref.values();
```

Output

```
[10.0, 20.0, 30.0]
```

CHAPTER

ELEVEN

DATAFRAMEPLOTACCESSOR API REFERENCE

CHAPTER

TWELVE

PRACTICAL DATAFRAMEPLOTACCESSOR API REFERENCE

12.1 API Reference Style

This page documents the public plotting accessor returned by `DataFrame::plot()` and `Atlas::plot()`.

12.2 `df.plot(...)` Defaults And Options

12.2.1 Signature

```
DataFramePlotAccessor plot(self)
```

```
DataFramePlotAccessor plot(self, DataFramePlotOptions options)
```

12.2.2 What `DataFramePlotOptions` Controls

`default_x_feature` (**`std::optional<std::string>`, default `nullopt`**)
Default x-axis feature used when a plot call does not provide an explicit `x_feature`.

`default_hue_feature` (**`std::optional<std::string>`, default `nullopt`**)
Default grouping/color feature used by plot types that support hue/group splits.

`default_bins` (**`std::int64_t`, default `20`**)
Default bin count for histogram-style plots when bins are not explicitly passed.

`datetime_label_view` (**`DateTimeLabelView`, default `DateTimeLabelView::Full`**)
Datetime tick label rendering mode (`Full`, `Date`, or `Time`) for datetime axes.

`max_labeled_ticks` (**`std::int64_t`, default `14`**)
Upper bound on how many tick labels are rendered to avoid dense, unreadable axes.

`x_tick_rotation_deg` (**`std::optional<double>`, default `nullopt`**)
Optional x-axis tick-label rotation (degrees).

`y_tick_rotation_deg` (**`std::optional<double>`, default `nullopt`**)
Optional y-axis tick-label rotation (degrees).

`x_label_rotation_deg` (**`std::optional<double>`, default `nullopt`**)
Optional x-axis label rotation (degrees).

`y_label_rotation_deg` (**`std::optional<double>`, default `nullopt`**)
Optional y-axis label rotation (degrees).

`labeled_tick_rotation_deg` (**`std::optional<double>`, default `nullopt`**)
Legacy x-axis tick-label rotation alias. Prefer `x_tick_rotation_deg`.

`width` (**`std::optional<double>`, default `nullopt`**)
Optional figure width override.

height (**std::optional<double>**, **default nullopt**)
Optional figure height override.

use_tight_layout (**bool**, **default false**)
When enabled, applies tight layout spacing rules to reduce label clipping.

tight_layout_padding (**double**, **default 6.0**)
Padding used by tight layout when `use_tight_layout=true`.

12.2.3 Usage Pattern

```
using namespace pluss::table;

DataFramePlotOptions opts{};
opts.default_x_feature = "timestamp";
opts.default_hue_feature = "sector";
opts.default_bins = 30;
opts.datetime_label_view = DateTimeLabelView::Date;
opts.max_labeled_ticks = 10;
opts.x_tick_rotation_deg = 35.0;
opts.y_tick_rotation_deg = 20.0;
opts.x_label_rotation_deg = 0.0;
opts.y_label_rotation_deg = -90.0;
opts.width = 13.0;
opts.height = 7.0;
opts.use_tight_layout = true;
opts.tight_layout_padding = 4.0;

auto plot = df.plot(opts);
auto line_fig = plot.line({"open", "close"});
auto hist_fig = plot.hist("returns");
```

12.2.4 Output

```
Figure(...)
Figure(...)
```

12.3 Method-Level Options Reference

All method options are regular C++ structs. Create, edit fields, then pass into the corresponding `plot.*(...)` call.

```
pluss::table::ActPlotOptions o{};
o.max_lag = 60;
o.confidence_level = 0.99;
auto fig = df.plot().act("returns", o);
```

12.3.1 LinePlotOptions

`LinePlotOptions(alpha=1.0, line_width=1.6, y_limit_method=pluss::pl::LimitMethod::Padded)`

- `alpha` (`double`): line opacity.
- `line_width` (`double`): line thickness.

- y_limit_method (std::optional<pluss::pl::LimitMethod>): y-axis limit strategy.

12.3.2 StepPlotOptions

StepPlotOptions(alpha=1.0, line_width=1.5)

- alpha (double): step trace opacity.
- line_width (double): step trace thickness.

12.3.3 AreaPlotOptions

AreaPlotOptions(alpha=0.88, line_width=1.3)

- alpha (double): fill opacity.
- line_width (double): boundary line thickness.

12.3.4 AreaStackedPlotOptions

AreaStackedPlotOptions(alpha=0.7, line_width=1.2)

- alpha (double): stacked area opacity.
- line_width (double): boundary line thickness.

12.3.5 ScatterPlotOptions

ScatterPlotOptions(alpha=0.92, line_width=1.1)

- alpha (double): marker opacity.
- line_width (double): marker edge width.

12.3.6 BarPlotOptions

BarPlotOptions(ci_enabled=false, ci_level=0.95, horizontal=false)

- ci_enabled (bool): enable confidence interval bars.
- ci_level (double): confidence level when CI is enabled.
- horizontal (bool): draw horizontal bars.

12.3.7 BarStackedPlotOptions

BarStackedPlotOptions(alpha=0.65, line_width=1.0)

- alpha (double): stacked bar opacity.
- line_width (double): bar edge thickness.

12.3.8 BoxPlotOptions

BoxPlotOptions(alpha=0.95, line_width=1.1)

- alpha (double): box body opacity.
- line_width (double): outline thickness.

12.3.9 ViolinPlotOptions

`ViolinPlotOptions(alpha=0.92, line_width=1.1, width=0.8, bw_adjust=1.0, cut=2.0, scale=pluss::pl::ViolinScale::Area, inner=pluss::pl::ViolinInner::Box, side=pluss::pl::ViolinSide::Full)`

- `alpha` (`double`): violin fill opacity.
- `line_width` (`double`): outline thickness.
- `width` (`double`): violin width scale.
- `bw_adjust` (`double`): KDE bandwidth adjustment.
- `cut` (`double`): tail extension in bandwidth units.
- `scale` (`pluss::pl::ViolinScale`): area/count/width scaling behavior.
- `inner` (`pluss::pl::ViolinInner`): inner annotation style.
- `side` (`pluss::pl::ViolinSide`): full/left/right rendering.

12.3.10 HistPlotOptions

`HistPlotOptions(bins=20, alpha=0.95, line_width=1.0)`

- `bins` (`std::int64_t`): histogram bin count.
- `alpha` (`double`): bar opacity.
- `line_width` (`double`): bar edge thickness.

12.3.11 DensityPlotOptions

`DensityPlotOptions(points=128, alpha=1.0, line_width=1.5)`

- `points` (`std::int64_t`): sampled KDE points.
- `alpha` (`double`): curve opacity.
- `line_width` (`double`): curve thickness.

12.3.12 HeatmapPlotOptions

`HeatmapPlotOptions(annotate_levels=false, scale_guide=true, borderless=false)`

- `annotate_levels` (`bool`): print values in cells.
- `scale_guide` (`bool`): show color scale legend.
- `borderless` (`bool`): remove cell border strokes.

12.3.13 CorrHeatmapPlotOptions

`CorrHeatmapPlotOptions(annotate_levels=true, scale_guide=true, borderless=false)`

- `annotate_levels` (`bool`): print correlation values in cells.
- `scale_guide` (`bool`): show color scale legend.
- `borderless` (`bool`): remove cell borders.

12.3.14 ScatterMatrixPlotOptions

`ScatterMatrixPlotOptions(corner=false, pair={})`

- `corner` (`bool`): render only one matrix triangle.
- `pair` (`pluss::pl::flash::PairPlotOptions`): low-level pair-plot styling options.

12.3.15 ParallelPlotOptions

`ParallelPlotOptions(labels={}, normalize=true, max_points=1200, alpha=0.24, trace_width=0.9f, show_grid=true, show_legend=true)`

- `labels` (`std::vector<std::string>`): display labels for classes/groups.
- `normalize` (`bool`): scale features before plotting.
- `max_points` (`std::int64_t`): cap row count rendered.
- `alpha` (`double`): polyline opacity.
- `trace_width` (`float`): polyline thickness.
- `show_grid` (`bool`): show axis grid lines.
- `show_legend` (`bool`): show legend.

12.3.16 AndrewsCurvesPlotOptions

`AndrewsCurvesPlotOptions(labels={}, normalize=true, points=220, max_curves=600, alpha=0.28, trace_width=0.9f, show_grid=true, show_legend=true)`

- `labels` (`std::vector<std::string>`): class labels.
- `normalize` (`bool`): normalize feature scales.
- `points` (`std::int64_t`): points per curve.
- `max_curves` (`std::int64_t`): max curves rendered.
- `alpha` (`double`): curve opacity.
- `trace_width` (`float`): curve thickness.
- `show_grid` (`bool`): show grid.
- `show_legend` (`bool`): show legend.

12.3.17 RadvizPlotOptions

`RadvizPlotOptions(labels={}, normalize=true, max_points=3000, marker_area=18.0, trace_width=0.55f, show_grid=true, show_anchors=true, show_legend=true)`

- `labels` (`std::vector<std::string>`): class labels.
- `normalize` (`bool`): normalize features before projection.
- `max_points` (`std::int64_t`): max points rendered.
- `marker_area` (`double`): marker size.
- `trace_width` (`float`): marker edge width.
- `show_grid` (`bool`): show grid.
- `show_anchors` (`bool`): show anchor positions.

- show_legend (bool): show legend.

12.3.18 LagPlotOptions

LagPlotOptions(lag=1, max_points=2500, marker_area=16.0, trace_width=0.55f)

- lag (std::int64_t): lag step.
- max_points (std::int64_t): max points rendered.
- marker_area (double): marker size.
- trace_width (float): marker edge width.

12.3.19 ActPlotOptions

ActPlotOptions(max_lag=40, show_confidence_band=true, confidence_level=0.95, trace_width=1.2f)

- max_lag (std::int64_t): max lag for ACF/PACF style plot.
- show_confidence_band (bool): draw confidence band.
- confidence_level (double): confidence level for band.
- trace_width (float): trace line width.

12.3.20 BootstrapPlotOptions

BootstrapPlotOptions(samples=300, sample_size=0, seed=7, bins=24, statistic=BootstrapStatistic::Mean, trace_width=0.85f)

- samples (std::int64_t): bootstrap draw count.
- sample_size (std::int64_t): sample size per draw; 0 means default dataset size.
- seed (std::uint64_t): RNG seed.
- bins (int): histogram bins for bootstrap distribution.
- statistic (BootstrapStatistic): one of Mean, Median, Std.
- trace_width (float): trace width.

12.3.21 PiePlotOptions

PiePlotOptions(show_label=true, show_percent=true, show_value=false, precision=1, separator=" | ")

- show_label (bool): show labels.
- show_percent (bool): show percent text.
- show_value (bool): show raw values.
- precision (int): decimal precision for labels.
- separator (std::string): label text separator.

12.3.22 AutoPlotOptions

`AutoPlotOptions()`

- No fields currently. Use default automatic chart selection.

12.3.23 3D Shared Option Structs

Plot3DSceneOptions

`Plot3DSceneOptions(preset=nullopt, theme=nullopt, camera_preset=nullopt, camera=nullopt, lighting=nullopt, grid_style=nullopt)`

- `preset` (`std::optional<pluss::pl::flash::ScenePreset3D>`): scene preset.
- `theme` (`std::optional<pluss::pl::flash::ThemePreset3D>`): 3D theme preset.
- `camera_preset` (`std::optional<pluss::pl::flash::CameraPreset3D>`): camera preset.
- `camera` (`std::optional<pluss::pl::flash::Camera3DOptions>`): explicit camera config.
- `lighting` (`std::optional<pluss::pl::flash::Lighting3DOptions>`): lighting config.
- `grid_style` (`std::optional<pluss::pl::GridStyle>`): grid style.

Plot3DTraceOptions

`Plot3DTraceOptions(preset=nullopt, shade=nullopt, lighting=nullopt, material=nullopt, line_width=nullopt, transparent=nullopt)`

- `preset` (`std::optional<pluss::pl::flash::TracePreset3D>`): trace style preset.
- `shade` (`std::optional<pluss::pl::ShadeMode>`): shading mode.
- `lighting` (`std::optional<pluss::pl::LightingMode>`): lighting mode.
- `material` (`std::optional<pluss::pl::MaterialProfile>`): material profile.
- `line_width` (`std::optional<float>`): line width override.
- `transparent` (`std::optional<bool>`): transparency override.

12.3.24 Scatter3DPlotOptions

`Scatter3DPlotOptions(scene={}, trace={}, default_marker_size=12.0)`

- `scene` (`Plot3DSceneOptions`): scene options.
- `trace` (`Plot3DTraceOptions`): trace options.
- `default_marker_size` (`double`): marker size fallback.

12.3.25 Line3DPlotOptions

`Line3DPlotOptions(scene={}, trace={})`

- `scene` (`Plot3DSceneOptions`): scene options.
- `trace` (`Plot3DTraceOptions`): trace options.

12.3.26 Hist3DPlotOptions

`Hist3DPlotOptions(bins_x=26, bins_y=20, normalize=false, smooth=false, smooth_passes=1, as_surface=false, as_wireframe=false, scene={}, trace={})`

- `bins_x` (`std::int64_t`): x-axis bin count.
- `bins_y` (`std::int64_t`): y-axis bin count.
- `normalize` (`bool`): normalize counts to density-like scale.
- `smooth` (`bool`): enable smoothing.
- `smooth_passes` (`std::int64_t`): smoothing passes.
- `as_surface` (`bool`): render as surface.
- `as_wireframe` (`bool`): render as wireframe.
- `scene` (`Plot3DSceneOptions`): scene options.
- `trace` (`Plot3DTraceOptions`): trace options.

12.3.27 Surface3DPlotOptions

`Surface3DPlotOptions(scene={}, trace={})`

- `scene` (`Plot3DSceneOptions`): scene options.
- `trace` (`Plot3DTraceOptions`): trace options.

12.3.28 Wireframe3DPlotOptions

`Wireframe3DPlotOptions(scene={}, trace={})`

- `scene` (`Plot3DSceneOptions`): scene options.
- `trace` (`Plot3DTraceOptions`): trace options.

12.3.29 Bonus: BriefPlotOptions (used by plot.brief)

`BriefPlotOptions(title="table", precision=4, include_header=true, max_cases=10, transpose=false, include_case_index=true, features_per_table=0, tables_per_row=1, table_gap_spaces=4)`

- `title` (`std::string`): figure title.
- `precision` (`int`): numeric precision.
- `include_header` (`bool`): include feature header row.
- `max_cases` (`std::int64_t`): max rows shown.
- `transpose` (`bool`): transpose preview table.
- `include_case_index` (`bool`): show case index.
- `features_per_table` (`std::int64_t`): chunk width per table block.
- `tables_per_row` (`std::int64_t`): number of blocks per row.
- `table_gap_spaces` (`std::int64_t`): spacing between blocks.

12.3.30 act

Signature

```
pluss::pl::Studio act(self,  std::string_view feature, const ActPlotOptions& options =
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`options` : `const ActPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `act` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.act();
```

Output

```
Figure(...)
```

12.3.31 andrews_curves

Signature

```
pluss::pl::Studio andrews_curves(self,  std::initializer_list<std::string_view> features␣
↪=
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `andrews_curves` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.andrews_curves();
```

Output

```
Figure(...)
```

12.3.32 area

Signature

```
pluss::pl::Studio area(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature = std::nullopt) const
```

Parameters

`y_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `area` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio area(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio area(self, std::string_view y_feature, std::optional<std::string_view> x_feature, const AreaPlotOptions& options) const`
- `pluss::pl::Studio area(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const AreaPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.area();
```

Output

```
Figure(...)
```

12.3.33 area

Signature

```
pluss::pl::Studio area(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature, const AreaPlotOptions& options) const
```

Parameters

`y_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

`options` : `const AreaPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `area` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio area(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio area(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio area(self, std::string_view y_feature, std::optional<std::string_view> x_feature, const AreaPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
```

(continues on next page)

(continued from previous page)

```
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.area();
```

Output

```
Figure(...)
```

12.3.34 area

Signature

```
pluss::pl::Studio area(self, std::string_view y_feature, std::optional<std::string_view>␣
↪x_feature = std::nullopt) const
```

Parameters

`y_feature` : `std::string_view` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `area` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio area(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio area(self, std::string_view y_feature, std::optional<std::string_view> x_feature, const AreaPlotOptions& options) const`
- `pluss::pl::Studio area(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const AreaPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.area();
```

Output

```
Figure(...)
```

12.3.35 area

Signature

```
pluss::pl::Studio area(self, std::string_view y_feature, std::optional<std::string_view>␣
↪x_feature, const AreaPlotOptions& options) const
```

Parameters

`y_feature` : `std::string_view` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

`options` : `const AreaPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `area` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio area(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio area(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio area(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const AreaPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.area();
```

Output

```
Figure(...)
```

12.3.36 area_stacked

Signature

```
pluss::pl::Studio area_stacked(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature = std::nullopt) const
```

Parameters

y_features : std::initializer_list<std::string_view> See the signature for the supported argument form.

x_feature : std::optional<std::string_view> See the signature for the supported argument form.

Returns

pluss::pl::Studio Value produced by this call.

Notes

Applies area_stacked to the current DataFramePlotAccessor object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- pluss::pl::Studio area_stacked(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const AreaStackedPlotOptions& options) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.area_stacked();
```

Output

```
Figure(...)
```

12.3.37 area_stacked

Signature

```
pluss::pl::Studio area_stacked(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature, const AreaStackedPlotOptions& options) const
```

Parameters

`y_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

`options` : `const AreaStackedPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `area_stacked` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio area_stacked(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.area_stacked();
```

Output

```
Figure(...)
```

12.3.38 auto_plot

Signature

```
pluss::pl::Studio auto_plot(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `auto_plot` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio auto_plot(self, std::initializer_list<std::string_view> features, const AutoPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.auto_plot();
```

Output

```
Figure(...)
```

12.3.39 auto_plot

Signature

```
pluss::pl::Studio auto_plot(self, std::initializer_list<std::string_view> features,␣
↪const AutoPlotOptions& options) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`options` : `const AutoPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `auto_plot` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio auto_plot(self, std::initializer_list<std::string_view> features =`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.auto_plot();
```

Output

```
Figure(...)
```

12.3.40 bar

Signature

```
pluss::pl::Studio bar(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature = std::nullopt) const
```

Parameters

`y_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a bar chart from one feature, optionally using another feature as the x-axis.

See also

- `pluss::pl::Studio bar(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio bar(self, std::string_view y_feature, std::optional<std::string_view> x_feature, const BarPlotOptions& options) const`
- `pluss::pl::Studio bar(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const BarPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();
```

(continues on next page)

(continued from previous page)

```
auto fig = plot.bar("sales", std::nullopt);
```

Output

```
Figure(...)
```

12.3.41 bar

Signature

```
pluss::pl::Studio bar(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature, const BarPlotOptions& options) const
```

Parameters

`y_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

`options` : `const BarPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a bar chart from one feature, optionally using another feature as the x-axis.

See also

- `pluss::pl::Studio bar(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio bar(self, std::string_view y_feature, std::optional<std::string_view> x_feature, const BarPlotOptions& options) const`
- `pluss::pl::Studio bar(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.bar("sales", std::nullopt);
```

Output

```
Figure(...)
```

12.3.42 bar

Signature

```
pluss::pl::Studio bar(self, std::string_view y_feature, std::optional<std::string_view>␣
↪x_feature = std::nullopt) const
```

Parameters

`y_feature` : `std::string_view` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a bar chart from one feature, optionally using another feature as the x-axis.

See also

- `pluss::pl::Studio bar(self, std::string_view y_feature, std::optional<std::string_view> x_feature, const BarPlotOptions& options) const`
- `pluss::pl::Studio bar(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio bar(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const BarPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.bar("sales", std::nullopt);
```

Output

```
Figure(...)
```

12.3.43 bar

Signature

```
pluss::pl::Studio bar(self, std::string_view y_feature, std::optional<std::string_view>␣
↪x_feature, const BarPlotOptions& options) const
```

Parameters

`y_feature` : `std::string_view` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

`options` : `const BarPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a bar chart from one feature, optionally using another feature as the x-axis.

See also

- `pluss::pl::Studio bar(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio bar(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio bar(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const BarPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.bar("sales", std::nullopt);
```

Output

```
Figure(...)
```

12.3.44 bar_stacked

Signature

```
pluss::pl::Studio bar_stacked(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature = std::nullopt) const
```

Parameters

`y_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `bar_stacked` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio bar_stacked(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const BarStackedPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.bar_stacked();
```

Output

```
Figure(...)
```

12.3.45 bar_stacked

Signature

```
pluss::pl::Studio bar_stacked(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature, const BarStackedPlotOptions& options) const
```

Parameters

`y_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

`options` : `const BarStackedPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `bar_stacked` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio bar_stacked(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.bar_stacked();
```

Output

```
Figure(...)
```

12.3.46 bootstrap

Signature

```
pluss::pl::Studio bootstrap(self,  std::string_view feature, const BootstrapPlotOptions&␣
↪options =
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`options` : `const BootstrapPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `bootstrap` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto plot = df.plot();

auto out = plot.bootstrap();
```

Output

```
Figure(...)
```

12.3.47 box

Signature

```
pluss::pl::Studio box(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `box` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio box(self, std::initializer_list<std::string_view> features, const BoxPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.box();
```

Output

```
Figure(...)
```

12.3.48 box

Signature

```
pluss::pl::Studio box(self, std::initializer_list<std::string_view> features, const␣
↪BoxPlotOptions& options) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`options` : `const BoxPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `box` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio box(self, std::initializer_list<std::string_view> features =`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.box();
```

Output

```
Figure(...)
```

12.3.49 brief

Signature

```
pluss::pl::Studio brief(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `brief` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio& brief(self, pluss::pl::Studio& figure, std::initializer_list<std::string_view> features =`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.brief();
```

Output

```
Figure(...)
```

12.3.50 `brief`

Signature

```
pluss::pl::Studio& brief(self, pluss::pl::Studio& figure, std::initializer_list
↪<std::string_view> features =
```

Parameters

`figure` : `pluss::pl::Studio&` See the signature for the supported argument form.

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio&` Value produced by this call.

Notes

Applies `brief` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio brief(self, std::initializer_list<std::string_view> features =`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.brief();
```

Output

```
Figure(...)
```

12.3.51 corr_heatmap

Signature

```
pluss::pl::Studio corr_heatmap(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `corr_heatmap` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio corr_heatmap(self, std::initializer_list<std::string_view> features, const CorrHeatmapPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.corr_heatmap();
```

Output

```
Figure(...)
```

12.3.52 corr_heatmap

Signature

```
pluss::pl::Studio corr_heatmap(self, std::initializer_list<std::string_view> features,␣
↪const CorrHeatmapPlotOptions& options) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`options` : `const CorrHeatmapPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `corr_heatmap` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio corr_heatmap(self, std::initializer_list<std::string_view> features =`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.corr_heatmap();
```

Output

```
Figure(...)
```

12.3.53 density

Signature

```
pluss::pl::Studio density(self, std::initializer_list<std::string_view> features, const␣
↪DensityPlotOptions& options) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`options` : `const DensityPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `density` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio density(self, std::string_view feature, std::int64_t points = 128) const`
- `pluss::pl::Studio density(self, std::initializer_list<std::string_view> features, std::int64_t points = 128) const`
- `pluss::pl::Studio density(self, std::string_view feature, const DensityPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.density();
```

Output

```
Figure(...)
```

12.3.54 density

Signature

```
pluss::pl::Studio density(self, std::initializer_list<std::string_view> features,␣
↪std::int64_t points = 128) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`points` : `std::int64_t` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `density` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio density(self, std::string_view feature, std::int64_t points = 128) const`
- `pluss::pl::Studio density(self, std::string_view feature, const DensityPlotOptions& options) const`
- `pluss::pl::Studio density(self, std::initializer_list<std::string_view> features, const DensityPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.density();
```

Output

```
Figure(...)
```

12.3.55 `density`

Signature

```
pluss::pl::Studio density(self, std::string_view feature, const DensityPlotOptions&␣
↪options) const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`options` : `const DensityPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `density` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio density(self, std::string_view feature, std::int64_t points = 128) const`
- `pluss::pl::Studio density(self, std::initializer_list<std::string_view> features, std::int64_t points = 128) const`
- `pluss::pl::Studio density(self, std::initializer_list<std::string_view> features, const DensityPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.density();
```

Output

```
Figure(...)
```

12.3.56 density

Signature

```
pluss::pl::Studio density(self, std::string_view feature, std::int64_t points = 128)␣
↪const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`points` : `std::int64_t` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `density` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- pluss::pl::Studio density(self, std::initializer_list<std::string_view> features, std::int64_t points = 128) const
- pluss::pl::Studio density(self, std::string_view feature, const DensityPlotOptions& options) const
- pluss::pl::Studio density(self, std::initializer_list<std::string_view> features, const DensityPlotOptions& options) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.density();
```

Output

```
Figure(...)
```

12.3.57 heatmap

Signature

```
pluss::pl::Studio heatmap(self, std::initializer_list<std::string_view> features) const
```

Parameters

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

pluss::pl::Studio Value produced by this call.

Notes

Applies heatmap to the current DataFramePlotAccessor object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- pluss::pl::Studio heatmap(self, std::initializer_list<std::string_view> features, const HeatmapPlotOptions& options) const
- pluss::pl::Studio heatmap(self, std::string_view case_feature, std::string_view feature_feature, std::string_view value_feature, DataFrame::GroupOp op = DataFrame::GroupOp::Mean) const
- pluss::pl::Studio heatmap(self, std::string_view case_feature, std::string_view feature_feature, std::string_view value_feature, DataFrame::GroupOp op, const HeatmapPlotOptions& options) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.heatmap();
```

Output

```
Figure(...)
```

12.3.58 heatmap

Signature

```
pluss::pl::Studio heatmap(self, std::initializer_list<std::string_view> features, const␣
↪HeatmapPlotOptions& options) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`options` : `const HeatmapPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `heatmap` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio heatmap(self, std::initializer_list<std::string_view> features) const`
- `pluss::pl::Studio heatmap(self, std::string_view case_feature, std::string_view feature_feature, std::string_view value_feature, DataFrame::GroupOp op = DataFrame::GroupOp::Mean) const`
- `pluss::pl::Studio heatmap(self, std::string_view case_feature, std::string_view feature_feature, std::string_view value_feature, DataFrame::GroupOp op, const HeatmapPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
```

(continues on next page)

(continued from previous page)

```
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.heatmap();
```

Output

```
Figure(...)
```

12.3.59 heatmap

Signature

```
pluss::pl::Studio heatmap(self, std::string_view case_feature, std::string_view feature_
↪feature, std::string_view value_feature, DataFrame::GroupOp op =␣
↪DataFrame::GroupOp::Mean) const
```

Parameters

`case_feature` : `std::string_view` See the signature for the supported argument form.

`feature_feature` : `std::string_view` See the signature for the supported argument form.

`value_feature` : `std::string_view` See the signature for the supported argument form.

`op` : `DataFrame::GroupOp` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `heatmap` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio heatmap(self, std::initializer_list<std::string_view> features) const`
- `pluss::pl::Studio heatmap(self, std::initializer_list<std::string_view> features, const HeatmapPlotOptions& options) const`
- `pluss::pl::Studio heatmap(self, std::string_view case_feature, std::string_view feature_feature, std::string_view value_feature, DataFrame::GroupOp op, const HeatmapPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
```

(continues on next page)

(continued from previous page)

```
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.heatmap();
```

Output

```
Figure(...)
```

12.3.60 heatmap

Signature

```
pluss::pl::Studio heatmap(self, std::string_view case_feature, std::string_view feature_
↪feature, std::string_view value_feature, DataFrame::GroupOp op, const␣
↪HeatmapPlotOptions& options) const
```

Parameters

`case_feature` : `std::string_view` See the signature for the supported argument form.

`feature_feature` : `std::string_view` See the signature for the supported argument form.

`value_feature` : `std::string_view` See the signature for the supported argument form.

`op` : `DataFrame::GroupOp` See the signature for the supported argument form.

`options` : `const HeatmapPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `heatmap` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio heatmap(self, std::initializer_list<std::string_view> features) const`
- `pluss::pl::Studio heatmap(self, std::initializer_list<std::string_view> features, const HeatmapPlotOptions& options) const`
- `pluss::pl::Studio heatmap(self, std::string_view case_feature, std::string_view feature_feature, std::string_view value_feature, DataFrame::GroupOp op = DataFrame::GroupOp::Mean) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.heatmap();
```

Output

```
Figure(...)
```

12.3.61 hist

Signature

```
pluss::pl::Studio hist(self, const HistPlotOptions& options) const
```

Parameters

`options` : `const HistPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a histogram for one feature using the requested number of bins.

See also

- `pluss::pl::Studio hist(self, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, std::string_view feature, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, std::initializer_list<std::string_view> features, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, std::string_view feature, const HistPlotOptions& options) const`
- `pluss::pl::Studio hist(self, std::initializer_list<std::string_view> features, const HistPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
```

(continues on next page)

(continued from previous page)

```
auto plot = df.plot();

auto fig = plot.hist("sales", 12);
```

Output

```
Figure(...)
```

12.3.62 hist

Signature

```
pluss::pl::Studio hist(self, std::initializer_list<std::string_view> features, const␣
↪HistPlotOptions& options) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`options` : `const HistPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a histogram for one feature using the requested number of bins.

See also

- `pluss::pl::Studio hist(self, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, std::string_view feature, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, std::initializer_list<std::string_view> features, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, const HistPlotOptions& options) const`
- `pluss::pl::Studio hist(self, std::string_view feature, const HistPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.hist("sales", 12);
```

Output

```
Figure(...)
```

12.3.63 hist

Signature

```
pluss::pl::Studio hist(self, std::initializer_list<std::string_view> features,␣
↪std::int64_t bins = 20) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`bins` : `std::int64_t` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a histogram for one feature using the requested number of bins.

See also

- `pluss::pl::Studio hist(self, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, std::string_view feature, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, const HistPlotOptions& options) const`
- `pluss::pl::Studio hist(self, std::string_view feature, const HistPlotOptions& options) const`
- `pluss::pl::Studio hist(self, std::initializer_list<std::string_view> features, const HistPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.hist("sales", 12);
```

Output

```
Figure(...)
```

12.3.64 hist

Signature

```
pluss::pl::Studio hist(self, std::int64_t bins = 20) const
```

Parameters

bins : std::int64_t See the signature for the supported argument form.

Returns

pluss::pl::Studio Value produced by this call.

Notes

Builds a histogram for one feature using the requested number of bins.

See also

- pluss::pl::Studio hist(self, std::string_view feature, std::int64_t bins = 20) const
- pluss::pl::Studio hist(self, std::initializer_list<std::string_view> features, std::int64_t bins = 20) const
- pluss::pl::Studio hist(self, const HistPlotOptions& options) const
- pluss::pl::Studio hist(self, std::string_view feature, const HistPlotOptions& options) const
- pluss::pl::Studio hist(self, std::initializer_list<std::string_view> features, const HistPlotOptions& options) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.hist("sales", 12);
```

Output

```
Figure(...)
```

12.3.65 hist

Signature

```
pluss::pl::Studio hist(self, std::string_view feature, const HistPlotOptions& options)␣
↪const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`options` : `const HistPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a histogram for one feature using the requested number of bins.

See also

- `pluss::pl::Studio hist(self, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, std::string_view feature, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, std::initializer_list<std::string_view> features, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, const HistPlotOptions& options) const`
- `pluss::pl::Studio hist(self, std::initializer_list<std::string_view> features, const HistPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.hist("sales", 12);
```

Output

```
Figure(...)
```

12.3.66 `hist`

Signature

```
pluss::pl::Studio hist(self, std::string_view feature, std::int64_t bins = 20) const
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`bins` : `std::int64_t` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a histogram for one feature using the requested number of bins.

See also

- `pluss::pl::Studio hist(self, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, std::initializer_list<std::string_view> features, std::int64_t bins = 20) const`
- `pluss::pl::Studio hist(self, const HistPlotOptions& options) const`
- `pluss::pl::Studio hist(self, std::string_view feature, const HistPlotOptions& options) const`
- `pluss::pl::Studio hist(self, std::initializer_list<std::string_view> features, const HistPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.hist("sales", 12);
```

Output

```
Figure(...)
```

12.3.67 lag

Signature

```
pluss::pl::Studio lag(self, std::string_view feature, const LagPlotOptions& options =
```

Parameters

`feature` : `std::string_view` See the signature for the supported argument form.

`options` : `const LagPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `lag` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.lag();
```

Output

```
Figure(...)
```

12.3.68 `line`

Signature

```
pluss::pl::Studio line(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature = std::nullopt) const
```

Parameters

`y_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a line chart for one or more y-features, optionally against a chosen x-feature.

See also

- `pluss::pl::Studio line(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio line(self, std::string_view y_feature, std::optional<std::string_view> x_feature, const LinePlotOptions& options) const`
- `pluss::pl::Studio line(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const LinePlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.line({"sales"}, std::nullopt);
```

Output

```
Figure(...)
```

12.3.69 line

Signature

```
pluss::pl::Studio line(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature, const LinePlotOptions& options) const
```

Parameters

`y_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

`options` : `const LinePlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a line chart for one or more y-features, optionally against a chosen x-feature.

See also

- `pluss::pl::Studio line(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio line(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio line(self, std::string_view y_feature, std::optional<std::string_view> x_feature, const LinePlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
```

(continues on next page)

(continued from previous page)

```
});
auto plot = df.plot();

auto fig = plot.line({"sales"}, std::nullopt);
```

Output

```
Figure(...)
```

12.3.70 line

Signature

```
pluss::pl::Studio line(self, std::string_view y_feature, std::optional<std::string_view>␣
↪x_feature = std::nullopt) const
```

Parameters

`y_feature` : `std::string_view` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a line chart for one or more y-features, optionally against a chosen x-feature.

See also

- `pluss::pl::Studio line(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio line(self, std::string_view y_feature, std::optional<std::string_view> x_feature, const LinePlotOptions& options) const`
- `pluss::pl::Studio line(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const LinePlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.line("sales");
```

Output

```
Figure(...)
```

12.3.71 line

Signature

```
pluss::pl::Studio line(self, std::string_view y_feature, std::optional<std::string_view>␣
↪x_feature, const LinePlotOptions& options) const
```

Parameters

`y_feature` : `std::string_view` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

`options` : `const LinePlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a line chart for one or more y-features, optionally against a chosen x-feature.

See also

- `pluss::pl::Studio line(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio line(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio line(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const LinePlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.line("sales");
```

Output

```
Figure(...)
```

12.3.72 parallel

Signature

```
pluss::pl::Studio parallel(self,  std::initializer_list<std::string_view> features =
```

Parameters

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

pluss::pl::Studio Value produced by this call.

Notes

Applies parallel to the current DataFramePlotAccessor object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.parallel();
```

Output

```
Figure(...)
```

12.3.73 pie

Signature

```
pluss::pl::Studio pie(self, std::string_view value_feature, std::optional<std::string_
↪view> label_feature = std::nullopt) const
```

Parameters

value_feature : std::string_view See the signature for the supported argument form.

label_feature : std::optional<std::string_view> See the signature for the supported argument form.

Returns

pluss::pl::Studio Value produced by this call.

Notes

Applies `pie` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio pie(self, std::string_view value_feature, std::optional<std::string_view> label_feature, const PiePlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.pie();
```

Output

```
Figure(...)
```

12.3.74 `pie`

Signature

```
pluss::pl::Studio pie(self, std::string_view value_feature, std::optional<std::string_
↪view> label_feature, const PiePlotOptions& options) const
```

Parameters

`value_feature` : `std::string_view` See the signature for the supported argument form.

`label_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

`options` : `const PiePlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `pie` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio pie(self, std::string_view value_feature, std::optional<std::string_view> label_feature = std::nullopt) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.pie();
```

Output

```
Figure(...)
```

12.3.75 radviz

Signature

```
pluss::pl::Studio radviz(self,  std::initializer_list<std::string_view> features =
```

Parameters

features : std::initializer_list<std::string_view> See the signature for the supported argument form.

Returns

pluss::pl::Studio Value produced by this call.

Notes

Applies radviz to the current DataFramePlotAccessor object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.radviz();
```

Output

```
Figure(...)
```

12.3.76 scatter

Signature

```
pluss::pl::Studio scatter(self, std::string_view x_feature, std::string_view y_feature,␣
↪std::optional<std::string_view> hue_feature = std::nullopt) const
```

Parameters

x_feature : std::string_view See the signature for the supported argument form.

y_feature : std::string_view See the signature for the supported argument form.

hue_feature : std::optional<std::string_view> See the signature for the supported argument form.

Returns

pluss::pl::Studio Value produced by this call.

Notes

Builds a scatter plot using one feature for x and another for y.

See also

- pluss::pl::Studio scatter(self, std::string_view x_feature, std::string_view y_feature, std::optional<std::string_view> hue_feature, const ScatterPlotOptions& options) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.scatter("id", "sales");
```

Output

```
Figure(...)
```

12.3.77 scatter

Signature

```
pluss::pl::Studio scatter(self, std::string_view x_feature, std::string_view y_feature,␣
↪std::optional<std::string_view> hue_feature, const ScatterPlotOptions& options) const
```

Parameters

`x_feature` : `std::string_view` See the signature for the supported argument form.

`y_feature` : `std::string_view` See the signature for the supported argument form.

`hue_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

`options` : `const ScatterPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Builds a scatter plot using one feature for x and another for y.

See also

- `pluss::pl::Studio scatter(self, std::string_view x_feature, std::string_view y_feature, std::optional<std::string_view> hue_feature = std::nullopt) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto fig = plot.scatter("id", "sales");
```

Output

```
Figure(...)
```

12.3.78 `scatter_matrix`

Signature

```
pluss::pl::Studio scatter_matrix(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `scatter_matrix` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio scatter_matrix(self, std::initializer_list<std::string_view> features, ScatterMatrixTriangle triangle) const`
- `pluss::pl::Studio scatter_matrix(self, std::initializer_list<std::string_view> features, const ScatterMatrixPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.scatter_matrix();
```

Output

```
Figure(...)
```

12.3.79 `scatter_matrix`

Signature

```
pluss::pl::Studio scatter_matrix(self, std::initializer_list<std::string_view> features,␣
↪const ScatterMatrixPlotOptions& options) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`options` : `const ScatterMatrixPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `scatter_matrix` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio scatter_matrix(self, std::initializer_list<std::string_view> features =`
- `pluss::pl::Studio scatter_matrix(self, std::initializer_list<std::string_view> features, ScatterMatrixTriangle triangle) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.scatter_matrix();
```

Output

```
Figure(...)
```

12.3.80 scatter_matrix

Signature

```
pluss::pl::Studio scatter_matrix(self, std::initializer_list<std::string_view> features,␣
↪ScatterMatrixTriangle triangle) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`triangle` : `ScatterMatrixTriangle` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `scatter_matrix` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio scatter_matrix(self, std::initializer_list<std::string_view> features =`
- `pluss::pl::Studio scatter_matrix(self, std::initializer_list<std::string_view> features, const ScatterMatrixPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.scatter_matrix();
```

Output

```
Figure(...)
```

12.3.81 step

Signature

```
pluss::pl::Studio step(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature = std::nullopt) const
```

Parameters

`y_features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `step` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio step(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio step(self, std::string_view y_feature, std::optional<std::string_view> x_feature, const StepPlotOptions& options) const`
- `pluss::pl::Studio step(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const StepPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.step();
```

Output

```
Figure(...)
```

12.3.82 step

Signature

```
pluss::pl::Studio step(self, std::initializer_list<std::string_view> y_features,␣
↪std::optional<std::string_view> x_feature, const StepPlotOptions& options) const
```

Parameters

y_features : std::initializer_list<std::string_view> See the signature for the supported argument form.

x_feature : std::optional<std::string_view> See the signature for the supported argument form.

options : const StepPlotOptions& See the signature for the supported argument form.

Returns

pluss::pl::Studio Value produced by this call.

Notes

Applies step to the current DataFramePlotAccessor object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- pluss::pl::Studio step(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const
- pluss::pl::Studio step(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const
- pluss::pl::Studio step(self, std::string_view y_feature, std::optional<std::string_view> x_feature, const StepPlotOptions& options) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.step();
```

Output

```
Figure(...)
```

12.3.83 step

Signature

```
pluss::pl::Studio step(self, std::string_view y_feature, std::optional<std::string_view>␣
↪x_feature = std::nullopt) const
```

Parameters

`y_feature` : `std::string_view` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `step` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio step(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio step(self, std::string_view y_feature, std::optional<std::string_view> x_feature, const StepPlotOptions& options) const`
- `pluss::pl::Studio step(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const StepPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.step();
```

Output

```
Figure(...)
```

12.3.84 `step`

Signature

```
pluss::pl::Studio step(self, std::string_view y_feature, std::optional<std::string_view>␣
↪x_feature, const StepPlotOptions& options) const
```

Parameters

`y_feature` : `std::string_view` See the signature for the supported argument form.

`x_feature` : `std::optional<std::string_view>` See the signature for the supported argument form.

`options` : `const StepPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `step` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio step(self, std::string_view y_feature, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio step(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature = std::nullopt) const`
- `pluss::pl::Studio step(self, std::initializer_list<std::string_view> y_features, std::optional<std::string_view> x_feature, const StepPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.step();
```

Output

```
Figure(...)
```

12.3.85 `violin`

Signature

```
pluss::pl::Studio violin(self, std::initializer_list<std::string_view> features =
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `violin` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- pluss::pl::Studio violin(self, std::initializer_list<std::string_view> features, const ViolinPlotOptions& options) const
- pluss::pl::Studio violin(self, std::string_view value_feature, std::string_view group_feature) const
- pluss::pl::Studio violin(self, std::string_view value_feature, std::string_view group_feature, const ViolinPlotOptions& options) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.violin();
```

Output

```
Figure(...)
```

12.3.86 violin

Signature

```
pluss::pl::Studio violin(self, std::initializer_list<std::string_view> features, const␣
↪ViolinPlotOptions& options) const
```

Parameters

`features` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

`options` : `const ViolinPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `violin` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- pluss::pl::Studio violin(self, std::initializer_list<std::string_view> features =
- pluss::pl::Studio violin(self, std::string_view value_feature, std::string_view group_feature) const
- pluss::pl::Studio violin(self, std::string_view value_feature, std::string_view group_feature, const ViolinPlotOptions& options) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.violin();
```

Output

```
Figure(...)
```

12.3.87 violin

Signature

```
pluss::pl::Studio violin(self, std::string_view value_feature, std::string_view group_
↪feature) const
```

Parameters

`value_feature` : `std::string_view` See the signature for the supported argument form.

`group_feature` : `std::string_view` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `violin` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio violin(self, std::initializer_list<std::string_view> features =`
- `pluss::pl::Studio violin(self, std::initializer_list<std::string_view> features, const ViolinPlotOptions& options) const`
- `pluss::pl::Studio violin(self, std::string_view value_feature, std::string_view group_feature, const ViolinPlotOptions& options) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();
```

(continues on next page)

(continued from previous page)

```
auto out = plot.violin();
```

Output

```
Figure(...)
```

12.3.88 violin

Signature

```
pluss::pl::Studio violin(self, std::string_view value_feature, std::string_view group_
↪feature, const ViolinPlotOptions& options) const
```

Parameters

`value_feature` : `std::string_view` See the signature for the supported argument form.

`group_feature` : `std::string_view` See the signature for the supported argument form.

`options` : `const ViolinPlotOptions&` See the signature for the supported argument form.

Returns

`pluss::pl::Studio` Value produced by this call.

Notes

Applies `violin` to the current `DataFramePlotAccessor` object. Use the example and output together to see which part of the object changes and which structure is preserved.

See also

- `pluss::pl::Studio violin(self, std::initializer_list<std::string_view> features =`
- `pluss::pl::Studio violin(self, std::initializer_list<std::string_view> features, const ViolinPlotOptions& options) const`
- `pluss::pl::Studio violin(self, std::string_view value_feature, std::string_view group_feature) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto plot = df.plot();

auto out = plot.violin();
```

Output

```
Figure(...)
```

CHAPTER

THIRTEEN

DATAFRAMESQLACCESSOR API REFERENCE

CHAPTER

FOURTEEN

PRACTICAL DATAFRAMESQLACCESSOR API REFERENCE

14.1 API Reference Style

This page documents the public SQL accessor returned by `DataFrame::sql()`.

14.1.1 execute

Signature

```
DataFrame execute(self, std::string_view statement) const
```

Parameters

`statement` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Runs a SQL query against the current frame and returns the result as a new `DataFrame`.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto sql = df.sql();

auto out = sql.execute("SELECT id, sales FROM self WHERE sales > 95");
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

14.1.2 prepare

Signature

```
DataFrameSqlPrepared prepare(self, std::string_view statement) const
```

Parameters

`statement` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrameSqlPrepared` Value produced by this call.

Notes

Compiles a parameterized SQL statement that can be executed repeatedly with different bindings.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto sql = df.sql();

auto out = sql.prepare("SELECT id, sales FROM self WHERE sales > {min_sales}");
```

Output

```
Prepared statement ready
```

CHAPTER

FIFTEEN

DATAFRAMESQLPREPARED API REFERENCE

CHAPTER

SIXTEEN

PRACTICAL DATAFRAMESQLPREPARED API REFERENCE

16.1 API Reference Style

This page documents the public prepared-statement helper returned by `DataFrame::prepare_sql()` or `DataFrameSqlAccessor::prepare()`.

16.1.1 execute

Signature

```
DataFrame execute(self) const
```

Parameters

None

Returns

`DataFrame` Value produced by this call.

Notes

Runs the prepared SQL statement with the supplied parameters and returns the result as a new `DataFrame`.

See also

- `DataFrame execute(self, std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam> params) const`
- `DataFrame execute(self, std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam> params, std::unordered_map<std::string, std::string> identifier_params) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto stmt = df.prepare_sql("SELECT id, sales FROM self WHERE sales > {min_sales}");

auto out = stmt.execute();
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

16.1.2 execute

Signature

```
DataFrame execute(self, std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam>␣
↪params) const
```

Parameters

params : std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam> See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Runs the prepared SQL statement with the supplied parameters and returns the result as a new DataFrame.

See also

- DataFrame execute(self) const
- DataFrame execute(self, std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam> params, std::unordered_map<std::string, std::string> identifier_params) const

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto stmt = df.prepare_sql("SELECT id, sales FROM self WHERE sales > {min_sales}");

auto out = stmt.execute({{"min_sales", 95.0}});
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

16.1.3 execute

Signature

```
DataFrame execute(self, std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam>␣
↪params, std::unordered_map<std::string, std::string> identifier_params) const
```

Parameters

`params` : `std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam>` See the signature for the supported argument form.

`identifier_params` : `std::unordered_map<std::string, std::string>` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Runs the prepared SQL statement with the supplied parameters and returns the result as a new `DataFrame`.

See also

- `DataFrame execute(self) const`
- `DataFrame execute(self, std::unordered_map<std::string, DataFrame::SqlOptions::SqlParam> params) const`

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});
auto stmt = df.prepare_sql("SELECT id, sales FROM self WHERE sales > {min_sales}");

auto out = stmt.execute({{"min_sales", 95.0}}, {{"table_name", "self"}});
```

Output

```
id  sales  city
1   100.0  Accra
3    90.0  Kumasi
```

CHAPTER

SEVENTEEN

THREAD AND ATLAS GUIDE

17.1 Thread

`Thread` is a typed column wrapper around `Series`, `Note`, `Timeline`, or `TimeIndex`. Use it when you want column operations without carrying the full `DataFrame` around.

```
auto sales = pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}));
auto cities = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto cleaned = sales.fillna(0.0).clip(0.0, 150.0);
auto avg = cleaned.mean();
auto city_mask = cities.contains("Accra");
```

Notes:

- `Thread` is the easiest bridge point between the Pluss domain types and table verbs
- when you want the original typed payload back, use `as_series()`, `as_note()`, `as_timeline()`, or `as_timeindex()`

17.2 Row-Space Helpers

```
auto head = sales.head(2);
auto tail = sales.tail(2);
auto slice = sales.slice(0, 2);
auto taken = sales.take(std::array<std::int64_t, 2>{0, 2});
auto filtered = sales.filter(city_mask);
```

17.3 Atlas

`Atlas` is the frame-stack model in PlussMic. It is useful when you want a small set of aligned `DataFrame` objects with an optional depth axis.

```
auto f0 = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema"}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0}))},
});

auto f1 = pluss::table::DataFrame({
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema"}))},
```

(continues on next page)

(continued from previous page)

```
    {"sales", pluss::table::Thread(pluss::make_series({130.0, 125.0}))},
});

auto labels = pluss::table::Thread(pluss::make_note({"train", "test"}));
auto atlas = pluss::table::Atlas({f0, f1}, labels);

atlas.show();
atlas.head(1).show();
auto sorted = atlas.sort_depth();
auto first = atlas.frame(0);
```

Popup behavior:

- `atlas.show()` opens an atlas viewer when a GUI backend is available
- the atlas popup shows atlas-level summary cards plus a frame selector
- the main panel renders one selected `DataFrame` slice at a time
- if no GUI backend is available, `atlas.show()` falls back to terminal rendering

17.4 Atlas To Kindle

```
auto archive = atlas.kindelize();
auto roundtrip = archive.tabelize();
```

CHAPTER

EIGHTEEN

THREAD API REFERENCE

CHAPTER

NINETEEN

PRACTICAL THREAD API REFERENCE

19.1 API Reference Style

This page documents the full public `Thread` surface with direct typed-column examples.

19.1.1 abs

Signature

```
Thread abs(self) const
```

Parameters

None

Returns

`Thread` Value produced by this call.

Notes

Returns a numeric thread whose values are the absolute values of the current numeric payload.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.abs();
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.2 as_note

Signature

```
const pluss::Note& as_note(self) const
```

Parameters

None

Returns

`const pluss::Note&` Value produced by this call.

Notes

Interprets or converts the current payload as a note/text thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.as_note();
```

Output

```
["Accra", "Tema", "Kumasi"]
```

19.1.3 as_series

Signature

```
const pluss::Series& as_series(self) const
```

Parameters

None

Returns

`const pluss::Series&` Value produced by this call.

Notes

Interprets or converts the current payload as a numeric series thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.as_series();
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.4 as_timeindex

Signature

```
const pluss::TimeIndex& as_timeindex(self) const
```

Parameters

None

Returns

const pluss::TimeIndex& Value produced by this call.

Notes

Interprets or converts the current payload as a time-index thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.as_timeindex();
```

Output

```
["2024-01-01T09:10:00Z", "2024-02-01T11:00:00Z", "2024-03-05T15:30:00Z"]
```

19.1.5 as_timeline

Signature

```
const pluss::Timeline& as_timeline(self) const
```

Parameters

None

Returns

const pluss::Timeline& Value produced by this call.

Notes

Interprets or converts the current payload as a timeline thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.as_timeline();
```

Output

```
["2024-01-01T09:10:00Z", "2024-02-01T11:00:00Z", "2024-03-05T15:30:00Z"]
```

19.1.6 astype

Signature

```
Thread astype(self, pluss::codem::ScalarType dtype) const
```

Parameters

dtype : pluss::codem::ScalarType See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.astype(pluss::codem::ScalarType::Float64);
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.7 bfill

Signature

```
Thread bfill(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.bfill();
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.8 cast

Signature

```
Thread cast(self, ThreadCastTarget target, std::optional<pluss::codem::ScalarType>␣
↪series_dtype = std::nullopt) const
```

Parameters

`target` : `ThreadCastTarget` See the signature for the supported argument form.

`series_dtype` : `std::optional<pluss::codem::ScalarType>` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"1.0", "2.0", "3.0"}));

auto out = thread.cast(pluss::table::ThreadCastTarget::Series,␣
↪pluss::codem::ScalarType::Float64);
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.9 clear_validity_mask

Signature

```
Thread clear_validity_mask(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

Returns a thread with the validity mask removed, so all entries are treated as valid again.

Examples

```
auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.clear_validity_mask();
```

19.1.10 clear_validity_mask_

Signature

```
Thread& clear_validity_mask_(self)
```

Parameters

None

Returns

Thread& Value produced by this call.

Notes

Removes the validity mask in place, so all entries are treated as valid again.

Examples

```
auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

thread.clear_validity_mask_();
```

19.1.11 clip

Signature

```
Thread clip(self, scalar min_value, scalar max_value) const
```

Parameters

`min_value` : `scalar` See the signature for the supported argument form.

`max_value` : `scalar` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

Clamps numeric values to the supplied lower and upper bounds.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.clip(0.0, 25.0);
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.12 `compare`

Signature

```
Thread compare(self, const Thread& other, ThreadBinaryCompareOp op) const
```

Parameters

`other` : `const Thread&` See the signature for the supported argument form.

`op` : `ThreadBinaryCompareOp` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `Thread compare(self, scalar other, ThreadBinaryCompareOp op) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto other = pluss::table::Thread(pluss::make_series({15.0, 10.0, 35.0}));
auto out = thread.compare(other, pluss::table::ThreadBinaryCompareOp::Gt);
```

Output

```
1
```

19.1.13 compare

Signature

```
Thread compare(self, scalar other, ThreadBinaryCompareOp op) const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

`op` : `ThreadBinaryCompareOp` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This produces a comparison-style result you can inspect directly or reuse as a mask.

See also

- `Thread compare(self, const Thread& other, ThreadBinaryCompareOp op) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.compare(15.0, pluss::table::ThreadBinaryCompareOp::Gt);
```

Output

```
1
```

19.1.14 compare_rows

Signature

```
int compare_rows(self, std::int64_t lhs_row, std::int64_t rhs_row, bool nulls_last =␣
↪true) const
```

Parameters

`lhs_row` : `std::int64_t` See the signature for the supported argument form.

`rhs_row` : `std::int64_t` See the signature for the supported argument form.

`nulls_last` : `bool` See the signature for the supported argument form.

Returns

`int` Value produced by this call.

Notes

Compares two positions inside the same thread and returns the comparison result.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.compare_rows(0, 1, true);
```

Output

```
1
```

19.1.15 contains

Signature

```
pluss::Series contains(self, std::string_view needle) const
```

Parameters

`needle` : `std::string_view` See the signature for the supported argument form.

Returns

`pluss::Series` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.contains("Accra");
```

Output

```
[true, false, true]
```

19.1.16 corr

Signature

```
scalar corr(const Thread& other, ThreadCorrMethod method = ThreadCorrMethod::Pearson)␣
↪const
```

Parameters

`other` : `const Thread&` See the signature for the supported argument form.

`method` : `ThreadCorrMethod` See the signature for the supported argument form.

Returns

`double` Value produced by this call.

Notes

Applies `corr` to the current `Thread` object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.corr();
```

19.1.17 count

Signature

```
pluss::Series count(self, bool count_true = true) const
```

Parameters

`count_true` : `bool` See the signature for the supported argument form.

Returns

`pluss::Series` Value produced by this call.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.count();
```

19.1.18 day

Signature

```
Thread day(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.day();
```

Output

```
[1, 2, 3]
```

19.1.19 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.defined();
```

Output

```
true
```

19.1.20 distinct

Signature

```
Thread distinct(self, bool dropna = true) const
```

Parameters

dropna : bool See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

Applies distinct to the current Thread object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.distinct();
```

19.1.21 dropna

Signature

```
Thread dropna(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.dropna();
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.22 dtype_name

Signature

```
std::string dtype_name(self) const
```

Parameters

None

Returns

std::string Rendered string result.

Notes

Returns the scalar dtype name used by the current numeric payload.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.dtype_name();
```

Output

```
Float64
```

19.1.23 exp

Signature

```
Thread exp(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

Applies the elementwise exponential to a numeric thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.exp();
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.24 ffill

Signature

```
Thread ffill(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.ffill();
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.25 fillna

Signature

```
Thread fillna(self, const char* value) const
```

Parameters

value : const char* See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Thread fillna(self, scalar value) const
- Thread fillna(self, std::string_view value) const
- Thread fillna(self, const pluss::Timeline& value) const
- Thread fillna(self, const pluss::TimeIndex& value) const

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.fillna("Unknown");
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.26 fillna

Signature

```
Thread fillna(self, const pluss::TimeIndex& value) const
```

Parameters

value : const pluss::TimeIndex& See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Thread fillna(self, scalar value) const
- Thread fillna(self, std::string_view value) const
- Thread fillna(self, const char* value) const
- Thread fillna(self, const pluss::Timeline& value) const

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeindex({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z"}));

auto out = thread.fillna(pluss::make_timeindex("2024-01-01T00:00:00Z"));
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.27 fillna

Signature

```
Thread fillna(self, const pluss::Timeline& value) const
```

Parameters

`value` : `const pluss::Timeline&` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Thread fillna(self, scalar value) const`
- `Thread fillna(self, std::string_view value) const`
- `Thread fillna(self, const char* value) const`
- `Thread fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.fillna(pluss::make_timeline("2024-01-01T00:00:00Z"));
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.28 fillna

Signature

```
Thread fillna(self, scalar value) const
```

Parameters

`value` : `scalar` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Thread fillna(self, std::string_view value) const`
- `Thread fillna(self, const char* value) const`
- `Thread fillna(self, const pluss::Timeline& value) const`
- `Thread fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.fillna(0.0);
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.29 `fillna`

Signature

```
Thread fillna(self, std::string_view value) const
```

Parameters

`value` : `std::string_view` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Thread fillna(self, scalar value) const`
- `Thread fillna(self, const char* value) const`
- `Thread fillna(self, const pluss::Timeline& value) const`
- `Thread fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.fillna("Unknown");
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.30 fillna_

Signature

```
Thread& fillna_(self, const char* value)
```

Parameters

`value` : `const char*` See the signature for the supported argument form.

Returns

`Thread&` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Thread& fillna_(self, scalar value)`
- `Thread& fillna_(self, std::string_view value)`
- `Thread& fillna_(self, const pluss::Timeline& value)`
- `Thread& fillna_(self, const pluss::TimeIndex& value)`

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

thread.fillna_("Unknown");
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.31 fillna_

Signature

```
Thread& fillna_(self, const pluss::TimeIndex& value)
```

Parameters

`value` : `const pluss::TimeIndex&` See the signature for the supported argument form.

Returns

`Thread&` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Thread& fillna_(self, scalar value)`
- `Thread& fillna_(self, std::string_view value)`
- `Thread& fillna_(self, const char* value)`
- `Thread& fillna_(self, const pluss::Timeline& value)`

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeindex({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z"}));

thread.fillna_(pluss::make_timeindex("2024-01-01T00:00:00Z"));
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.32 `fillna_`

Signature

```
Thread& fillna_(self, const pluss::Timeline& value)
```

Parameters

`value` : `const pluss::Timeline&` See the signature for the supported argument form.

Returns

`Thread&` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Thread& fillna_(self, scalar value)`
- `Thread& fillna_(self, std::string_view value)`
- `Thread& fillna_(self, const char* value)`

- Thread& fillna_(self, const pluss::TimeIndex& value)

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

thread.fillna_(pluss::make_timeline("2024-01-01T00:00:00Z"));
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.33 fillna_

Signature

```
Thread& fillna_(self, scalar value)
```

Parameters

`value` : `scalar` See the signature for the supported argument form.

Returns

`Thread&` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Thread& fillna_(self, std::string_view value)
- Thread& fillna_(self, const char* value)
- Thread& fillna_(self, const pluss::Timeline& value)
- Thread& fillna_(self, const pluss::TimeIndex& value)

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

thread.fillna_(0.0);
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.34 fillna_

Signature

```
Thread& fillna_(self, std::string_view value)
```

Parameters

value : std::string_view See the signature for the supported argument form.

Returns

Thread& Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Thread& fillna_(self, scalar value)
- Thread& fillna_(self, const char* value)
- Thread& fillna_(self, const pluss::Timeline& value)
- Thread& fillna_(self, const pluss::TimeIndex& value)

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

thread.fillna_("Unknown");
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.35 filter

Signature

```
Thread filter(self, const pluss::Series& bool_mask) const
```

Parameters

bool_mask : const pluss::Series& See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

Keeps only the entries where the supplied boolean mask is true.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = thread.filter(mask);
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.36 has_validity_mask

Signature

```
bool has_validity_mask(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the thread currently carries a validity mask.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.has_validity_mask();
```

19.1.37 head

Signature

```
Thread head(self, std::int64_t n) const
```

Parameters

n : std::int64_t Number of rows, entries, or depth items to keep in the preview.

Returns

Thread Value produced by this call.

Notes

Returns the first `n` entries of the thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.head(2);
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.38 hour

Signature

```
Thread hour(self) const
```

Parameters

None

Returns

`Thread` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.hour();
```

Output

```
[1, 2, 3]
```

19.1.39 is_valid

Signature

```
bool is_valid(self, std::int64_t row) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

Returns

`bool` Boolean status or membership result.

Notes

Checks whether the value at one position is marked valid by the thread validity mask.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.is_valid(0);
```

Output

```
true
```

19.1.40 `isfinite`

Signature

```
Thread isfinite(self) const
```

Parameters

None

Returns

`Thread` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.isfinite();
```

Output

```
[true, false, true]
```

19.1.41 isnan

Signature

```
Thread isnan(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.isnan();
```

Output

```
[true, false, true]
```

19.1.42 isnull

Signature

```
Thread isnull(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.isnull();
```

Output

```
[true, false, true]
```

19.1.43 kind

Signature

```
ThreadKind kind(self) const
```

Parameters

None

Returns

ThreadKind Value produced by this call.

Notes

Returns which payload kind this thread currently stores, such as Series, Note, Timeline, or TimeIndex.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.kind();
```

Output

```
Series
```

19.1.44 log

Signature

```
Thread log(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

Applies the elementwise natural logarithm to a numeric thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.log();
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.45 lower

Signature

```
Thread lower(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.lower();
```

Output

```
["Accra", "Tema", "Kumasi"]
```

19.1.46 max

Signature

```
pluss::Series max(self) const
```

Parameters

None

Returns

pluss::Series Value produced by this call.

Notes

Returns the maximum value of a numeric thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.max();
```

Output

```
20.0
```

19.1.47 mean

Signature

```
pluss::Series mean(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

Returns the mean value of a numeric thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.mean();
```

Output

```
20.0
```

19.1.48 min

Signature

```
pluss::Series min(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

Returns the minimum value of a numeric thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.min();
```

Output

```
20.0
```

19.1.49 month

Signature

```
Thread month(self) const
```

Parameters

None

Returns

`Thread` Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.month();
```

Output

```
[1, 2, 3]
```

19.1.50 notnull

Signature

```
Thread notnull(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.notnull();
```

Output

```
[true, false, true]
```

19.1.51 numeric_binary

Signature

```
Thread numeric_binary(self, const Thread& other, ThreadBinaryMathOp op) const
```

Parameters

other : const Thread& See the signature for the supported argument form.

op : ThreadBinaryMathOp See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- Thread numeric_binary(self, scalar other, ThreadBinaryMathOp op, bool reverse = false) const

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.numeric_binary(10.0, pluss::table::ThreadBinaryMathOp::Add, false);
```

Output

```
[20.0, 30.0, 40.0]
```

19.1.52 numeric_binary

Signature

```
Thread numeric_binary(self, scalar other, ThreadBinaryMathOp op, bool reverse = false)␣
↪const
```

Parameters

`other` : `scalar` See the signature for the supported argument form.

`op` : `ThreadBinaryMathOp` See the signature for the supported argument form.

`reverse` : `bool` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

This applies arithmetic across the compatible numeric payloads in the current object.

See also

- `Thread numeric_binary(self, const Thread& other, ThreadBinaryMathOp op) const`

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.numeric_binary(10.0, pluss::table::ThreadBinaryMathOp::Add, false);
```

Output

```
[20.0, 30.0, 40.0]
```

19.1.53 replace

Signature

```
Thread replace(self, std::string_view from, std::string_view to) const
```

Parameters

`from` : `std::string_view` See the signature for the supported argument form.

`to` : `std::string_view` See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = thread.replace("Tema", "Tamale");
```

Output

```
["Accra", "Tema", "Kumasi"]
```

19.1.54 shift

Signature

```
Thread shift(self, std::int64_t periods = 1) const
```

Parameters

periods : std::int64_t See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

Applies shift to the current Thread object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.shift();
```

19.1.55 show

Signature

```
void show(self, const display::DisplayOptions& options =
```

Parameters

options : const display::DisplayOptions& See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- `void show(self, std::ostream& os, const display::DisplayOptions& options =`

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

thread.show();
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.56 show

Signature

```
void show(self, std::ostream& os, const display::DisplayOptions& options =
```

Parameters

`os` : `std::ostream&` See the signature for the supported argument form.

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- `void show(self, const display::DisplayOptions& options =`

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

thread.show();
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.57 size

Signature

```
std::int64_t size(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.size();
```

Output

```
3
```

19.1.58 slice

Signature

```
Thread slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

start : std::int64_t Inclusive starting position of the slice.

stop : std::int64_t Exclusive ending position of the slice.

Returns

Thread Value produced by this call.

Notes

Returns a contiguous slice [start, stop) of the thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.slice(0, 2);
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.59 std

Signature

```
pluss::Series std(self, bool unbiased = true) const
```

Parameters

unbiased : bool See the signature for the supported argument form.

Returns

pluss::Series Value produced by this call.

Notes

Returns the standard deviation of a numeric thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.std(true);
```

Output

```
20.0
```

19.1.60 sum

Signature

```
pluss::Series sum(self) const
```

Parameters

None

Returns

pluss::Series Value produced by this call.

Notes

Returns the sum of a numeric thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.sum();
```

Output

```
20.0
```

19.1.61 tail

Signature

```
Thread tail(self, std::int64_t n) const
```

Parameters

n : std::int64_t Number of rows, entries, or depth items to keep in the preview.

Returns

Thread Value produced by this call.

Notes

Returns the last n entries of the thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.tail(2);
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.62 take

Signature

```
Thread take(self, std::span<const std::int64_t> indices) const
```

Parameters

indices : std::span<const std::int64_t> See the signature for the supported argument form.

Returns

Thread Value produced by this call.

Notes

Returns the entries at the requested integer positions.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

std::vector<std::int64_t> picks{0, 2};
auto out = thread.take(picks);
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.63 tally

Signature

```
DataFrameView tally(self, std::int64_t top_n = 0, bool dropna = true, bool render =␣
↪true) const
```

Parameters

top_n : std::int64_t See the signature for the supported argument form.

dropna : bool See the signature for the supported argument form.

render : bool See the signature for the supported argument form.

Returns

DataFrameView Value produced by this call.

Notes

Use this when you want a summary view instead of carrying the full input forward unchanged.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.tally();
```

19.1.64 to_note_weekday

Signature

```
Thread to_note_weekday(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.to_note_weekday();
```

Output

```
[1, 2, 3]
```

19.1.65 to_string

Signature

```
std::string to_string(self, const display::DisplayOptions& options =
```

Parameters

options : const display::DisplayOptions& See the signature for the supported argument form.

Returns

std::string Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.to_string();
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.66 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.unique();
```

Output

```
true
```

19.1.67 validity_mask

Signature

```
const std::optional<pluss::Series>& validity_mask(self) const
```

Parameters

None

Returns

const std::optional<pluss::Series>& Value produced by this call.

Notes

Returns the current validity mask as a boolean series.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.validity_mask();
```

19.1.68 value

Signature

```
ThreadValue value(self) const
```

Parameters

None

Returns

ThreadValue Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.value();
```

Output

```
10.0
```

19.1.69 values

Signature

```
const Thread& values(self) const
```

Parameters

None

Returns

const Thread& Value produced by this call.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.values();
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.70 var

Signature

```
pluss::Series var(self, bool unbiased = true) const
```

Parameters

`unbiased` : `bool` See the signature for the supported argument form.

Returns

`pluss::Series` Value produced by this call.

Notes

Returns the variance of a numeric thread.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.var(true);
```

Output

```
20.0
```

19.1.71 window

Signature

```
Thread window(self, std::int64_t window, ThreadWindowOp op, std::int64_t min_periods =␣
↪1) const
```

Parameters

`window` : `std::int64_t` See the signature for the supported argument form.

`op` : `ThreadWindowOp` See the signature for the supported argument form.

`min_periods` : `std::int64_t` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

Computes a window-based transformation such as a rolling mean over the thread values.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.window(2, pluss::table::ThreadWindowOp::RollingMean, 1);
```

Output

```
[10.0, 20.0, 30.0]
```

19.1.72 with_validity_mask

Signature

```
Thread with_validity_mask(self, const pluss::Series& validity_mask) const
```

Parameters

`validity_mask` : `const pluss::Series&` See the signature for the supported argument form.

Returns

`Thread` Value produced by this call.

Notes

Returns a new thread with the supplied validity mask attached.

Examples

```
auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = thread.with_validity_mask(mask);
```

19.1.73 with_validity_mask_

Signature

```
Thread& with_validity_mask_(self, const pluss::Series& validity_mask)
```

Parameters

`validity_mask` : `const pluss::Series&` See the signature for the supported argument form.

Returns

`Thread&` Value produced by this call.

Notes

Attaches the supplied validity mask in place.

Examples

```
auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

thread.with_validity_mask_(mask);
```

19.1.74 year

Signature

```
Thread year(self) const
```

Parameters

None

Returns

Thread Value produced by this call.

Notes

This is part of the time-aware surface and expects timeline or time-index semantics.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪02-01T11:00:00Z", "2024-03-05T15:30:00Z"}));

auto out = thread.year();
```

Output

```
[2024, 2024, 2024]
```

CHAPTER

TWENTY

THREADVALUE API REFERENCE

CHAPTER

TWENTYONE

PRACTICAL THREADVALUE API REFERENCE

21.1 API Reference Style

This page documents the public scalar-like `ThreadValue` wrapper returned by thread and column-value accessors.

21.1.1 as_note

Signature

```
pluss::Note as_note(self) const
```

Parameters

None

Returns

`pluss::Note` Value produced by this call.

Notes

Converts the wrapped scalar-like value into a `pluss::Note`.

Examples

```
auto thread = pluss::table::Thread(pluss::make_note({"Accra"}));
auto value = thread.value();

auto out = value.as_note();
```

Output

```
["Accra"]
```

21.1.2 as_series

Signature

```
pluss::Series as_series(self) const
```

Parameters

None

Returns

`pluss::Series` Value produced by this call.

Notes

Converts the wrapped scalar-like value into a `pluss::Series`.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.as_series();
```

Output

```
[10.0]
```

21.1.3 as_timeindex

Signature

```
pluss::TimeIndex as_timeindex(self) const
```

Parameters

None

Returns

`pluss::TimeIndex` Value produced by this call.

Notes

Converts the wrapped scalar-like value into a `pluss::TimeIndex`.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeindex({"2024-01-01T09:10:00Z"}));
auto value = thread.value();

auto out = value.as_timeindex();
```

Output

```
["2024-01-01T09:10:00Z"]
```

21.1.4 as_timeline

Signature

```
pluss::Timeline as_timeline(self) const
```

Parameters

None

Returns

`pluss::Timeline` Value produced by this call.

Notes

Converts the wrapped scalar-like value into a `pluss::Timeline`.

Examples

```
auto thread = pluss::table::Thread(pluss::make_timeline({"2024-01-01T09:10:00Z"}));
auto value = thread.value();

auto out = value.as_timeline();
```

Output

```
["2024-01-01T09:10:00Z"]
```

21.1.5 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.defined();
```

Output

```
true
```

21.1.6 kind

Signature

```
ThreadKind kind(self) const
```

Parameters

None

Returns

ThreadKind Value produced by this call.

Notes

Returns the underlying thread kind carried by this value wrapper.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.kind();
```

Output

```
Series
```

21.1.7 pluss::Note

Signature

```
operator pluss::Note(self) const
```

Parameters

None

Returns

operator Value produced by this call.

Notes

Uses the implicit conversion operator to materialize the wrapped value as a `pluss::Note`.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.pluss::Note();
```

21.1.8 pluss::Series

Signature

```
operator pluss::Series(self) const
```

Parameters

None

Returns

operator Value produced by this call.

Notes

Uses the implicit conversion operator to materialize the wrapped value as a `pluss::Series`.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.pluss::Series();
```

21.1.9 pluss::TimeIndex

Signature

```
operator pluss::TimeIndex(self) const
```

Parameters

None

Returns

operator Value produced by this call.

Notes

Uses the implicit conversion operator to materialize the wrapped value as a `pluss::TimeIndex`.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.pluss::TimeIndex();
```

21.1.10 pluss::Timeline

Signature

```
operator pluss::Timeline(self) const
```

Parameters

None

Returns

operator Value produced by this call.

Notes

Uses the implicit conversion operator to materialize the wrapped value as a pluss::Timeline.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.pluss::Timeline();
```

21.1.11 thread

Signature

```
const Thread& thread(self) const
```

Parameters

None

Returns

const Thread& Value produced by this call.

Notes

Returns the wrapped single-value Thread object.

Examples

```
auto thread = pluss::table::Thread(pluss::make_series({10.0}));
auto value = thread.value();

auto out = value.thread();
```

Output

```
[10.0]
```

CHAPTER

TWENTYTWO

ATLAS API REFERENCE

CHAPTER

TWENTYTHREE

PRACTICAL ATLAS API REFERENCE

23.1 API Reference Style

This page documents the full public `Atlas` surface for frame-stack workflows.

23.1.1 aggregate_over_frames

Signature

```
DataFrame aggregate_over_frames(self, const std::function<DataFrame(const std::vector
↪<DataFrame>&)>& fn) const
```

Parameters

`fn` : `const std::function<DataFrame(const std::vector<DataFrame>&)>&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Reduces the full frame stack to one output by applying an aggregation function across the contained frames.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.aggregate_over_frames([](const std::vector<pluss::table::DataFrame>&␣
↪frames) { return frames.front(); });
```

Output

```
sales
10.0
20.0
```

23.1.2 align_depth

Signature

```
Atlas align_depth(self, const Thread& labels) const
```

Parameters

`labels` : `const Thread&` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

Reorders frames so their depth labels match the supplied label order.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labels = pluss::table::Thread(pluss::make_note({"baseline", "stress"}));
auto out = atlas.align_depth(labels);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

23.1.3 axis0_labels

Signature

```
const std::optional<Thread>& axis0_labels(self) const
```

Parameters

None

Returns

const std::optional<Thread>& Value produced by this call.

Notes

Returns the current labels used to address fields, slots, rows, or depth items.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.axis0_labels();
```

Output

```
["train", "test"]
```

23.1.4 column_names

Signature

```
const std::vector<std::string>& column_names(self) const
```

Parameters

None

Returns

const std::vector<std::string>& Borrowed list of names.

Notes

Returns the current labels used to address fields, slots, rows, or depth items.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.column_names();
```

Output

```
["sales"]
```

23.1.5 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.defined();
```

Output

```
true
```

23.1.6 depth

Signature

```
std::int64_t depth(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.depth();
```

Output

```
2
```

23.1.7 empty

Signature

```
bool empty(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the current object has no rows, entries, slots, or depth items.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.empty();
```

Output

```
false
```

23.1.8 frame

Signature

```
DataFrame frame(self, std::int64_t index) const
```

Parameters

`index` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.frame(0);
```

Output

```
sales
10.0
20.0
```

23.1.9 frames

Signature

```
const std::vector<DataFrame>& frames(self) const
```

Parameters

None

Returns

const std::vector<DataFrame>& Borrowed collection of frames.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.frames();
```

Output

```
[frame0, frame1]
```

23.1.10 head

Signature

```
Atlas head(self, std::int64_t n = 5) const
```

Parameters

n : std::int64_t Number of rows, entries, or depth items to keep in the preview.

Returns

Atlas Value produced by this call.

Notes

Returns the first n frames from the atlas depth axis.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.head(1);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

23.1.11 iloc

Signature

```
DataFrame iloc(self, std::int64_t index) const
```

Parameters

`index` : `std::int64_t` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.iloc(0);
```

Output

```
sales
10.0
20.0
```

23.1.12 kindelize

Signature

```
pluss::kindle::Archive kindelize(self) const
```

Parameters

None

Returns

`pluss::kindle::Archive` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `pluss::kindle::Archive kindelize(self, const pluss::kindle::KindelizePolicy& policy) const`

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.kindelize();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

23.1.13 kindelize

Signature

```
pluss::kindle::Archive kindelize(self, const pluss::kindle::KindelizePolicy& policy)␣
↪const
```

Parameters

`policy` : `const pluss::kindle::KindelizePolicy&` See the signature for the supported argument form.

Returns

`pluss::kindle::Archive` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `pluss::kindle::Archive kindelize(self) const`

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.kindelize();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

23.1.14 `loc`

Signature

```
DataFrame loc(self, const pluss::Timeline& label) const
```

Parameters

`label` : `const pluss::Timeline&` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- DataFrame loc(self, std::string_view label) const
- DataFrame loc(self, std::int64_t label) const

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labeled = pluss::table::Atlas({f0, f1}, pluss::table::Thread(pluss::make_note({
↪"train", "test"})));
auto out = labeled.loc("test");
```

Output

```
sales
10.0
20.0
```

23.1.15 loc

Signature

```
DataFrame loc(self, std::int64_t label) const
```

Parameters

label : std::int64_t See the signature for the supported argument form.

Returns

DataFrame Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- DataFrame loc(self, std::string_view label) const
- DataFrame loc(self, const pluss::Timeline& label) const

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
```

(continues on next page)

(continued from previous page)

```
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labeled = pluss::table::Atlas({f0, f1}, pluss::table::Thread(pluss::make_note({
↪"train", "test"})));
auto out = labeled.loc("test");
```

Output

```
sales
10.0
20.0
```

23.1.16 loc

Signature

```
DataFrame loc(self, std::string_view label) const
```

Parameters

`label` : `std::string_view` See the signature for the supported argument form.

Returns

`DataFrame` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- `DataFrame loc(self, std::int64_t label) const`
- `DataFrame loc(self, const pluss::Timeline& label) const`

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labeled = pluss::table::Atlas({f0, f1}, pluss::table::Thread(pluss::make_note({
↪"train", "test"})));
auto out = labeled.loc("test");
```

Output

```
sales
10.0
20.0
```

23.1.17 map_frames

Signature

```
Atlas map_frames(self, const std::function<DataFrame(const DataFrame&)>& fn) const
```

Parameters

`fn` : `const std::function<DataFrame(const DataFrame&)>&` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

Applies the same function to each frame and returns the transformed atlas.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.map_frames([](const pluss::table::DataFrame& frame) { return frame.
↪head(1, false); });
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

23.1.18 ncases

Signature

```
std::int64_t ncases(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

Applies ncases to the current Atlas object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.ncases();
```

23.1.19 nfeats

Signature

```
std::int64_t nfeats(self) const
```

Parameters

None

Returns

std::int64_t Integer count, size, or positional result.

Notes

Applies nfeats to the current Atlas object. Use the example and output together to see which part of the object changes and which structure is preserved.

Examples

```cpp
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.nfeats();
```

23.1.20 plot

Signature

```
AtlasPlotAccessor plot(self) const
```

Parameters

None

Returns

`AtlasPlotAccessor` Value produced by this call.

Notes

This opens the plotting accessor for the current data object.

See also

- `AtlasPlotAccessor plot(self, const AtlasPlotOptions& options) const`

Examples

```cpp
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto fig = atlas.plot(0).line("sales");
```

Output

```
Figure(line)
```

23.1.21 plot

Signature

```
AtlasPlotAccessor plot(self, const AtlasPlotOptions& options) const
```

Parameters

`options` : `const AtlasPlotOptions&` See the signature for the supported argument form.

Returns

`AtlasPlotAccessor` Value produced by this call.

Notes

This opens the plotting accessor for the current data object.

See also

- `AtlasPlotAccessor plot(self) const`

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto fig = atlas.plot(0).line("sales");
```

Output

```
Figure(line)
```

23.1.22 relabel_depth

Signature

```
Atlas relabel_depth(self, const Thread& labels) const
```

Parameters

`labels` : `const Thread&` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

Replaces the depth-label axis with the supplied labels while keeping frame contents unchanged.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labels = pluss::table::Thread(pluss::make_note({"baseline", "stress"}));
auto out = atlas.relabel_depth(labels);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

23.1.23 show

Signature

```
void show(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- `void show(self, std::ostream& os, const display::DisplayOptions& options =`

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

atlas.show();
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

23.1.24 show

Signature

```
void show(self, std::ostream& os, const display::DisplayOptions& options =
```

Parameters

`os` : `std::ostream&` See the signature for the supported argument form.

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

See also

- `void show(self, const display::DisplayOptions& options =`

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

atlas.show();
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

23.1.25 slice_frames

Signature

```
Atlas slice_frames(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` Inclusive starting position of the slice.

`stop` : `std::int64_t` Exclusive ending position of the slice.

Returns

`Atlas` Value produced by this call.

Notes

Returns a contiguous depth slice `[start, stop)` from the atlas.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.slice_frames(0, 1);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

23.1.26 sort_depth

Signature

```
Atlas sort_depth(self, bool ascending = true, bool nulls_last = true) const
```

Parameters

`ascending` : `bool` See the signature for the supported argument form.

`nulls_last` : `bool` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

Sorts frames by their current depth labels.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto labeled = pluss::table::Atlas({f0, f1}, pluss::table::Thread(pluss::make_note({"test
↪", "train"})));
auto out = labeled.sort_depth(true, true);
```

Output

```
frame0:
sales
10.0
20.0
```

(continues on next page)

(continued from previous page)

```
frame1:
sales
30.0
40.0
```

23.1.27 squeeze

Signature

```
DataFrame squeeze(self) const
```

Parameters

None

Returns

`DataFrame` Value produced by this call.

Notes

Collapses a depth-1 atlas back into a single `DataFrame`.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.squeeze();
```

Output

```
sales
10.0
20.0
```

23.1.28 tabelize

Signature

```
static Atlas tabelize(const pluss::kindle::Archive& archive)
```

Parameters

`archive` : `const pluss::kindle::Archive&` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `static Atlas tabelize(const pluss::kindle::Archive& archive, const pluss::kindle::TabelizePolicy& policy)`

Examples

```
using namespace pluss::kindle::literals;
auto c0 = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_series(
↪{10.0, 20.0}))});
auto c1 = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_series(
↪{30.0, 40.0}))});
auto archive = pluss::kindle::Archive({c0, c1});
auto out = pluss::table::Atlas::tabelize(archive);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

23.1.29 `tabelize`

Signature

```
static Atlas tabelize(const pluss::kindle::Archive& archive, const␣
↪pluss::kindle::TabelizePolicy& policy)
```

Parameters

`archive` : `const pluss::kindle::Archive&` See the signature for the supported argument form.

`policy` : `const pluss::kindle::TabelizePolicy&` See the signature for the supported argument form.

Returns

`Atlas` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- `static Atlas tabelize(const pluss::kindle::Archive& archive)`

Examples

```
using namespace pluss::kindle::literals;
auto c0 = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_series(
↪{10.0, 20.0}))});
auto c1 = pluss::kindle::Catalog({"sales"_k = pluss::kindle::Stream(pluss::make_series(
↪{30.0, 40.0}))});
auto archive = pluss::kindle::Archive({c0, c1});
auto out = pluss::table::Atlas::tabelize(archive);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

23.1.30 `tail`

Signature

```
Atlas tail(self, std::int64_t n = 5) const
```

Parameters

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

Returns

`Atlas` Value produced by this call.

Notes

Returns the last `n` frames from the atlas depth axis.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
```

(continues on next page)

(continued from previous page)

```
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.tail(1);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

23.1.31 to_string

Signature

```
std::string to_string(self, const display::DisplayOptions& options =
```

Parameters

`options` : `const display::DisplayOptions&` See the signature for the supported argument form.

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.to_string();
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

23.1.32 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = atlas.unique();
```

Output

```
true
```

CHAPTER

TWENTYFOUR

KINDLE MODEL GUIDE

24.1 When To Use Kindle Types

The kindle model is stream-first:

- `Stream`: one typed vector plus optional validity mask and case labels
- `Catalog`: named collection of streams
- `Archive`: depth stack of catalogs

This is the right model when case alignment and stream-native transforms matter more than table presentation.

24.2 Stream Basics

```
auto metric = pluss::kindle::Stream(
    pluss::make_series({10.0, 20.0, 30.0}),
    std::nullopt,
    pluss::table::Thread(pluss::make_note({"cs0", "cs1", "cs2"})));

metric = metric.with_meta("source", "demo").with_meta("unit", "km");

auto cleaned = metric.fillna(0.0).clip(0.0, 35.0);
auto filtered = cleaned.filter(cleaned.as_series().gt(15.0));
```

24.3 Catalog Basics

```
using namespace pluss::kindle::literals;

auto id = pluss::kindle::Stream(pluss::make_series({1, 2, 3}));
auto city = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"}));
auto value = pluss::kindle::Stream(pluss::make_series({10.0, 15.0, 20.0}));

auto catalog = pluss::kindle::Catalog({
    "id"_k = id,
    "city"_k = city,
    "value"_k = value,
});

catalog.select({"id", "value"}).show();
catalog.sort("value").show();
```

24.4 Text And Similarity Helpers

```
auto text_catalog = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto clean = text_catalog.text_preprocess("text");
auto keywords = text_catalog.extract_keywords("text", 5);
auto tfidf = text_catalog.tfidf("text");
```

24.5 Archive Basics

```
auto c0 = catalog;
auto c1 = catalog.rename({{"value", "value_test"}});
auto depth = pluss::kindle::Stream(pluss::make_note({"train", "test"}));

auto archive = pluss::kindle::Archive({c0, c1}, depth);

auto first = archive.catalog(0);
auto mapped = archive.map_catalogs([](const pluss::kindle::Catalog& c) {
  return c.head(2);
});
```

24.6 Convert Back To Table Model

```
pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Outer;
policy.index_mode = pluss::kindle::IndexMode::Positional;

auto df = catalog.tabelize(policy);
auto atlas = archive.tabelize(policy);
```

CHAPTER

TWENTYFIVE

STREAM API REFERENCE

CHAPTER

TWENTYSIX

PRACTICAL STREAM API REFERENCE

26.1 API Reference Style

This page documents the full public `Stream` surface for the Kindle model.

26.1.1 `abs`

Signature

```
Stream abs(self) const
```

Parameters

None

Returns

`Stream` Value produced by this call.

Notes

Returns a numeric stream whose values are the absolute values of the current numeric payload.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.abs();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.2 `as_note`

Signature

```
const pluss::Note& as_note(self) const
```

Parameters

None

Returns

`const pluss::Note&` Value produced by this call.

Notes

Interprets or converts the current payload as a note/text stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = stream.as_note();
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

26.1.3 as_series

Signature

```
const pluss::Series& as_series(self) const
```

Parameters

None

Returns

`const pluss::Series&` Value produced by this call.

Notes

Interprets or converts the current payload as a numeric series stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.as_series();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.4 as_timeindex

Signature

```
const pluss::TimeIndex& as_timeindex(self) const
```

Parameters

None

Returns

const pluss::TimeIndex& Value produced by this call.

Notes

Interprets or converts the current payload as a time-index stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_timeindex({"2024-01-01T09:10:00Z", "2024-
↪01-01T09:40:00Z"}));

auto out = stream.as_timeindex();
```

Output

```
Timeline([...])
```

26.1.5 as_timeline

Signature

```
const pluss::Timeline& as_timeline(self) const
```

Parameters

None

Returns

const pluss::Timeline& Value produced by this call.

Notes

Interprets or converts the current payload as a timeline stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪01-01T09:40:00Z", "2024-01-01T10:05:00Z"}));

auto out = stream.as_timeline();
```

Output

```
Timeline([...])
```

26.1.6 astype

Signature

```
Stream astype(self, pluss::codem::ScalarType dtype) const
```

Parameters

`dtype` : `pluss::codem::ScalarType` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.astype(pluss::codem::ScalarType::Float64);
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.7 bfill

Signature

```
Stream bfill(self) const
```

Parameters

`None`

Returns

`Stream` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.bfill();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.8 cases

Signature

```
const std::optional<pluss::table::Thread>& cases(self) const
```

Parameters

None

Returns

const std::optional<pluss::table::Thread>& Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.cases();
```

Output

```
["cs0", "cs1", "cs2"]
```

26.1.9 cast

Signature

```
Stream cast(self, pluss::table::ThreadCastTarget target, std::optional
↪<pluss::codem::ScalarType> series_dtype = std::nullopt) const
```

Parameters

target : pluss::table::ThreadCastTarget See the signature for the supported argument form.

series_dtype : std::optional<pluss::codem::ScalarType> See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_note({"1.0", "2.0", "3.0"}));

auto out = stream.cast(pluss::table::ThreadCastTarget::Series,␣
↪pluss::codem::ScalarType::Float64);
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.10 clear_cases

Signature

```
Stream clear_cases(self) const
```

Parameters

None

Returns

`Stream` Value produced by this call.

Notes

Returns a stream with its case labels removed.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.clear_cases();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.11 clear_cases_

Signature

```
Stream& clear_cases_(self)
```

Parameters

None

Returns

Stream& Value produced by this call.

Notes

Removes the stream case labels in place.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.clear_cases_();
```

26.1.12 clear_validity_mask

Signature

```
Stream clear_validity_mask(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

Returns a stream with the validity mask removed, so all entries are treated as valid again.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.clear_validity_mask();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.13 clear_validity_mask_

Signature

```
Stream& clear_validity_mask_(self)
```

Parameters

None

Returns

Stream& Value produced by this call.

Notes

Removes the validity mask in place, so all entries are treated as valid again.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.clear_validity_mask_();
```

26.1.14 clip

Signature

```
Stream clip(self, scalar min_value, scalar max_value) const
```

Parameters

min_value : scalar See the signature for the supported argument form.

max_value : scalar See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

Clamps numeric stream values to the supplied lower and upper bounds.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.clip(0.0, 25.0);
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.15 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.defined();
```

Output

```
true
```

26.1.16 drop_meta

Signature

```
Stream drop_meta(self, std::string_view key) const
```

Parameters

key : std::string_view Name of the target slot or stream.

Returns

Stream Value produced by this call.

Notes

Returns a stream with one metadata key removed.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));
stream = stream.with_meta("unit", "km");

auto out = stream.drop_meta("unit");
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.17 drop_meta_

Signature

```
Stream& drop_meta_(self, std::string_view key)
```

Parameters

key : std::string_view Name of the target slot or stream.

Returns

Stream& Value produced by this call.

Notes

Removes one metadata key in place.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));
stream = stream.with_meta("unit", "km");

stream.drop_meta_("unit");
```

26.1.18 dropna

Signature

```
Stream dropna(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.dropna();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.19 dtype_name

Signature

```
std::string dtype_name(self) const
```

Parameters

None

Returns

std::string Rendered string result.

Notes

Returns the scalar dtype name used by the current numeric payload.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.dtype_name();
```

Output

```
Float64
```

26.1.20 exp

Signature

```
Stream exp(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

Applies the elementwise exponential to a numeric stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.exp();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.21 ffill

Signature

```
Stream ffill(self) const
```

Parameters

None

Returns

`Stream` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.ffill();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.22 fillna

Signature

```
Stream fillna(self, const char* value) const
```

Parameters

`value` : `const char*` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Stream fillna(self, scalar value) const
- Stream fillna(self, std::string_view value) const
- Stream fillna(self, const pluss::Timeline& value) const
- Stream fillna(self, const pluss::TimeIndex& value) const

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = stream.fillna("Unknown");
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

26.1.23 fillna

Signature

```
Stream fillna(self, const pluss::TimeIndex& value) const
```

Parameters

value : const pluss::TimeIndex& See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Stream fillna(self, scalar value) const
- Stream fillna(self, std::string_view value) const
- Stream fillna(self, const char* value) const
- Stream fillna(self, const pluss::Timeline& value) const

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_timeindex({"2024-01-01T09:10:00Z", "2024-
↪01-01T09:40:00Z"}));

auto out = stream.fillna(pluss::make_timeindex("2024-01-01T00:00:00Z"));
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

26.1.24 fillna

Signature

```
Stream fillna(self, const pluss::Timeline& value) const
```

Parameters

`value` : `const pluss::Timeline&` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Stream fillna(self, scalar value) const`
- `Stream fillna(self, std::string_view value) const`
- `Stream fillna(self, const char* value) const`
- `Stream fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_timeline({"2024-01-01T09:10:00Z", "2024-
↪01-01T09:40:00Z", "2024-01-01T10:05:00Z"}));

auto out = stream.fillna(pluss::make_timeline("2024-01-01T00:00:00Z"));
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

26.1.25 fillna

Signature

```
Stream fillna(self, scalar value) const
```

Parameters

`value` : `scalar` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Stream fillna(self, std::string_view value) const`
- `Stream fillna(self, const char* value) const`
- `Stream fillna(self, const pluss::Timeline& value) const`
- `Stream fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.fillna(0.0);
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

26.1.26 `fillna`

Signature

```
Stream fillna(self, std::string_view value) const
```

Parameters

`value` : `std::string_view` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Stream fillna(self, scalar value) const`
- `Stream fillna(self, const char* value) const`
- `Stream fillna(self, const pluss::Timeline& value) const`
- `Stream fillna(self, const pluss::TimeIndex& value) const`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"}));

auto out = stream.fillna("Unknown");
```

Output

```
Note(["Accra", "Tema", "Kumasi"])
```

26.1.27 filter

Signature

```
Stream filter(self, const pluss::Series& bool_mask) const
```

Parameters

bool_mask : const pluss::Series& See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

Keeps only the entries where the supplied boolean mask is true.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = stream.filter(mask);
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.28 from_thread

Signature

```
static Stream from_thread(const pluss::table::Thread& thread, std::optional
↪<pluss::table::Thread> cases = std::nullopt)
```

Parameters

thread : const pluss::table::Thread& See the signature for the supported argument form.

cases : std::optional<pluss::table::Thread> Case-label thread to attach to the current object.

Returns

`Stream` Value produced by this call.

Notes

Builds a Kindle stream from a table thread.

Examples

```cpp
auto thread = pluss::table::Thread(pluss::make_series({10.0, 20.0, 30.0}));

auto out = pluss::kindle::Stream::from_thread(thread);
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.29 has_cases

Signature

```cpp
bool has_cases(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

Reports whether the stream currently carries explicit case labels.

Examples

```cpp
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.has_cases();
```

Output

```
true
```

26.1.30 has_meta

Signature

```cpp
bool has_meta(self, std::string_view key) const
```

Parameters

key : `std::string_view` Name of the target slot or stream.

Returns

`bool` Boolean status or membership result.

Notes

Reports whether a metadata key exists on the stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream = stream.with_meta("unit", "km");
auto out = stream.has_meta("unit");
```

Output

```
true
```

26.1.31 has_validity_mask

Signature

```
bool has_validity_mask(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

Reports whether the stream currently carries a validity mask.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.has_validity_mask();
```

Output

```
true
```

26.1.32 head

Signature

```
Stream head(self, std::int64_t n) const
```

Parameters

n : std::int64_t Number of rows, entries, or depth items to keep in the preview.

Returns

Stream Value produced by this call.

Notes

Returns the first n entries of the stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.head();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.33 isnull

Signature

```
Stream isnull(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.isnull();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.34 kind

Signature

```
StreamKind kind(self) const
```

Parameters

None

Returns

StreamKind Value produced by this call.

Notes

Returns which payload kind this stream currently stores, such as Series, Note, Timeline, or TimeIndex.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.kind();
```

Output

```
Series
```

26.1.35 log

Signature

```
Stream log(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

Applies the elementwise natural logarithm to a numeric stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.log();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.36 max

Signature

```
Stream max(self) const
```

Parameters

None

Returns

`Stream` Value produced by this call.

Notes

Returns the maximum value of a numeric stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.max();
```

Output

```
Series([20.0])
```

26.1.37 mean

Signature

```
Stream mean(self) const
```

Parameters

None

Returns

`Stream` Value produced by this call.

Notes

Returns the mean value of a numeric stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.mean();
```

Output

```
Series([20.0])
```

26.1.38 meta

Signature

```
const std::unordered_map<std::string, std::string>& meta(self) const
```

Parameters

None

Returns

const std::unordered_map<std::string, std::string>& Value produced by this call.

Notes

Returns the value stored under one metadata key.

See also

- std::optional<std::string> meta(self, std::string_view key) const

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream = stream.with_meta("unit", "km");
auto out = stream.meta("unit");
```

Output

```
km
```

26.1.39 meta

Signature

```
std::optional<std::string> meta(self, std::string_view key) const
```

Parameters

key : `std::string_view` Name of the target slot or stream.

Returns

`std::optional<std::string>` Value produced by this call.

Notes

Returns the value stored under one metadata key.

See also

- `const std::unordered_map<std::string, std::string>& meta(self) const`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream = stream.with_meta("unit", "km");
auto out = stream.meta("unit");
```

Output

```
km
```

26.1.40 `min`

Signature

```
Stream min(self) const
```

Parameters

None

Returns

`Stream` Value produced by this call.

Notes

Returns the minimum value of a numeric stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.min();
```

Output

```
Series([20.0])
```

26.1.41 notnull

Signature

```
Stream notnull(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.notnull();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.42 show

Signature

```
void show(self, std::ostream& os) const
```

Parameters

os : std::ostream& See the signature for the supported argument form.

Returns

void Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.show(std::cout);
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.43 size

Signature

```
std::int64_t size(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.size();
```

Output

```
3
```

26.1.44 slice

Signature

```
Stream slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` Inclusive starting position of the slice.

`stop` : `std::int64_t` Exclusive ending position of the slice.

Returns

`Stream` Value produced by this call.

Notes

Returns a contiguous slice `[start, stop)` of the stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.slice(0, 2);
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.45 `std`

Signature

```
Stream std(self, bool unbiased = true) const
```

Parameters

`unbiased` : `bool` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

Returns the standard deviation of a numeric stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.std(true);
```

Output

```
Series([20.0])
```

26.1.46 `sum`

Signature

```
Stream sum(self) const
```

Parameters

None

Returns

Stream Value produced by this call.

Notes

Returns the sum of a numeric stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.sum();
```

Output

```
Series([20.0])
```

26.1.47 tail

Signature

```
Stream tail(self, std::int64_t n) const
```

Parameters

n : std::int64_t Number of rows, entries, or depth items to keep in the preview.

Returns

Stream Value produced by this call.

Notes

Returns the last n entries of the stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.tail();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.48 take

Signature

```
Stream take(self, std::span<const std::int64_t> indices) const
```

Parameters

`indices` : `std::span<const std::int64_t>` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

Returns the entries at the requested integer positions.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

std::vector<std::int64_t> picks{0, 2};
auto out = stream.take(picks);
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.49 to_json

Signature

```
std::string to_json(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.to_json();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.50 to_string

Signature

```
std::string to_string(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.to_string();
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.51 to_thread

Signature

```
pluss::table::Thread to_thread(self) const
```

Parameters

None

Returns

`pluss::table::Thread` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- pluss::table::Thread to_thread(self, std::int64_t start, std::int64_t stop) const

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.to_thread();
```

Output

```
[10.0, 20.0, 30.0]
```

26.1.52 to_thread

Signature

```
pluss::table::Thread to_thread(self, std::int64_t start, std::int64_t stop) const
```

Parameters

start : std::int64_t Inclusive starting position of the slice.

stop : std::int64_t Exclusive ending position of the slice.

Returns

pluss::table::Thread Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

See also

- pluss::table::Thread to_thread(self) const

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.to_thread(0, 2);
```

Output

```
[10.0, 20.0, 30.0]
```

26.1.53 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.unique();
```

Output

```
true
```

26.1.54 validity_mask

Signature

```
const std::optional<pluss::Series>& validity_mask(self) const
```

Parameters

None

Returns

const std::optional<pluss::Series>& Value produced by this call.

Notes

Returns the current validity mask as a boolean series.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.validity_mask();
```

Output

```
[true, false, true]
```

26.1.55 var

Signature

```
Stream var(self, bool unbiased = true) const
```

Parameters

unbiased : bool See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

Returns the variance of a numeric stream.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.var(true);
```

Output

```
Series([20.0])
```

26.1.56 with_cases

Signature

```
Stream with_cases(self, const pluss::table::Thread& cases) const
```

Parameters

cases : const pluss::table::Thread& Case-label thread to attach to the current object.

Returns

Stream Value produced by this call.

Notes

Returns a new stream with the supplied case labels attached.

Examples

```
auto cases = pluss::table::Thread(pluss::make_note({"cs0", "cs1", "cs2"}));
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.with_cases(cases);
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.57 with_cases_

Signature

```
Stream& with_cases_(self, const pluss::table::Thread& cases)
```

Parameters

`cases` : `const pluss::table::Thread&` Case-label thread to attach to the current object.

Returns

`Stream&` Value produced by this call.

Notes

Attaches the supplied case labels in place.

Examples

```
auto cases = pluss::table::Thread(pluss::make_note({"cs0", "cs1", "cs2"}));
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.with_cases_(cases);
```

26.1.58 with_meta

Signature

```
Stream with_meta(self, std::initializer_list<std::pair<std::string_view, std::string_
↪view>> items) const
```

Parameters

`items` : `std::initializer_list<std::pair<std::string_view, std::string_view>>` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

Returns a new stream with one metadata key/value attached.

See also

- `Stream with_meta(self, std::string key, std::string value) const`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.with_meta("unit", "km");
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.59 with_meta

Signature

```
Stream with_meta(self, std::string key, std::string value) const
```

Parameters

`key` : `std::string` Name of the target slot or stream.

`value` : `std::string` See the signature for the supported argument form.

Returns

`Stream` Value produced by this call.

Notes

Returns a new stream with one metadata key/value attached.

See also

- `Stream with_meta(self, std::initializer_list<std::pair<std::string_view, std::string_view>> items) const`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.with_meta("unit", "km");
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.60 with_meta_

Signature

```
Stream& with_meta_(self, std::initializer_list<std::pair<std::string_view, std::string_
↪view>> items)
```

Parameters

`items` : `std::initializer_list<std::pair<std::string_view, std::string_view>>` See the signature for the supported argument form.

Returns

`Stream&` Value produced by this call.

Notes

Adds or updates one metadata key/value in place.

See also

- `Stream& with_meta_(self, std::string key, std::string value)`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.with_meta_("unit", "km");
```

26.1.61 with_meta_

Signature

```
Stream& with_meta_(self, std::string key, std::string value)
```

Parameters

`key` : `std::string` Name of the target slot or stream.

`value` : `std::string` See the signature for the supported argument form.

Returns

`Stream&` Value produced by this call.

Notes

Adds or updates one metadata key/value in place.

See also

- `Stream& with_meta_(self, std::initializer_list<std::pair<std::string_view, std::string_view>> items)`

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.with_meta_("unit", "km");
```

26.1.62 with_validity_mask

Signature

```
Stream with_validity_mask(self, const pluss::Series& mask) const
```

Parameters

mask : const pluss::Series& See the signature for the supported argument form.

Returns

Stream Value produced by this call.

Notes

Returns a new stream with the supplied validity mask attached.

Examples

```
auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = stream.with_validity_mask(mask);
```

Output

```
Series([10.0, 20.0, 30.0])
```

26.1.63 with_validity_mask_

Signature

```
Stream& with_validity_mask_(self, const pluss::Series& mask)
```

Parameters

mask : const pluss::Series& See the signature for the supported argument form.

Returns

Stream& Value produced by this call.

Notes

Attaches the supplied validity mask in place.

Examples

```
auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

stream.with_validity_mask_(mask);
```

CHAPTER

TWENTYSEVEN

CATALOG API REFERENCE

CHAPTER

TWENTYEIGHT

PRACTICAL CATALOG API REFERENCE

28.1 API Reference Style

This page documents the full public `Catalog` surface for named stream collections.

28.1.1 add_text_features

Signature

```
Catalog add_text_features(self, std::string_view field) const
```

Parameters

`field` : `std::string_view` Name of the target stream in the catalog.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.add_text_features("text");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.2 aggregate_text_stats

Signature

```
Catalog aggregate_text_stats(self, std::string_view field, std::string_view stat_type)␣
↪const
```

Parameters

`field` : `std::string_view` Name of the target stream in the catalog.

`stat_type` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.aggregate_text_stats("text", "mean");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

(continues on next page)

(continued from previous page)

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.3 apply

Signature

```
Catalog apply(self, std::string_view field, const std::function<Stream(const Stream&)>&
↪func) const
```

Parameters

`field` : `std::string_view` Name of the target stream in the catalog.

`func` : `const std::function<Stream(const Stream&)>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Applies a function to one named stream and returns the updated catalog.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.apply("metric", [](const pluss::kindle::Stream& s) { return s.clip(0.0, 25.
↪0); });
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.4 apply_

Signature

```
Catalog& apply_(self, std::string_view field, const std::function<Stream(const Stream&)>&
↪ func)
```

Parameters

`field` : `std::string_view` Name of the target stream in the catalog.

`func` : `const std::function<Stream(const Stream&)>&` See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

Applies a function to one named stream in place.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.apply_("metric", [](const pluss::kindle::Stream& s) { return s.clip(0.0, 25.0); });
```

28.1.5 apply_all

Signature

```
Catalog apply_all(self, const std::function<Stream(const Stream&)>& fn) const
```

Parameters

`fn` : `const std::function<Stream(const Stream&)>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Applies the same function to every stream and returns the updated catalog.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.apply_all([](const pluss::kindle::Stream& s) { return s.kind() ==␣
↪pluss::kindle::StreamKind::Series ? s.clip(0.0, 25.0) : s; });
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.6 apply_all_

Signature

```
Catalog& apply_all_(self, const std::function<Stream(const Stream&)>& fn)
```

Parameters

`fn` : `const std::function<Stream(const Stream&)>&` See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

Applies the same function to every stream in place.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});
```

(continues on next page)

(continued from previous page)

```
ct.apply_all_([](const pluss::kindle::Stream& s) { return s.kind() ==␣
↪pluss::kindle::StreamKind::Series ? s.clip(0.0, 25.0) : s; });
```

28.1.7 astype

Signature

```
Catalog astype(self, std::initializer_list<std::string_view> keys,␣
↪pluss::codem::ScalarType dtype) const
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

dtype : pluss::codem::ScalarType See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.astype({"metric"}, pluss::codem::ScalarType::Float64);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.8 at

Signature

```
Stream at(self, std::string_view key) const
```

Parameters

key : std::string_view Name of the target slot or stream.

Returns

Stream Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.at("metric");
```

Output

```
Series([10.0, 20.0, 30.0])
```

28.1.9 begin

Signature

```
const_iterator begin(self) const
```

Parameters

None

Returns

const_iterator Value produced by this call.

Notes

Exposes iterator boundaries so the object can be traversed with a range-based loop or manual iteration.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

for (const auto& key : ct) {
  (void)key;
}
```

28.1.10 bfill

Signature

```
Catalog bfill(self, std::initializer_list<std::string_view> keys =
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

Catalog Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.bfill();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
```

(continues on next page)

(continued from previous page)

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.11 cast_note

Signature

```
Catalog cast_note(self, std::initializer_list<std::string_view> keys) const
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

Catalog Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.cast_note({"metric"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.12 cast_series

Signature

```
Catalog cast_series(self, std::initializer_list<std::string_view> keys, std::optional
↪<pluss::codem::ScalarType> dtype = std::nullopt) const
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

dtype : std::optional<pluss::codem::ScalarType> See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
using namespace pluss::kindle::literals;
auto ct = pluss::kindle::Catalog({"metric"_k = pluss::kindle::Stream(pluss::make_note({
↪"10.0", "20.0", "30.0"}))});

auto out = ct.cast_series({"metric"}, pluss::codem::ScalarType::Float64);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.13 cast_timeindex

Signature

```
Catalog cast_timeindex(self, std::initializer_list<std::string_view> keys) const
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.cast_timeindex({"metric"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.14 cast_timeline

Signature

```
Catalog cast_timeline(self, std::initializer_list<std::string_view> keys) const
```

Parameters

`keys` : `std::initializer_list<std::string_view>` Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Notes

This changes the semantic or numeric type so later operations route through the correct implementation path.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.cast_timeline({"metric"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.15 compute_similarity

Signature

```
Catalog compute_similarity(self, std::string_view field1, std::string_view field2,␣
↪std::string_view method = "cosine") const
```

Parameters

`field1` : `std::string_view` See the signature for the supported argument form.

`field2` : `std::string_view` See the signature for the supported argument form.

`method` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
```

(continues on next page)

(continued from previous page)

```
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.compute_similarity("text", "text", "cosine");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.16 count

Signature

```
std::int64_t count(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.count();
```

Output

```
3
```

28.1.17 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.defined();
```

Output

```
true
```

28.1.18 display

Signature

```
void display(self) const
```

Parameters

None

Returns

void Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.display();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.19 drop

Signature

```
Catalog drop(self, const std::vector<std::string>& field_names) const
```

Parameters

`field_names` : `const std::vector<std::string>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Removes the named streams and keeps the remaining catalog structure intact.

See also

- `Catalog drop(self, std::string_view field_name) const`
- `Catalog drop(self, std::initializer_list<std::string_view> keys) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
```

(continues on next page)

(continued from previous page)

```
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

std::vector<std::string> names{"city"};
auto out = ct.drop(names);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.20 drop

Signature

```
Catalog drop(self, std::initializer_list<std::string_view> keys) const
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

Catalog Value produced by this call.

Notes

Removes the named streams and keeps the remaining catalog structure intact.

See also

- Catalog drop(self, std::string_view field_name) const
- Catalog drop(self, const std::vector<std::string>& field_names) const

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});
```

(continues on next page)

(continued from previous page)

```
auto out = ct.drop({"city"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.21 drop

Signature

```
Catalog drop(self, std::string_view field_name) const
```

Parameters

`field_name` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Removes the named streams and keeps the remaining catalog structure intact.

See also

- `Catalog drop(self, const std::vector<std::string>& field_names) const`
- `Catalog drop(self, std::initializer_list<std::string_view> keys) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.drop("city");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.22 dropna

Signature

```
Catalog dropna(self, std::initializer_list<std::string_view> keys =
```

Parameters

`keys` : `std::initializer_list<std::string_view>` Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.dropna();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.23 empty

Signature

```
bool empty(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the current object has no rows, entries, slots, or depth items.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.empty();
```

Output

```
false
```

28.1.24 end

Signature

```
const_iterator end(self) const
```

Parameters

None

Returns

const_iterator Value produced by this call.

Notes

Exposes iterator boundaries so the object can be traversed with a range-based loop or manual iteration.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

for (const auto& key : ct) {
  (void)key;
}
```

28.1.25 extract_keywords

Signature

```
Catalog extract_keywords(self, std::string_view field, int top_n = 10) const
```

Parameters

`field` : `std::string_view` Name of the target stream in the catalog.

`top_n` : `int` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.extract_keywords("text", 5);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
```

(continues on next page)

(continued from previous page)

```
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.26 `ffill`

Signature

```
Catalog ffill(self, std::initializer_list<std::string_view> keys =
```

Parameters

`keys` : `std::initializer_list<std::string_view>` Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.ffill();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.27 fillna

Signature

```
Catalog fillna(self, const char* value, std::initializer_list<std::string_view> keys =
```

Parameters

value : const char* See the signature for the supported argument form.

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

Catalog Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Catalog fillna(self, scalar value, std::initializer_list<std::string_view> keys =
- Catalog fillna(self, std::string_view value, std::initializer_list<std::string_view> keys =
- Catalog fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> keys =
- Catalog fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> keys =

Examples

```
using namespace pluss::kindle::literals;
auto ct = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note({
↪"Accra", "Tema", "Kumasi"}))});

auto out = ct.fillna("Unknown", {"city"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.28 `fillna`

Signature

```
Catalog fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_
↪view> keys =
```

Parameters

`value` : `const pluss::TimeIndex&` See the signature for the supported argument form.

`keys` : `std::initializer_list<std::string_view>` Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Catalog fillna(self, scalar value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, std::string_view value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const char* value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> keys =`

Examples

```
using namespace pluss::kindle::literals;
auto ct = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note({
↪"Accra", "Tema", "Kumasi"}))});

auto out = ct.fillna("Unknown", {"city"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.29 fillna

Signature

```
Catalog fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_
↪view> keys =
```

Parameters

value : const pluss::Timeline& See the signature for the supported argument form.

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

Catalog Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Catalog fillna(self, scalar value, std::initializer_list<std::string_view> keys =
- Catalog fillna(self, std::string_view value, std::initializer_list<std::string_view> keys =
- Catalog fillna(self, const char* value, std::initializer_list<std::string_view> keys =
- Catalog fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> keys =

Examples

```
using namespace pluss::kindle::literals;
auto ct = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note({
↪"Accra", "Tema", "Kumasi"}))});

auto out = ct.fillna("Unknown", {"city"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.30 `fillna`

Signature

```
Catalog fillna(self, scalar value, std::initializer_list<std::string_view> keys =
```

Parameters

`value` : `scalar` See the signature for the supported argument form.

`keys` : `std::initializer_list<std::string_view>` Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- `Catalog fillna(self, std::string_view value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const char* value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> keys =`
- `Catalog fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> keys =`

Examples

```
using namespace pluss::kindle::literals;
auto ct = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note({
↪"Accra", "Tema", "Kumasi"}))});

auto out = ct.fillna("Unknown", {"city"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.31 fillna

Signature

```
Catalog fillna(self, std::string_view value, std::initializer_list<std::string_view>␣
↪keys =
```

Parameters

value : std::string_view See the signature for the supported argument form.

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

Catalog Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

See also

- Catalog fillna(self, scalar value, std::initializer_list<std::string_view> keys =
- Catalog fillna(self, const char* value, std::initializer_list<std::string_view> keys =
- Catalog fillna(self, const pluss::Timeline& value, std::initializer_list<std::string_view> keys =
- Catalog fillna(self, const pluss::TimeIndex& value, std::initializer_list<std::string_view> keys =

Examples

```
using namespace pluss::kindle::literals;
auto ct = pluss::kindle::Catalog({"city"_k = pluss::kindle::Stream(pluss::make_note({
↪"Accra", "Tema", "Kumasi"}))});

auto out = ct.fillna("Unknown", {"city"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.32 filter

Signature

```
Catalog filter(self, const pluss::Series& condition) const
```

Parameters

condition : const pluss::Series& See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

Keeps only the rows where the supplied mask stream is true.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = ct.filter(mask);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.33 filter_by_text

Signature

```
Catalog filter_by_text(self, std::string_view field, std::string_view pattern) const
```

Parameters

field : std::string_view Name of the target stream in the catalog.

pattern : std::string_view See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.filter_by_text("text", "rain");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.34 `from_json`

Signature

```
static Catalog from_json(std::string_view json_text)
```

Parameters

`json_text` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Builds a catalog from JSON input using the PlussMic JSON loader route.

Examples

```
auto out = pluss::kindle::Catalog::from_json(R"([{"id":1,"city":"Accra"},{"id":2,"city":
↪"Tema"}])");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.35 get

Signature

```
std::optional<Stream> get(self, std::string_view key) const
```

Parameters

key : std::string_view Name of the target slot or stream.

Returns

std::optional<Stream> Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.get("metric");
```

Output

```
Series([10.0, 20.0, 30.0])
```

28.1.36 get_stream

Signature

```
std::optional<Stream> get_stream(self, std::string_view key) const
```

Parameters

key : std::string_view Name of the target slot or stream.

Returns

std::optional<Stream> Value produced by this call.

Notes

Returns one named stream from the catalog.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.get_stream("metric");
```

Output

```
Series([10.0, 20.0, 30.0])
```

28.1.37 group_by

Signature

```
std::unordered_map<std::string, Catalog> group_by(self, std::string_view key_field) const
```

Parameters

key_field : std::string_view See the signature for the supported argument form.

Returns

std::unordered_map<std::string, Catalog> Value produced by this call.

Notes

Splits the catalog into sub-catalogs keyed by the distinct values in the grouping stream.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
```

(continues on next page)

(continued from previous page)

```
});

auto out = ct.group_by("city");
```

Output

```
{"Accra": Catalog(...), "Tema": Catalog(...)}
```

28.1.38 has

Signature

```
bool has(self, std::string_view key) const
```

Parameters

key : std::string_view Name of the target slot or stream.

Returns

bool Boolean status or membership result.

Notes

Checks whether a named stream exists.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.has("metric");
```

Output

```
true
```

28.1.39 has_stream

Signature

```
bool has_stream(self, std::string_view key) const
```

Parameters

key : std::string_view Name of the target slot or stream.

Returns

bool Boolean status or membership result.

Notes

Checks whether a named stream exists.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.has_stream("metric");
```

Output

```
true
```

28.1.40 head

Signature

```
Catalog head(self, std::int64_t n) const
```

Parameters

n : std::int64_t Number of rows, entries, or depth items to keep in the preview.

Returns

Catalog Value produced by this call.

Notes

Returns the first n rows across every stream in the catalog.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
```

(continues on next page)

(continued from previous page)

```
});

auto out = ct.head(2);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.41 iloc

Signature

```
Catalog iloc(self, std::int64_t row) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- `Catalog iloc(self, std::int64_t start, std::int64_t stop) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.iloc(1);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.42 `iloc`

Signature

```
Catalog iloc(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` Inclusive starting position of the slice.

`stop` : `std::int64_t` Exclusive ending position of the slice.

Returns

`Catalog` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- `Catalog iloc(self, std::int64_t row) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.iloc(0, 2);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

(continues on next page)

(continued from previous page)

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.43 info

Signature

```
void info(self) const
```

Parameters

None

Returns

void Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.info();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.44 isnull

Signature

```
Catalog isnull(self, std::initializer_list<std::string_view> keys =
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

Catalog Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.isnull();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.45 keys

Signature

```
const std::vector<std::string>& keys(self) const
```

Parameters

None

Returns

const std::vector<std::string>& Borrowed list of names.

Notes

Returns the current labels used to address fields, slots, rows, or depth items.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.keys();
```

Output

```
["id", "metric", "city"]
```

28.1.46 kindelize

Signature

```
static Catalog kindelize(const pluss::table::DataFrame& frame, const KindelizePolicy&␣
↪policy =
```

Parameters

`frame` : `const pluss::table::DataFrame&` See the signature for the supported argument form.

`policy` : `const KindelizePolicy&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto df = pluss::table::DataFrame({
    {"id", pluss::table::Thread(pluss::make_series({1, 2, 3}))},
    {"sales", pluss::table::Thread(pluss::make_series({100.0, 120.0, 90.0}))},
    {"city", pluss::table::Thread(pluss::make_note({"Accra", "Tema", "Kumasi"}))},
});

auto out = pluss::kindle::Catalog::kindelize(df);
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.47 load

Signature

```
static Catalog load(std::string_view source_type, std::string_view filepath)
```

Parameters

`source_type` : `std::string_view` Storage format selector such as `"csv"`, `"xlsx"`, `"json"`, or `"parquet"`.

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::kindle::Catalog::load("csv", "path/to/data.csv");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.48 load

Signature

```
static Catalog load(std::string_view source_type, std::string_view filepath, const␣
↪pluss::loader::Config& config)
```

Parameters

`source_type` : `std::string_view` Storage format selector such as `"csv"`, `"xlsx"`, `"json"`, or `"parquet"`.

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load(std::string_view source_type, std::string_view filepath)`

Examples

```
auto out = pluss::kindle::Catalog::load("csv", "path/to/data.csv");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.49 `load_csv`

Signature

```
static Catalog load_csv(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`

Examples

```
auto out = pluss::kindle::Catalog::load_csv("path/to/data.csv");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.50 load_csv

Signature

```
static Catalog load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions&␣
↪options)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`options` : `const pluss::loader::ReadCsvOptions&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_csv(std::string_view filepath)`

Examples

```
pluss::loader::ReadCsvOptions options;
auto out = pluss::kindle::Catalog::load_csv("path/to/data.csv", options);
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.51 load_csv_with_key

Signature

```
static Catalog load_csv_with_key(std::string_view filepath, std::string_view key_field,␣
↪const pluss::loader::ReadCsvOptions& options =
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`key_field` : `std::string_view` See the signature for the supported argument form.

`options` : `const pluss::loader::ReadCsvOptions&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

Examples

```
auto out = pluss::kindle::Catalog::load_csv_with_key("path/to/data.csv", "id");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.52 `load_excel`

Signature

```
static Catalog load_excel(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`

Examples

```
auto out = pluss::kindle::Catalog::load_excel("path/to/data.xlsx");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.53 load_excel

Signature

```
static Catalog load_excel(std::string_view filepath, const␣
↪pluss::loader::ReadExcelOptions& options)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`options` : `const pluss::loader::ReadExcelOptions&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_excel(std::string_view filepath)`

Examples

```
pluss::loader::ReadExcelOptions options;
auto out = pluss::kindle::Catalog::load_excel("path/to/data.xlsx", options);
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.54 load_json

Signature

```
static Catalog load_json(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_json(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::kindle::Catalog::load_json("path/to/data.json");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.55 load_json

Signature

```
static Catalog load_json(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_json(std::string_view filepath)`

Examples

```
auto out = pluss::kindle::Catalog::load_json("path/to/data.json", {"nrows", 16});
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.56 load_parquet

Signature

```
static Catalog load_parquet(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_parquet(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::kindle::Catalog::load_parquet("path/to/data.parquet");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.57 load_parquet

Signature

```
static Catalog load_parquet(std::string_view filepath, const pluss::loader::Config&␣
↪config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_parquet(std::string_view filepath)`

Examples

```
auto out = pluss::kindle::Catalog::load_parquet("path/to/data.parquet", {"nrows", 16});
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.58 load_txt

Signature

```
static Catalog load_txt(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_txt(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::kindle::Catalog::load_txt("path/to/data.txt");
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.59 `load_txt`

Signature

```
static Catalog load_txt(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Catalog load_txt(std::string_view filepath)`

Examples

```
auto out = pluss::kindle::Catalog::load_txt("path/to/data.txt", {"nrows", 16});
```

Output

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.60 masked_select

Signature

```
Catalog masked_select(self, const pluss::Series& mask) const
```

Parameters

mask : const pluss::Series& See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

Keeps only the rows where the supplied mask stream is true.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = ct.masked_select(mask);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.61 max

Signature

```
Stream max(self, std::string_view field) const
```

Parameters

field : std::string_view Name of the target stream in the catalog.

Returns

`Stream` Value produced by this call.

Notes

Computes the maximum value of one named numeric stream.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.max("metric");
```

Output

```
Series([20.0])
```

28.1.62 mean

Signature

```
Stream mean(self, std::string_view field) const
```

Parameters

`field` : `std::string_view` Name of the target stream in the catalog.

Returns

`Stream` Value produced by this call.

Notes

Computes the mean value of one named numeric stream.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.mean("metric");
```

Output

```
Series([20.0])
```

28.1.63 merge

Signature

```
Catalog merge(self, const Catalog& other, std::string_view on, std::string_view how =
↪"inner") const
```

Parameters

other : const Catalog& See the signature for the supported argument form.

on : std::string_view See the signature for the supported argument form.

how : std::string_view See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

Combines two catalogs by matching rows on a key stream.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

using namespace pluss::kindle::literals;
auto right = pluss::kindle::Catalog({"id"_k = pluss::kindle::Stream(pluss::make_series(
↪{1, 2, 3})), "score"_k = pluss::kindle::Stream(pluss::make_series({0.9, 0.7, 0.8}))});
auto out = ct.merge(right, "id", "inner");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.64 min

Signature

```
Stream min(self, std::string_view field) const
```

Parameters

`field` : `std::string_view` Name of the target stream in the catalog.

Returns

`Stream` Value produced by this call.

Notes

Computes the minimum value of one named numeric stream.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.min("metric");
```

Output

```
Series([20.0])
```

28.1.65 nfields

Signature

```
std::int64_t nfields(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.nfields();
```

Output

```
3
```

28.1.66 note

Signature

```
Catalog note(self) const
```

Parameters

None

Returns

Catalog Value produced by this call.

Notes

Returns one named stream interpreted as a note/text stream.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.note();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

(continues on next page)

(continued from previous page)

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.67 notnull

Signature

```
Catalog notnull(self, std::initializer_list<std::string_view> keys =
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

Catalog Value produced by this call.

Notes

This is part of the missing-data cleanup path and is usually applied before summaries, grouping, or export.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.notnull();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.68 nstreams

Signature

```
std::int64_t nstreams(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.nstreams();
```

Output

```
3
```

28.1.69 query

Signature

```
Catalog query(self, const QueryFn& fn) const
```

Parameters

`fn` : `const QueryFn&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Keeps only the rows selected by the supplied row-wise query function or expression.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.query([](const pluss::kindle::Catalog& c) { return c.stream("metric").as_
↪series().gt(15.0); });
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.70 `rename`

Signature

```
Catalog rename(self, const std::unordered_map<std::string, std::string>& mapping) const
```

Parameters

`mapping` : `const std::unordered_map<std::string, std::string>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Renames one or more streams without changing their values.

See also

- `Catalog rename(self, std::initializer_list<std::pair<std::string_view, std::string_view>> mapping) const`
- `Catalog rename(self, std::string_view from, std::string_view to) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

std::unordered_map<std::string, std::string> renames{{"metric", "distance_km"}};
auto out = ct.rename(renames);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.71 `rename`

Signature

```
Catalog rename(self, std::initializer_list<std::pair<std::string_view, std::string_view>>
↪ mapping) const
```

Parameters

`mapping` : `std::initializer_list<std::pair<std::string_view, std::string_view>>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Renames one or more streams without changing their values.

See also

- `Catalog rename(self, std::string_view from, std::string_view to) const`
- `Catalog rename(self, const std::unordered_map<std::string, std::string>& mapping) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.rename({{"metric", "distance_km"}});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.72 `rename`

Signature

```
Catalog rename(self, std::string_view from, std::string_view to) const
```

Parameters

`from` : `std::string_view` See the signature for the supported argument form.

`to` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Renames one or more streams without changing their values.

See also

- `Catalog rename(self, std::initializer_list<std::pair<std::string_view, std::string_view>> mapping) const`
- `Catalog rename(self, const std::unordered_map<std::string, std::string>& mapping) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.rename("metric", "distance_km");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.73 rename_

Signature

```
Catalog& rename_(self, std::string_view from, std::string_view to)
```

Parameters

`from` : `std::string_view` See the signature for the supported argument form.

`to` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

Renames one or more streams in place without changing their values.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});
```

(continues on next page)

(continued from previous page)

```
ct.rename_("metric", "distance_km");
```

28.1.74 sample

Signature

```
Catalog sample(self, std::int64_t n, std::optional<std::uint64_t> random_state =␣
↪std::nullopt) const
```

Parameters

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

`random_state` : `std::optional<std::uint64_t>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Returns a random sample of rows across all streams.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.sample(2, 7);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.75 save_as

Signature

```
void save_as(self, std::string_view target_type, std::string_view filename, const␣
↪pluss::loader::Config& config =
```

Parameters

`target_type` : `std::string_view` See the signature for the supported argument form.

`filename` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the catalog using the explicitly supplied output format selector.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_as("json", "path/to/output.json");
```

Output

```
saved: out.*
```

28.1.76 save_csv

Signature

```
void save_csv(self, std::string_view filename) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the catalog to CSV after aligning its streams into a table-like representation.

See also

- `void save_csv(self, std::string_view filename, const pluss::loader::Config& config) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

28.1.77 save_csv

Signature

```
void save_csv(self, std::string_view filename, const pluss::loader::Config& config) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the catalog to CSV after aligning its streams into a table-like representation.

See also

- `void save_csv(self, std::string_view filename) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

28.1.78 save_excel

Signature

```
void save_excel(self, std::string_view filename) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the catalog to an Excel workbook after aligning its streams into a table-like representation.

See also

- `void save_excel(self, std::string_view filename, const pluss::loader::Config& config) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

28.1.79 save_excel

Signature

```
void save_excel(self, std::string_view filename, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the catalog to an Excel workbook after aligning its streams into a table-like representation.

See also

- `void save_excel(self, std::string_view filename) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

28.1.80 save_json

Signature

```
void save_json(self, std::string_view filename) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the catalog to JSON for interchange or later reload.

See also

- `void save_json(self, std::string_view filename, const pluss::loader::Config& config) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_json("path/to/output.json");
```

Output

```
saved: out.*
```

28.1.81 save_json

Signature

```
void save_json(self, std::string_view filename, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the catalog to JSON for interchange or later reload.

See also

- `void save_json(self, std::string_view filename) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_json("path/to/output.json");
```

Output

```
saved: out.*
```

28.1.82 save_parquet

Signature

```
void save_parquet(self, std::string_view filename) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the catalog to Parquet for compact typed storage.

See also

- `void save_parquet(self, std::string_view filename, const pluss::loader::Config& config) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_parquet("path/to/output.parquet");
```

Output

```
saved: out.*
```

28.1.83 save_parquet

Signature

```
void save_parquet(self, std::string_view filename, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the catalog to Parquet for compact typed storage.

See also

- `void save_parquet(self, std::string_view filename) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_parquet("path/to/output.parquet");
```

Output

```
saved: out.*
```

28.1.84 save_txt

Signature

```
void save_txt(self, std::string_view filename) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes a text rendering of the catalog for inspection or logging.

See also

- `void save_txt(self, std::string_view filename, const pluss::loader::Config& config) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_txt("path/to/output.txt");
```

Output

```
saved: out.*
```

28.1.85 `save_txt`

Signature

```
void save_txt(self, std::string_view filename, const pluss::loader::Config& config) const
```

Parameters

`filename` : `std::string_view` See the signature for the supported argument form.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes a text rendering of the catalog for inspection or logging.

See also

- `void save_txt(self, std::string_view filename) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.save_txt("path/to/output.txt");
```

Output

```
saved: out.*
```

28.1.86 `select`

Signature

```
Catalog select(self, const std::vector<std::string>& field_names) const
```

Parameters

`field_names` : `const std::vector<std::string>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Keeps only the named streams and preserves their shared row alignment.

See also

- `Catalog select(self, std::initializer_list<std::string_view> field_names) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

std::vector<std::string> names{"id", "metric"};
auto out = ct.select(names);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.87 select

Signature

```
Catalog select(self, std::initializer_list<std::string_view> field_names) const
```

Parameters

`field_names` : `std::initializer_list<std::string_view>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Keeps only the named streams and preserves their shared row alignment.

See also

- `Catalog select(self, const std::vector<std::string>& field_names) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.select({"id", "metric"});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

(continues on next page)

(continued from previous page)

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.88 `series`

Signature

```
Catalog series(self) const
```

Parameters

None

Returns

`Catalog` Value produced by this call.

Notes

Returns one named stream interpreted as a numeric series stream.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.series();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.89 `set`

Signature

```
Catalog set(self, std::string_view key, const Stream& value) const
```

Parameters

key : std::string_view Name of the target slot or stream.

value : const Stream& See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

Adds or replaces one named stream and returns the updated catalog.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.set_stream("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1})));
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.90 set_

Signature

```
Catalog& set_(self, std::string_view key, const Stream& value)
```

Parameters

key : std::string_view Name of the target slot or stream.

value : const Stream& See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

Adds or replaces one named stream in place.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.set_stream_("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1})));
```

28.1.91 `set_stream`

Signature

```
Catalog set_stream(self, std::string_view key, const Stream& value) const
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

`value` : `const Stream&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Adds or replaces one named stream and returns the updated catalog.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.set_stream("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1})));
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.92 set_stream_

Signature

```
Catalog& set_stream_(self, std::string_view key, const Stream& value)
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

`value` : `const Stream&` See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

Adds or replaces one named stream in place.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.set_stream_("flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1})));
```

28.1.93 set_streams

Signature

```
Catalog set_streams(self, std::initializer_list<NamedStream> items) const
```

Parameters

`items` : `std::initializer_list<NamedStream>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Adds or replaces several named streams and returns the updated catalog.

See also

- `Catalog set_streams(self, std::initializer_list<std::pair<std::string_view, Stream>> items) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

using namespace pluss::kindle::literals;
auto out = ct.set_streams({
    "flag"_k = pluss::kindle::Stream(pluss::make_series({1, 0, 1})),
});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.94 set_streams

Signature

```
Catalog set_streams(self, std::initializer_list<std::pair<std::string_view, Stream>>␣
↪items) const
```

Parameters

`items` : `std::initializer_list<std::pair<std::string_view, Stream>>` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Adds or replaces several named streams and returns the updated catalog.

See also

- `Catalog set_streams(self, std::initializer_list<NamedStream> items) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.set_streams({
    {"flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1}))},
});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.95 `set_streams_`

Signature

```
Catalog& set_streams_(self, std::initializer_list<NamedStream> items)
```

Parameters

`items` : `std::initializer_list<NamedStream>` See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

Adds or replaces several named streams in place.

See also

- `Catalog& set_streams_(self, std::initializer_list<std::pair<std::string_view, Stream>> items)`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

using namespace pluss::kindle::literals;
ct.set_streams_({
    "flag"_k = pluss::kindle::Stream(pluss::make_series({1, 0, 1})),
});
```

28.1.96 `set_streams_`

Signature

```
Catalog& set_streams_(self, std::initializer_list<std::pair<std::string_view, Stream>>␣
↪items)
```

Parameters

`items` : `std::initializer_list<std::pair<std::string_view, Stream>>` See the signature for the supported argument form.

Returns

`Catalog&` Value produced by this call.

Notes

Adds or replaces several named streams in place.

See also

- `Catalog& set_streams_(self, std::initializer_list<NamedStream> items)`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

ct.set_streams_({
    {"flag", pluss::kindle::Stream(pluss::make_series({1, 0, 1}))},
});
```

28.1.97 show

Signature

```
void show(self, std::ostream& os) const
```

Parameters

os : std::ostream& See the signature for the supported argument form.

Returns

void Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.show();
```

28.1.98 slice

Signature

```
Catalog slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` Inclusive starting position of the slice.

`stop` : `std::int64_t` Exclusive ending position of the slice.

Returns

`Catalog` Value produced by this call.

Notes

Returns a contiguous row slice `[start, stop)` across every stream.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.slice(0, 2);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.99 `sort`

Signature

```
Catalog sort(self, std::string_view by_field, bool ascending = true) const
```

Parameters

`by_field` : `std::string_view` See the signature for the supported argument form.

`ascending` : `bool` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Reorders rows using one stream as the sort key, while keeping the other streams aligned to the same row order.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.sort("metric", false);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.100 sort_by

Signature

```
Catalog sort_by(self, const std::vector<std::string>& fields, const std::vector<bool>&␣
↪ascending =
```

Parameters

`fields` : `const std::vector<std::string>&` See the signature for the supported argument form.

`ascending` : `const std::vector<bool>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Reorders rows using multiple sort keys, while keeping all streams aligned.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.sort_by({"city", "id"}, {true, false});
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.101 `stream`

Signature

```
Stream stream(self, std::string_view key) const
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

Returns

`Stream` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.stream("metric");
```

Output

```
Series([10.0, 20.0, 30.0])
```

28.1.102 streams

Signature

```
std::vector<Stream> streams(self) const
```

Parameters

None

Returns

`std::vector<Stream>` Materialized collection of stream objects.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.streams();
```

Output

```
[Stream(id), Stream(metric), Stream(city)]
```

28.1.103 sum

Signature

```
Stream sum(self, std::string_view field) const
```

Parameters

`field` : `std::string_view` Name of the target stream in the catalog.

Returns

`Stream` Value produced by this call.

Notes

Computes the sum of one named numeric stream.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.sum("metric");
```

Output

```
Series([20.0])
```

28.1.104 `tabelize`

Signature

```
pluss::table::DataFrame tabelize(self, const TabelizePolicy& policy =
```

Parameters

`policy` : `const TabelizePolicy&` See the signature for the supported argument form.

Returns

`pluss::table::DataFrame` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Inner;
auto out = ct.tabelize(policy);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

28.1.105 tail

Signature

```
Catalog tail(self, std::int64_t n) const
```

Parameters

n : std::int64_t Number of rows, entries, or depth items to keep in the preview.

Returns

Catalog Value produced by this call.

Notes

Returns the last n rows across every stream in the catalog.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.tail(2);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.106 take

Signature

```
Catalog take(self, std::span<const std::int64_t> indices) const
```

Parameters

indices : std::span<const std::int64_t> See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

Returns the rows at the requested integer positions across every stream.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

std::vector<std::int64_t> picks{0, 2};
auto out = ct.take(picks);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.107 text_preprocess

Signature

```
Catalog text_preprocess(self, std::string_view field) const
```

Parameters

field : std::string_view Name of the target stream in the catalog.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.text_preprocess("text");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.108 `tfidf`

Signature

```
Catalog tfidf(self, std::string_view field) const
```

Parameters

`field` : `std::string_view` Name of the target stream in the catalog.

Returns

`Catalog` Value produced by this call.

Notes

This is part of the text-processing surface for note-backed or text-oriented data.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "text"_k = pluss::kindle::Stream(pluss::make_note({
        "heavy rain on road",
        "clear weather downtown",
        "storm warning issued",
    })),
});

auto out = ct.tfidf("text");
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.109 `timeindex`

Signature

```
Catalog timeindex(self) const
```

Parameters

None

Returns

`Catalog` Value produced by this call.

Notes

Returns one named stream interpreted as a time-index stream.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});
```

(continues on next page)

(continued from previous page)

```
auto out = ct.timeindex();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.110 `timeline`

Signature

```
Catalog timeline(self) const
```

Parameters

None

Returns

`Catalog` Value produced by this call.

Notes

Returns one named stream interpreted as a timeline stream.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.timeline();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

(continues on next page)

(continued from previous page)

```
out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.111 to_json

Signature

```
std::string to_json(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.to_json();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.112 to_string

Signature

```
std::string to_string(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.to_string();
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.113 to_table

Signature

```
pluss::table::DataFrame to_table(self, const TabelizePolicy& policy =
```

Parameters

`policy` : `const TabelizePolicy&` See the signature for the supported argument form.

Returns

`pluss::table::DataFrame` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Inner;
auto out = ct.to_table(policy, true);
```

Output

```
id  sales  city
1   100.0  Accra
2   120.0  Tema
3    90.0  Kumasi
```

28.1.114 `transform`

Signature

```
Catalog transform(self, const std::function<Stream(const Stream&)>& fn) const
```

Parameters

`fn` : `const std::function<Stream(const Stream&)>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Rewrites one stream, several streams, or all streams by applying a user function.

See also

- `Catalog transform(self, std::string_view field, const std::function<Stream(const Stream&)>& fn) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.transform([](const pluss::kindle::Stream& s) { return s.kind() ==␣
↪pluss::kindle::StreamKind::Series ? s.clip(0.0, 25.0) : s; });
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.115 transform

Signature

```
Catalog transform(self, std::string_view field, const std::function<Stream(const Stream&
↪)>& fn) const
```

Parameters

`field` : `std::string_view` Name of the target stream in the catalog.

`fn` : `const std::function<Stream(const Stream&)>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Rewrites one stream, several streams, or all streams by applying a user function.

See also

- `Catalog transform(self, const std::function<Stream(const Stream&)>& fn) const`

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.transform("metric", [](const pluss::kindle::Stream& s) { return s.clip(0.0,
↪ 25.0); });
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

28.1.116 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = ct.unique();
```

Output

```
true
```

28.1.117 where

Signature

```
Catalog where(self, const pluss::Series& condition) const
```

Parameters

`condition` : `const pluss::Series&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Keeps only the rows where the supplied mask stream is true.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto mask = pluss::make_series({true, false, true}, pluss::codem::ScalarType::Bool);
auto out = ct.where(mask);
```

Output

```
ct.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])

out.head():
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

CHAPTER

TWENTYNINE

ARCHIVE API REFERENCE

CHAPTER

THIRTY

PRACTICAL ARCHIVE API REFERENCE

30.1 API Reference Style

This page documents the full public `Archive` surface for depth-stacked catalogs.

30.1.1 aggregate_over_catalogs

Signature

```
Catalog aggregate_over_catalogs(self, const std::function<Catalog(const std::vector
↪<Catalog>&)>& fn) const
```

Parameters

`fn` : `const std::function<Catalog(const std::vector<Catalog>&)>&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Reduces the full catalog stack to one output by applying an aggregation function across the contained catalogs.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.aggregate_over_catalogs([](const std::vector<pluss::kindle::Catalog>&␣
↪catalogs) { return catalogs.front(); });
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

30.1.2 align_depth

Signature

```
Archive align_depth(self, const Stream& labels) const
```

Parameters

`labels` : `const Stream&` See the signature for the supported argument form.

Returns

`Archive` Value produced by this call.

Notes

Reorders catalogs so their depth labels match the supplied label order.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto labels = pluss::kindle::Stream(pluss::make_note({"baseline", "stress"}));
auto out = archive.align_depth(labels);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

30.1.3 catalog

Signature

```
Catalog catalog(self, std::int64_t i) const
```

Parameters

`i` : `std::int64_t` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.catalog(0);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

30.1.4 `catalogs`

Signature

```
const std::vector<Catalog>& catalogs(self) const
```

Parameters

None

Returns

`const std::vector<Catalog>&` Borrowed collection of catalogs.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.catalogs();
```

Output

```
[Catalog(train), Catalog(test)]
```

30.1.5 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.defined();
```

Output

```
true
```

30.1.6 depth

Signature

```
std::int64_t depth(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.depth();
```

Output

```
2
```

30.1.7 depth_labels

Signature

```
const std::optional<Stream>& depth_labels(self) const
```

Parameters

None

Returns

const std::optional<Stream>& Value produced by this call.

Notes

Returns the current labels used to address fields, slots, rows, or depth items.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.depth_labels();
```

Output

```
["train", "test"]
```

30.1.8 empty

Signature

```
bool empty(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the current object has no rows, entries, slots, or depth items.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
```

(continues on next page)

(continued from previous page)

```
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.empty();
```

Output

```
false
```

30.1.9 head

Signature

```
Archive head(self, std::int64_t n = 5) const
```

Parameters

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

Returns

`Archive` Value produced by this call.

Notes

Returns the first `n` catalogs from the archive depth axis.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.head(1);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

30.1.10 iloc

Signature

```
Catalog iloc(self, std::int64_t i) const
```

Parameters

i : std::int64_t See the signature for the supported argument form.

Returns

Catalog Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.iloc(0);
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

30.1.11 kindelize

Signature

```
static Archive kindelize(const pluss::table::Atlas& atlas, const KindelizePolicy& policy␣
↪=
```

Parameters

atlas : const pluss::table::Atlas& See the signature for the supported argument form.

policy : const KindelizePolicy& See the signature for the supported argument form.

Returns

`Archive` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto f0 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({10.0, 20.0}))},
});
auto f1 = pluss::table::DataFrame({
    {"sales", pluss::table::Thread(pluss::make_series({30.0, 40.0}))},
});
auto atlas = pluss::table::Atlas({f0, f1});

auto out = pluss::kindle::Archive::kindelize(atlas);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

30.1.12 `loc`

Signature

```
Catalog loc(self, const pluss::Timeline& label) const
```

Parameters

`label` : `const pluss::Timeline&` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- `Catalog loc(self, std::string_view label) const`
- `Catalog loc(self, std::int64_t label) const`

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
```

(continues on next page)

(continued from previous page)

```
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.loc("test");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

30.1.13 loc

Signature

```
Catalog loc(self, std::int64_t label) const
```

Parameters

`label` : `std::int64_t` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- `Catalog loc(self, std::string_view label) const`
- `Catalog loc(self, const pluss::Timeline& label) const`

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));
```

(continues on next page)

(continued from previous page)

```
auto out = archive.loc("test");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

30.1.14 loc

Signature

```
Catalog loc(self, std::string_view label) const
```

Parameters

`label` : `std::string_view` See the signature for the supported argument form.

Returns

`Catalog` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- `Catalog loc(self, std::int64_t label) const`
- `Catalog loc(self, const pluss::Timeline& label) const`

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.loc("test");
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

30.1.15 map_catalogs

Signature

```
Archive map_catalogs(self, const std::function<Catalog(const Catalog&)>& fn) const
```

Parameters

`fn` : `const std::function<Catalog(const Catalog&)>&` See the signature for the supported argument form.

Returns

`Archive` Value produced by this call.

Notes

Applies the same function to each catalog and returns the transformed archive.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.map_catalogs([](const pluss::kindle::Catalog& catalog) { return␣
↪catalog.head(1); });
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

30.1.16 relabel_depth

Signature

```
Archive relabel_depth(self, const Stream& labels) const
```

Parameters

`labels` : `const Stream&` See the signature for the supported argument form.

Returns

`Archive` Value produced by this call.

Notes

Replaces the depth-label axis with the supplied labels while keeping catalog contents unchanged.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto labels = pluss::kindle::Stream(pluss::make_note({"baseline", "stress"}));
auto out = archive.relabel_depth(labels);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

30.1.17 show

Signature

```
void show(self, std::ostream& os) const
```

Parameters

`os` : `std::ostream&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
```

(continues on next page)

(continued from previous page)

```
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

archive.show(std::cout);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

30.1.18 slice_depth

Signature

```
Archive slice_depth(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` Inclusive starting position of the slice.

`stop` : `std::int64_t` Exclusive ending position of the slice.

Returns

`Archive` Value produced by this call.

Notes

Returns a contiguous depth slice `[start, stop)` from the archive.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.slice_depth(0, 1);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

30.1.19 `sort_depth`

Signature

```
Archive sort_depth(self, bool ascending = true, bool nulls_last = true) const
```

Parameters

`ascending` : `bool` See the signature for the supported argument form.

`nulls_last` : `bool` See the signature for the supported argument form.

Returns

`Archive` Value produced by this call.

Notes

Sorts catalogs by their current depth labels.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.sort_depth(true, true);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

30.1.20 `tabelize`

Signature

```
pluss::table::Atlas tabelize(self, const TabelizePolicy& policy =
```

Parameters

`policy` : `const TabelizePolicy&` See the signature for the supported argument form.

Returns

`pluss::table::Atlas` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Inner;
auto out = archive.tabelize(policy);
```

Output

```
frame0:
sales
10.0
20.0

frame1:
sales
30.0
40.0
```

30.1.21 `tail`

Signature

```
Archive tail(self, std::int64_t n = 5) const
```

Parameters

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

Returns

`Archive` Value produced by this call.

Notes

Returns the last `n` catalogs from the archive depth axis.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.tail(1);
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

30.1.22 `to_string`

Signature

```
std::string to_string(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
```

(continues on next page)

(continued from previous page)

```
↪"train", "test"})));

auto out = archive.to_string();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

30.1.23 to_table

Signature

```
pluss::table::Atlas to_table(self, const TabelizePolicy& policy =
```

Parameters

`policy` : `const TabelizePolicy&` See the signature for the supported argument form.

Returns

`pluss::table::Atlas` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

pluss::kindle::TabelizePolicy policy;
policy.align = pluss::kindle::AlignPolicy::Inner;
auto out = archive.to_table(policy, true);
```

Output

```
frame0:
sales
10.0
20.0
```

(continues on next page)

(continued from previous page)

```
frame1:
sales
30.0
40.0
```

30.1.24 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = archive.unique();
```

Output

```
true
```

CHAPTER

THIRTYONE

BUNDLE MODEL GUIDE

31.1 When To Use Bundle Types

The bundle model is the heterogeneous record layer:

- `Slot`: sequence of heterogeneous Pluss values
- `Registry`: named set of slots
- `Aisle`: depth stack of registries

Use it when one field can hold different domain types, or when you want a lighter record/batch abstraction than `DataFrame`.

31.2 Slot

```
auto ids = pluss::bundle::Slot({
    pluss::make_series(1, pluss::codem::ScalarType::Int64),
    pluss::make_series(2, pluss::codem::ScalarType::Int64),
    pluss::make_series(3, pluss::codem::ScalarType::Int64),
});

ids.push_(pluss::make_series(4, pluss::codem::ScalarType::Int64));
auto even = ids.filter([](const pluss::bundle::Slot::Value& v) {
  return std::get<pluss::Series>(v).to_vec<std::int64_t>()[0] % 2 == 0;
});
```

31.3 Registry

```
auto cities = pluss::bundle::Slot({
    pluss::make_note("Accra"),
    pluss::make_note("Tema"),
    pluss::make_note("Kumasi"),
});

auto registry = pluss::bundle::Registry({
    {"id", ids},
    {"city", cities},
});

auto selected = registry.select({"id"});
```

(continues on next page)

(continued from previous page)

```
auto renamed = registry.rename({{"city", "location"}});
auto profiled = registry.type_profiles();
```

31.4 Aisle

```
auto train = registry;
auto test = registry.rename({{"city", "city_test"}});
auto labels = pluss::bundle::Slot({pluss::make_note("train"), pluss::make_note("test")});

auto aisle = pluss::bundle::Aisle({train, test}, labels);

auto first = aisle.head(1);
auto combined = aisle.concat_depth(first);
```

31.5 Kindle Bridge

```
auto stream = ids.to_kindle();
auto catalog = registry.to_kindle();
auto archive = aisle.to_kindle();

auto back_slot = pluss::bundle::Slot::from_kindle(stream);
auto back_registry = pluss::bundle::Registry::from_kindle(catalog);
auto back_aisle = pluss::bundle::Aisle::from_kindle(archive);
```

CHAPTER

THIRTYTWO

SLOT API REFERENCE

CHAPTER

THIRTYTHREE

PRACTICAL SLOT API REFERENCE

33.1 API Reference Style

This page documents the full public `Slot` surface for one-dimensional heterogeneous bundles.

33.1.1 as_kindle

Signature

```
pluss::kindle::Stream as_kindle(self) const
```

Parameters

None

Returns

`pluss::kindle::Stream` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.as_kindle();
```

Output

```
Series([10.0, 20.0, 30.0])
```

33.1.2 at

Signature

```
const Value& at(self, std::int64_t i) const
```

Parameters

`i` : `std::int64_t` See the signature for the supported argument form.

Returns

`const Value&` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.at(0);
```

Output

```
Series(1)
```

33.1.3 concat

Signature

```
Slot concat(self, const Slot& other) const
```

Parameters

`other` : `const Slot&` See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

Appends the values from another slot and returns the combined slot.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto other = pluss::bundle::Slot({pluss::make_series(4), pluss::make_series(5)});
auto out = slot.concat(other);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.4 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.defined();
```

Output

```
true
```

33.1.5 empty

Signature

```
bool empty(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the current object has no rows, entries, slots, or depth items.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.empty();
```

Output

```
false
```

33.1.6 filter

Signature

```
Slot filter(self, const std::function<bool(const Value&)>& predicate) const
```

Parameters

`predicate` : `const std::function<bool(const Value&)>&` See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

Keeps only the slot values that satisfy the supplied predicate.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_note("ok"),␣
↪pluss::make_series(3)});

auto out = slot.filter([](const pluss::bundle::Slot::Value& value) { return std::holds_
↪alternative<pluss::Series>(value); });
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.7 from_kindle

Signature

```
static Slot from_kindle(const pluss::kindle::Stream& stream)
```

Parameters

`stream` : `const pluss::kindle::Stream&` See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto stream = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0}));

auto out = pluss::bundle::Slot::from_kindle(stream);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.8 head

Signature

```
Slot head(self, std::int64_t n) const
```

Parameters

n : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

Returns

`Slot` Value produced by this call.

Notes

Returns the first `n` values from the slot.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.head(2);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.9 kindelize

Signature

```
pluss::kindle::Stream kindelize(self) const
```

Parameters

None

Returns

`pluss::kindle::Stream` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.kindelize();
```

Output

```
Series([10.0, 20.0, 30.0])
```

33.1.10 push

Signature

```
Slot push(self, const Value& value) const
```

Parameters

`value` : `const Value&` See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

Returns a new slot with one additional value appended.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.push(pluss::make_note("tail"));
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.11 push_

Signature

```
Slot& push_(self, const Value& value)
```

Parameters

`value` : `const Value&` See the signature for the supported argument form.

Returns

`Slot&` Value produced by this call.

Notes

Appends one additional value to the slot in place.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

slot.push_(pluss::make_note("tail"));
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.12 show

Signature

```
void show(self, std::ostream& os) const
```

Parameters

`os` : `std::ostream&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

slot.show(std::cout);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.13 size

Signature

```
std::int64_t size(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.size();
```

Output

```
3
```

33.1.14 slice

Signature

```
Slot slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` Inclusive starting position of the slice.

`stop` : `std::int64_t` Exclusive ending position of the slice.

Returns

`Slot` Value produced by this call.

Notes

Returns a contiguous positional slice `[start, stop)` from the slot.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.slice(0, 2);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.15 tail

Signature

```
Slot tail(self, std::int64_t n) const
```

Parameters

n : std::int64_t Number of rows, entries, or depth items to keep in the preview.

Returns

Slot Value produced by this call.

Notes

Returns the last n values from the slot.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.tail(2);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.16 take

Signature

```
Slot take(self, std::span<const std::int64_t> indices) const
```

Parameters

indices : std::span<const std::int64_t> See the signature for the supported argument form.

Returns

`Slot` Value produced by this call.

Notes

Returns the values at the requested integer positions.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

std::vector<std::int64_t> picks{0, 2};
auto out = slot.take(picks);
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.17 to_json

Signature

```
std::string to_json(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.to_json();
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.18 to_kindle

Signature

```
pluss::kindle::Stream to_kindle(self) const
```

Parameters

None

Returns

`pluss::kindle::Stream` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.to_kindle();
```

Output

```
Series([10.0, 20.0, 30.0])
```

33.1.19 `to_string`

Signature

```
std::string to_string(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.to_string();
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.20 transform

Signature

```
Slot transform(self, const std::function<Value(const Value&)>& fn) const
```

Parameters

fn : const std::function<Value(const Value&)>& See the signature for the supported argument form.

Returns

Slot Value produced by this call.

Notes

Applies a value-wise transform to each slot entry and returns the updated slot.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.transform([](const pluss::bundle::Slot::Value& value) ->␣
↪pluss::bundle::Slot::Value { if (std::holds_alternative<pluss::Series>(value)) return␣
↪std::get<pluss::Series>(value).add(10); return value; });
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

33.1.21 type_counts

Signature

```
std::unordered_map<std::string, std::int64_t> type_counts(self) const
```

Parameters

None

Returns

std::unordered_map<std::string, std::int64_t> Value produced by this call.

Notes

Counts how many slot entries belong to each stored pluss type.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_note("ok"),␣
↪pluss::make_series(3)});

auto out = slot.type_counts();
```

Output

```
{"Series": 2, "Note": 1}
```

33.1.22 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.unique();
```

Output

```
true
```

33.1.23 values

Signature

```
const std::vector<Value>& values(self) const
```

Parameters

None

Returns

`const std::vector<Value>&` Value produced by this call.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

Examples

```
auto slot = pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)});

auto out = slot.values();
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

CHAPTER

THIRTYFOUR

REGISTRY API REFERENCE

CHAPTER

THIRTYFIVE

PRACTICAL REGISTRY API REFERENCE

35.1 API Reference Style

This page documents the full public `Registry` surface for named bundle slots.

35.1.1 apply

Signature

```
Registry apply(self, std::initializer_list<std::string_view> keys, const std::function
↪<Slot(const Slot&)>& fn) const
```

Parameters

`keys` : `std::initializer_list<std::string_view>` Collection of slot or stream names to keep, drop, or rewrite together.

`fn` : `const std::function<Slot(const Slot&)>&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Applies a function to one or more named slots and returns the updated registry.

See also

- `Registry apply(self, std::string_view key, const std::function<Slot(const Slot&)>& fn) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.apply({"ids"}, [](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.2 apply

Signature

```
Registry apply(self, std::string_view key, const std::function<Slot(const Slot&)>& fn)␣
↪const
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

`fn` : `const std::function<Slot(const Slot&)>&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Applies a function to one or more named slots and returns the updated registry.

See also

- `Registry apply(self, std::initializer_list<std::string_view> keys, const std::function<Slot(const Slot&)>& fn) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.apply("ids", [](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
```

(continues on next page)

(continued from previous page)

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.3 apply_

Signature

```
Registry& apply_(self, std::initializer_list<std::string_view> keys, const std::function
↪<Slot(const Slot&)>& fn)
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

fn : const std::function<Slot(const Slot&)>& See the signature for the supported argument form.

Returns

Registry& Value produced by this call.

Notes

Applies a function to one or more named slots in place.

See also

- Registry& apply_(self, std::string_view key, const std::function<Slot(const Slot&)>& fn)

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.apply_({"ids"}, [](const pluss::bundle::Slot& slot) { return slot; });
```

35.1.4 apply_

Signature

```
Registry& apply_(self, std::string_view key, const std::function<Slot(const Slot&)>& fn)
```

Parameters

key : std::string_view Name of the target slot or stream.

fn : const std::function<Slot(const Slot&)>& See the signature for the supported argument form.

Returns

Registry& Value produced by this call.

Notes

Applies a function to one or more named slots in place.

See also

- Registry& apply_(self, std::initializer_list<std::string_view> keys, const std::function<Slot(const Slot&)>& fn)

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.apply_("ids", [](const pluss::bundle::Slot& slot) { return slot; });
```

35.1.5 apply_all

Signature

```
Registry apply_all(self, const std::function<Slot(const Slot&)>& fn) const
```

Parameters

fn : const std::function<Slot(const Slot&)>& See the signature for the supported argument form.

Returns

Registry Value produced by this call.

Notes

Applies the same function to every slot and returns the updated registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.apply_all([](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.6 as_kindle

Signature

```
pluss::kindle::Catalog as_kindle(self) const
```

Parameters

None

Returns

`pluss::kindle::Catalog` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.as_kindle();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

35.1.7 at

Signature

```
Slot at(self, std::string_view key) const
```

Parameters

key : `std::string_view` Name of the target slot or stream.

Returns

`Slot` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.at("ids");
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

35.1.8 begin

Signature

```
const_iterator begin(self) const
```

Parameters

None

Returns

`const_iterator` Value produced by this call.

Notes

Exposes iterator boundaries so the object can be traversed with a range-based loop or manual iteration.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});
```

(continues on next page)

(continued from previous page)

```
for (const auto& key : rg) {
  (void)key;
}
```

35.1.9 clear

Signature

```
Registry clear(self) const
```

Parameters

None

Returns

Registry Value produced by this call.

Notes

Removes all slots and returns an empty registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.clear();
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.10 clear_

Signature

```
Registry& clear_(self)
```

Parameters

None

Returns

Registry& Value produced by this call.

Notes

Removes all slots in place.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.clear_();
```

35.1.11 combine

Signature

```
Registry combine(self, const Registry& other, std::string suffix = "_other") const
```

Parameters

other : const Registry& See the signature for the supported argument form.

suffix : std::string See the signature for the supported argument form.

Returns

Registry Value produced by this call.

Notes

Merges slots from another registry into this one, applying the collision suffix when keys overlap.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto other = pluss::bundle::Registry({{"flag", pluss::bundle::Slot({pluss::make_
↪series(true, pluss::codem::ScalarType::Bool), pluss::make_series(false,␣
↪pluss::codem::ScalarType::Bool), pluss::make_series(true,␣
```

(continues on next page)

(continued from previous page)

```
↪pluss::codem::ScalarType::Bool)})}});
auto out = rg.combine(other, "_rhs");
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.12 concat_values

Signature

```
Registry concat_values(self, std::string_view key, const Slot& other) const
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

`other` : `const Slot&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Appends extra values onto one named slot and returns the updated registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto extra = pluss::bundle::Slot({pluss::make_series(4), pluss::make_series(5)});
auto out = rg.concat_values("ids", extra);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
```

(continues on next page)

(continued from previous page)

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.13 concat_values_

Signature

```
Registry& concat_values_(self, std::string_view key, const Slot& other)
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

`other` : `const Slot&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

Appends extra values onto one named slot in place.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto extra = pluss::bundle::Slot({pluss::make_series(4), pluss::make_series(5)});
rg.concat_values_("ids", extra);
```

35.1.14 defined

Signature

```
bool defined(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.defined();
```

Output

```
true
```

35.1.15 drop

Signature

```
Registry drop(self, const std::vector<std::string>& keys) const
```

Parameters

`keys` : `const std::vector<std::string>&` Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Registry` Value produced by this call.

Notes

Removes the named slots and keeps the remaining registry structure intact.

See also

- `Registry drop(self, std::initializer_list<std::string_view> keys) const`
- `Registry drop(self, std::string_view key) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::string> names{"city"};
auto out = rg.drop(names);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.16 drop

Signature

```
Registry drop(self, std::initializer_list<std::string_view> keys) const
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

Registry Value produced by this call.

Notes

Removes the named slots and keeps the remaining registry structure intact.

See also

- Registry drop(self, const std::vector<std::string>& keys) const
- Registry drop(self, std::string_view key) const

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.drop({"city"});
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
```

(continues on next page)

(continued from previous page)

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.17 drop

Signature

```
Registry drop(self, std::string_view key) const
```

Parameters

key : std::string_view Name of the target slot or stream.

Returns

Registry Value produced by this call.

Notes

Removes the named slots and keeps the remaining registry structure intact.

See also

- Registry drop(self, std::initializer_list<std::string_view> keys) const
- Registry drop(self, const std::vector<std::string>& keys) const

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.drop("city");
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.18 drop_empty_slots

Signature

```
Registry drop_empty_slots(self) const
```

Parameters

None

Returns

`Registry` Value produced by this call.

Notes

Removes slots whose payload is empty and returns the reduced registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.drop_empty_slots();
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.19 drop_empty_slots_

Signature

```
Registry& drop_empty_slots_(self)
```

Parameters

None

Returns

`Registry&` Value produced by this call.

Notes

Removes empty slots in place.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.drop_empty_slots_();
```

35.1.20 empty

Signature

```
bool empty(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the current object has no rows, entries, slots, or depth items.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.empty();
```

Output

```
false
```

35.1.21 end

Signature

```
const_iterator end(self) const
```

Parameters

None

Returns

const_iterator Value produced by this call.

Notes

Exposes iterator boundaries so the object can be traversed with a range-based loop or manual iteration.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

for (const auto& key : rg) {
  (void)key;
}
```

35.1.22 ensure_stream_compatible

Signature

```
void ensure_stream_compatible(self, std::string_view where = "Registry") const
```

Parameters

where : std::string_view See the signature for the supported argument form.

Returns

void Performs the side effect and returns no value.

Notes

Validates that the registry can be converted to a Kindle catalog without incompatible slot contents.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.ensure_stream_compatible("demo");
```

Output

```
no error
```

35.1.23 field_names

Signature

```
const std::vector<std::string>& field_names(self) const
```

Parameters

None

Returns

const std::vector<std::string>& Borrowed list of names.

Notes

Returns the current labels used to address fields, slots, rows, or depth items.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.field_names();
```

Output

```
["ids", "city"]
```

35.1.24 filter_values

Signature

```
Registry filter_values(self, std::string_view key, const std::function<bool(const␣
↪Slot::Value&)>& predicate) const
```

Parameters

key : std::string_view Name of the target slot or stream.

predicate : const std::function<bool(const Slot::Value&)>& See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Keeps only the values in one named slot that satisfy the supplied predicate.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.filter_values("ids", [](const pluss::bundle::Slot::Value& value) { return␣
↪std::holds_alternative<pluss::Series>(value); });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.25 `filter_values_`

Signature

```
Registry& filter_values_(self, std::string_view key, const std::function<bool(const␣
↪Slot::Value&)>& predicate)
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

`predicate` : `const std::function<bool(const Slot::Value&)>&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

Filters the values in one named slot in place.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.filter_values_("ids", [](const pluss::bundle::Slot::Value& value) { return std::holds_
↪alternative<pluss::Series>(value); });
```

35.1.26 from_kindle

Signature

```
static Registry from_kindle(const pluss::kindle::Catalog& catalog)
```

Parameters

`catalog` : `const pluss::kindle::Catalog&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
using namespace pluss::kindle::literals;

auto ct = pluss::kindle::Catalog({
    "id"_k = pluss::kindle::Stream(pluss::make_series({1, 2, 3})),
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0, 30.0})),
    "city"_k = pluss::kindle::Stream(pluss::make_note({"Accra", "Tema", "Kumasi"})),
});

auto out = pluss::bundle::Registry::from_kindle(catalog);
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.27 get

Signature

```
std::optional<Slot> get(self, std::string_view key) const
```

Parameters

key : std::string_view Name of the target slot or stream.

Returns

std::optional<Slot> Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.get("ids");
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

35.1.28 get_slot

Signature

```
std::optional<Slot> get_slot(self, std::string_view key) const
```

Parameters

key : std::string_view Name of the target slot or stream.

Returns

std::optional<Slot> Value produced by this call.

Notes

Returns one named slot from the registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
```

(continues on next page)

(continued from previous page)

```
↪pluss::make_note("Kumasi")})},
});

auto out = rg.get_slot("ids");
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

35.1.29 has

Signature

```
bool has(self, std::string_view key) const
```

Parameters

key : `std::string_view` Name of the target slot or stream.

Returns

`bool` Boolean status or membership result.

Notes

Checks whether a named slot exists.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.has("ids");
```

Output

```
true
```

35.1.30 has_all

Signature

```
bool has_all(self, std::initializer_list<std::string_view> keys) const
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

bool Boolean status or membership result.

Notes

Checks whether every requested slot name exists.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.has_all({"ids", "city"});
```

Output

```
true
```

35.1.31 has_any

Signature

```
bool has_any(self, std::initializer_list<std::string_view> keys) const
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

bool Boolean status or membership result.

Notes

Checks whether at least one requested slot name exists.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
```

(continues on next page)

(continued from previous page)

```
});

auto out = rg.has_any({"ids", "city"});
```

Output

```
true
```

35.1.32 has_slot

Signature

```
bool has_slot(self, std::string_view key) const
```

Parameters

key : std::string_view Name of the target slot or stream.

Returns

bool Boolean status or membership result.

Notes

Checks whether a named slot exists.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.has_slot("ids");
```

Output

```
true
```

35.1.33 head

Signature

```
Registry head(self, std::int64_t n) const
```

Parameters

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

Returns

`Registry` Value produced by this call.

Notes

Returns the first `n` values from every slot in the registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.head(2);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.34 `iloc`

Signature

```
Registry iloc(self, std::int64_t row) const
```

Parameters

`row` : `std::int64_t` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- `Registry iloc(self, std::int64_t start, std::int64_t stop) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.iloc(1);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.35 `iloc`

Signature

```
Registry iloc(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` Inclusive starting position of the slice.

`stop` : `std::int64_t` Exclusive ending position of the slice.

Returns

`Registry` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

See also

- `Registry iloc(self, std::int64_t row) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
```

(continues on next page)

(continued from previous page)

```
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.iloc(0, 2);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.36 is_stream_compatible

Signature

```
bool is_stream_compatible(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the registry can be converted to a Kindle catalog without incompatible slot contents.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.is_stream_compatible();
```

Output

```
true
```

35.1.37 keys

Signature

```
const std::vector<std::string>& keys(self) const
```

Parameters

None

Returns

const std::vector<std::string>& Borrowed list of names.

Notes

Returns the current labels used to address fields, slots, rows, or depth items.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.keys();
```

Output

```
["ids", "city"]
```

35.1.38 kindelize

Signature

```
pluss::kindle::Catalog kindelize(self) const
```

Parameters

None

Returns

pluss::kindle::Catalog Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.kindelize();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

35.1.39 load

Signature

```
static Registry load(std::string_view source_type, std::string_view filepath)
```

Parameters

`source_type` : `std::string_view` Storage format selector such as `"csv"`, `"xlsx"`, `"json"`, or `"parquet"`.

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load(std::string_view source_type, std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::bundle::Registry::load("csv", "path/to/data.csv");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.40 load

Signature

```
static Registry load(std::string_view source_type, std::string_view filepath, const␣
↪pluss::loader::Config& config)
```

Parameters

`source_type` : `std::string_view` Storage format selector such as `"csv"`, `"xlsx"`, `"json"`, or `"parquet"`.

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load(std::string_view source_type, std::string_view filepath)`

Examples

```
auto out = pluss::bundle::Registry::load("csv", "path/to/data.csv");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.41 load_csv

Signature

```
static Registry load_csv(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`
- `static Registry load_csv(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::bundle::Registry::load_csv("path/to/data.csv");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.42 load_csv

Signature

```
static Registry load_csv(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_csv(std::string_view filepath)`
- `static Registry load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions& options)`

Examples

```
auto out = pluss::bundle::Registry::load_csv("path/to/data.csv", {"nrows", 16});
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.43 load_csv

Signature

```
static Registry load_csv(std::string_view filepath, const pluss::loader::ReadCsvOptions&␣
↪options)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`options` : `const pluss::loader::ReadCsvOptions&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_csv(std::string_view filepath)`
- `static Registry load_csv(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
pluss::loader::ReadCsvOptions options;
auto out = pluss::bundle::Registry::load_csv("path/to/data.csv", options);
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.44 load_excel

Signature

```
static Registry load_excel(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`
- `static Registry load_excel(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::bundle::Registry::load_excel("path/to/data.xlsx");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.45 load_excel

Signature

```
static Registry load_excel(std::string_view filepath, const pluss::loader::Config&␣
↪config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_excel(std::string_view filepath)`
- `static Registry load_excel(std::string_view filepath, const pluss::loader::ReadExcelOptions& options)`

Examples

```
auto out = pluss::bundle::Registry::load_excel("path/to/data.xlsx", {"nrows", 16});
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.46 load_excel

Signature

```
static Registry load_excel(std::string_view filepath, const␣
↪pluss::loader::ReadExcelOptions& options)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`options` : `const pluss::loader::ReadExcelOptions&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_excel(std::string_view filepath)`
- `static Registry load_excel(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
pluss::loader::ReadExcelOptions options;
auto out = pluss::bundle::Registry::load_excel("path/to/data.xlsx", options);
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.47 load_json

Signature

```
static Registry load_json(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_json(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::bundle::Registry::load_json("path/to/data.json");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.48 load_json

Signature

```
static Registry load_json(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_json(std::string_view filepath)`

Examples

```
auto out = pluss::bundle::Registry::load_json("path/to/data.json", {"nrows", 16});
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.49 `load_parquet`

Signature

```
static Registry load_parquet(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_parquet(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::bundle::Registry::load_parquet("path/to/data.parquet");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.50 load_parquet

Signature

```
static Registry load_parquet(std::string_view filepath, const pluss::loader::Config&␣
↪config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_parquet(std::string_view filepath)`

Examples

```
auto out = pluss::bundle::Registry::load_parquet("path/to/data.parquet", {"nrows", 16});
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.51 load_txt

Signature

```
static Registry load_txt(std::string_view filepath)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_txt(std::string_view filepath, const pluss::loader::Config& config)`

Examples

```
auto out = pluss::bundle::Registry::load_txt("path/to/data.txt");
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.52 `load_txt`

Signature

```
static Registry load_txt(std::string_view filepath, const pluss::loader::Config& config)
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Loads the object from disk using the corresponding PlussMic loader route.

See also

- `static Registry load_txt(std::string_view filepath)`

Examples

```
auto out = pluss::bundle::Registry::load_txt("path/to/data.txt", {"nrows", 16});
```

Output

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.53 map

Signature

```
Registry map(self, const std::function<Slot(const Slot&)>& fn) const
```

Parameters

fn : const std::function<Slot(const Slot&)>& See the signature for the supported argument form.

Returns

Registry Value produced by this call.

Notes

Transforms every slot with the same function and returns the updated registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.map([](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.54 map_

Signature

```
Registry& map_(self, const std::function<Slot(const Slot&)>& fn)
```

Parameters

fn : const std::function<Slot(const Slot&)>& See the signature for the supported argument form.

Returns

Registry& Value produced by this call.

Notes

Transforms every slot in place with the same function.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.map_([](const pluss::bundle::Slot& slot) { return slot; });
```

35.1.55 nfields

Signature

```
std::int64_t nfields(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.nfields();
```

Output

```
2
```

35.1.56 nslots

Signature

```
std::int64_t nslots(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.nslots();
```

Output

```
2
```

35.1.57 push

Signature

```
Registry push(self, std::string_view key, const Slot::Value& value) const
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

`value` : `const Slot::Value&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Appends one new value to a named slot and returns the updated registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
```

(continues on next page)

(continued from previous page)

```
});

auto out = rg.push("ids", pluss::make_series(4));
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.58 push_

Signature

```
Registry& push_(self, std::string_view key, const Slot::Value& value)
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

`value` : `const Slot::Value&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

Appends one new value to a named slot in place.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.push_("ids", pluss::make_series(4));
```

35.1.59 push_all

Signature

```
Registry push_all(self, const Slot::Value& value) const
```

Parameters

`value` : `const Slot::Value&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Appends the same new value to every slot and returns the updated registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.push_all(pluss::make_note("ok"));
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.60 push_all_

Signature

```
Registry& push_all_(self, const Slot::Value& value)
```

Parameters

`value` : `const Slot::Value&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

Appends the same new value to every slot in place.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.push_all_(pluss::make_note("ok"));
```

35.1.61 `rename`

Signature

```
Registry rename(self, const std::unordered_map<std::string, std::string>& mapping) const
```

Parameters

`mapping` : `const std::unordered_map<std::string, std::string>&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Renames one or more slots without changing their stored values.

See also

- `Registry rename(self, std::initializer_list<std::pair<std::string_view, std::string_view>> mapping) const`
- `Registry rename(self, std::string_view from, std::string_view to) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::unordered_map<std::string, std::string> renames{{"ids", "ids_new"}};
auto out = rg.rename(renames);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

(continues on next page)

(continued from previous page)

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.62 `rename`

Signature

```
Registry rename(self, std::initializer_list<std::pair<std::string_view, std::string_view>
↪> mapping) const
```

Parameters

`mapping` : `std::initializer_list<std::pair<std::string_view, std::string_view>>` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Renames one or more slots without changing their stored values.

See also

- `Registry rename(self, const std::unordered_map<std::string, std::string>& mapping) const`
- `Registry rename(self, std::string_view from, std::string_view to) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.rename({{"ids", "ids_new"}});
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.63 rename

Signature

```
Registry rename(self, std::string_view from, std::string_view to) const
```

Parameters

`from` : `std::string_view` See the signature for the supported argument form.

`to` : `std::string_view` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Renames one or more slots without changing their stored values.

See also

- `Registry rename(self, std::initializer_list<std::pair<std::string_view, std::string_view>> mapping) const`
- `Registry rename(self, const std::unordered_map<std::string, std::string>& mapping) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.rename("ids", "ids_new");
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.64 rename_

Signature

```
Registry& rename_(self, std::string_view from, std::string_view to)
```

Parameters

`from` : `std::string_view` See the signature for the supported argument form.

`to` : `std::string_view` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

Renames one or more slots in place without changing their stored values.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.rename_("ids", "ids_new");
```

35.1.65 `sample`

Signature

```
Registry sample(self, std::int64_t n, std::optional<std::uint64_t> random_state =␣
↪std::nullopt) const
```

Parameters

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

`random_state` : `std::optional<std::uint64_t>` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Returns a random sample of positions across every slot.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.sample(2, 7);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.66 save_as

Signature

```
void save_as(self, std::string_view target_type, std::string_view filepath, const␣
↪pluss::loader::Config& config =
```

Parameters

`target_type` : `std::string_view` See the signature for the supported argument form.

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the registry using the explicitly supplied output format selector.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_as("json", "path/to/output.json");
```

Output

```
saved: out.*
```

35.1.67 save_csv

Signature

```
void save_csv(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the registry to CSV after materializing each slot into a tabular export form.

See also

- `void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

35.1.68 save_csv

Signature

```
void save_csv(self, std::string_view filepath, const pluss::loader::Config& config) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the registry to CSV after materializing each slot into a tabular export form.

See also

- `void save_csv(self, std::string_view filepath) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_csv("path/to/output.csv");
```

Output

```
saved: out.*
```

35.1.69 save_excel

Signature

```
void save_excel(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the registry to an Excel workbook after materializing each slot into a tabular export form.

See also

- `void save_excel(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
```

(continues on next page)

(continued from previous page)

```
});

rg.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

35.1.70 save_excel

Signature

```
void save_excel(self, std::string_view filepath, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the registry to an Excel workbook after materializing each slot into a tabular export form.

See also

- `void save_excel(self, std::string_view filepath) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_excel("path/to/output.xlsx");
```

Output

```
saved: out.*
```

35.1.71 save_json

Signature

```
void save_json(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the registry to JSON for interchange or later reload.

See also

- `void save_json(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_json("path/to/output.json");
```

Output

```
saved: out.*
```

35.1.72 save_json

Signature

```
void save_json(self, std::string_view filepath, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the registry to JSON for interchange or later reload.

See also

- `void save_json(self, std::string_view filepath) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_json("path/to/output.json");
```

Output

```
saved: out.*
```

35.1.73 save_parquet

Signature

```
void save_parquet(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the registry to Parquet for compact typed storage.

See also

- `void save_parquet(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_parquet("path/to/output.parquet");
```

Output

```
saved: out.*
```

35.1.74 save_parquet

Signature

```
void save_parquet(self, std::string_view filepath, const pluss::loader::Config& config)␣
↪const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes the registry to Parquet for compact typed storage.

See also

- `void save_parquet(self, std::string_view filepath) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_parquet("path/to/output.parquet");
```

Output

```
saved: out.*
```

35.1.75 save_txt

Signature

```
void save_txt(self, std::string_view filepath) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes a text rendering of the registry for inspection or logging.

See also

- `void save_txt(self, std::string_view filepath, const pluss::loader::Config& config) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_txt("path/to/output.txt");
```

Output

```
saved: out.*
```

35.1.76 save_txt

Signature

```
void save_txt(self, std::string_view filepath, const pluss::loader::Config& config) const
```

Parameters

`filepath` : `std::string_view` File location to read from or write to. Use a neutral project path such as `path/to/data.csv`.

`config` : `const pluss::loader::Config&` See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

Writes a text rendering of the registry for inspection or logging.

See also

- `void save_txt(self, std::string_view filepath) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.save_txt("path/to/output.txt");
```

Output

```
saved: out.*
```

35.1.77 `select`

Signature

```
Registry select(self, const std::vector<std::string>& keys) const
```

Parameters

`keys` : `const std::vector<std::string>&` Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Registry` Value produced by this call.

Notes

Keeps only the named slots and preserves their shared positional alignment.

See also

- `Registry select(self, std::initializer_list<std::string_view> keys) const`
- `Registry select(self, std::string_view key) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::string> names{"ids"};
auto out = rg.select(names);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.78 `select`

Signature

```
Registry select(self, std::initializer_list<std::string_view> keys) const
```

Parameters

`keys` : `std::initializer_list<std::string_view>` Collection of slot or stream names to keep, drop, or rewrite together.

Returns

`Registry` Value produced by this call.

Notes

Keeps only the named slots and preserves their shared positional alignment.

See also

- `Registry select(self, const std::vector<std::string>& keys) const`
- `Registry select(self, std::string_view key) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.select({"ids"});
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.79 `select`

Signature

```
Registry select(self, std::string_view key) const
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

Returns

`Registry` Value produced by this call.

Notes

Keeps only the named slots and preserves their shared positional alignment.

See also

- `Registry select(self, std::initializer_list<std::string_view> keys) const`
- `Registry select(self, const std::vector<std::string>& keys) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});
```

(continues on next page)

(continued from previous page)

```
auto out = rg.select("ids");
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.80 set_slot

Signature

```
Registry set_slot(self, std::string_view key, const Slot& value) const
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

`value` : `const Slot&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Adds or replaces one named slot and returns the updated registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.set_slot("score", pluss::bundle::Slot({pluss::make_series(0.9),␣
↪pluss::make_series(0.7), pluss::make_series(0.8)}));
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
```

(continues on next page)

(continued from previous page)

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.81 set_slot_

Signature

```
Registry& set_slot_(self, std::string_view key, const Slot& value)
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

`value` : `const Slot&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

Adds or replaces one named slot in place.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.set_slot_("score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.
↪7), pluss::make_series(0.8)}));
```

35.1.82 set_slots

Signature

```
Registry set_slots(self, std::initializer_list<std::pair<std::string, Slot>> items) const
```

Parameters

`items` : `std::initializer_list<std::pair<std::string, Slot>>` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Adds or replaces several named slots in one call and returns the updated registry.

See also

- `Registry set_slots(self, std::initializer_list<std::pair<std::string_view, Slot>> items) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.set_slots({
    {"score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.7),␣
↪pluss::make_series(0.8)})},
});
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.83 `set_slots`

Signature

```
Registry set_slots(self, std::initializer_list<std::pair<std::string_view, Slot>> items)␣
↪const
```

Parameters

`items` : `std::initializer_list<std::pair<std::string_view, Slot>>` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Adds or replaces several named slots in one call and returns the updated registry.

See also

- Registry set_slots(self, std::initializer_list<std::pair<std::string, Slot>> items) const

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.set_slots({
    {"score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.7),␣
↪pluss::make_series(0.8)})},
});
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.84 set_slots_

Signature

```
Registry& set_slots_(self, std::initializer_list<std::pair<std::string, Slot>> items)
```

Parameters

`items` : `std::initializer_list<std::pair<std::string, Slot>>` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

Adds or replaces several named slots in place.

See also

- Registry& set_slots_(self, std::initializer_list<std::pair<std::string_view, Slot>> items)

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.set_slots_({
    {"score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.7),␣
↪pluss::make_series(0.8)})},
});
```

35.1.85 set_slots_

Signature

```
Registry& set_slots_(self, std::initializer_list<std::pair<std::string_view, Slot>>␣
↪items)
```

Parameters

`items` : `std::initializer_list<std::pair<std::string_view, Slot>>` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

Adds or replaces several named slots in place.

See also

- `Registry& set_slots_(self, std::initializer_list<std::pair<std::string, Slot>> items)`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.set_slots_({
    {"score", pluss::bundle::Slot({pluss::make_series(0.9), pluss::make_series(0.7),␣
↪pluss::make_series(0.8)})},
});
```

35.1.86 show

Signature

```
void show(self, std::ostream& os) const
```

Parameters

os : std::ostream& See the signature for the supported argument form.

Returns

void Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.show(std::cout);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.87 slice

Signature

```
Registry slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

start : std::int64_t Inclusive starting position of the slice.

stop : std::int64_t Exclusive ending position of the slice.

Returns

Registry Value produced by this call.

Notes

Returns a contiguous positional slice `[start, stop)` across every slot.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.slice(0, 2);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.88 `slot`

Signature

```
Slot slot(self, std::string_view key) const
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

Returns

`Slot` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.slot("ids");
```

Output

```
Slot([Series(1), Series(2), Series(3)])
```

35.1.89 slot_sizes

Signature

```
std::unordered_map<std::string, std::int64_t> slot_sizes(self) const
```

Parameters

None

Returns

`std::unordered_map<std::string, std::int64_t>` Value produced by this call.

Notes

Returns the current length of each slot.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.slot_sizes();
```

Output

```
{"ids": 3, "city": 3}
```

35.1.90 slots

Signature

```
std::vector<Slot> slots(self) const
```

Parameters

None

Returns

`std::vector<Slot>` Materialized collection of slot objects.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.slots();
```

Output

```
[Slot(ids), Slot(city)]
```

35.1.91 tail

Signature

```
Registry tail(self, std::int64_t n) const
```

Parameters

`n` : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

Returns

`Registry` Value produced by this call.

Notes

Returns the last `n` values from every slot in the registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.tail(2);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

(continues on next page)

(continued from previous page)

```
out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.92 take

Signature

```
Registry take(self, const std::vector<std::int64_t>& indices) const
```

Parameters

`indices` : `const std::vector<std::int64_t>&` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Returns the values at the requested integer positions across every slot.

See also

- `Registry take(self, std::span<const std::int64_t> indices) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::int64_t> picks{0, 2};
auto out = rg.take(picks);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.93 take

Signature

```
Registry take(self, std::span<const std::int64_t> indices) const
```

Parameters

`indices` : `std::span<const std::int64_t>` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Returns the values at the requested integer positions across every slot.

See also

- `Registry take(self, const std::vector<std::int64_t>& indices) const`

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::int64_t> picks{0, 2};
auto out = rg.take(picks);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.94 take_values

Signature

```
Registry take_values(self, std::string_view key, std::span<const std::int64_t> indices)␣
↪const
```

Parameters

key : std::string_view Name of the target slot or stream.

indices : std::span<const std::int64_t> See the signature for the supported argument form.

Returns

Registry Value produced by this call.

Notes

Selects values from one named slot by integer position and returns the updated registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::int64_t> picks{0, 2};
auto out = rg.take_values("ids", picks);
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.95 take_values_

Signature

```
Registry& take_values_(self, std::string_view key, std::span<const std::int64_t> indices)
```

Parameters

key : std::string_view Name of the target slot or stream.

indices : std::span<const std::int64_t> See the signature for the supported argument form.

Returns

Registry& Value produced by this call.

Notes

Selects values from one named slot by integer position in place.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

std::vector<std::int64_t> picks{0, 2};
rg.take_values_("ids", picks);
```

35.1.96 to_json

Signature

```
std::string to_json(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.to_json();
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.97 to_kindle

Signature

```
pluss::kindle::Catalog to_kindle(self) const
```

Parameters

None

Returns

`pluss::kindle::Catalog` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.to_kindle();
```

Output

```
id     -> Series([1, 2, 3])
metric -> Series([10.0, 20.0, 30.0])
city   -> Note(["Accra", "Tema", "Kumasi"])
```

35.1.98 to_string

Signature

```
std::string to_string(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.to_string();
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.99 transform_values

Signature

```
Registry transform_values(self, std::string_view key, const std::function
↪<Slot::Value(const Slot::Value&)>& fn) const
```

Parameters

key : std::string_view Name of the target slot or stream.

fn : const std::function<Slot::Value(const Slot::Value&)>& See the signature for the supported argument form.

Returns

Registry Value produced by this call.

Notes

Rewrites the values inside one named slot with a user function and returns the updated registry.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.transform_values("ids", [](const pluss::bundle::Slot::Value& value) ->␣
↪pluss::bundle::Slot::Value { if (std::holds_alternative<pluss::Series>(value)) return␣
↪std::get<pluss::Series>(value).add(10); return value; });
```

Output

```
rg.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])

out.head():
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

35.1.100 transform_values_

Signature

```
Registry& transform_values_(self, std::string_view key, const std::function
↪<Slot::Value(const Slot::Value&)>& fn)
```

Parameters

key : `std::string_view` Name of the target slot or stream.

fn : `const std::function<Slot::Value(const Slot::Value&)>&` See the signature for the supported argument form.

Returns

`Registry&` Value produced by this call.

Notes

Rewrites the values inside one named slot in place.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

rg.transform_values_("ids", [](const pluss::bundle::Slot::Value& value) ->␣
↪pluss::bundle::Slot::Value { if (std::holds_alternative<pluss::Series>(value)) return␣
↪std::get<pluss::Series>(value).add(10); return value; });
```

35.1.101 type_profiles

Signature

```
std::unordered_map<std::string, std::unordered_map<std::string, std::int64_t>> type_
↪profiles(self) const
```

Parameters

None

Returns

std::unordered_map<std::string, std::unordered_map<std::string, std::int64_t>> Value produced by this call.

Notes

Summarizes the value types stored inside each slot.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});

auto out = rg.type_profiles();
```

Output

```
{"ids": {"Series": 3}, "city": {"Note": 3}}
```

35.1.102 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Examples

```
auto rg = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2),␣
↪pluss::make_series(3)})},
    {"city", pluss::bundle::Slot({pluss::make_note("Accra"), pluss::make_note("Tema"),␣
↪pluss::make_note("Kumasi")})},
});
```

(continues on next page)

(continued from previous page)

```
auto out = rg.unique();
```

Output

```
true
```

CHAPTER

THIRTYSIX

AISLE API REFERENCE

CHAPTER

THIRTYSEVEN

PRACTICAL AISLE API REFERENCE

37.1 API Reference Style

This page documents the full public `Aisle` surface for depth-stacked registries.

37.1.1 `apply_slot`

Signature

```
Aisle apply_slot(self, std::string_view key, const std::function<Slot(const Slot&)>& fn)␣
↪const
```

Parameters

`key` : `std::string_view` Name of the target slot or stream.

`fn` : `const std::function<Slot(const Slot&)>&` See the signature for the supported argument form.

Returns

`Aisle` Value produced by this call.

Notes

Applies a slot transform to one named key inside every registry and returns the updated aisle.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.apply_slot("ids", [](const pluss::bundle::Slot& slot) { return slot; });
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

37.1.2 as_kindle

Signature

```
pluss::kindle::Archive as_kindle(self) const
```

Parameters

None

Returns

`pluss::kindle::Archive` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.as_kindle();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

37.1.3 concat_depth

Signature

```
Aisle concat_depth(self, const Aisle& other) const
```

Parameters

`other` : `const Aisle&` See the signature for the supported argument form.

Returns

`Aisle` Value produced by this call.

Notes

Appends the registries from another aisle onto the current depth axis.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.concat_depth(aisle);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

37.1.4 `defined`

Signature

```
bool defined(self) const
```

Parameters

`None`

Returns

`bool` Boolean status or membership result.

Notes

Reports whether the current object owns a concrete payload that can be used safely in later operations.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.defined();
```

Output

```
true
```

37.1.5 depth

Signature

```
std::int64_t depth(self) const
```

Parameters

None

Returns

`std::int64_t` Integer count, size, or positional result.

Notes

Returns a size/count summary for the current object without changing it.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.depth();
```

Output

```
2
```

37.1.6 empty

Signature

```
bool empty(self) const
```

Parameters

None

Returns

`bool` Boolean status or membership result.

Notes

Reports whether the current object has no rows, entries, slots, or depth items.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.empty();
```

Output

```
false
```

37.1.7 from_kindle

Signature

```
static Aisle from_kindle(const pluss::kindle::Archive& archive)
```

Parameters

`archive` : `const pluss::kindle::Archive&` See the signature for the supported argument form.

Returns

`Aisle` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
using namespace pluss::kindle::literals;

auto c0 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({10.0, 20.0})),
});
auto c1 = pluss::kindle::Catalog({
    "metric"_k = pluss::kindle::Stream(pluss::make_series({30.0, 40.0})),
});
auto archive = pluss::kindle::Archive({c0, c1}, pluss::kindle::Stream(pluss::make_note({
↪"train", "test"})));

auto out = pluss::bundle::Aisle::from_kindle(archive);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

37.1.8 head

Signature

```
Aisle head(self, std::int64_t n) const
```

Parameters

n : std::int64_t Number of rows, entries, or depth items to keep in the preview.

Returns

Aisle Value produced by this call.

Notes

Returns the first n registries from the aisle depth axis.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.head(1);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

37.1.9 kindelize

Signature

```
pluss::kindle::Archive kindelize(self) const
```

Parameters

None

Returns

`pluss::kindle::Archive` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.kindelize();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

37.1.10 `labels`

Signature

```
const std::optional<Slot>& labels(self) const
```

Parameters

None

Returns

`const std::optional<Slot>&` Value produced by this call.

Notes

Returns the current labels used to address fields, slots, rows, or depth items.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.labels();
```

Output

```
["train", "test"]
```

37.1.11 map

Signature

```
Aisle map(self, const std::function<Registry(const Registry&)>& fn) const
```

Parameters

fn : const std::function<Registry(const Registry&)>& See the signature for the supported argument form.

Returns

Aisle Value produced by this call.

Notes

Applies the same function to each registry and returns the transformed aisle.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.map([](const pluss::bundle::Registry& registry) { return registry; });
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

37.1.12 registries

Signature

```
const std::vector<Registry>& registries(self) const
```

Parameters

None

Returns

`const std::vector<Registry>&` Borrowed collection of registries.

Notes

Materializes the current members as a collection you can inspect or reuse elsewhere.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.registries();
```

Output

```
[Registry(depth0), Registry(depth1)]
```

37.1.13 `registry`

Signature

```
Registry registry(self, std::int64_t i) const
```

Parameters

`i` : `std::int64_t` See the signature for the supported argument form.

Returns

`Registry` Value produced by this call.

Notes

Fetches one value, member, or slice from the current object without changing the source object.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.registry(0);
```

Output

```
ids  -> Slot([Series(1), Series(2), Series(3)])
city -> Slot([Note("Accra"), Note("Tema"), Note("Kumasi")])
```

37.1.14 select

Signature

```
Aisle select(self, std::initializer_list<std::string_view> keys) const
```

Parameters

keys : std::initializer_list<std::string_view> Collection of slot or stream names to keep, drop, or rewrite together.

Returns

Aisle Value produced by this call.

Notes

Keeps only the named keys inside every registry in the aisle.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.select({"ids"});
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

37.1.15 show

Signature

```
void show(self, std::ostream& os) const
```

Parameters

os : std::ostream& See the signature for the supported argument form.

Returns

`void` Performs the side effect and returns no value.

Notes

This renders a human-readable view for inspection instead of creating a new analytic object.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

aisle.show(std::cout);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

37.1.16 `slice`

Signature

```
Aisle slice(self, std::int64_t start, std::int64_t stop) const
```

Parameters

`start` : `std::int64_t` Inclusive starting position of the slice.

`stop` : `std::int64_t` Exclusive ending position of the slice.

Returns

`Aisle` Value produced by this call.

Notes

Returns a contiguous depth slice `[start, stop)` from the aisle.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});
```

(continues on next page)

(continued from previous page)

```
auto out = aisle.slice(0, 1);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

37.1.17 tail

Signature

```
Aisle tail(self, std::int64_t n) const
```

Parameters

n : `std::int64_t` Number of rows, entries, or depth items to keep in the preview.

Returns

`Aisle` Value produced by this call.

Notes

Returns the last `n` registries from the aisle depth axis.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.tail(1);
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

37.1.18 to_json

Signature

```
std::string to_json(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.to_json();
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

37.1.19 to_kindle

Signature

```
pluss::kindle::Archive to_kindle(self) const
```

Parameters

None

Returns

`pluss::kindle::Archive` Value produced by this call.

Notes

This is an explicit bridge between PlussMic models so you can switch to the representation that fits the next task.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
```

(continues on next page)

(continued from previous page)

```
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.to_kindle();
```

Output

```
train -> Catalog(metric -> Series([10.0, 20.0]))
test  -> Catalog(metric -> Series([30.0, 40.0]))
```

37.1.20 to_string

Signature

```
std::string to_string(self) const
```

Parameters

None

Returns

`std::string` Rendered string result.

Notes

Serializes or renders the current object into a text form you can inspect, log, or export.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.to_string();
```

Output

```
depth0 -> Registry(ids -> Slot([Series(1), Series(2)]))
depth1 -> Registry(ids -> Slot([Series(3), Series(4)]))
```

37.1.21 unique

Signature

```
bool unique(self) const
```

Parameters

None

Returns

bool Boolean status or membership result.

Notes

Reports whether the object satisfies the uniqueness constraint expected by its model, such as distinct labels or keys.

Examples

```
auto r0 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(1), pluss::make_series(2)})},
});
auto r1 = pluss::bundle::Registry({
    {"ids", pluss::bundle::Slot({pluss::make_series(3), pluss::make_series(4)})},
});
auto aisle = pluss::bundle::Aisle({r0, r1});

auto out = aisle.unique();
```

Output

```
true
```

CHAPTER

THIRTYEIGHT

BUILD OUTPUTS

38.1 HTML

```
cd docs/sphinx
make html
```

Expected output:

- `_build/html/index.html`

38.2 PDF

```
cd docs/sphinx
make pdf
```

Expected output:

- `_build/latex/plussmic_user_docs.pdf`

38.3 Notes

- HTML uses the same custom left-sidebar/top-bar shell as the PlussQit docs.
- PDF is built through `latex` plus `latexmk`.
- Run the commands from an environment where Sphinx and the PDF toolchain are already available.
- If HTML looks stale after a rebuild, hard refresh the browser once.